Michael R. Berthold · Christian Borgelt ·
Frank Höppner · Frank Klawonn ·
Rosaria Silipo

Guide to Intelligent Data Science

How to Intelligently Make Use of Real Data

Second Edition

Springer

Contents

1 Introduction 1
1.1 Motivation 1
1.1.1 Data and Knowledge 2
1.1.2 Tycho Brahe and Johannes Kepler 4
1.1.3 Intelligent Data Science 6
1.2 The Data Science Process 7
1.3 Methods, Tasks, and Tools 11
1.4 How to Read This Book 13
References 14

2 Practical Data Science: An Example 15
2.1 The Setup 15
2.2 Data Understanding and Pattern Finding 16
2.3 Explanation Finding 19
2.4 Predicting the Future 21
2.5 Concluding Remarks 23

3 Project Understanding 25
3.1 Determine the Project Objective 26
3.2 Assess the Situation 28
3.3 Determine Analysis Goals 30
3.4 Further Reading 31
References 32

4 Data Understanding 33
4.1 Attribute Understanding 34
4.2 Data Quality 37
4.3 Data Visualization 40
4.3.1 Methods for One and Two Attributes 40
4.3.2 Methods for Higher-Dimensional Data 48
4.4 Correlation Analysis 62
4.5 Outlier Detection 65

4.5.1 Outlier Detection for Single Attributes . . . 66
4.5.2 Outlier Detection for Multidimensional Data . . . 68
4.6 Missing Values . . . 69
4.7 A Checklist for Data Understanding . . . 72
4.8 Data Understanding in Practice . . . 73
4.8.1 Visualizing the Iris Data . . . 74
4.8.2 Visualizing a Three-Dimensional Data Set on a Two-Coordinate Plot . . . 82
References . . . 82

5 Principles of Modeling . . . 85
5.1 Model Classes . . . 86
5.2 Fitting Criteria and Score Functions . . . 89
5.2.1 Error Functions for Classification Problems . . . 91
5.2.2 Measures of Interestingness . . . 93
5.3 Algorithms for Model Fitting . . . 93
5.3.1 Closed-Form Solutions . . . 93
5.3.2 Gradient Method . . . 94
5.3.3 Combinatorial Optimization . . . 96
5.3.4 Random Search, Greedy Strategies, and Other Heuristics . . . 96
5.4 Types of Errors . . . 100
5.4.1 Experimental Error . . . 102
5.4.2 Sample Error . . . 109
5.4.3 Model Error . . . 110
5.4.4 Algorithmic Error . . . 111
5.4.5 Machine Learning Bias and Variance . . . 111
5.4.6 Learning Without Bias? . . . 112
5.5 Model Validation . . . 112
5.5.1 Training and Test Data . . . 112
5.5.2 Cross-Validation . . . 114
5.5.3 Bootstrapping . . . 114
5.5.4 Measures for Model Complexity . . . 115
5.5.5 Coping with Unbalanced Data . . . 121
5.6 Model Errors and Validation in Practice . . . 121
5.6.1 Scoring Models for Classification . . . 122
5.6.2 Scoring Models for Numeric Predictions . . . 124
5.7 Further Reading . . . 125
References . . . 125

6 Data Preparation . . . 127
6.1 Select Data . . . 127
6.1.1 Feature Selection . . . 128
6.1.2 Dimensionality Reduction . . . 133
6.1.3 Record Selection . . . 134
6.2 Clean Data . . . 136
6.2.1 Improve Data Quality . . . 136

6.2.2 Missing Values . . . 137
6.2.3 Remove Outliers . . . 139
6.3 Construct Data . . . 140
6.3.1 Provide Operability . . . 140
6.3.2 Assure Impartiality . . . 142
6.3.3 Maximize Efficiency . . . 144
6.4 Complex Data Types . . . 147
6.5 Data Integration . . . 148
6.5.1 Vertical Data Integration . . . 149
6.5.2 Horizontal Data Integration . . . 150
6.6 Data Preparation in Practice . . . 152
6.6.1 Removing Empty or Almost Empty Attributes and Records in a Data Set . . . 152
6.6.2 Normalization and Denormalization . . . 153
6.6.3 Backward Feature Elimination . . . 154
6.7 Further Reading . . . 155
References . . . 155

7 Finding Patterns . . . 157
7.1 Hierarchical Clustering . . . 159
7.1.1 Overview . . . 160
7.1.2 Construction . . . 162
7.1.3 Variations and Issues . . . 164
7.2 Notion of (Dis-)Similarity . . . 167
7.3 Prototype- and Model-Based Clustering . . . 173
7.3.1 Overview . . . 174
7.3.2 Construction . . . 175
7.3.3 Variations and Issues . . . 178
7.4 Density-Based Clustering . . . 181
7.4.1 Overview . . . 181
7.4.2 Construction . . . 182
7.4.3 Variations and Issues . . . 184
7.5 Self-organizing Maps . . . 187
7.5.1 Overview . . . 187
7.5.2 Construction . . . 188
7.6 Frequent Pattern Mining and Association Rules . . . 189
7.6.1 Overview . . . 191
7.6.2 Construction . . . 192
7.6.3 Variations and Issues . . . 199
7.7 Deviation Analysis . . . 206
7.7.1 Overview . . . 206
7.7.2 Construction . . . 207
7.7.3 Variations and Issues . . . 210
7.8 Finding Patterns in Practice . . . 211
7.8.1 Hierarchical Clustering . . . 211

7.8.2 *k*-Means and DBSCAN . . . 211
7.8.3 Association Rule Mining . . . 214
7.9 Further Reading . . . 214
References . . . 215

8 Finding Explanations . . . 219
8.1 Decision Trees . . . 220
8.1.1 Overview . . . 221
8.1.2 Construction . . . 222
8.1.3 Variations and Issues . . . 225
8.2 Bayes Classifiers . . . 230
8.2.1 Overview . . . 230
8.2.2 Construction . . . 231
8.2.3 Variations and Issues . . . 235
8.3 Regression . . . 241
8.3.1 Overview . . . 241
8.3.2 Construction . . . 243
8.3.3 Variations and Issues . . . 246
8.3.4 Two-Class Problems . . . 254
8.3.5 Regularization for Logistic Regression . . . 255
8.4 Rule learning . . . 258
8.4.1 Propositional Rules . . . 258
8.4.2 Inductive Logic Programming or First-Order Rules . . . 265
8.5 Finding Explanations in Practice . . . 267
8.5.1 Decision Trees . . . 267
8.5.2 Naïve Bayes . . . 268
8.5.3 Logistic Regression . . . 269
8.6 Further Reading . . . 270
References . . . 271

9 Finding Predictors . . . 273
9.1 Nearest-Neighbor Predictors . . . 275
9.1.1 Overview . . . 275
9.1.2 Construction . . . 277
9.1.3 Variations and Issues . . . 279
9.2 Artificial Neural Networks . . . 282
9.2.1 Overview . . . 283
9.2.2 Construction . . . 286
9.2.3 Variations and Issues . . . 290
9.3 Deep Learning . . . 292
9.3.1 Recurrent Neural Networks and Long-Short Term Memory Units . . . 293
9.3.2 Convolutional Neural Networks . . . 295
9.3.3 More Deep Learning Networks: Generative-Adversarial Networks (GANs) . . . 296
9.4 Support Vector Machines . . . 297

9.4.1 Overview . . . 298
9.4.2 Construction . . . 302
9.4.3 Variations and Issues . . . 303
9.5 Ensemble Methods . . . 304
9.5.1 Overview . . . 304
9.5.2 Construction . . . 306
9.5.3 Variations and Issues . . . 309
9.6 Finding Predictors in Practice . . . 312
9.6.1 *k* Nearest Neighbor (kNN) . . . 312
9.6.2 Artificial Neural Networks and Deep Learning . . . 312
9.6.3 Support Vector Machine (SVM) . . . 313
9.6.4 Random Forest and Gradient Boosted Trees . . . 314
9.7 Further Reading . . . 315
References . . . 315

10 Deployment and Model Management . . . 319
10.1 Model Deployment . . . 319
10.1.1 Interactive Applications . . . 320
10.1.2 Model Scoring as a Service . . . 320
10.1.3 Model Representation Standards . . . 320
10.1.4 Frequent Causes for Deployment Failures . . . 321
10.2 Model Management . . . 322
10.2.1 Model Updating and Retraining . . . 323
10.2.2 Model Factories . . . 324
10.3 Model Deployment and Management in Practice . . . 324
10.3.1 Deployment to a Dashboard . . . 325
10.3.2 Deployment as REST Service . . . 326
10.3.3 Integrated Deployment . . . 327
References . . . 328

A Statistics . . . 329
A.1 Terms and Notation . . . 330
A.2 Descriptive Statistics . . . 331
A.2.1 Tabular Representations . . . 331
A.2.2 Graphical Representations . . . 332
A.2.3 Characteristic Measures for One-Dimensional Data . . . 335
A.2.4 Characteristic Measures for Multidimensional Data . . . 342
A.2.5 Principal Component Analysis . . . 344
A.3 Probability Theory . . . 350
A.3.1 Probability . . . 350
A.3.2 Basic Methods and Theorems . . . 353
A.3.3 Random Variables . . . 359
A.3.4 Characteristic Measures of Random Variables . . . 365
A.3.5 Some Special Distributions . . . 369
A.4 Inferential Statistics . . . 375
A.4.1 Random Samples . . . 376

A.4.2 Parameter Estimation 376
A.4.3 Hypothesis Testing 388

B KNIME 395
B.1 Installation and Overview 395
B.2 Building Workflows 398
B.3 Example Workflow 400

References 409

Index 411

Symbols

A, A_i	attribute, variable [e.g., $A_1 = color$, $A_2 = price$, $A_3 = category$]
ω	a possible value of an attribute [e.g., $\omega = red$]
Ω, dom$(\cdot)$	set of possible values of an attribute [e.g., $\Omega_1 = \Omega_{\text{color}} = \text{dom}(A_i) = \{red, blue, green\}$]
$\mathcal{A}$	set of all attributes [e.g., $\mathcal{A} = \{color, price, category\}$]
m	number of considered attributes [e.g., 3]
x	a specific value of an attribute [e.g., $x_2 = x_{\text{price}} = 4000$]
$\mathcal{X}$	space of possible data records [e.g., $\mathcal{X} = \Omega_{A_1} \times \cdots \times \Omega_{A_m}$]
$\mathcal{D}$	set of all records, data set, $\mathcal{D} \subseteq \mathcal{X}$ [e.g., $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$]
n	number of records in data set
$\mathbf{x}$	record in database [e.g., $\mathbf{x} = (x_1, x_2, x_3) = (red, 4000, luxury)$]
$\mathbf{x}_A$	attribute A of record $\mathbf{x}$ [e.g., $\mathbf{x}_{\text{price}} = 4000$]
$\mathbf{x}_{2,A}$	attribute A of record $\mathbf{x}_2$
$\mathcal{D}_{A=v}$	set of all records $\mathbf{x} \in \mathcal{D}$ with $\mathbf{x}_A = v$
C	a selected categorical target attribute [e.g., $C = A_3 = category$]
Ω_C	set of all possible classes [e.g., $\Omega_C = \{\text{quits, stays, unknown}\}$]
Y	a selected continuous target attribute [e.g., $Y = A_2 = price$]
$\mathcal{C}$	cluster (set of associated data objects) [e.g., $\mathcal{C} \subseteq \mathcal{D}$]
c	number of clusters
$\mathcal{P}$	partition, set of clusters $\{\mathcal{C}_1, \ldots, \mathcal{C}_c\}$
$p_{i\|j}$	membership degree of data #j to cluster #i
$[p_{i\|j}]$	membership matrix
$d_\cdot$	distance function, metric (d_E: Euclidean)
$[d_{i,j}]$	distance matrix

Introduction 1

In this introductory chapter we provide a brief overview of some core ideas of data science and their motivation. In a first step we carefully distinguish between "data" and "knowledge" in order to obtain clear notions that help us to work out why it is usually not enough to simply collect data and why we have to strive to turn them into knowledge. As an illustration, we consider a well-known example from the history of science. In a second step we characterize the data science process, also often referred to as the knowledge discovery process, in which modeling is one important step. We characterize standard data science tasks and summarize the catalog of methods to tackle them.

1.1 Motivation

Every year that passes brings us more powerful computers, faster and cheaper storage media, and higher bandwidth data connections. Due to these technological advancements, it is possible nowadays to collect and store enormous amounts of data with little effort and at impressively low costs. As a consequence, more and more companies, research centers, and governmental institutions create huge archives of tables, documents, images, and sounds in electronic form. Since for centuries lack of data has been a core hindrance to scientific and economic progress, we feel compelled to think that we can solve—at least in principle—basically any problem we are faced with if only we have enough data.

However, a closer examination of the matter reveals that this is an illusion. Data alone, regardless of how voluminous they are, are not enough. Even though large databases allow us to retrieve many different single pieces of information and to compute (simple) aggregations (like average monthly sales in Berlin), general patterns, structures, and regularities often go undetected. We may say that in the vast amount of data stored in some data repositories we cannot see the wood (the patterns) for the trees (the individual data records). However, it is most often exactly these patterns, regularities, and trends that are particularly valuable if one desires, for example, to increase sales in a supermarket. Suppose, for instance, a supermarket manager discovers that by analyzing sales and customer records certain products

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_1

are frequently bought together. In such a case sales can sometimes be stimulated by cleverly arranging these products on the shelves of the market (they may, for example, be placed close to each other, or may be offered as a bundle, in order to invite even more customers to buy them together).

Unfortunately, it turns out to be harder than may be expected at first sight to actually discover such patterns and regularities and thus to exploit a larger part of the information that is contained in the available data. In contrast to the overwhelming flood of data there was, at least at the beginning, a lack of tools by which raw data could be transformed into useful information. More than 20 years ago John Naisbett aptly characterized the situation by saying [4]: "We are drowning in information, but starving for knowledge." As a consequence, a new research area has been developed, which has become known under the name of *data science*. The goal of this area was to meet the challenge to develop tools that can help humans to find potentially useful patterns in their data and to solve the problems they are facing by making better use of the data they have. Today, more than 20 years later, a lot of progress has been made, and a considerable number of methods and implementations of these techniques in software tools have been developed. Still it is not the tools alone, but the *intelligent composition* of human intuition with the computational power, of sound background knowledge with computer-aided modeling, of critical reflection with convenient automatic model construction, that leads *data science* projects to success [2]. In this book we try to provide a hands-on approach to many basic data science techniques and how they are used to solve data science problems if relevant data is available.

1.1.1 Data and Knowledge

In this book we distinguish carefully between *data* and *knowledge*. Statements like "Columbus discovered America in 1492" or "Mister Smith owns a VW Beetle" are **data**. Note that we ignore whether we already know these statements or whether we have any concrete use for them at the moment. The essential property of these statements we focus on here is that they refer to single events, objects, people, points in time, etc. That is, they generally refer to single instances or individual cases. As a consequence, their domain of application and thus their utility is necessarily limited.

In contrast to this, **knowledge** consists of statements like "All masses attract each other" or "Every day at 7:30 a.m. a train with destination Rome departs from Zurich main station." Again, we neglect the relevance of these statements for our current situation and whether we already know them. Rather, we focus on the essential property that they do *not* refer to single instances or individual cases but are general rules or (physical) laws. Hence, if they are true, they have a large domain of application. Even more importantly, though, they allow us to make predictions and are thus highly useful (at least if they are relevant to us).

We have to admit, though, that in daily life we also call statements like "Columbus discovered America in 1492" knowledge (actually, this particular statement is used as a kind of prototypical example of knowledge). However, we neglect here

this vernacular and rather fuzzy use of the notion "knowledge" and express our regrets that it is not possible to find a terminology that is completely consistent with everyday speech. Neither single statements about individual cases nor collections of such statements qualify, in our use of the term, as knowledge.

Summarizing, we can characterize data and knowledge as follows:

data

- refer to single instances
  (single objects, people, events, points in time, etc.)
- describe individual properties
- are often available in large amounts
  (databases, archives)
- are often easy to collect or to obtain
  (e.g., scanner cashiers in supermarkets, Internet)
- do not allow us to make predictions or forecasts

knowledge

- refers to *classes* of instances
  (*sets* of objects, people, events, points in time, etc.)
- describes general patterns, structures, laws, principles, etc.
- consists of as few statements as possible
  (this is actually an explicit goal, see below)
- is often difficult and time consuming to find or to obtain
  (e.g., natural laws, education)
- allows us to make predictions and forecasts

These characterizations make it very clear that generally knowledge is much more valuable than (raw) data. Its generality and the possibility to make predictions about the properties of new cases are the main reasons for this superiority.

It is obvious, though, that not all kinds of knowledge are equally valuable as any other. Not all general statements are equally important, equally substantial, equally significant, or equally useful. Therefore knowledge has to be assessed, so that we do not drown in a sea of irrelevant knowledge. The following list (which we do not claim to be complete) lists some of the most important criteria:

criteria to assess knowledge

- correctness (probability, success in tests)
- generality (domain and conditions of validity)
- usefulness (relevance, predictive power)
- comprehensibility (simplicity, clarity, parsimony)
- novelty (previously unknown, unexpected)

In the domain of science, the focus is on correctness, generality, and simplicity (parsimony) are in the focus: one way of characterizing science is to say that it is the search for a minimal, correct description of the world. In economy and industry,

however, the emphasis is placed on usefulness, comprehensibility, and novelty: The main goal is to gain a competitive edge and thus to increase revenues. Nevertheless, neither of the two areas can afford to neglect the other criteria.

1.1.2 Tycho Brahe and Johannes Kepler

We illustrate the considerations of the previous section with an (at least partially) well-known example from the history of science. In the sixteenth century studying the stars and the planetary motions was one of the core areas of research. Among its proponents was Tycho Brahe (1546–1601), a Danish nobleman and astronomer, who in 1576 and 1584, with the financial help of King Frederic II, built two observatories on the island of Ven, about 32 km north-east of Copenhagen. He had access to the best astronomical instruments of his time (but no telescopes, which were used only later by Galileo Galilei (1564–1642) and Johannes Kepler (see below) to observe celestial bodies), and he used them to determine the positions of the sun, the moon, and the planets with a precision of less than one angle minute. With this precision he managed to surpass all measurements that had been carried out before and to actually reach the theoretical limit for observations with the unaided eye (that is, without the help of telescopes). Working carefully and persistently, he recorded the motions of the celestial bodies over several years.

Stated plainly, Tycho Brahe collected data about our planetary system, fairly large amounts of data, at least from the point of view of the sixteenth century. However, he failed to find a consistent scheme to combine them, could not discern a clear underlying pattern—partially because he stuck too closely to the geocentric system (the earth is in the center, and all planets, the sun, and the moon revolve around the earth). He could tell the precise location of Mars on any given day of the year 1582, but he could not connect its locations on different days by a clear and consistent theory. All hypotheses he tried did not fit his highly precise data. For example, he developed the so-called Tychonic planetary system (the earth is in the center, the sun and the moon revolve around the earth, and the other planets revolve around the sun on circular orbits). Although temporarily popular in the seventeenth century, this system did not stand the test of time. From a modern point of view, we may say that Tycho Brahe had a "data science problem" (or "knowledge discovery problem"). He had obtained the necessary data but could not extract the hidden knowledge.

This problem was solved later by Johannes Kepler (1571–1630), a German astronomer and mathematician, who worked as an assistant of Tycho Brahe. Contrary to Brahe, he advocated the Copernican planetary system (the sun is in the center, the earth and all other planets revolve around the sun in circular orbits) and tried all his life to reveal the laws that govern the motions of the celestial bodies. His approach was almost radical for his time, because he strove to find a mathematical description. He started his investigations with the data Tycho Brahe had collected and which he extended in later years. After several fruitless trials and searches and long and cumbersome calculations (imagine no pocket calculators), Kepler finally succeeded. He managed to combine Tycho Brahe's data into three simple laws, which nowadays bear his name as **Kepler's laws**. After having realized in 1604 already that

the course of Mars is an ellipse, he published the first two of these laws in his work "Astronomia Nova" in 1609 [7] and the third law ten years later in his magnum opus "Harmonices Mundi" [5, 8]:

1. The orbit of every planet (including the earth) is an ellipse, with the sun at a focal point.
2. A line from the sun to the planet sweeps out equal areas during equal intervals of time.
3. The squares of the orbital periods of any two planets relate to each other like the cubes of the semimajor axes of their respective orbits: $T_1^2/T_2^2 = a_1^3/a_2^3$, and therefore generally $T \sim a^{\frac{3}{2}}$.

Tycho Brahe had collected a large amount of astronomical data, and Johannes Kepler found the underlying laws that can explain them. He discovered the hidden knowledge and thus became one of the first "data scientists" in history.

Today the works of Tycho Brahe are almost forgotten—few have even heard his name. His catalogs of celestial data are merely of historical interest. No textbook on astronomy contains excerpts from his measurements—and this is only partially due to the better measurement technology we have available today. His observations and precise measurements are raw data and thus suffer from a decisive drawback: they do not provide any insight into the underlying mechanisms and thus do not allow us to make predictions. Kepler's laws, on the other hand, are treated in basically all astronomy and physics textbooks, because they state the principles according to which planets and comets move. They combine all of Brahe's observations and measurements in three simple statements. In addition, they permit us to make predictions: if we know the location and the speed of a planet relative to the sun at any given moment, we can compute its future course by drawing on Kepler's laws.

How did Johannes Kepler find the simple astronomical laws that bear his name? How did he discover them in Tycho Brahe's long tables and voluminous catalogs, thus revolutionizing astronomy? We know fairly little about his searches and efforts. He must have tried a large number of hypotheses, most of them failing. He must have carried out long and cumbersome computations, repeating some of them several times to eliminate errors. It is likely that exceptional mathematical talent, hard and tenacious work, and a significant amount of good luck finally led him to success. What we can be sure of is that he did not possess a universally applicable procedure or method to discover physical or astronomical laws.

Even today we are not much further: there is still no silver bullet to hit on the right solution. It is still much easier to collect data, with which we are virtually swamped in today's "information society" (whatever this popular term actually means) than to discover knowledge. Automatic measurement instruments and scanners, digital cameras and computers, and an abundance of other automatic and semiautomatic devices have even relieved us of the burden of manual data collection. In addition, modern databases and the ability to store data in the cloud allow us to keep ever increasing amounts of data and to retrieve and to sample them easily. John Naisbett was perfectly right: "We are drowning in information, but starving for knowledge."

It took a distinguished researcher like Johannes Kepler several years (actually half a lifetime) to evaluate the data that Tycho Brahe had collected—data that from a modern point of view are negligibly few and of which Kepler actually analyzed closely only those about the orbit of Mars. Given this, how can we hope today to cope with the enormous amounts of data we are faced with every day? "Manual" analyses (like Kepler's) have long ceased to be feasible. Simple aids, like the visualization of data in charts and diagrams, even though highly useful and certainly a first and important step, quickly reach their limits. Thus, if we refuse to surrender to the flood of data, we are forced to develop and employ computer-aided techniques, with which data science can be simplified or even automated to some degree. These are the methods that have been and still are developed in the research areas of statistics, machine learning, data analysis, knowledge discovery in databases, and data mining. Even though these methods are far from replacing human beings like Johannes Kepler, especially since a mindless (or unintelligent) application can easily produce artifacts and misleading results, it is not entirely implausible to assume that Kepler, if he had been supported by these methods and tools, could have reached his goal a little earlier.

1.1.3 Intelligent Data Science

Many people associate any kind of data science with **statistics** (see also Appendix A, which provides a brief review). Statistics has a long history and originated from collecting and analyzing data about the population and the state in general.

Statistics can be divided into *descriptive* and *inferential statistics*. **Descriptive statistics** summarizes data without making specific assumptions about the data, often by characteristic values like the (empirical) mean or by diagrams like histograms. **Inferential statistics** provides more rigorous methods than descriptive statistics that are based on certain assumptions about the data generating random process. The conclusions drawn in inferential statistics are only valid if these assumptions are satisfied.

Typically, in statistics the first step of the data science process is to *design the experiment* that defines how data should be collected in order to be able to carry out a reliable analysis based on the obtained data. To capture this important issue, we distinguish between *experimental* and *observational studies*. In an **experimental study** one can control and manipulate the data generating process. For instance, if we are interested in the effects of certain diets on the health status of a person, we might ask different groups of people to stick to different diets. Thus we have a certain control over the data generating process. In this experimental study, we can decide which and how many people should be assigned to a certain diet.

In an **observational study** one cannot control the data generating process. For the same dietary study as above, we might simply ask people on the street what they normally eat. Then we have no control about which kinds of diets we get data and how many people we will have for each diet in our data.

No matter whether the study is experimental or observational, there are usually independence assumptions involved, and the data we collect should be representative. The main reason is that inferential statistics is often applied to *hypothesis testing* where, based on the collected data, we desire to either confirm or reject some hypothesis about the considered domain. In this case representative data and certain independencies are required in order to ensure that the test decisions are valid.

In contrast to hypothesis testing, **exploratory data analysis** is concerned with *generating hypotheses* from the collected data. In exploratory data analysis there are no or at least considerably weaker model assumptions about the data generating process. Most of the methods presented in this book fall into this category, since they are mostly universal methods designed to achieve a certain goal but are not based on a rigorous model as in inferential statistics.

The typical situation we assume in this book is that we already have the data. They might not have been collected in the best way, or in the way we would have collected them had we been able to design the experiment in advance. Therefore, it is often difficult to make specific assumptions about the data generating process. We are also mostly goal-oriented—that is, we ask questions like "Which customers will yield the highest profit"?—and search for methods that can help us to answer such questions or to solve our problems.

The opportunity of analyzing large real world data repositories that were initially collected for different purposes came with the availability of powerful tools and technologies that can process and analyze massive amounts of data, which is nowadays called **data science**.

In this book we strove to provide a comprehensive guide to intelligent data science, outlining the process and its phases, presenting methods and algorithms for various tasks and purposes, and illustrating them using a well known, freely available software platform. In this way we hope to offer a good starting point for anyone who wishes to become more familiar with the area of data science.

1.2 The Data Science Process

There are at least two typical situations in which data science may help us to find solutions to certain problems or provide answers to questions that arise. In the first case, the problem at hand is by no means new, but it is already solved as a matter of routine (e.g., approval of credit card applications, technical inspection during quality assurance, machine control by a plant operator, etc.). If data have been collected for the past cases together with the result that was finally achieved (such as poor customer performance, malfunction of parts, etc.), such historical data may be used to revise and optimize the presently used strategy to reach a decision. In the second case, a certain question arises for the first time, and only little experience is available, or the experience is not directly applicable to this new question (e.g., starting with a new product, preventing abuse of servers, evaluating a large experiment or survey). In such cases, it is supposed that data from related situations may be helpful to generalize the new problem or that unknown relationships can be discovered from the data to gain insights into this unfamiliar area.

What if we have no data at all? This situation does not usually occur in practice, since in most cases there is always *some* data. Especially in businesses huge amounts of data have been collected and stored for operational reasons in the past (e.g., billing, logistics, warranty claims) that may now be used to optimize various decisions or offer new options (e.g., predicting customer performance, reducing stock on hand, tracking causes of defects). So the right question should be: How do we know if we have enough *relevant* data? This question is not answered easily. If it actually turns out that the data are not sufficient, one option is to acquire new data to solve the problem. However, as already pointed out in the preceding section, the experimental design of data acquisition is beyond the scope of this book.

Historically there have been several proposals about what the data science process should look like, such as SEMMA (an acronym for *sample, explore, modify, model, assess*), CRISP-DM (an acronym for *CRoss Industry Standard Process for Data Mining* as defined by the CRISP-DM consortium) [3], or the KDD-process [4] (see [9] for a detailed comparison). In this book, we are going to adopt the CRISP-DM process, which has been developed by a consortium of large companies, and still represents the most widely used process model for data science today.

Thus, the data science process consists of six phases as shown in Fig. 1.1. Most of these phases are usually executed more than once, and the most frequent phase transitions are shown by arrows. The main objective of the first **project understanding** step (see Chap. 3) is to identify the potential benefit, as well as the risks and efforts of a successful project, such that a deliberate decision on conducting the full project can be made. The envisaged solution is also transferred from the project domain to a more technical, data-centered notion. This first phase is usually called business understanding, but we stick to the more general term *project understanding* to emphasize that our problem at hand may as well be purely technical in nature or a research project rather than economically motivated.

Next we need to make sure that we will have sufficient data at hand to tackle the problem. While we cannot know this for sure until the end of the project, we at least have to convince ourselves that there are enough relevant data. To achieve this, we proceed in the **data understanding** phase (see Chap. 4) with a review of the available databases and the information contained in the database fields, a visual assessment of the basic relationships between attributes, a data quality audit, an inspection of abnormal cases (outliers), etc. For instance, outliers appear to be abnormal in some sense and are often caused by faulty insertion, but sometimes they give surprising insights on closer inspection. Some techniques respond very sensitively to outliers, which is why they should be treated with special care. Another aspect is empty fields which may occur in the data for various reasons—ignoring them may introduce a systematic error in the results. By getting familiar with the data, typically first insights and hypotheses are gained. If we do not believe that the data suffice to solve the problem, it may be necessary to revise the project's objective.

So far, we have not changed any field of our data. However, this will be required to get the data into a shape that enables us to apply modeling tools. In the **data preparation** phase (Chap. 6) the data are selected, corrected, modified, even new attributes are generated, such that the prepared data sets best suit the problem and the

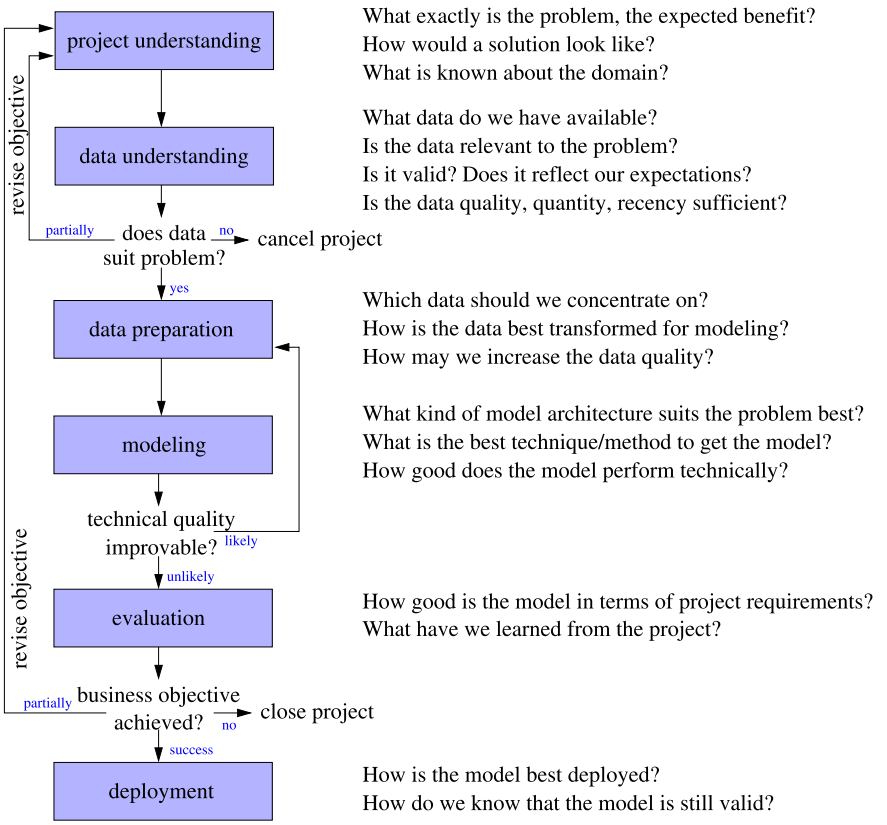


Fig. 1.1 Overview of the data science process together with typical questions to be asked in the respective phases

envisaged modeling technique. Basically all deficiencies that have been identified in the data understanding phase require special actions. Often the outliers and missing values are replaced by estimated values or true values obtained from other sources. We may restrict further analysis to certain variables and to a selection of the records from the full data set. Redundant and irrelevant data can give many techniques an unnecessarily hard time.

Once the data are prepared, we select and apply **modeling** tools to extract *knowledge* out of the data in the form of a *model* (Chaps. 5 and 7–9). Depending on what we want to do with the model, we may choose techniques that are easily interpretable (to gain insights) or less demonstrative black-box models, which may perform better. If we are not pleased with the results but are confident that the model can be improved, we step back to the data preparation phase and, say, generate new attributes from the existing ones, to support the modeling technique or to apply different techniques. Background knowledge may provide hints on useful *transformations* that simplify the representation of the solution.

Compared to the modeling itself, which is typically supported by efficient tools and algorithms, the data understanding and preparation phases take considerable part of the overall project time as they require a close manual inspection of the data, investigations into the relationships between different data sources, often even the analysis of the process that generated the data. New insights promote new ideas for feature generation or alter the subset of selected data, in which case the data preparation and modeling phases are carried out multiple times. The number of steps is not predetermined but influenced by the process and findings itself.

When the technical benchmarks cannot be improved anymore, the obtained results are analyzed in the **deployment** phase (Chap. 10) from the perspective of the problem owner. At this point, the project may stop due to unsatisfactory results, the objectives may be revised in order to succeed under a slightly different setting, or the found and optimized model may be deployed.

After **deployment**, which ranges from writing a report to the creation of a software system that applies the model automatically to aid or make decisions, the project is not necessarily finished. If the project results are used continuously over time, an additional monitoring and model management phase is necessary: During the analysis, a number of assumptions will be made, and the correctness of the derived model (and the decisions that rely on the model) depends on them. So we better verify from time to time that these assumptions still hold to prevent decision-making on outdated information.

In the literature one can find attempts to create cost models that estimate the costs associated with a data science project. Without going into the details, the major key factors that remained in a reduced cost model derived from 40 projects were [10]:

- the number of tables and attributes,
- the dispersion of the attributes (only a few vs. many values),
- the number of external data sources,
- the type of the model (prediction being the most expensive),
- the attribute type mixture (mixture of numeric and nonnumeric), and
- the familiarity of the staff with data science projects in general, the project domain in particular, and the software suites.

While there is not much we can do about the problem size, the goal of this book is to increase the familiarity with data science projects by going through each of the phases and providing first instructions to get along with the software suites.

It is obvious that this process requires lots of familiarity with the methods used, as well as experience with data science projects in general. So in recent years the question has arisen as to how less trained people can use data science techniques and how parts of or the entire process can be automated. Obviously, in order to apply sophisticated techniques, some understanding of the underlying theory is required—otherwise the results are potentially nonsensical. This is why we added the term "intelligent" in the title of this book. Our aim is to introduce sufficient details about the main steps of the data science process so that they can be applied in a thoughtful, intelligent way [6].

To address this, techniques for automated machine learning have been suggested that are able to automatically create, train, and test machine learning models with as little expert input as possible. There are different algorithms and strategies to do this which vary by complexity and performance, but the main idea is empowering business analysts to train a great number of models and deliver the best one with just a small amount of configuration [1, 11, 12]. These methods can work surprisingly well if the data are in appropriate shape but face limitations for less typical or more complex tasks. Systems for automation of other steps of the data science process appear increasingly, but they face similar limitations. More complex data science problems are likely to call for a higher degree of data science expertise in conjunction with business or problem insights. For example, the domain expert can add some unique knowledge about the data treatment and filtering before continuing with the machine learning process. Also, when the data domain becomes more complex than simple tabular data, e.g., includes text, images or time series, the human expert can contribute custom techniques for data preparation, data partitioning, and feature engineering.

The focus of this book is not on techniques to eliminate the need for data science expertise, but instead on providing deeper insights into the methods so that they can be applied in an intelligent way. This should not eliminate the use of some automation techniques where appropriate—but that decision alone already requires some level of data science expertise.

1.3 Methods, Tasks, and Tools

Problem Categories Every data science problem is different. To avoid the effort of inventing a completely new solution for each problem, it is helpful to think of different problem categories and consider them as building blocks from which a solution may be composed. These categories also help categorize the large number of different algorithms that solve specific tasks. Over the years, the following set of method categories has been established [4]:

- **Classification**
  Predict the outcome of an experiment with a finite number of possible results (like *yes*/*no* or *unacceptable*/*acceptable*/*good*/*very good*). We may be interested in a prediction because the true result will emerge in the future or because it is expensive, difficult, or cumbersome to determine it.
  Typical questions: *Is this customer credit-worthy? Will this customer respond to our mailing? Will the technical quality be acceptable?*
- **Regression**
  Regression is, just like classification, also a prediction task, but this time the value of interest is numerical in nature.
  Typical questions: *How will the EUR/USD exchange rate develop? How much money will the customer spend for vacation next year? How much will the machine's temperature change within the next cycle?*

- **Clustering, segmentation**
  Summarize the data to get a better overview by forming groups of similar cases (called clusters or segments). Instead of examining a large number of similar records, we need to inspect the group summary only. We may also obtain some insight into the structure of the whole data set. Cases that do not belong to any group may be considered as abnormal or outliers.
  Typical questions: *Do my customers divide into different groups? How many operating points does the machine have, and what do they look like?*
- **Association analysis**
  Find any correlations or associations to better understand or describe the interdependencies of all the attributes. The focus is on *relationships* between all attributes rather than on a single target variable or the cases (full record).
  Typical questions: *Which optional equipment of a car often goes together? How do the various qualities influence each other?*
- **Deviation analysis**
  Knowing already the major trends or structures, find any exceptional subgroup that behaves differently with respect to some target attribute.
  Typical questions: *Under which circumstances does the system behave differently? Which properties do those customers share who do not follow the crowd?*

The most frequent categories are *classification* and *regression*, because decision making always becomes much easier if reliable predictions of the near future are available. When a completely new area or domain is explored, cluster analysis and association analysis may help identify relationships among attributes or records. Once the major relationships are understood (e.g., by a domain expert), a deviation analysis can help focus on *exceptional situations* that deviate from regularity.

Catalog of Methods There are various methods in each of these categories to find reliable answers to the questions raised above. However, there is no such thing as a *single golden method* that works perfectly for all problems. To convey some idea which method may be best suited for a given problem, we will discuss various methods in Chaps. 7–9. However, in order to organize these chapters, we did not rely on the problem categories collected above, as some methods can be used likewise for more than one problem type. We rather used the intended task of the data analysis as a grouping criterion:

- **Finding patterns** (Chap. 7)
  If the domain (and therefore the data) is new to us or if we expect to find interesting relationships, we explore the data for new, previously unknown patterns. We want to get a full picture and do not concentrate on a single target attribute, yet. We may apply methods from, for instance, segmentation, clustering, association analysis, or deviation analysis.
- **Finding explanations** (Chap. 8)
  We have a special interest in some target variable and wonder why and how it varies from case to case. The primary goal is to gain new insights (knowledge) that may influence our decision making, but we do not necessarily intend au-

tomation. We may apply methods from, for instance, classification, regression, association analysis, or deviation analysis.

- **Finding predictors** (Chap. 9)
  We have a special interest in the prediction of some target variable, but it (possibly) represents only one building block of our full problem, so we do not really care about the *how* and *why* but are just interested in the best-possible prediction. We may apply methods from, for instance, classification or regression.

Tools for Data Science As already mentioned, the key to success is often the proper combination of data preparation and modeling techniques. Data analysis software suites are of great help as they reduce data formatting efforts and ease method linking. There is a (growing) list of commercial and free software suites and tools which we do not attempt to summarize here. Many online resources provide comparative summaries.

Although the choice of the software suite has considerable impact on the project time (usability) and can help avoid errors (because some of them are easily spotted using powerful visualization capabilities), the suites cannot take over the full analysis process. They provide at best an initial starting point (by means of analysis templates or project wizards), but in most cases the key factor is the intelligent combination of tools and background knowledge (regarding the project domain and the utilized tools). The suites exhibit different strengths, some focus on supporting the human data analyst by sophisticated graphical user interfaces, graphical configuration and reporting, while others are better suited for batch processing and automatization.

1.4 How to Read This Book

In the next chapter we will take a glimpse at the data science process by looking over the shoulder of Stan and Laura as they analyze their data (while only one of them actually follows the data science process). The chapter is intended to give an impression of what will be discussed in much greater detail throughout the book. The subsequent chapters follow the data science process stages: we analyze the problem first in Chap. 3 (project understanding) and then investigate whether the available data suit our purposes in terms of size, representativeness, and quality in Chap. 4 (data understanding). If we are confident that the data are worth carrying out the analysis, we discuss the data preparation (Chap. 6) as the last step before we enter the modeling phase (Chaps. 7–9). As already mentioned, data preparation is already tailored to the methods we are going to use for modeling; therefore, we have to introduce the principles of modeling already in Chap. 5. Deployment and monitoring is briefly addressed in Chap. 10. Readers who, over the years, have lost some of their statistical knowledge can (partially) recover it in Appendix A. The statistics appendix is not just a glossary of terms to quickly look up details but also serves as a book within the book for a few preparative lessons on statistics before delving into the chapters about data science.

Most chapters contain a section that equips the reader with the necessary information for some first hands-on experience using KNIME Analytics Platform. We have settled on KNIME Analytics Platform because it supports the composition of complex workflows in a graphical user interface. Appendix B provides a brief introduction to KNIME Analytics Platform. The workflows discussed in this book—and many more—are available for download at the book's website. We will continuously update this material and also provide examples using the popular data science languages R and Python.

References

1. Berthold, M.R.: Principles of guided analytics. KNIME Blog (2018)
2. Berthold, M., Hand, D.: Intelligent Data Analysis. Springer, Berlin (2009)
3. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., Wirth, R.: Cross Industry Standard Process for Data Mining 1.0, Step-by-Step Data Mining Guide. CRISP-DM consortium (2000)
4. Fayyad, U.M., Piatetsky-Shapiro, G., Smyth, P., Uthurusamy, R. (eds.): Advances in Knowledge Discovery and Data Mining. AAAI Press/MIT Press, Menlo Park/Cambridge (1996)
5. Feynman, R.P., Leighton, R.B., Sands, M.: The Feynman Lectures on Physics. Mechanics, Radiation, and Heat, vol. 1. Addison-Wesley, Reading (1963)
6. Hand, D.: Intelligent data analysis: issues and opportunities. In: Proc. 2nd Int. Symp. on Advances in Intelligent Data Analysis, pp. 1–14. Springer, Berlin (1997)
7. Kepler, J.: Astronomia Nova, aitiologetos seu physica coelestis, tradita commentariis de motibus stellae martis, ex observationibus Tychonis Brahe. (New Astronomy, Based upon Causes, or Celestial Physics, Treated by Means of Commentaries on the Motions of the Star Mars, from the Observations of Tycho Brahe) (1609); English edition: New Astronomy. Cambridge University Press, Cambridge (1992)
8. Kepler, J.: Harmonices Mundi (1619); English edition: The Harmony of the World. American Philosophical Society, Philadelphia (1997)
9. Kurgan, L.A., Musilek, P.: A survey of knowledge discovery and data mining process models. Knowl. Eng. Rev. **21**(1), 1–24 (2006)
10. Marban, O., Menasalvas, E., Fernandez-Baizan, C.: A cost model to estimate the effort of data mining process (DMCoMo). Inf. Syst. **33**, 133–150 (2008)
11. Tamagnini, P., Schmid, S., Dietz, C.: How to automate Machine Learning. KNIME Blog (2018)
12. Thornton, C., Hutter, F., Hoos, H., Leyton-Brown, K.: Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. In: KDD '13 Proc. 19th ACM SIGKDD, vol. 19, pp. 847–855 (2013)

Practical Data Science: An Example

2

Before talking about the full-fledged data science process and diving into the details of individual methods, this chapter demonstrates some typical pitfalls one encounters when analyzing real-world data. We start our journey through the data science process by looking over the shoulders of two (pseudo) data scientists, Stan and Laura, working on some hypothetical data science problems in a sales environment. Being differently skilled, they show how things should and should not be done. Throughout the chapter, a number of typical problems that data analysts meet in real work situations are demonstrated as well. We will skip algorithmic and other details here and only briefly mention the intention behind applying some of the processes and methods. They will be discussed in depth in subsequent chapters.

2.1 The Setup

Disclaimer The data and the application scenario used in this chapter are fictional. However, the underlying problems are motivated by actual problems which are encountered in real-world data science scenarios. Explaining particular applicational setups would have been entirely out of the scope of this book since, in order to understand the actual issue, a bit of domain knowledge is often helpful if not required. Please keep this in mind when reading the following. The goal of this chapter is to show (and sometimes slightly exaggerate) pitfalls encountered in real-world data science setups and not the reality in a supermarket chain. We are painfully aware that people familiar with this domain will find some of the encountered problems strange, to say the least. Have fun.

The Data For the following examples, we will use an artificial set of data sources from a hypothetical supermarket chain. The data set consists of a few tables, which have already been extracted from an in-house database:[1]

[1] Often just getting the data is a problem of its own. Data science assumes that you have access to the data you need—an assumption which is, unfortunately, frequently not true.

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_2

- **Customers**—Data about customers, stemming mostly from information collected when these customers signed up for frequent shopper cards;
- **Products**—A list of products with their categories and prices;
- **Purchases**—A list of products together with the date they were purchased and the customer card ID used during checkout.

The Analysts Stan and Laura are responsible for the analytics of the southern and northern parts, respectively, of a large supermarket chain. They were recently hired to help better understand customer groups and behavior and try to increase revenue in the local stores. As is unfortunately all too common, over the years the stores have already begun all sorts of data acquisition operations, but in recent years quite a lot of this data has been merged—however, still without a clear picture in mind. Many other stores had started to issue frequent shopping cards, so the directors of marketing of the southern and northern markets decided to launch a similar program. Lots of data have been recorded, and Stan and Laura now face the challenge to fit existing data to the questions posed. Together with their managers, they have sat down and defined three data science tasks to be addressed in the following year:

- Differentiating the different customer groups and their behavior to better understand their impact on the overall revenue,
- Identifying connections between products to allow for cross-selling campaigns, and
- Helping design a marketing campaign to attract core customers to increase their purchases.

Stan is a representative of the typical self-taught data science newbie with little experience on the job and some more applied knowledge about the different techniques, whereas Laura has some training in statistics, data processing, and data science process planning.

2.2 Data Understanding and Pattern Finding

The first analysis task is a standard data science setup: customer segmentation—find out which types of customers exist in your database and try to link them to the revenue they create. This can be used later to care for clientele who are responsible for the largest revenue source or foster groups of customers who are underrepresented. Grouping (or *clustering*) records in a database is the predominant method to find such customer segments: the data are partitioned into smaller subsets, each forming a more coherent group than the overall database contains. We will go into much more detail on this type of data science methods in Chap. 7. For now it suffices to know that some of the most prominent clustering methods return one typical example for each cluster. This essentially allows us to reduce a large data set to a small number of representative examples for the subgroups contained in the database.

Table 2.1 Stan's clustering result

Cluster-id	Age	Customer revenue
1	46.5	€ 1,922.07
2	39.4	€ 11,162.20
3	39.1	€ 7,279.59
4	46.3	€ 419.23
5	39.0	€ 4,459.30

The Naive Approach Stan quickly jumps onto the challenge, creates a dump of the database containing customer purchases and their birth date, and computes the age of the customers based on their birth date and the current day. He realizes that he is interested in customer clusters and therefore needs to somehow aggregate the individual purchases to their respective "owner." He uses an aggregating operator in his database to compute the total price of the shopping baskets for each customer. Stan then applies a well-known clustering algorithm which results in five prototypical examples, as shown in Table 2.1.

Stan is puzzled—he was expecting the clustering algorithm to return reasonably meaningful groups, but this result looks as if all shoppers are around 40–50 years old but spend vastly different amounts of money on products. He looks into some of the customers' data in some of these clusters but seems unable to find any interesting relations or any reason why some seem to buy substantially more than others. He changes some of the algorithm's settings, such as the number of clusters created, but the results are similarly uninteresting.

The Sound Approach Laura takes a different approach. Routinely she first tries to understand the available data and validates that some basic assumptions are in fact true. She uses a basis data summarization tool to report the different values for the string attributes. The distribution of first names seems to match the frequencies she would expect. Names such as "Michael" and "Maria" are most frequent, and "Rosemarie" and "Anneliese" appear a lot less often. The frequencies of the occupations also roughly match her expectations: the majority of the customers are employees, while the second and third groups are students and freelancers, respectively. She proceeds to checking the attributes holding numbers. In order to check the age of the customers, she also computes the customers' ages from their birth date and checks minimum and maximum. She spots a number of customers who obviously reported a wrong birthday, because they are unbelievably young. As a consequence, she decides to filter the data to only include people between the ages of 18 and 100. In order to explore the data more quickly, she reduces the overall customer data set to 5,000 records by random sampling and then plots a so-called histogram, which shows different ranges of the attribute *age* and how many customers fall into that range. Figure 2.1 shows the result of this analysis.

This view confirms Laura's assumptions—the majority of shoppers is middle aged, and the number of shoppers continuously declines toward higher age groups. She creates a second histogram to better inspect the subtle, but strange, cliff at around age 48 using finer setting for the bins. Figure 2.2 shows the result of this analysis.

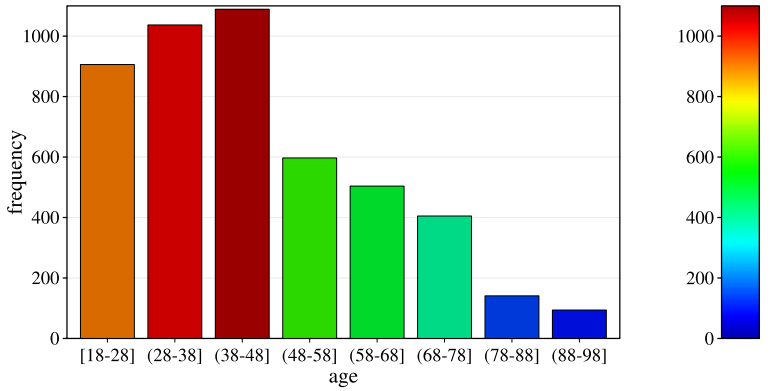


Fig. 2.1 A histogram for the distribution of the value of attribute *age* using 8 bins

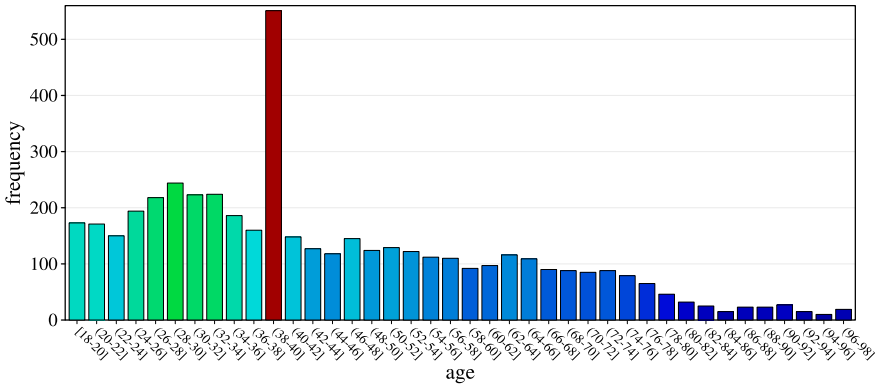


Fig. 2.2 A histogram for the distribution of the value of attribute *age* using 40 bins

Surprised, she notices the huge peak in the bin of ages 38–40. She discusses this observation with colleagues and the administrator of the shopping card database. They have no explanation for this odd concentration of 40-year-old people either (as of 2010). After a few other investigations, a colleague of the person who—before his retirement—designed the data entry forms suspects that this may have to do with the coding of missing birth dates. And, as it turns out, this is in fact the case: forms where people entered no or obviously nonsensical birth dates were entered into the form as zero values. For technical reasons, these zeros were then converted into the Java 0-date which turns out to be January 1, 1970. So these people all turn up with the same birth date in the customer database and in turn have the same age after the conversion Laura performed initially. Laura marks those entries in her database as "missing" in order to be able to distinguish them in future analyses.

Similarly, she inspects the shopping basket and product database and cleans up a number of other outliers and oddities. She then proceeds with the customer segmentation task. As in her previous data science projects, Laura first writes down her

Table 2.2 Laura's clustering result

Cluster	Age	Avg. cart price	Avg. purchases/month
1	75.3	€ 19.–	5.6
2	42.1	€ 78.–	7.8
3	38.1	€ 112.–	9.3
4	30.6	€ 16.–	4.8
5	44.7	€ 45.–	3.7

domain knowledge in form of a cognitive map, indicating relationships and dependencies between the attributes of her database. Having thus recalled the interactions between the variables of interest, she is well aware that the length of the customer's history and the number of overall shopping trips affect the overall basket price, and so she settles on the average basket price as a better estimator for the value of a particular customer. She also considers distinguishing the different product categories, realizing that those, of course, also potentially affect the average price. For the first step, she adds the average number of purchases per month, another indicator for the revenue a customer brings in. Data aggregation is now a bit more complex, but the modern data science tool she is using allows her to do the required joining and pivoting operations effortlessly. Laura knows that clustering algorithms are very sensitive to attributes with very different magnitudes, so she normalizes the three attributes to make sure that all three contribute equally to the clustering result. Running the same clustering algorithm that Stan was using, with the same setting for the number of clusters to be found, she gets the result shown in Table 2.2.

Obviously, there is a cluster (#1) of older customers who have a relatively small average basket price. There is also another group of customers (#4) which seems to correlate to younger shoppers, also purchasing smaller baskets. The middle-aged group varies wildly in price, however. Laura realizes that this matches her assumption about family status—people with families will likely buy more products and hence combine more products into more expensive baskets, which seems to explain the difference between clusters #2/#3 and cluster #5. The latter also seem to shop significantly less often. She goes back and validates some of these assumptions by looking at shopping frequency and average basket size as well, and also determines the overall impact on store revenues for these different groups. She finally discusses these results with her marketing and campaign specialists to develop strategies to foster the customer groups which bring in the largest chunk of revenue and develop the ones which seem to be underrepresented.

2.3 Explanation Finding

The second analysis goal is another standard shopping basket analysis problem: find product dependencies in order to better plan campaigns.

The Naive Approach Stan recently read in a book on practical data science how association rules can arbitrarily find such connections in market basket data. He

runs the association rule mining algorithm in his favorite data science tool with the default settings and inspects the results. Among the top-ranked generated rules, sorted by their confidence, Stan finds the following output:

```
'foie gras' (p1231) <- 'champagne Don Huberto' (p2149),
      'truffle oil de Rossini' (p578) [s=1E-5, c=75%]
'Tortellini De Cecco 500g' (p3456)'
   <- 'De Cecco Sugo Siciliana' (p8764) [s=1E-5, c=60%]
```

He quickly infers that this representation must mean that foie gras is bought whenever champagne and truffle oil are bought together and similarly for the other rule. Stan knows that the confidence measure c is important, as it indicates the strength of the dependency (the first rule holds in 3 out of 4 cases). He considers the second measure of frequency s to be less important and deliberately ignores its fairly small value. The two rules shown above are followed by a set of other, similarly luxury/culinary product-oriented rules. Stan concludes that luxury products are clearly the most important products on the shelf and recommends to his marketing manager to launch a campaign to advertise some of the products on the right-hand side of these rules (champagne, truffle oil) to increase the sales of the left side (foie gras). In parallel, he increases orders for these products, expecting a recognizable increase in sales. He proudly sends the results of his analysis to Laura.

The Sound Approach Laura is puzzled by those nonintuitive results. She reruns the analysis and notices the support values of the rules extracted by Stan—some of the rules Stan extracted have a remarkably high confidence, and some almost forecast shopping behavior. However, they have very low support values, meaning that only a small number of shopping baskets containing the products were ever observed. The rules that Stan found are not representative at all for his customer base. To confirm this, she runs a quick query on her database and sees that, indeed, there is essentially no influence on the overall revenue.

She notices that the problem of low support is caused by the fact that Stan ran the analysis on product IDs, so in effect he was forcing the rules to differentiate between brands of champagne and truffle oil. She reruns the analysis based on the product categories instead, ranks them by a mix of support and confidence, and finds a number of association rules with substantially higher support:

```
tomatoes <- capers, pasta  [s=0.007, c=32%]
tomatoes <- apples  [s=0.013, c=22%]
```

Laura focuses on rules with a much higher support measure s than before and also realizes that the confidence measure c is significantly higher than one would expect by chance. The first rule seems to be triggered by a recent fashion of Italian cooking, whereas the apple/tomato-rule is a known aspect.

However, she is still irritated by one of the rules discovered by Stan, which has a higher than suspected confidence despite a relatively low support. Are there some gourmets among the customers who prefer a very specific set of products? Rerunning this analysis on the shopping card owners yields almost the same results, so the (potential) gourmets appear among their regular customers. Just to be sure, she

inspects how many different customers (resp. shopping cards) occur for baskets that support this rule. As she had conjectured, there is a very limited number of customers that seem to have a strong affection for these products. Those few customers have bought this combination frequently, thus inflating the overall support measure (which refers to shopping baskets). This means that the support in terms of the *number of customers* is even smaller than the support in terms of *number of shopping baskets*. The response to any kind of special promotion would fall even shorter than expected from Stan's rule.

Apparently the time period in which the analyzed data have been collected influences the results. Thinking about it, she develops an idea how to learn about changes in the customers shopping behavior: She identifies a few rules, some rather promising, other well-known facts, and decides to monitor those combinations on a regular basis (say, quarterly). She got to know that a chain of liquor stores will soon open a number of shops close to the own markets, so she picks some rules with beverages in their conclusion part to see if the opening has any impact on the established shopping patterns of the own customers. As she fears a loss of potential sales, she plans a comparison of rules obtained not only over time but also among markets in the vicinity of such stores versus the other markets. She wonders whether promoting the products in the rule's antecedent may help bring back the customer and decides to discuss this with the marketing & sales team to determine if and where appropriate campaigns should be launched, once she has the results of her analysis.

2.4 Predicting the Future

The third and final analysis goal we consider in this brief overview is a forecasting or prediction problem. The idea is to find some relationship in our existing data that can help us predict if and how customers will react to coupon mailings and how this will affect our future revenue.

The Naive Approach Stan believes that no detailed analysis is required for this problem and notices that it is fairly straightforward to monitor success. At a competitor he has seen how discount coupons attract customers to purchase additional products. So he suggests launching a coupon campaign that gives customers a discount of 10% if they purchase products for more than €50. This coupon is mailed to all customers on record. Throughout the course of the next month, he carefully monitors his database and is positively surprised when he sees that his campaign is obviously working: the average price of shopping baskets is going up in comparison with previous months. However, at the end of the quarter he is shocked to see that overall revenues for the past quarter actually fell. His management is finally fed up with the lack of performance and fires Stan.

The Sound Approach Laura, who is promoted to head of analytics for the northern and southern supermarket chain, first cancels Stan's campaign and looks into the underlying data. She quickly realizes that even though quite a few customers did in fact use the coupons and increased their shopping baskets, their average number

of baskets per month actually went down—so quite a few people seem to have simply combined smaller shopping trips to be able to benefit from the discount offer. However, for some shoppers, the combined monthly shopping basket value did go up markedly, so there might be value here. Laura wonders how she can discriminate between those customers who simply use the coupons to discount their existing purchases and those who are actually enticed to purchase additional items. She notices that one of the earlier generated customer segments correlates better than others with the group of customers whose revenue went up—this fraction of customers is significantly higher than in the other groups. She considers using this very simple, manually designed predictor for a future campaign but wants to first make sure that she cannot do better with some smarter techniques. She decides that in the end it is not so important if she can actually understand the extracted model but only how well it performs.

To provide good starting points for the modeling technique, she decides to generate a few potentially informative attributes first. Models that rely on thousands of details typically perform poorly, so providing how often every product has been bought by the customer in the last month is not an option for her. To get robust models, she wants to aggregate the tiny bits of information, but what kind of aggregation could be helpful? She returns to her cognitive map to review the dependencies. One aspect is the availability of competitors: She reckons that customers may have alternative (possibly specialized) markets nearby but have been attracted by the coupon this time, keeping them away from the competitors. She decides to aggregate the money spent by the customer per month for a number of product types (such as *beverages*, thinking of the chain of liquor stores again). She conjectures that customers that perform well on average, but underperform in a specific segment only, may be enticed by the coupon to buy products for the underperforming segment also. Providing the segment performance before and after Stan's campaign should help a predictor detect such dependencies if they exist.

The cognitive map brings another idea to her mind: People who appreciate the full assortment but live somewhat further away from the own stores may see the coupon as a kind of travel compensation. So she adds a variable expressing a coarse estimation of the distance between the customer home and the nearest available market (which is only possible for the shopping card owners). She continues to use her cognitive map to address many different aspects and creates attributes that may help verify her hypotheses. She then investigates the generated attributes visually and also technically by means of feature selection methods.

After selecting the most promising attributes, she trains a classifier to distinguish the groups. She uses part of the data to simulate an independent test scenario and thereby evaluates the expected impact of a campaign—are the costs created by sending coupons to customers who do not purchase additional products offset by customers buying additional items? After some additional model fine-tuning, she reaches satisfactory performance. She discusses the results with the marketing&sales team and deploys the prediction system to control the coupon mailings for the next quarter. She keeps monitoring the performance of these coupon campaigns over future quarters and updates her model sporadically.

2.5 Concluding Remarks

In this chapter we have, very briefly and informally, touched upon a number of issues data scientists may encounter while making sense of real-world data. Many other problems can arise, and many more methods for data science exist in the academic literature and in real-world data science tools. We will attempt at covering the most prominent and most often used examples in the following chapters.

Note that one of the biggest problems data scientists very often have is that the data they get is not suited to answer the questions they are asked. For instance, if we were supposed to use the data in our customer database to find out how to differentiate Asian shopping behavior from European, we would have a very hard time. These data can only be used to distinguish between different types of European shoppers because it contains data from European markets only. Note also that we are (why ever) assuming that we used a nice, representative sample of all different types of European shoppers to generate the data—very often this is not the case, and the data themselves are already biased and will bias our analysis results—in this example we could be heavily biased by the type of supermarket chain we used to record the data in the first place. An upscale delicatessen supermarket will have dramatically different shopping patterns than a downscale discounter. We will be discussing these points later in more depth as well.

Project Understanding

3

We are at the beginning of a series of interdependent steps, where the project understanding phase marks the first. In this initial phase of the data analysis project, we have to map a problem onto one or many data analysis tasks. In a nutshell, we conjecture that the nature of the problem at hand can be adequately captured by some data sets (that still have to be identified or constructed), that appropriate modeling techniques can successfully be applied to learn the relationships in the data, and finally that the gained insights or models can be transferred back to the real case and applied successfully. This endeavor relies on a number of assumptions and is threatened by several risks, so the goal of the project understanding phase is to assess the main objective, the potential benefits, as well as the constraints, assumptions, and risks. While the number of data analysis projects is rapidly expanding, the failure rate is still high, so this phase should be carried out seriously to rate the chances of success realistically. The project understanding phase should be carried out with care to keep the project on the right track.

We have already sketched the data analysis process (CRISP-DM in Sect. 1.2). There is a clear order among the steps in the sense that for a later step, all precedent steps must have been executed. However, this does not mean that we can run once through all steps to deterministically achieve the desired results. There are many options and decisions to be made. Most of them will rely on our (subjective and dynamic) understanding of the problem at hand. The line of argument will not always be from an earlier phase to a later one. For instance, if a regression problem has to be solved, the analyst may decide that a certain method seems to be a promising choice for the modeling phase. From the characteristics of this technique he knows that all input data have to be transformed into numerical data, which has to be carried out beforehand (data preparation phase). This requires a careful look at the multivalued ordinally scaled attributes already in the data understanding phase to see how the order of the values is best preserved. If it is not considered in time, it may happen that later, in the evaluation phase, it turns out that the project owner expected to gain insights into the input–output relationship rather than having a black-box model only. If the analyst had considered this requirement beforehand, he might have chosen a different method. Changing this decision at any point later than in this initial

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_3

Table 3.1 Problems faced in data analysis projects, excerpt from [1]

Problem source	Project owner perspective	Analyst perspective
Communication	Project owner does not understand the technical terms of the analyst	Analyst does not understand the terms of the domain of the project owner
Lack of understanding	Project owner was not sure what the analyst could do or achieve Models of analyst were different from what the project owner envisioned	Analyst found it hard to understand how to help the project owner
Organization	Requirements had to be adopted in later stages as problems with the data became evident	Project owner was an unpredictable group (not so concerned with the project)

project understanding phase often renders some (if not most) of the earlier work in data understanding, data preparation, and modeling useless. While the time spent on project and data understanding compared to data preparation and modeling is small (20% : 80%), the importance to success is just the opposite (80% : 20%) [4].

3.1 Determine the Project Objective

As a first step, a *primary* objective (not a long list but one or two) and some success criteria in terms of the project domain have to be determined (who will decide which results are desired and whether the original project goal was achieved or not). This is much easier said than done, especially if the analysis is not carried out by the domain expert himself. In such cases the project owner and the analyst *speak different languages* which may cause misunderstandings and confusion. In the worst case, the communication problems lead to very soft project goals, just vague enough to allow every stakeholder to see his own perspective somehow accounted for. At the end, all of a sudden the stakeholders recognize that the results do not fit their expectations. The challenge here is usually not a matter of technical but of communicative competence.

Table 3.1 shows some of the typical problems that occur in such projects. To overcome language confusion, a glossary of terms, definition, acronyms, and abbreviations is inevitable. Knowing the terms still does not imply an understanding of the project domain, objectives, constraints, and influencing factors. One interviewing technique that may help getting most out of the expert is to rephrase all of her statements, which often provokes additional relativizing statements. Another technique is to use explorative tools such as mind or cognitive maps to sketch beliefs, experiences, and known factors, and how they influence each other.

An example of a **cognitive map** in the shopping domain considered in Sect. 2 is given in Fig. 3.1. Each node of this graph represents a property of the considered product or the customer. The variable of interest is placed in the center: How often will a certain product be found in the shopping basket of the customer? This depends on various factors, which are placed around this node. The direction of influence is

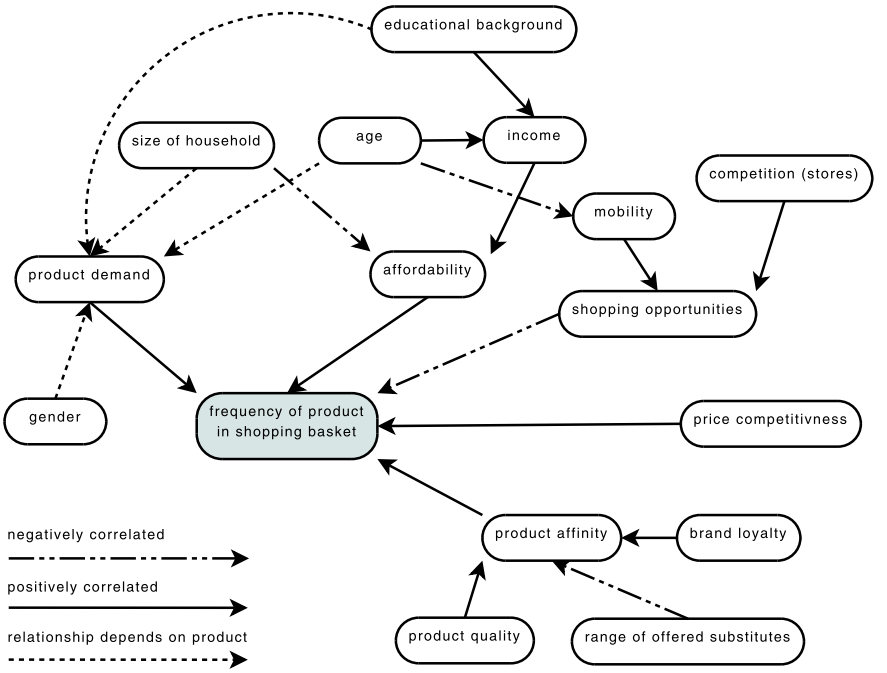


Fig. 3.1 A cognitive map for the shopping domain: How often will a certain product occur in a shopping basket of some customer? The positive correlation between *income* and *affordability* reads like *the higher the income, the higher the affordability*, whereas an example of a negative correlation reads like *the broader the range of offered substitutes, the lower the product affinity*

given by the arrows, and the line style indicates the way in which the variables influence each other: The higher the customer's affinity to the product, the more often it will be found in the basket. The author of the cognitive map conjectures that the product affinity itself is positively influenced by a high product quality and the customer's brand loyalty (a loyal customer is less likely to buy substitute products). On the other hand, the broader the range of offered substitutes, the more likely a customer may try out a different product. Other relationships depend on the product itself: The higher the demand of a certain product, the more often it will be found in the shopping basket, but the demand itself may, depending on the product, vary with gender (e.g., razor blades, hairspray), age (e.g., denture cleaner), or family status (e.g., napkins, fruit juice). The development of such a map supports the domain understanding and adjustment of expectations.

While constructing a cognitive map, a few rules should be adhered to: First, keep the map clear, only direct dependencies should be included in the graph. For instance, the size of the household influences the target variable, but only indirectly via the generated product demand and the affordability, and therefore there is no direct connection from *size of household* to *frequency of product in shopping basket*. Secondly, the labels of the nodes should be chosen carefully, so that they are easily

Table 3.2 Clarifying the primary objectives

Objective	Increase revenues (per campaign and/or per customer) in direct mailing campaigns by personalized offer and individual customer selection
Deliverable	Software that automatically selects a specified number of customers from the database to whom the mailing shall be sent, runtime max. half-day for database of current size
Success criteria	Improve order rate by 5% or total revenues by 5%, measured within 4 weeks after mailing was sent, compared to rate of last 3 mailings

interpretable when plugged into the relationship templates such as *the higher . . . , the more . . . .* As an example, the node *size of household* could have been named *family status*, but then it is not quite clear what *the more family status . . .* actually means.

Once an understanding of the domain has been achieved, the problem and primary objective have to be identified (see Table 3.2). Again, it is often useful to discuss or model the current solution first, for instance, by using techniques from software engineering (business process modeling, UML use cases, etc.) [3]. When the current solution has been elaborated, its advantages and disadvantages can be explored and discussed. Often, the primary objective is assumed to be known beforehand, probably the project would not have been initiated without having identified a problem first. But as there are many different ways to attack a problem, the objective should be precise about the direction to follow. A general statement about the goal is easily made ("model the profitable customers to increase the sales"), but it is often not precise enough (how do we precisely identify a profitable customer?) and not actionable (how exactly shall this model help increase the sales?). To render the objective more precise, it is necessary to sketch the target use already at this early stage. Thus it becomes clear what kind of result has to be delivered, which may range from a written technical report with interesting findings to a user-friendly software that uses the final model to automatize decisions.

From the perspective of the project owner, some of these elaborate steps may appear unnecessary—they master their domain already, after all. However, these steps must be considered as a preparation of the closely linked data understanding phase (see the next section). All the identified factors, situations, and relationships that are assumed to be relevant must be present and recognizable in the data. If they cannot be found in the data, either there is a misconception in the project understanding or (even worse) the data are not substantial or detailed enough to reflect the important relations of the real-world problem. In both cases, it would be fatal to miss this point and proceed unworried.

3.2 Assess the Situation

The next step is to estimate the chances of a successful data analysis project. This requires the review of available resources, requirements, and risks. The most important resources are data and knowledge, that is, databases and experts who can provide background information (about the domain in general and about the databases

in particular). Besides a plain listing of databases and personnel, it is important to clarify the access to both: If the data are stored in an operative system, mining the data may paralyze the applications using it. To become independent, it is advisable to provide a database dump. Experts are typically busy and difficult to grasp—but an inaccessible knowledge source is useless. A sufficiently large number of time slots for meetings should be arranged.

Based on the domain exploration (cognitive map, business process model, etc.), a list of explicit and implicit assumptions and risks is created to judge the chances of a successful project and guide the next steps. Data analysis lives on data. This list shall help convince ourselves that the data are meaningful and relevant to the project. Why should we undertake this effort? We will see whether we can build a model from these data later anyway. Unfortunately, this is only half of the truth. After reviewing a number of reports in a data analysis competition, Charles Elkan noted that "*when something surprising happens, rather than question the expectations, people typically believe that they should have done something slightly different*" [2]. Expecting that the problem can be solved with the given data may lead to continuously changing and "optimizing" the model—rather than taking the possibility into account that the data are not appropriate for this problem. In order to avoid this pitfall, the conjectured relations and expert-proven connections can help us verify that the given data satisfy our needs—or to put forward good reasons why the project will probably fail. This is particularly important as in many projects the available data have not been collected to serve the purpose that is intended now. To prevent us from carrying out an expensive project having almost no prospect of success, we have to carefully track all assumptions and verify them as soon as possible. Typical requirements and assumptions include:

- Requirements and constraints
  - *model requirements*,
    e.g., model has to be explanatory (because decisions must be justified clearly);
  - *ethical, political, legal issues*,
    e.g., variables such as gender, age, race must not be used;
  - *technical constraints*,
    e.g., applying the technical solution must not take more than n seconds;
- Assumptions
  - *representativeness*
    If conclusions about a specific target group are to be derived, a sufficiently large number of cases from this group must be contained in the database, and the sample in the database must be representative for the whole population.
  - *informativeness*
    To cover all aspects by the model, most of the influencing factors (identified in the cognitive map) should be represented by attributes in the database.
  - *good data quality*
    The relevant data must be of good quality (correct, complete, up-to-date) and unambiguous thanks to the available documentation.
  - *presence of external factors*

> We may assume that the external world does not change constantly—for instance, in a marketing project we may assume that the competitors do not change their current strategy or product portfolio at all.

Every assumption inherently represents a risk (there might be other risks though). If possible, a contingency should be sketched in case the assumption turns out to be invalid, including options such as the acquisition of additional data sources.

3.3 Determine Analysis Goals

Finally, the primary objective must be transformed into a more technical data mining goal. An architecture for the envisaged solution has to be found, composed of building blocks as discussed in Sect. 1.3 (data analysis tasks). For instance, this architecture might contain a component responsible for grouping the customers according to some readily available attributes first; another component finds interesting deviating subgroups in each of the groups; a third component predicts some variable of interest based on the customer data and the membership to the respective groups and subgroups. The better this architecture fits the actual situation, the better the chances of finding a model class that will prove successful in practice. To achieve this analogy, the discussions about the project domain are of great help.

Again there is the danger of accepting a reasonable architecture quickly, underestimating or even ignoring the great impact on the overall effort. Suppose that a company wants to increase the sales of some high-end product by direct mailing. One approach is to develop a model that predicts who will buy this product using the company's own customer database. Such a model might be interesting to interpret (useful for a report), but if it is used to decide to whom a mailing should be sent, most of the customers may have the product already (within the same customer database). Applying the model to people not being in the database is impossible as we lack the information about them that is needed by the model. The predictive model may also find out that customers buying the product were loyal customers for many years—but *artificially* increasing the duration of the customer relationship to support the purchase of the product is unfortunately impossible. If a foreseeable result is ignored or a misconception w.r.t. the desired use of the model is not recognized, considerable time may be wasted with building a correct model that turns out to be useless in the end.

For each of the building blocks, we can select a model class and technique to derive a model of this class automatically from data. There is nothing like *the unique best method for predictive tasks*, because they all have their individual weaknesses and strengths and it is impossible to combine all their properties or remove all biases (see Chap. 5). Although the final decision about the modeling technique will be made in the modeling phase, it should be clear already at this point of the analysis which properties the model should have and why. The methods and tools optimize the technical aspects of the model quality (such as accuracy, see also Chap. 5). Other aspects are often difficult to formalize and thus to optimize (such as interestingness

or interpretability), so that the choice of the model class has the greatest influence on these properties. Desirable properties may be, for instance:

- **Interpretability**
  If the goal of the analysis is a report that sketches possible explanations for a certain situation, the ultimate goal is to understand the delivered model. For some *black-box models*, it is hard to comprehend how the final decision is made, and their model lacks interpretability.
- **Reproduceability/stability**
  If the analysis is carried out more than once, we may achieve similar performance—but not necessarily similar models. This does no harm if the model is used as a black box, but hinders a direct comparison of subsequent models to investigate their differences.
- **Model flexibility/adequacy**
  A flexible model can adapt to more (complicated) situations than an inflexible model, which typically makes more assumptions about the real world and requires less parameters. If the problem domain is complex, the model learned from data must also be complex to be successful. However, with flexible models the risk of overfitting increases (will be discussed in Chap. 5).
- **Runtime**
  If restrictive runtime requirements are given (either for building or applying the model), this may exclude some computationally expensive approaches.
- **Interestingness** and **use of expert knowledge**
  The more an expert already knows, the more challenging it is to "surprise" him or her with new findings. Some techniques looking for associations (see Sect. 7.6) are known for their large number of findings, many of them redundant and thus uninteresting. So if there is a possibility of including any kind of previous knowledge, this may ease the search for the best model considerably, on the one hand, and may prevent us from rediscovering too many well-known artifacts.

When discussing the various modeling techniques in Chaps. 7–9, we will give hints which properties they possess. The final choice is then up to the analyst.

3.4 Further Reading

The books by Dorian Pyle [4, 5] offer many suggestions and constructive hints for carrying out the project understanding phase; [5] contains a step-by-step workflow for business understanding and data mining consisting of various *action boxes*. An organizationally grounded framework to formally implement the business understanding phase of data mining projects is presented in [6]. In [1] a template set for educing and documenting project requirements is proposed.

References

1. Britos, P., Dieste, O., García-Martínez, R.: Requirements elicitation in data mining for business intelligence projects. In: Advances in Information Systems Research, Education and Practice, pp. 139–150. IEEE Press, Piscataway (2008)
2. Elkan, C.: Magical thinking in data mining: lessons from coil challenge 2000. In: Proc. 7th Int. Conf. on Knowledge Discovery and Data Mining (KDD), pp. 426–431. ACM Press, New York (2001)
3. Marban, O., Segovia, J., Menasalvas, E., Fernandez-Baizan, C.: Towards data mining engineering: a software engineering approach. Inf. Syst. **34**, 87–107 (2009)
4. Pyle, D.: Data Preparation for Data Mining. Morgan Kaufmann, San Mateo (1999)
5. Pyle, D.: Business Modeling and Data Mining. Morgan Kaufmann, San Mateo (2003)
6. Sharma, S., Osei-Bryson, K.-M.: Framework for formal implementation of the business understanding phase of data mining projects. Expert Syst. Appl. **36**, 4114–4124 (2009)

Data Understanding

4

The main goal of data understanding is to gain general insights about the data that will potentially be helpful for the further steps in the data analysis process, but data understanding should not be driven exclusively by the goals and methods to be applied in later steps. Although these requirements should be kept in mind during data understanding, one should approach the data from a neutral point of view. Never trust any data as long as you have not carried out some simple plausibility checks. Methods for such plausibility checks will be discussed in this chapter. At the end of the data understanding phase, we know much better whether the assumptions we made during the project understanding phase concerning representativeness, informativeness, data quality, and the presence or absence of external factors are justified.

We first take a general look at single attributes in Sect. 4.1 and ask questions like: What kind of attributes do we have, and what do their domains look like? What is the precision of numerical values? Is the domain of an attribute stable over time, or does it change? We also need to assess the data quality. Methods and criteria for this purpose are introduced in Sect. 4.2.

Data understanding requires taking a closer look at the data. However, this does not mean that we must browse through seemingly endless columns of numbers and other values. In this way we would probably overlook most of the important facts. Looking at the data refers to visualization techniques (Sect. 4.3) that can be used to get a quick overview of basic characteristics of the data and enable us to check the plausibility of the data to a certain extent. Visualization techniques are suitable for the analysis of single attributes and of attributes in combination. Apart from the pure visualization, it is also recommended to compute simple statistical measures for correlation between attributes as described in Sect. 4.4.

Outliers, values, or records that are very different from all others should be identified with methods described in Sect. 4.5. They might cause difficulties for some of the methods applied in later steps, or they might be wrong values due to data quality problems. Missing values (see Sect. 4.6) can lead to similar problems as outliers, and by simply ignoring missing values we might obtain wrong data analysis results, so we must be aware of whether we have to deal with missing values and, if we have to, of what kind the missing values are.

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_4

Data understanding is also a step that is required for data preparation. For example, data understanding will help us identify and characterize outliers and missing values. However, how to treat them—whether to leave them as they are, to exclude them from further analysis steps, or to replace them with more plausible values—is a task for data preparation.

Throughout this chapter we will use the Iris data set [2, 7]: a set of 150 data points describing three different types of iris flowers (Iris setosa, Iris virginica, and Iris versicolor) using four different attributes measuring the length and width of the sepal and petal leaves. This is a classic data set with a few very simple and obvious properties which lends itself naturally to demonstrate how the several different data analysis methods work. However, readers should not let themselves be fooled into believing that any real-world data sets will ever display just nice and well pronounced features. Quite the opposite: In real-world data the odd and undesired effects often far outweigh the interesting ones.

4.1 Attribute Understanding

In most cases, we assume that the data can be described in terms of a table or data matrix whose rows contain the **instances**, records, or **data objects** and whose columns represent the **attributes**, **features**, or **variables**. The data might not be stored directly in one table but in different tables from which the attributes of interest need to be extracted and joined into a single table. For instance, when we are interested in the correlation between the age of a customer and the number of budget products they buy, we would need a table with two columns representing the attributes *age* and *number of budget products bought*. The age of the customers can be extracted directly from the customer table (computing it from the birth date if necessary), whereas the number of budget products a customer has bought is not stored in any specific table but must be counted in the list of items he has bought.

An attribute or its **domain**—the set of possible values for the attribute—can have different principal characteristics. One of the most basic is the **scale type**: An attribute can be *nominal* (or *categorical*), *ordinal*, or *numeric* (details are provided below). Numeric attributes are further subdivided depending on whether they are *discrete* or *continuous* and whether they have an *interval*, *ratio*, or *absolute scale*.

An attribute is called **nominal** or **categorical** if its domain is a finite set. The possible values for a categorical attribute are often considered as classes or categories. For a purely categorical attribute, there is no additional structure on the domain. This means that two values or categories are either equal or different, but not more or less similar. Examples for categorical attributes are gender with the values *F* (female) and *M* (male) in the customer data set or species in the Iris data set.

For categorical attributes, there are sometimes different levels of **granularity**, and it is necessary to make a choice as to which level the model for the analysis of the data should be built upon. The different levels of granularities form a hierarchy of more and more refined domains. A typical example for such refined levels of granularity are product categories. The general category might have the values

drinks, food, . . ., a more refined level for the drinks might distinguish between *water, beer, wine, . . .*, and these categories might even be more refined by incorporating the producer. Even the same producer might offer different brands and variants of water, beer, or wine. This is still not the lowest level of refinement. We might even distinguish identical drinks by the size of the container in which they are sold, for instance, 0.5, 1.0, and 1.5 l bottles. Of course, the most refined level will always provide the most detailed information. Nevertheless, it is often not very useful to carry out an analysis on this level, since it is impossible to extract general structures or rules when we restrict our analysis on the refined level. A general rule like "Customers tend to buy wine and cheese together" might not be discovered on a refined level of granularity where no combination of a specific wine and specific cheese might be frequent. Therefore, it is crucial to choose an appropriate level of granularity for such attributes taking the aim of the analysis and the size of the data set into account. For smaller data sets, little statistical evidence can be found when we look at a very refined level, since the number of instances per value of the attribute decreases with the level of refinement. Especially, if a number of such attributes with different levels of granularity is considered, the number of possible combinations for the values grows extremely fast for refined levels. However, the choice of a suitable granularity is a task within data preparation. Nevertheless, we must make a decision here as well, as to which level we want to understand and take a closer look at the data. We might even do this on different levels of granularity.

Another problem that sometimes comes along with categorical attributes is a dynamic domain. For certain categorical attributes, the domain will never change. The range of possible values for the month of birth is *January, . . . , December*, and it seems quite improbable that it will be changed in the near future. The situation for products, however, is completely different. Certain products might not be offered by a shop anymore or might vanish completely from the market, whereas other new products enter the market, or already existing products are adopted by the shop.

Such a **dynamic domain** of an attribute can cause problems in the analysis later on. When products are analyzed on a long-term basis of, say, several years, then products that have entered the market just recently will not show significant (accumulated) sales numbers compared to products that have been in the market for decades. Therefore, categorical attributes can lead to undesired or even wrong analysis results when such problems like the level of granularity or dynamic domains are not taken into account. We identify such attributes already at this early stage and make sure that we do not forget to handle them later.

A specific type of categorical attributes are **ordinal** attributes with an additional linear ordering imposed on the domain. An attribute for university degrees with the values none, B.Sc., M.Sc., and Ph.D. represents an ordinal attribute. A Ph.D. is a higher degree than an M.Sc., and an M.Sc. is a higher degree than a B.Sc. However, the ordering does not say that the difference between a Ph.D. and an M.Sc. is the same as the difference between an M.Sc. and a B.Sc.

The domain of a **numerical** attribute are numbers. Numerical attributes can be **discrete**, usually taking integer values, or **continuous**, taking arbitrary real values. Discrete numerical attributes often result from counting processes like the number

of children or the number of times a customer has ordered from an online shop in the last 12 months.

Sometimes, categorical attributes are coded by numerical values. For instance, the three possible values *food, drinks, nonfood* of the attribute *general product category* might be coded by the numbers 1, 2, and 3. However, this does not turn the attribute *general product category* into a (discrete) numerical attribute. We should bear this fact in mind for later steps of the analysis to avoid that the attribute is suddenly interpreted as a numerical attribute: it does not make sense to carry out numerical operations like computing the mean on such coded categorical attributes. For a discrete attribute, though, especially when it represents some counting process, it is meaningful to calculate the mean value, even though the mean value will usually not be an integer number. It is meaningful to say that on average the customers buy products 2.6 times per year in our online shop. But it does not make sense that the average *general product category* we sell is 2.6, which we might obtain when we simply compute the mean value of the products we have sold based on the numerical coding of the general product categories.

In contrast to discrete numerical attributes, a continuous attribute can—at least theoretically—assume any real value. However, such numerical values will always be measured and represented with limited precision. It should be taken into account how precise these values are. Drastic round-off errors or truncations can lead to problems in later steps of the analysis. Suppose, for instance, that a cluster analysis is to be carried out later on and that there is one numerical attribute, say X, that is truncated to only one digit right after the decimal point, while all other numerical attributes were measured and stored with a higher precision. When comparing different records, such truncation for the attribute X influences their perceived similarity and might be a dominating factor for the further analysis only for this reason. Truncation errors and measurements with limited precision should be distinguished from values corrupted with noise. The problem of noise will be tackled in the context of data quality in Sect. 4.2 and will also be discussed in more detail in Chap. 5.

Numerical attributes can have an *interval*, a *ratio*, or an *absolute scale*. For an interval scale, the definition of what zero means is more or less arbitrary. The date is a typical example for an attribute measured on an interval scale. There are calendars with different definitions of the time point zero. For instance, the Unix standard time, counted in milliseconds, has its time point zero in the year 1970 of the Gregorian calendar. The same applies to temperature scales like Fahrenheit and Celsius degrees, where zero refers to different temperatures. Certain operations like quotients are not meaningful for interval scales. For example, it does not make sense to say that a temperature of 21 °C is three times as warm as 7 °C.[1]

In contrast to this, a **ratio scale** has a canonical zero value and thus allows us to compute meaningful ratios. Examples of ratio scales are height, distance, or duration. Distance can be measured in different units like meters, kilometers, or miles. But no matter which unit we choose, a distance of zero will always have the same

[1] Such a statement may make sense, though, for the Kelvin temperature scale, because on this scale the temperature is directly proportional to the average kinetic energy of the particles—and it is meaningful to compute ratios of energies.

meaning. Especially ratios, which do not make sense for interval scales, are often useful for ratio scales: The quotient of distances is independent of the measurement unit, so the distance 20 km is always twice as long as the distance 10 km, even if we change the unit kilometers to meters or miles. Whereas for a ratio scale, only the value zero has a canonical meaning and the meaning of other values depends on the choice of the measurement unit, for an **absolute scale**, there is a unique measurement unit. A typical example for an absolute scale is any kind of counting procedure.

4.2 Data Quality

The saying "garbage in, garbage out" applies to data analysis just as it does to any other area. The results of an analysis cannot be better than the quality of the data, therefore we should be concerned about the data quality before we carry out any deeper analysis with the data. **Data quality** refers to how well the data fit their intended use. There are various data quality dimensions.

Accuracy is defined as the closeness between the value in the data and the true value. For numerical attributes, accuracy means how exact the value in the data set is compared to the true value. Noise or limited precision in measurements can lead to reduced accuracy for numerical attributes. Limited precision is often obvious from the data set. For example, in the Iris data set all numerical values are measured with only one digit right after the decimal point. The magnitude of noise can be estimated when measurements for the same value have been taken repeatedly. Accuracy of numerical values can also be affected by wrong or erroneous measurements or simply by errors like transposition of digits when measurements are recorded manually. For categorical attributes, problems with accuracy can result from misspellings like "fmale" for a value of the attribute *gender*, and also from erroneous entries.

We distinguish between *syntactic* and *semantic accuracy*. **Syntactic accuracy** means that a considered value might not be correct, but it belongs at least to the domain of the corresponding attribute. For a categorical attribute like *gender* for which only the values *female* and *male* are admitted, "fmale" violates syntactic accuracy. For numerical attributes, syntactic accuracy does not only mean that the value must be a number and not a string or text. Also certain numerical values can be out of the range of syntactic accuracy. Attributes like *weight* or *duration* will admit only positive values, and therefore negative values would violate syntactic accuracy. Other numerical attributes have an interval as their range like [0, 100] for the percentage of votes for a candidate. Negative values and values larger than 100 should not occur. For integer-valued attributes like the number of items a customer has bought, floating-point values should be excluded.

Problems with **semantic accuracy** mean that a value might be in the domain of the corresponding attribute, but it is not correct. When the attribute *gender* has the value *female* for the customer John Smith, then this is not a question of syntactic

accuracy, since *female* is a possible value of the attribute *gender*. But it is obviously a wrong value for a person named "John".[2]

Discovering problems of syntactic accuracy in a data set is a relatively easy task. Once we know the domains of the attributes, we can easily verify, whether the values lie in the corresponding domains or not. A simple measure for syntactic accuracy is the fraction of values that lies in the domains of their corresponding attribute.

The verification of semantic accuracy is much more difficult or often even impossible. Another source for the same data would enable us to check our data, and differences not caused by problems with syntactic accuracy indicate problems with semantic accuracy. Sometimes also certain "business rules" are known for the data. For instance, if we find a record in our data set with the value *male* for the attribute *gender* and *yes* for the attribute *pregnant*, there must be a problem of semantic accuracy based on the known "business rule" that only women can be pregnant.

Whether or to what detail we check syntactic and semantic accuracy depends very much on how the data were generated. Especially, when data were entered manually, there is a higher chance of accuracy problems. In any case, it is recommended to carry out at least some simple tests to see whether there might be problems with accuracy. However, the usual practice is to keep these tests at a minimum and to find out later on that there are problems with accuracy, namely when the data analysis yields implausible results.

Throughout this book we normally assume that the data are already given, for example, as a database table. This is not the best point in time to cope with data quality problems. Chances of avoiding or reducing data quality problems are highest when the data are entered into the database. For instance, instead of letting a user type in the value of categorical attribute with the danger of misspellings, one could provide a fixed selection of values from which the user can choose.

Another dimension of data quality is **completeness** which can be divided into completeness with respect to attribute values and completeness with respect to records. Completeness with respect to attribute values refers to missing values (which will be discussed in Sect. 4.6). When missing values are explicitly marked as such, then a simple measure for this dimension of data quality is the fraction of missing values among all values. But we will see that missing values are not always directly recognizable, so that the fraction of known missing values might only provide a lower bound for the fraction of actually missing values.

Completeness with respect to records means that the data set contains the necessary information that is required for the analysis. Some records might simply be missing for some technical reasons. Data might have been lost because a few years ago the underlying database system was changed and only those data records were transferred to the new database that were considered to be important at that point in time. In a customer database, customers who had not bought anything for a longer time might not have been transferred to the new database (in order to

[2]Note, however, that the problem may also reside with the name. Maybe the name of the person was misspelled, and the correct name is "Joan Smith"—then the gender is actually *female*.

eliminate potential zombie customers), or older transactions were not stored anymore.

Very often the available data set itself is biased and not representative. Consider as an example a bank that provides mortgages to private customers. If the aim of the analysis is to predict whether future applicants of loans will return the loan, we must take into account that the sample is biased in the sense that we only have information about those customers who have been granted a loan. For those customers who have been denied the loan initially, we have no information whether they would have returned the loan or not. But these customers especially could be the ones for whom it is interesting to find a good scheme to predict the risk. For customers with high income and a safe job and no current debt, we need no sophisticated data analysis techniques to predict that there is a good chance that they will return the loan. Of course, it is impossible to obtain a representative sample in the statistical sense in this case. Such a sample would mean that we would have to provide loans to any customer for a certain period, no matter how bad their financial status is, and collect and evaluate these data. Unfortunately, this would be a method entailing almost guaranteed bankruptcy.

The same problems occur in many other areas. For a production plant, we usually have large amounts of data when it is running in a normal mode. For exceptional situations, we will have little or no data. We cannot ask for such data, for instance, by requiring to check what happens if, say, a nuclear plant operates at its limit.

In such cases we might encounter future situations for which we had no corresponding data in our sample. Such possible gaps in the data should be identified. One should be aware that the space of possible values is automatically covered sparsely by the data when we have a larger number of attributes. Consider a set of m numerical attributes, and we want to make sure that we have at least positive and negative values for each attribute in our data set. This does not require a large data set. But if we want to make sure that we have data for all combinations of positive and negative values for the considered attributes, this leads to 2^m possible combinations. If we have $m = 20$ attributes, we already have more than one million possible combinations of positive and negative values. Therefore, if we have a data set with one million records, we have on average one sample for each of these combinations. For a data set with 100,000 records, at least 90% of the combinations will not be covered.

Other problems can be caused by **unbalanced data**. As an example, consider a production line for goods for which an automatic quality control is to be installed. Based on suitable measurements, a classifier is to be constructed that sorts out parts with flaws or faults. The scrap rate in production is usually very small, so that our data might contain far less than 1% examples for parts with flaws or faults.

Timeliness refers to whether the available data are too old to provide up to date information or cannot be considered as representative for predictions of future data. Timeliness is often a problem in dynamically changing domains, where only recently collected data provide relevant information, while older data can be misleading and can indicate trends that have vanished or even reversed.

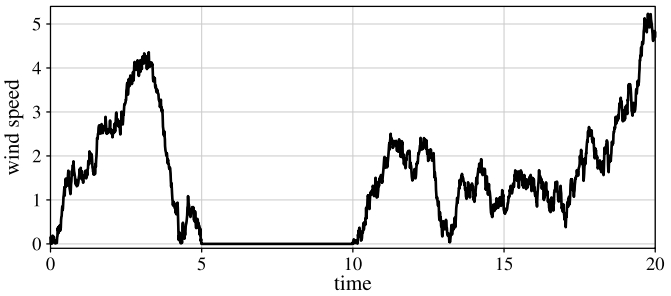


Fig. 4.1 Measured wind speeds with a period of a jammed sensor

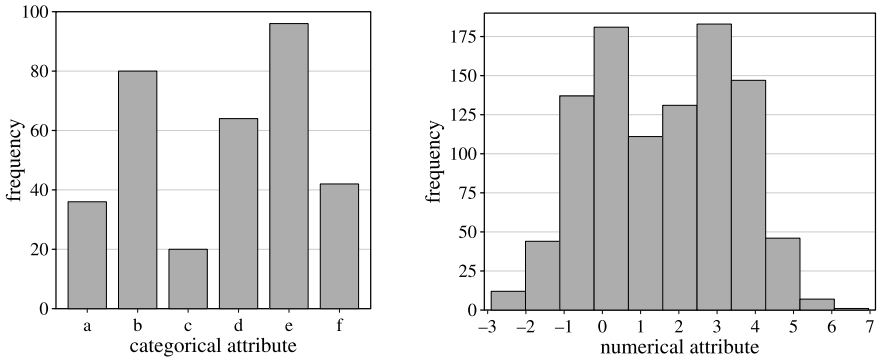


Fig. 4.2 A bar chart (categorical attribute, *left*) and a histogram (numerical attribute, *right*)

4.3 Data Visualization

According to Tukey [25], "there is no excuse for failing to plot and look" when one wants to handle a data analysis problem. The right plots of the data can provide valuable information as the simple time series plot in Fig. 4.1 shows, which enables us to discover zero values that are actually missing values. There are infinitely many ways to plot data, and it is not always easy to find the best ways of plotting a given data set. Nevertheless, there are some very useful standard data visualization techniques that will be discussed in the following.

4.3.1 Methods for One and Two Attributes

A **bar chart** is a simple way to depict the frequencies of the values of a categorical attribute. A simple example for a categorical attribute with six values a, b, c, d, e, and f is shown on the left in Fig. 4.2.

A **histogram** shows the frequency distribution for a numerical attribute. To this end, the range of the numerical attribute is discretized into a fixed number of intervals (called *bins*), usually of equal length. For each interval, the (absolute) frequency

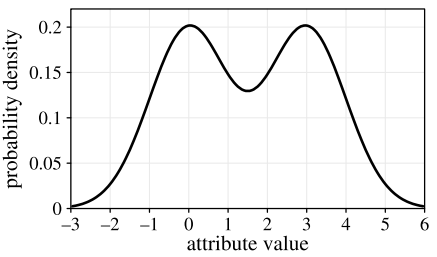


Fig. 4.3 Probability density function of a sample distribution, from which the data for the histogram in Fig. 4.2 was sampled

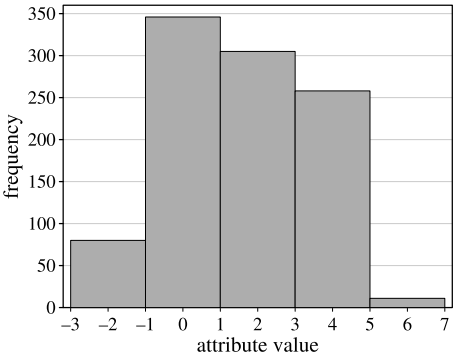


Fig. 4.4 A histogram with too few bins, for the same data set that was depicted in Fig. 4.2 on the right. As a consequence, the distribution looks unimodal but skewed

of values falling into it is indicated by the height of a bar as shown on the right in Fig. 4.2. From this histogram it can be read, for example, that a little bit more than 100 values lie in the vicinity of 1.

The histogram in Fig. 4.2 resulted from a sample of size 1000 from a mixture of two normal distributions with means 0 and 3, respectively, having both a standard deviation of 1. The density of this distribution is shown in Fig. 4.3.

But how should we choose the number of bins, and how much does this choice influence the result? Figure 4.4 shows a histogram with only five bins for the same data set underlying the histogram shown in Fig. 4.2. With only five bins, the two peaks of the original distribution are no longer visible, and one gets the wrong impression that the distribution is unimodal but skewed.

There is no generally best choice for the number of bins, but there are certain recommendations. **Sturges' rule** [23] proposes to choose the number k of bins according to the following formula:

$$k = \lceil \log_2(n) + 1 \rceil, \tag{4.1}$$

where n is the sample size. Although Sturges' rule is still very often used as a default in various statistics software packages, it is tailored to data from normal distributions and data sets of moderate size [22]. The number of bins of the histogram in Fig. 4.2 has been computed based on Sturges' rule. The size of the data set is $n = 1000$.

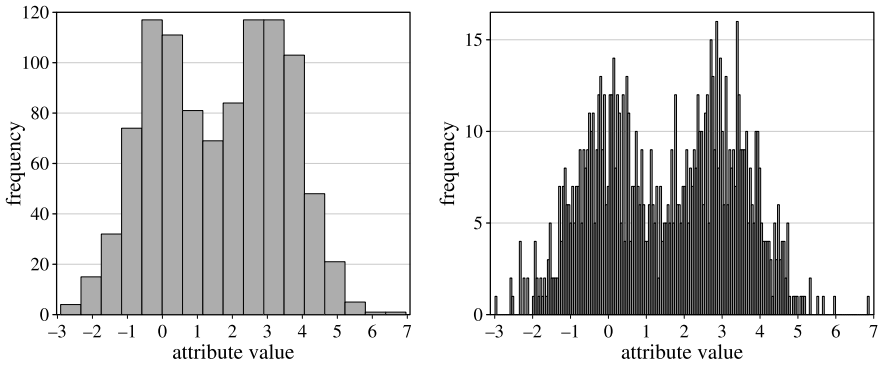


Fig. 4.5 Histograms with a suitable choice for the number of bins (*left*) and too many bins (*right*)

Assuming that, as in Sturges' rule, the bins have equal length, the number of bins can also be determined based on the length h of each bin:

$$k = \left\lceil \frac{\max_i\{x_i\} - \min_i\{x_i\}}{h} \right\rceil, \tag{4.2}$$

where $x_1, \ldots, x_n$ is the sample to be displayed. Reasonable values for h are [21]

$$h = \frac{3.5 \cdot s}{n^{\frac{1}{3}}}, \tag{4.3}$$

where s is the sample standard deviation, and [8]

$$h = \frac{2 \cdot \mathrm{IQR}(x)}{n^{\frac{1}{3}}}, \tag{4.4}$$

where $\mathrm{IQR}(x)$ is the interquartile range of the sample, that is, the length of the interval which covers the middle 50% of the data.

For the data set with the histogram displayed in Fig. 4.2, (4.3) yields $k = 16$, and (4.4) leads to $k = 17$. A histogram for the second choice (that is, for $k = 17$) is shown on the left in Fig. 4.5.

As we have seen in Fig. 4.4, the histogram can be misleading when the number of bins chosen is too small. Choosing a too high number of bins usually leads to a very scattered histogram in which it is difficult to distinguish true peaks from random peaks. An example is shown on the right in Fig. 4.5, where $k = 200$ was chosen for the number of bins for the same data underlying Fig. 4.2.

All of these methods (that is, (4.2), (4.3), and (4.4) for determining the number of bins or the length of the bins) are highly sensitive to outliers, since they divide the range between the smallest and the largest value of the sample into bins of equal size. A single outlier can make this range extremely large, so that for a smaller number of bins, the bins themselves become very large, and for a larger number of bins, most of the bins can be empty. To avoid this problem, one can either leave out

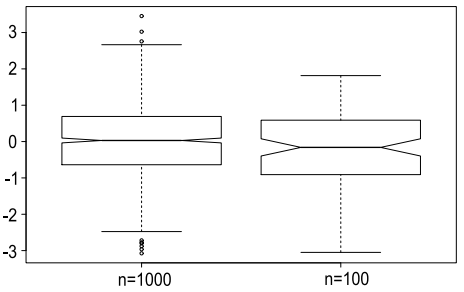


Fig. 4.6 Two boxplots for a sample from a standard normal distribution

extreme values from the sample (for instance, the 3% smallest and the 3% largest values) for calculating and displaying the histogram, or one can deviate from the principle of bins of equal length.

Boxplots are a very compact way to visualize and summarize main characteristics of a sample from a numerical attribute. Figure 4.6 shows two boxplots from samples from a standard normal distribution with mean 0 and variance 1. The left boxplot is based on sample of size $n = 1000$, whereas a sample of size $n = 100$ was used for the right boxplot.

The line in the middle of a boxplot indicates the sample median. The notch in the box is not always shown. It indicates a 95% confidence interval for the median. The box itself corresponds to the interquartile range covering the middle 50% of the data. The whiskers are drawn in the following way. The maximum length of each whisker is 1.5 times the length of the interquartile range. But if there is no data point at the maximum length of a whisker, the corresponding whisker is shortened until it reaches the next data point. Data points lying outside the whiskers are considered as outliers and are indicated in the form of small circles.

Comparing the two boxplots in Fig. 4.6, we can observe the following:

- Although both boxplots come from samples from the same normal distribution, they look different, since they are based on different samples.
- The notch of the left boxplot, representing a 95% confidence interval for the median, is much smaller than the notch of the right boxplot because of the larger sample size for the left boxplot.
- Theoretically, the whiskers for a sample from a symmetric distribution like the normal distribution should have roughly the same length. For the boxplot based on the smaller sample size, we can see that whiskers differ significantly in length, since—by chance—the largest value among the sample of 100 values was not greater than 2, whereas the smallest value was smaller than -3.
- In contrast to the boxplot on the left-hand side, the right boxplot does not contain any outliers. This is again due to the smaller sample size. The theoretical length of the interquartile range for a standard normal distribution is 1.349. Therefore, the probability of a point lying outside the (theoretical) range $[-2.698, 2.698]$ of the whiskers is almost 0.7%. Therefore, for a sample from a normal distribution of size $n = 1000$, we can expect roughly 7 outliers on average in a boxplot and less than one for a sample of size $n = 100$.

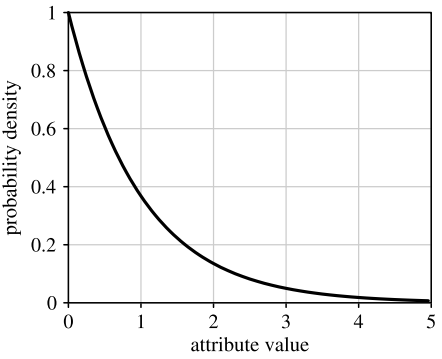


Fig. 4.7 The probability density function of the exponential distribution with $\lambda = 1$

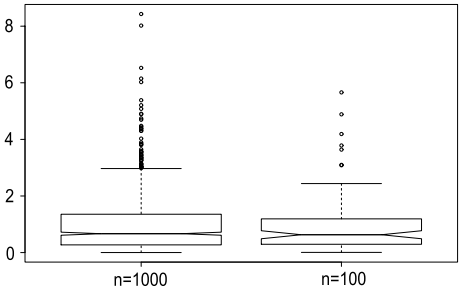


Fig. 4.8 Two boxplots for a sample from an exponential distribution

The boxplots of asymmetric distributions look completely different. If we sample from an exponential distribution, whose probability density function is shown in Fig. 4.7, we obtain boxplots as they are shown in Fig. 4.8. The boxplots on the left and right represent samples of sizes $n = 1000$ and $n = 100$, respectively.

Bar charts, histograms, and boxplots are visualizations for single attributes. In most cases, we have to deal with a number of attributes, and we are not only interested in the characteristics of single attributes but also in the relations and dependencies between the attributes. However, the display for visualizing the data is two-dimensional, and even if we use 3D-techniques from computer graphics, we cannot directly display more than two or three variables at the same time in a simple coordinate system, unless we use additional features such as symbols, color, and size. **Scatter plots** refer to displays where two attributes are plotted against each other. The two axes of the coordinate system represent the two considered attributes, and each instance in the data set is represented by a point, a circle, or any other symbol.

Simple scatter plots are not suited for larger data sets. For a data set with one million objects and a window size of 500×500 pixels, we would have on average four data objects per pixel. For larger data sets, many points or symbols in the scatter plot will be plotted at the same position, and we cannot distinguish whether a point in the scatter plot represents one or 100 objects. In the worst case, a scatter plot for a larger data set might simply look like the one in Fig. 4.9, providing only information about the range of the data, but no hint concerning the distribution of the data.

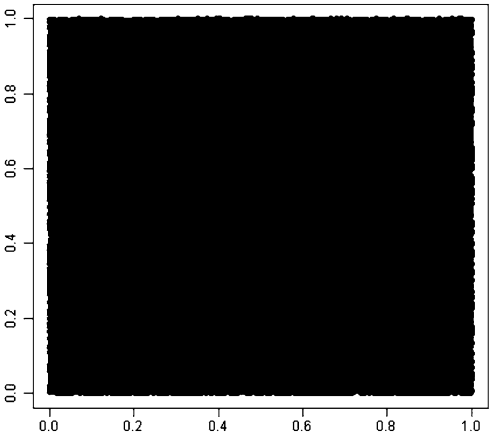


Fig. 4.9 A scatter plot of a data set with $n = 100{,}000$ instances

This can be amended by using **density plots** or plots based on **binning**. Using semitransparent points for plotting the data is one way to generate a density plot. Each plotted point is semitransparent, and the more points are plotted at the same place, the less transparent the image will become in this place. Binning was already used to generate histograms, and the principle is used for the scatter plots. The two-dimensional domain of the data for the scatter plot is partitioned into bins of the same size. Possible forms for the bins are rectangles or hexagons. The intensity of the color for the bin is chosen proportional to the number of data objects falling into the bin. Figure 4.10 shows a density plot on the left and a plot based on a hexagonal binning on the right for the same data set displayed in Fig. 4.9. Both plots indicate a higher density of the data around the point (0.6, 0.4), which cannot be seen in the simple scatter plot in Fig. 4.9.

Scatter plots can be enriched with further information in order to involve more attributes. Different plot symbols or colors can be used for plotting the points in order to include information about a categorical attribute. Color intensity and the size of the symbols are possible means to indicate the value of additional numerical attributes.

Figure 4.11 shows two scatter plots of the Iris data set—one displaying the sepal length versus the sepal width and the other one the petal length versus the petal width—in which different species are displayed by different colors. Both plots show that the red circles, representing the species Iris setosa, can be well distinguished from the other two species, Iris versicolor and Iris virginica, displayed as triangles and crosses, respectively. However, the left chart in Fig. 4.11 gives the impression that Iris virginica and Iris versicolor are very difficult to distinguish, at least when we only take the sepal length and the sepal width into account. When we consider the petal length and the petal width (right chart in Fig. 4.11), we can still see the overlap of the corresponding symbols for the species, but there is a clear tendency that Iris virginica tends to larger values than Iris versicolor for the petal length and width.

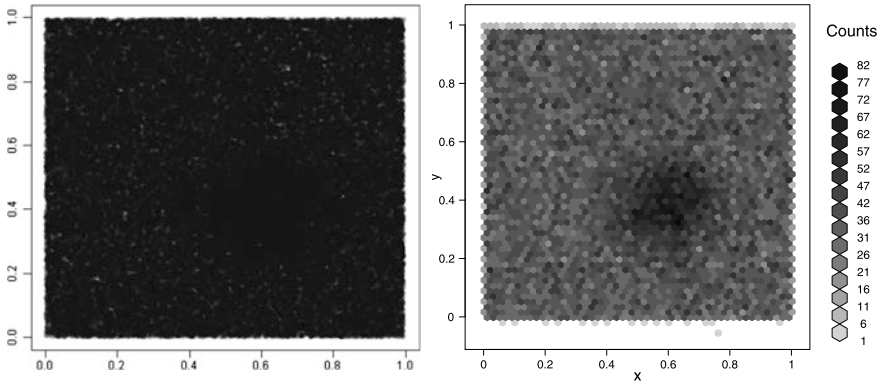


Fig. 4.10 Density plot (*left*) and a plot based on hexagonal binning (*right*) for the same data set as shown in Fig. 4.9

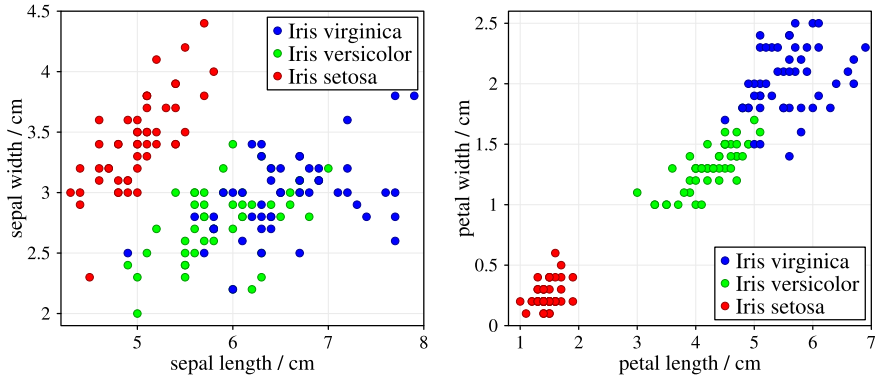


Fig. 4.11 Scatter plots of the iris data set for sepal length vs. sepal width (*left*) and for petal length vs. petal width (*right*). All quantities are measured in centimeters (cm)

Comparing the number of red circles in Figs. 4.11 (left and right), there seem to be less red circles on the right. But how can some of the objects suddenly vanish in the scatter plot? When we count the number of red circles, we see that in both scatter plots there are less than 50, although the data set contains 50 instances of Iris setosa that should be displayed by red circles. The circles are not missing in the scatter plots. Some circles are simply plotted at exactly the same position, since their measured sepal length and width or their measured petal length and width coincide. Recall that these values were only measured with a precision of just one digit right after the decimal point. To avoid this impression of seeing less objects than there actually are, one can add **jitter** to the scatter plot. Instead of plotting the symbols exactly at the coordinates specified by the values in the data set, we add a small random value to each original value in the data table. The left chart in Fig. 4.12 shows the resulting scatter plot with jitter where we have added random values from a uniform distribution on the interval $[-0.04, 0.04]$ to the original values. This ensures

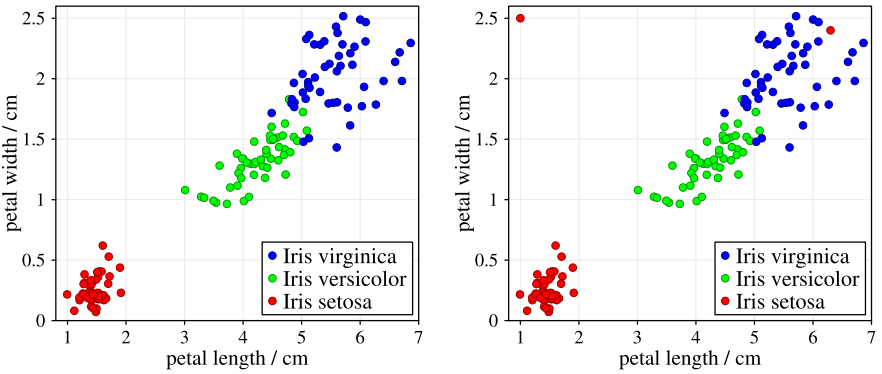


Fig. 4.12 The same scatter plot as in Fig. 4.11 on the right, but with jitter (*left*) and with jitter and two outliers (*right*; the outliers are the *red points* in the top left and top right corners)

that a point originally lying left or below another point will always remain left or below the other point, even when the jitter is added.

Jitter is essential when categorical attributes are used for the coordinate axes of a scatter plot, since categorical attributes have only a limited number of possible values, so that plotting of objects at exactly the same position occurs very often when no jitter is added.

From a scatter plot we can already extract important information. Consider again the scatter plot displayed in Fig. 4.12. We can see that the petal length and width are correlated. Objects with larger values for the petal length also tend to have larger values for the petal width. The scatter plot also shows that Iris setosa—the red circles in the scatter plot—can be easily distinguished from the other two species just on the basis of the petal length or width. The scatter plot does not indicate that the other two species cannot be separated clearly. It only shows that, solely based on the petal length and width, it is not possible to distinguish the two species perfectly. Outliers can also be discovered in scatter plots. The left chart in Fig. 4.12 does not have any outliers. In the right chart, however, we have added two artificial outliers. The data point in the upper left corner is a clear outlier with respect to the whole data set. Note that the values for the attributes petal length and width are both in the general range of the corresponding attributes in the data set. But there is no other object in the data set with a similar combination of these attribute values. The second outlier in the right chart of Fig. 4.12—the circle in the upper right corner—is not an outlier with respect to the values for the petal length and width or their combination. However, it is an outlier for the class Iris setosa displayed by red circles. Whenever such outliers are discovered, one should check the data or the data generating process again to ensure that the outliers are not due to erroneous data.

It should be noted that the scatter plots—like all other visualization techniques—are very useful tools to discover simple structures and patterns or peculiar deviations like outliers in a data set. But there is no guarantee that a scatter plot or any visualization technique will automatically show all or even any interesting or deviating

patterns in the data set. A scatter plot with no outliers does not mean that there are no outliers in the data set. It only means that there are no outliers with respect to the combination of the attributes displayed in the scatter plot. In this sense, visualization techniques are like test cases for computer programs. Test cases can discover errors in a program. But if the test cases have not indicated any errors, this does not imply that the program does not have any bugs. In the same way, a visualization technique might give hints to certain interesting patterns in the data set. But if one cannot see any interesting patterns in a visualization, it does not mean that there are no patterns in the data set.

4.3.2 Methods for Higher-Dimensional Data

In this section, we restrict our considerations to numerical attributes. Scatter plots are projections of the data set to a two-dimensional plane where the two dimensions of the projection plane correspond to two selected attributes. The data set can be considered as a subset of a space that has as many dimensions as the data set has attributes. How can such a higher-dimensional data set be represented in two or three dimensions? The idea is to preserve as much of the "structure" of the higher-dimensional data in the lower-dimensional representation. But what does it mean to preserve the structure? There is no unique answer to this question, and various approaches have been proposed to solve this problem. A very efficient method is principal component analysis that will be introduced in the next section.

4.3.2.1 Principal Component Analysis

Principal component analysis (**PCA**), which is also briefly described in the appendix on statistics, is a method from statistics to find a projection to a plane—or more generally, to a linear subspace—which preserves as much as possible of the original variance in the data. In order to restrict the search for the best projection plane to planes through the origin of the coordinate system, the data are first centered around the origin by subtracting the mean value for each attribute from the attribute values. In this way, the projection to the plane can be represented by a matrix M mapping the data points $\mathbf{x} \in \mathbb{R}^m$ to the plane by

$$\mathbf{y} = M \cdot (\mathbf{x} - \bar{\mathbf{x}}), \tag{4.5}$$

where $\bar{\mathbf{x}}$ denotes the (empirical) mean value (or vector of (empirical) mean values)

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{x}_i .$$

Also M is a $2 \times m$ matrix when PCA is used for visualizing the data on a plane. As we will see later on, PCA can be understood as a more general dimension reduction technique. Therefore, we do not restrict our considerations to projections to a plane of dimension 2 but to any linear subspace of dimension $q \leq m$. In this case, M is a

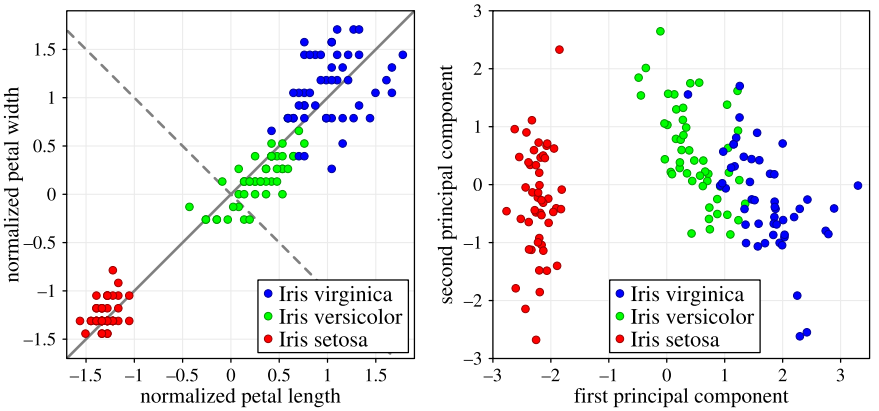


Fig. 4.13 *Left*: the first (*solid line*) and the second principal component (*dashed line*) of an example data set (Iris data). *Right*: the example data set projected to the space that is spanned by the first and second principal components (resulting from a PCA involving all four attributes)

$q \times m$ matrix. It should be noted that arbitrary matrices are not admitted for the projection, since only projections, but no scalings, are allowed. Scalings would enable the overall variance of the projection to be changed, which is not desired.

The solution of the optimization problem of finding the projection plane such that the variance of the projected data is maximized leads to an eigenvalue problem. The projection matrix M for PCA is given by

$$M = (\mathbf{v_1}, \ldots, \mathbf{v_q}),$$

where $\mathbf{v_1}, \ldots, \mathbf{v_q}$ are the normalized eigenvectors[3] of the **covariance matrix** of the data

$$C = \frac{1}{n-1} \sum_{i=1}^{n} (\mathbf{x_i} - \bar{\mathbf{x}})(\mathbf{x_i} - \bar{\mathbf{x}})^\top$$

for the q largest eigenvalues $\lambda_1 \geq \cdots \geq \lambda_q$.

The vectors $\mathbf{v_1}, \ldots, \mathbf{v_q}$ are called **principal components**. Figure 4.13 illustrates the concept of principal components on a two-dimensional example. Of course, for such an example, a dimension reduction is not necessary. There are two principal components. The vector corresponding to the largest eigenvalue is the first component, and its direction is indicated by the solid line. The dashed line represents the direction of the second component, corresponding to the second largest eigenvalue, which in this two-dimensional case is the smallest eigenvalue. It is obvious that the projection of the data set to the first principal component preserves a large fraction of the original variance, whereas a projection to the second principal component only would lead to a significant loss of the original variance.

[3] λ is called an eigenvalue of a matrix A if there is a nonzero vector $\mathbf{v}$ such that $A\mathbf{v} = \lambda\mathbf{v}$. The vector $\mathbf{v}$ is called an eigenvector to the eigenvalue λ.

Table 4.1 Preservation of the variance of the Iris data set depending onthe number of principal components

	Principal component			
	PC1	PC2	PC3	PC4
Proportion of variance	0.73	0.229	0.0367	0.00518
Cumulative proportion	0.73	0.958	0.9948	1.00000

The left chart in Fig. 4.13 is a plot of the petal length and width of the Iris data set. However, apart from the necessary centering of the data around the origin by subtracting the mean, the data have been **z-score standardized** by the transformation

$$x \mapsto \frac{x - \hat{\mu}_X}{\hat{\sigma}_X}, \tag{4.6}$$

where $\hat{\mu}_X$ and $\hat{\sigma}_X$ are the mean value and empirical standard deviation of attribute X (see also Sect. 6.3.2), respectively. Without standardization, the result of PCA would depend on the scaling of the attributes. If no standardization is carried out, the attribute with the largest variance can easily dominate the first principal component. For the example in Fig. 4.13 with z-score standardization, the first principal component is the vector $(\sqrt{2}/2, \sqrt{2}/2)$. If the petal length is measured in meters instead of centimeters, but petal width is still measured in centimeters, the first principal component without z-score standardization becomes the vector (0.0223, 0.9998), since the variance of the petal length has been decreased drastically by the scaling factor 0.01 resulting from the change from centimeters to meters, so that more or less only the petal width contributes to the variance in the data.

PCA can be used for visualization purposes by restricting to the first two principal components. More generally, PCA can carry out a dimension reduction to any lower-dimensional space; even more, PCA also provides information about over how many dimensions the data set actually spreads. This information can be extracted from the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_m$ of the covariance matrix. When we project the data to the first q principal components $v_1, \ldots, v_q$ corresponding to the eigenvalues $\lambda_1, \ldots, \lambda_q$, this projection will preserve a fraction of

$$\frac{\lambda_1 + \cdots + \lambda_q}{\lambda_1 + \cdots + \lambda_m} \tag{4.7}$$

of the variance of the original data. Table 4.1 shows the corresponding result of PCA applied to the Iris data set without the categorical attribute for the species. A projection of this four-dimensional data set to the first principal component, i.e., to only one dimension, covers already 73% of the variance of the original data set. A projection to a plane defined by the first two principal components covers already 95.8% of the variance. This means that the four numerical attributes of the Iris data set are not located on a two-dimensional plane in the four-dimensional space but do not deviate too much from the plane defined by the first two principal components. The right chart in Fig. 4.13 shows the projection of the Iris data set to the first two principal components where PCA was carried out after the z-score standardization had been applied.

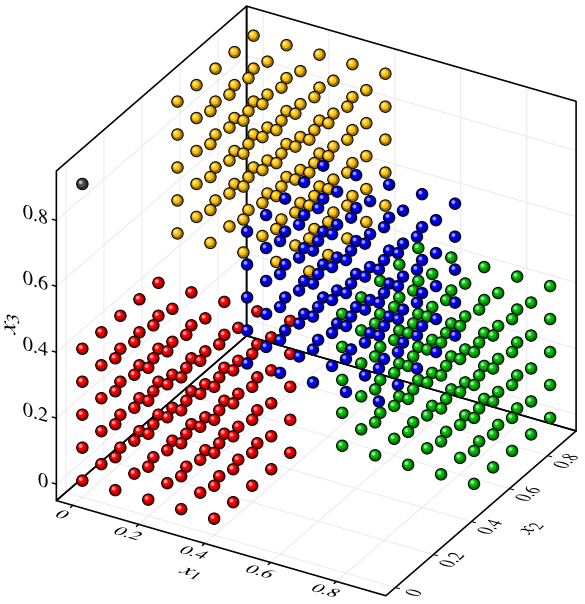


Fig. 4.14 A data set distributed over a cube in a chessboard-like pattern. The colors are only meant to make the different cubes more easily discernible. They do *not* indicate classes. Note the outlier in the upper left corner

The importance of the scaling effect carried out by the z-score standardization can also be observed by revisiting the example where we had considered only the petal length and width of the Iris data set. We had applied PCA to the original data and the data where we had changed the measurement of the petal length from centimeters to meters without scaling, resulting in the vector $(0.7071, 0.7071)$ as the first principal component for the original data and the vector $(0.0223, 0.9998)$ for the modified data. The variance preserved by the projection to the first principal component is 98.1% in the first case and 99.996% in the second case. In the latter case, the first principal component corresponds more or less to the petal width, since the variance of the petal length measured in meters is almost negligible, and therefore the projection can preserve close to 100% of the original variance.

In order to illustrate the advantages and limitations of PCA for visualization purposes, we consider an artificial three-dimensional data set illustrated in Fig. 4.14. The data fill the unit cube in chessboard-like manner. When the unit cube is divided into eight subcubes, these subcubes are alternately empty and filled with data. There is also one outlier close to the upper left corner of the surface in the front of the cube. The scatter plots resulting from projections to two axes of the coordinate system are shown in Fig. 4.15. All these scatter plots give the wrong impression that the data are uniformly distributed over a grid in the data space. The scatter plots provide neither a hint to the chessboard pattern in the three-dimensional data space nor to the single outlier.

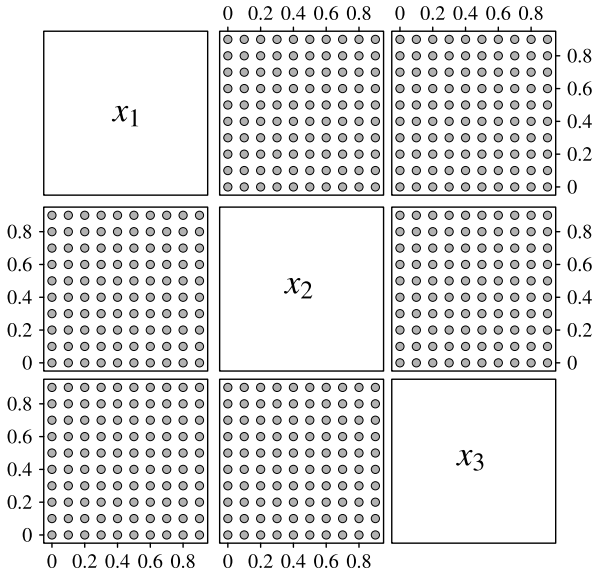


Fig. 4.15 The scatter plots for the data set shown in Fig. 4.14

Fig. 4.16 The projection to the first two principal components of the data set shown in Fig. 4.14

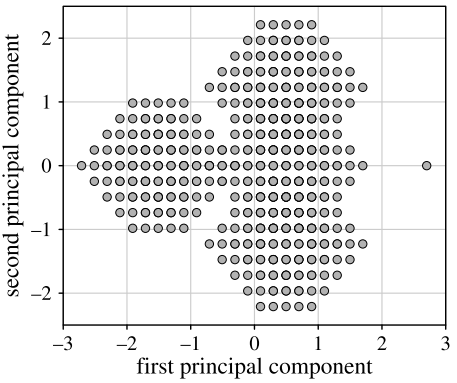


Figure 4.16 shows the projection to the first two principal components of the data set after a z-score standardization has been carried out. The outlier can now be identified easily. From Fig. 4.16 it is also obvious that the data cannot be distributed uniformly over the original three-dimensional space but that there must be some inherent pattern. Of course, it is impossible to recover the three-dimensional chessboard pattern in a two-dimensional projection completely.

4.3.2.2 Projection Pursuit

PCA has the advantage that the best projection with respect to the given criterion—the preservation of the variance—can be computed directly based on the eigenvectors of the covariance matrix. **Projection pursuit** [10] takes a different approach.

The projection of the data should show interesting aspects of the data. But what does interesting mean? Normally, for projection pursuit, interestingness of a projection is defined as the deviation from a normal distribution, according to the observation that most of the projections of high-dimensional data will resemble a normal distribution [6]. The more the projected data deviate from a normal distribution, the more interesting the projection. Various criteria [5, 9–12] can be defined to measure how much a projection deviates from a normal distribution.

In contrast to PCA, there is no way to find the most interesting projections with respect to the deviation from a normal distribution. Projection pursuit simply generates random projections and chooses those yielding the best values for the measures of interestingness.

4.3.2.3 Multidimensional Scaling

PCA is a method for dimension reduction that is based on two assumptions:

- The representation of the data in a lower-dimensional space should be obtained by a projection to a linear subspace.
- The criterion to evaluate the representation of the data in the lower-dimensional space is the preservation of the variance.

Projection pursuit shares the first assumption of PCA but replaces the second one by the deviation of the projection from the normal distribution. **Multidimensional scaling** (**MDS**) is also a dimension-reduction technique but differs in both assumptions from PCA and projection pursuit.

- MDS is not restricted to mappings in the form of simple projections. In contrast to PCA, MDS does not even construct an explicit mapping from the high-dimensional space to the low-dimensional space. It only positions the data points in the low-dimensional space.
- The representation of the data in the low-dimensional space constructed by MDS aims at preserving the distances between the data points and not the variance in the data set, for which PCA was designed.

For a data set with n data objects, MDS requires a distance matrix $[d_{ij}^{(X)}]_{1 \leq i,j \leq n}$, where $d_{ij}^{(X)}$ is the distance between data object i and data object j. MDS assumes that the distance is symmetric, i.e., $d_{ij}^{(X)} = d_{ji}^{(X)}$ for all $i, j \in \{1, \ldots, n\}$. The distances $d_{ii}^{(X)}$ must be zero, so that each data object has no distance to itself. MDS does not require the data, but only the distance matrix. Usually, the distance matrix is derived from the data by computing the Euclidean distances between the data objects (see Sect. 7.2 for more details on distance measures).

It is important to note that it is recommended to carry out a normalization of the data first, for instance, by applying a z-score standardization. Without such a normalization, MDS has to cope with the same problems as PCA. The variance and the difference between values of an attribute highly depend on the scaling or

the measurement unit of the attribute. In the same way as a single attribute or few attributes might dominate the variance, just because the corresponding measurement units tend to larger values, these attributes will contribute more to the Euclidean distance than attributes with very small values and ranges.

Since MDS is mainly used for visualization purposes, the original high-dimensional data should be represented by points in two or sometimes also in three dimensions. Each data object should be represented by a point in the low-dimensional space, usually in $\mathbb{R}^2$ or $\mathbb{R}^3$. Let us restrict our considerations for the moment to a two-dimensional visualization. Then MDS must define a point $\mathbf{p}_i = (p_x^{(i)}, p_y^{(i)})$ to be plotted in the plane for each data object x_i. The plot of these points should preserve the original distances $d_{ij}^{(X)}$ between the data points in the original high-dimensional space. Of course, it is usually impossible to preserve the distances exactly in the two-dimensional representation. Therefore, we require that the distances $d_{ij}^{(Y)} = \|\mathbf{p_i} - \mathbf{p_j}\|$ between the points in the two-dimensional representation should deviate as little as possible from the original distances $d_{ij}^{(X)}$. One way to measure this deviation is the sum of the squared errors between the original distances and the distances in the two-dimensional representation:

$$E_0 = \sum_{i=1}^{n} \sum_{j=i+1}^{n} \left(d_{ij}^{(Y)} - d_{ij}^{(X)}\right)^2. \tag{4.8}$$

This equation does not refer explicitly to the two-dimensional representation. The distances $d_{ij}^{(Y)}$ in the lower-dimensional space can be derived from points in $\mathbb{R}^2$, but the distances could also be computed for points in $\mathbb{R}^q$ for any $q \in \mathbb{N}$.

The sum of squared errors depends on the number of data objects and on the values for the original distances. For more data points and for larger distances $d_{ij}^{(X)}$, E_0 will tend to become larger as well. In order to obtain an error measure independent of these effects, E_0 is often normalized to

$$E_1 = \frac{1}{\sum_{i=1}^{n} \sum_{j=i+1}^{n} (d_{ij}^{(X)})^2} \sum_{i=1}^{n} \sum_{j=i+1}^{n} \left(d_{ij}^{(Y)} - d_{ij}^{(X)}\right)^2. \tag{4.9}$$

The factor $\frac{1}{\sum_{i=1}^{n} \sum_{j=i+1}^{n} (d_{ij}^{(X)})^2}$ is independent of the distances $d_{ij}^{(Y)}$ and therefore independent of the lower-dimensional representation of the data. It does not affect the result of the minimization of E_0 or E_1.

The relative error

$$E_2 = \sum_{i=1}^{n} \sum_{j=i+1}^{n} \left(\frac{d_{ij}^{(Y)} - d_{ij}^{(X)}}{d_{ij}^{(X)}} \right)^2 \tag{4.10}$$

is an alternative to the absolute error in (4.9). Very often, neither the absolute nor the relative error is considered for MDS, but a compromise between them given

by

$$E_3 = \frac{1}{\sum_{i=1}^{n}\sum_{j=i+1}^{n} d_{ij}^{(X)}} \sum_{i=1}^{n}\sum_{j=i+1}^{n} \frac{(d_{ij}^{(Y)} - d_{ij}^{(X)})^2}{d_{ij}^{(X)}}. \tag{4.11}$$

When MDS is carried out based on the error measure E_3, it is also called **Sammon mapping**. The error E_3 for a concrete representation of the data in the lower-dimensional space is called **stress**.

So far, we have only proposed error measures for MDS, but we still need to find a way to minimize these error measures. The minimization requires finding n suitable points in $\mathbb{R}^m$ and therefore involves $m \times n$ variables, where m is the dimension for the MDS representation of the data, and n is the number of data objects. This means that even a two-dimensional MDS representation of a small data set like the Iris data leads to a minimization problem with $2 \times 150 = 300$ parameters to be optimized. Unfortunately, there is no known analytical solution for any of the error measures in (4.8)–(4.11). Therefore, a heuristic optimization strategy is needed. Typically, a **gradient method** is applied. Gradient methods are discussed in more detail in Sect. 5.3.2. In order to minimize a differentiable function—here any of the error measures (4.8)–(4.11)—one can compute the vector of partial derivatives with respect to the parameters of the object function and then follow the opposite direction of the gradient, since the gradient points in the direction of steepest ascend and our aim is not to maximize but to minimize the error function. The gradient method starts with a random or a suitable initial solution, computes the gradient at the given solution, goes in the opposite direction of the gradient, and takes the resulting solution as the next solution. For this new solution, the gradient is computed again, and the process is repeated until no or little improvements can be achieved.

As an example, we compute the gradient for error measure E_1 in (4.9) with respect to one data point $\mathbf{y}_k$ in the lower-dimensional space:

$$\frac{\partial E_1}{\partial \mathbf{y_k}} = \frac{2}{\sum_{i=1}^{n}\sum_{j=i+1}^{n}(d_{ij}^{(X)})^2} \sum_{j\neq k} \big(d_{kj}^{(Y)} - d_{kj}^{(X)}\big) \frac{\mathbf{y_k} - \mathbf{y_j}}{d_{kj}^{(Y)}}, \tag{4.12}$$

where we have used

$$\frac{\partial d_{ij}^{(Y)}}{\partial \mathbf{y_k}} = \frac{\partial}{\partial \mathbf{y_k}} \|\mathbf{y_i} - \mathbf{y_j}\| = \begin{cases} \frac{\mathbf{y_k} - \mathbf{y_j}}{d_{kj}^{(Y)}} & \text{if } i = k, \\ 0 & \text{otherwise.} \end{cases} \tag{4.13}$$

The basic structure of the MDS algorithm is described in Table 4.2.

It is recommended to start with a good initialization, which can be obtained by first carrying out a principal component analysis and then projecting the data to the first q principal components. The stepwidth α determines how far to go in the opposite direction of the gradient. If α is chosen too large, the algorithm might never find a (local) minimum of the error function. Small values for α lead to slow convergence of the algorithm. It is also possible to use an adaptive stepwidth α that determines a new value for α in each step of the algorithm. The stop criterion can be

Table 4.2 Multidimensional scaling

Algorithm MDS($\mathcal{D}$)

input: data set $\mathcal{D} \subset \mathbb{R}^m$ with $|\mathcal{D}| = n$ or distance matrix $[d_{i,j}^{(X)}]_{1 \le i,j \le n}$
parameter: dimension q for the representation, stepwidth $\alpha > 0$, stop criterion SC
output: set Y of n points in $\mathbb{R}^q$

1 Initialize $Y = \{\mathbf{y_1}, \ldots, \mathbf{y_n}\} \subset \mathbb{R}^q$ randomly or better with a PCA projection
2 If the input is a data set, compute the distances $d_{ij}^{(X)}$ between the data objects
3 **do**
4 Compute $d_{i,j}^{(Y)} = \|\mathbf{y}_i - \mathbf{y}_j\|$ (for all $i, j = 1, \ldots, n$)
5 Compute $\partial E_1 / \partial \mathbf{y_k}$ according to (4.12) (for all $k = 1, \ldots, n$)
6 update $\mathbf{y_k^{new}} = \mathbf{y_k^{old}} - \alpha \cdot \partial E_1 / \partial \mathbf{y_k}$ (for all $k = 1, \ldots, n$)
7 **while** SC is not satisfied

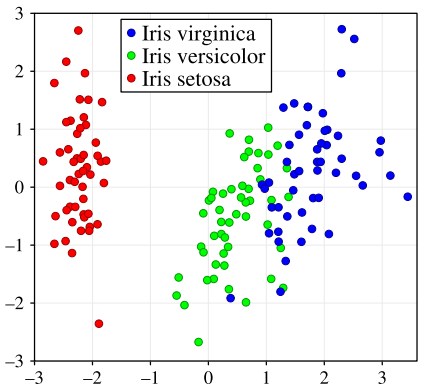


Fig. 4.17 The Sammon mapping of the Iris data, scatter plots of which are shown in Fig. 4.11 on p. 46

a fixed number of iterations or a lower bound for the change of the error function, i.e., when E_1 changes less than a given threshold in one step of the algorithm, the iteration scheme is terminated.

The algorithm is identical for the error measures (4.10) and (4.11) except for lines 5 and 6 where the gradient of E_1 has to be replaced by the gradients of the corresponding error measures.

Figure 4.17 shows the result of MDS in the form of the Sammon mapping applied to the Iris data set. The categorical attribute for the species has not been used for MDS. But the corresponding points resulting from the Sammon mapping are marked by colors corresponding to the Iris flower species. Before the distances $d_{ij}^{(X)}$ in the original four-dimensional space defined by the sepal and petal length and width are computed, a z-score standardization is applied to each of the four numerical attributes.

Before the MDS algorithm can be applied, we have to make sure that no distance between two different data objects is zero because these distances occur in the denominator of the gradient, so that a zero distance would lead to a division by zero error. There are two Iris flowers—number 102 and number 143—with exactly the

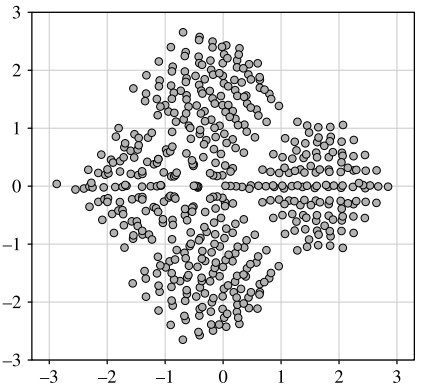


Fig. 4.18 The Sammon mapping of the data set that is shown in Fig. 4.14 on p. 51

same values for all attributes, leading to a distance of zero between them. Therefore, we removed one of them from the data set before carrying out the computations for MDS.

Figure 4.18 shows the result of the Sammon mapping applied to the "3D chessboard pattern" data set in Fig. 4.14, where we have again carried out z-score standardization in advance. The four cubes filled with data can even be recognized in this two-dimensional representation. However, the outlier—the left-most point—is not as well separated from the other data points as in the case of PCA in Fig. 4.16.

4.3.2.4 Variations of PCA and MDS

MDS differs from PCA in various aspects. MDS is based on the idea of preserving the distances among the original data objects, whereas PCA focuses on the variance.

PCA provides an explicit mapping from the space in which the data objects are located to the lower-dimensional space, whereas MDS provides only an explicit representation of the data objects in the lower-dimensional space. This means that when a new or hypothetical data object is considered, it can be represented immediately in case of PCA simply by projecting the data object to the corresponding principal components. This is not possible for MDS.

The computational complexity of PCA is lower than the complexity of MDS. The computation of the covariance matrix can be carried out in linear time with respect to the size of the data set. Once the covariance matrix has been calculated, the computation of the eigenvalues and eigenvectors depends only on the number of attributes, but not on the number of data anymore. The number of attributes is usually much smaller than the number of data objects. For MDS, it is necessary to consider the pairwise distances between data objects, leading to a quadratic complexity in the number of data objects. Although a quadratic complexity is often considered as feasible in computer science, it is unacceptable for larger data sets. Consider a data set with one million data objects. The distance matrix contains 10^{12} entries in this case. Since the distance matrix is symmetric and the entries in the diagonal are all zero, we only need to know $(10^{12} - 10^6)/2 = 4999995 \cdot 10^5$ entries. If we want to store the distances in 4-byte floating-point values, this would require more than 1800

gigabytes! There are various modifications of MDS [3] that try to overcome these complexity problems by sampling [16] or variations of the error measures [17, 18].

All these dimension-reduction methods generate scatter plots with abstract coordinate axes that do not correspond to attributes of the original data. In this way, the scatter plots of projection pursuit can even be extended and evaluated by the measures of interestingness proposed for projection pursuit and also other statistical measures and tests in order to select the most interesting visualizations [24].

4.3.2.5 t-distributed Stochastic Neighbor Embedding

t-distributed Stochastic Neighbor Embedding (t-SNE) [26] is a relatively recent technique, often used to graphically represent a complex and multidimensional data set on a two- or three-dimensional space. t-SNE is a nonlinear dimensionality reduction technique based on nonlinear local relationships among the data points. t-SNE defines probabilities

$$p_{ij} = \frac{p_{i|j} + p_{j|i}}{2n}$$

for any two data objects $\mathbf{x_i}$ and $\mathbf{x_j}$ where

$$p_{j|i} = \frac{\exp(-\|\mathbf{x_i} - \mathbf{x_j}\|^2)/2\sigma_i^2}{\sum_{k \neq i} \exp(-\|\mathbf{x_i} - \mathbf{x_k}\|^2)/2\sigma_i^2}$$

and n is the number of data points; $p_{j|i}$ stands for the probability that data object $\mathbf{x_i}$ would pick data object $\mathbf{x_j}$ as its neighbor. This probability is proportional to $\exp(-\|\mathbf{x_i} - \mathbf{x_j}\|^2/2\sigma_i^2)$. The parameter σ_i is adapted with respect to the local density of data points near $\mathbf{x_i}$. The higher the local density at $\mathbf{x_i}$, the smaller the value of σ_i.

Corresponding probabilities q_{ij} are defined for the points $\mathbf{y_i}$ in the lower-dimensional representation but instead of the Gaussian-like functions for the probabilities $p_{j|i}$ t- or Cauchy distribution-like functions are used:

$$q_{j|i} = \frac{(1 + \|\mathbf{y_i} - \mathbf{y_j}\|^2)^{-1}}{\sum_{k \neq i}(1 + \|\mathbf{y_i} - \mathbf{y_k}\|^2)^{-1}}.$$

The points $\mathbf{y_i}$ should be chosen in such a way that the probabilities p_{ij} and q_{ij} are as similar as possible where similarity is measured here based on the **Kullback–Leibler divergence**

$$KL(p||q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

t-SNE is primarily used for visualization. Indeed, the aggressive reduction of the data set dimensionality to just 2 or 3 coordinates, the transformation of the likelihood of pairs of data points into a visual neighborhood property, and the capability to represent strange shapes of the data groups thanks to the nonlinearity of the projection, make the t-SNE particularly suitable for most data visualization techniques.

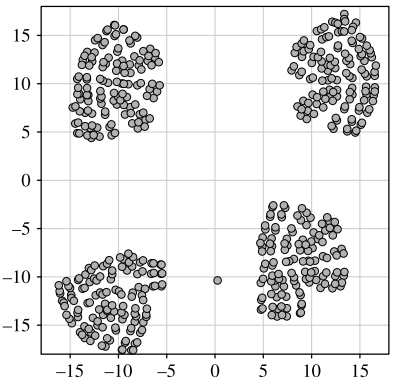


Fig. 4.19 The result of t-SNE for the data set that is shown in Fig. 4.14 on p. 51

Notice that the t-SNE technique works only on the current data set. It is not possible to export the model and apply it to new data. Indeed, all available data must be used to find the new low-dimensional space. As in Sammon mapping, new data cannot be projected.

Figure 4.19 shows the result of t-SNE applied to the cube data set shown in Fig. 4.14 on p. 51. The four original clusters are well preserved in the two-dimensional representation and the outlier is also clearly visible.

4.3.2.6 Parallel Coordinates

So far, the visualization techniques for multidimensional data we have introduced here are based on scatter plots with abstract coordinate axes. Since coordinate axes should be drawn pairwise perpendicular and only two axes can satisfy this condition on a plane and three axes in 3D-space, we cannot use more than three axes for scatter plot-like visualizations. **Parallel coordinates** draw the coordinate axes parallel to each other, so that there is no limitation for the number of axes to be displayed. For a data object, a polyline is drawn connecting the values of the data object for the attributes on the corresponding axes.

Figure 4.20 shows the plot of the Iris data set with parallel coordinates. The species is a categorical attribute and can only assume three possible values. The polylines for the different species are displayed with different colors. The plot clearly shows that the setosa has smaller values for the petal length and width than the other two species.

For larger data sets, it becomes more or less impossible to track the lines that correspond to a data object in parallel coordinates plots or even to discover general structures. It can be helpful to generate separate parallel coordinate plots for different subsets of the data. For instance, in the case of the Iris data set, we could generate a separate plot for each of the three species as shown in Fig. 4.21. In order to keep the three plots comparable, we have not rescaled the axes for each of the three plots. Normally, the axes will be scaled in such a way that the minimum and maximum of all values for the corresponding attribute of the displayed data are lowest and highest point of the axes.

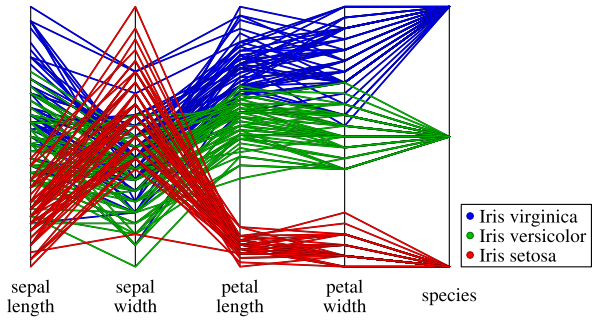


Fig. 4.20 Parallel coordinates plot for the Iris data set

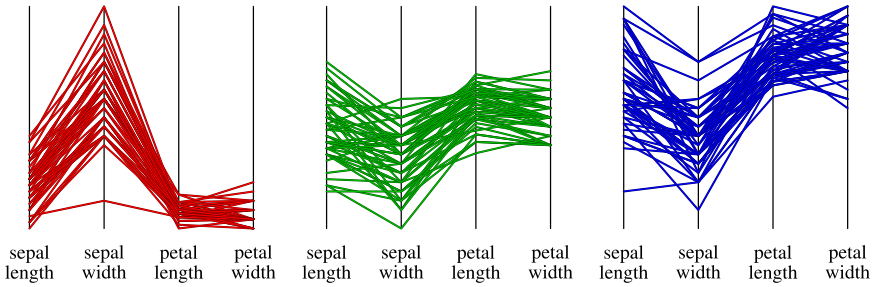


Fig. 4.21 Parallel coordinates plots for Iris setosa, Iris versicolor, and Iris virginica (*left* to *right*)

Fig. 4.22 Parallel coordinates plot for the data set shown in Fig. 4.14

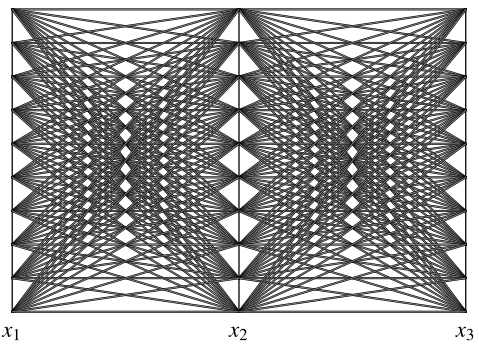


Figure 4.22 shows the parallel coordinates plot for the "3D chessboard pattern" data set in Fig. 4.14. This plot is identical to the plot that we would obtain if we had not used the "3D chessboard pattern," but had filled the cube uniformly with data.

4.3.2.7 Radar and Star Plots

Radar plots are based on a similar idea as parallel coordinates with the difference that the coordinate axes are not drawn as parallel lines, but in a star-like fashion intersecting in one point. Figure 4.23 shows a radar plot for the four numerical

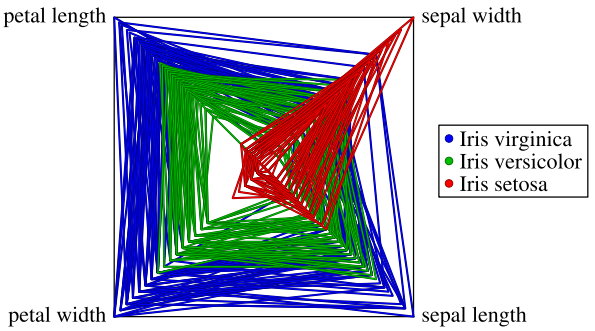


Fig. 4.23 Radar plot for the numerical attributes of the Iris data set

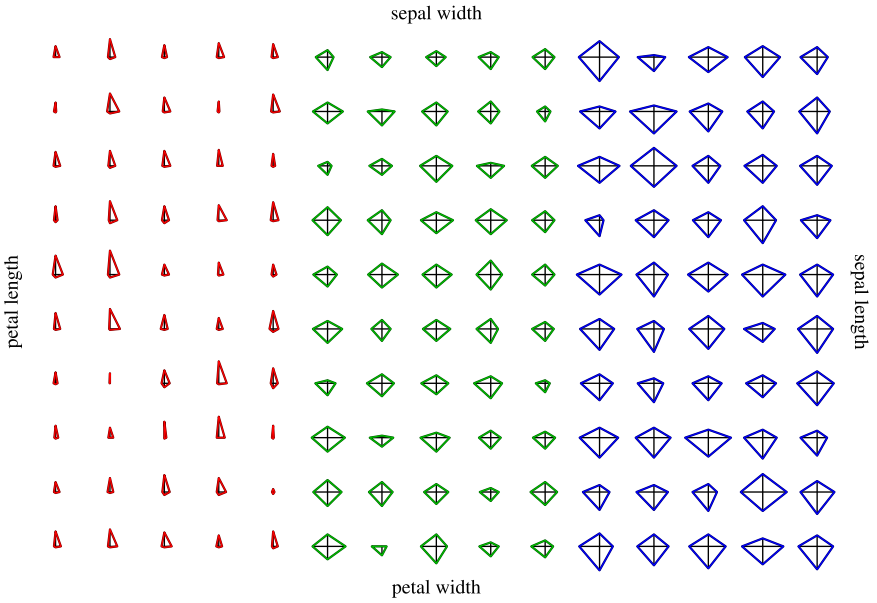


Fig. 4.24 Star plot for the numerical attributes of the Iris data set

attributes of the Iris data set. Due to the fact that radar plots sometimes resemble spider webs, they are also called **spider plots**.

Radar plots are only suited for smaller data sets. For such smaller data sets, it is sometimes better not to draw all data objects in the system of coordinate axes but to draw each data object separately, which is then called a **star plot**. A star plot for the numerical attributes of the Iris data set is shown in Fig. 4.24. The first 50 "stars" correspond to the objects from the species setosa, the next 50 to versicolor, and the last 50 to virginica. The star plot also shows clearly that setosa differs much from the two species.

Table 4.3 Pearson's correlation coefficients for the numerical attributesof the Iris data set

	Sepal length	Sepal width	Petal length	Petal width
Sepal length	1.000	−0.118	0.872	0.818
Sepal width	−0.118	1.000	−0.428	−0.366
Petal length	0.872	−0.428	1.000	0.963
Petal width	0.818	−0.366	0.963	1.000

Another interesting multidimensional hierarchical plot is the sunburst chart. This chart displays hierarchical nominal data in a radial layout. The circle in the chart center represents the hierarchy's root attribute. Sections further outside represent attributes that are located deeper in the hierarchy. Each section is connected to an attribute value and its area is derived from the accumulated value of all descending sections. This visualization technique allows seeing the different groups in the data set, their size and relations.

Figure 4.25 reports the sunburst chart for the adult data set. The adult data set is a CENSUS data set with circa 33k records of people living in the USA and described by classic demographic attributes such as age, gender, occupation, education, and other similar attributes. The adult data set is downloadable from the UC Irvine data repository.

4.4 Correlation Analysis

In the previous section, we have introduced methods to visualize the distribution of values for one, two, or even more attributes simultaneously. Scatter plots can uncover dependencies or correlations between two attributes. Instead of a purely visual approach, it is possible to compute measures of correlation between attributes to confirm expected dependencies or to discover unexpected correlations between attributes.

The (sample) **Pearson's correlation coefficient** is a measure for a linear relationship between two numerical attributes X and Y and is defined as

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{(n-1)s_x s_y}, \tag{4.14}$$

where $\bar{x}$ and $\bar{y}$ are the mean values of the attributes X and Y, respectively, and s_x and s_y are the corresponding (sample) standard deviations. Pearson's correlation coefficient yields values between -1 and 1. The larger the absolute value of the Pearson correlation coefficient, the stronger the linear relationship between the two attributes. For $|r_{xy}| = 1$, the values of X and Y lie exactly on a line. For $r_{xy} = 1$, the line has a positive slope, for $r_{xy} = -1$ a negative one. Table 4.3 lists the Pearson correlation coefficients for the numerical attributes of the Iris data set.

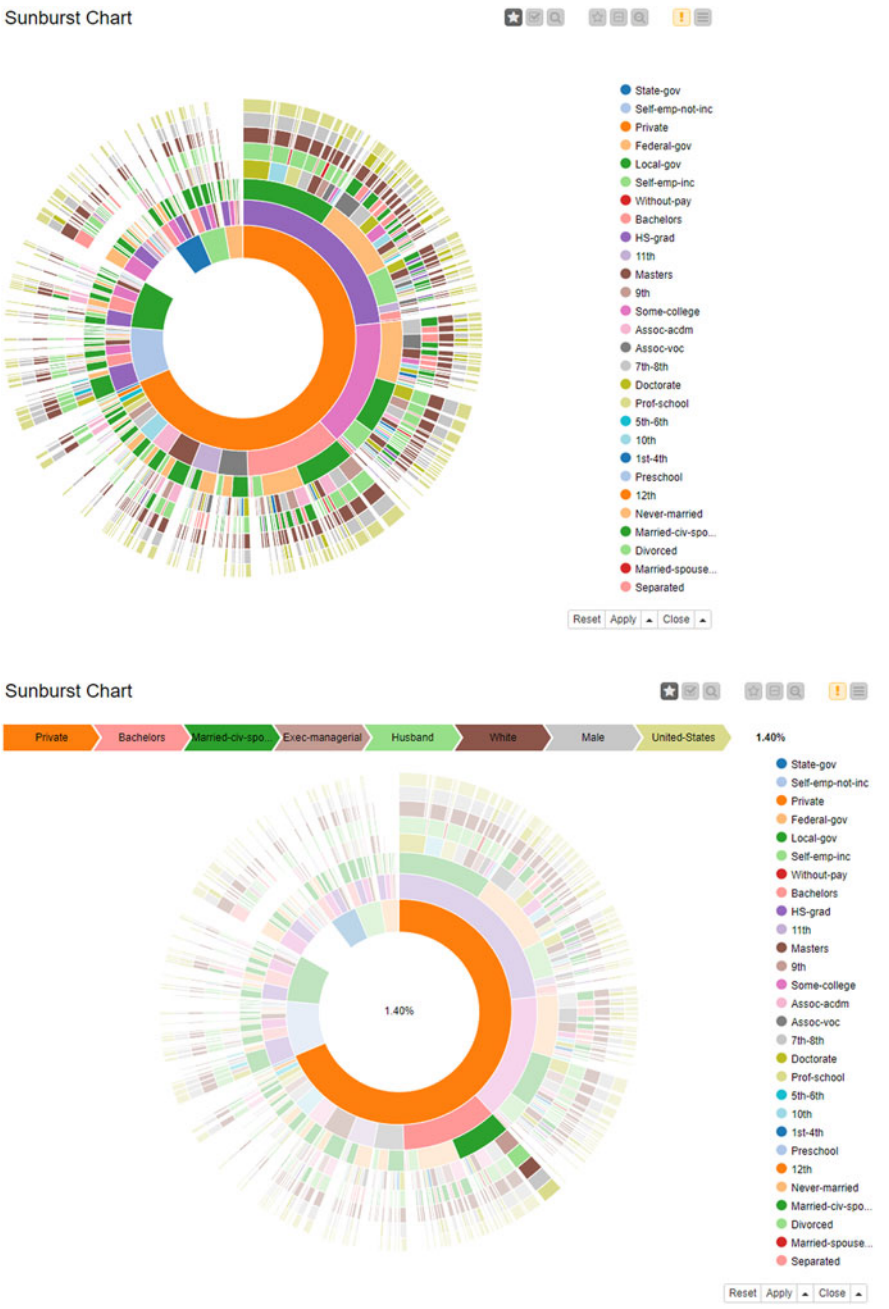


Fig. 4.25 Sunburst chart of the adult data set. On the top, full unselected categories. On the bottom, the selection is visible for workclass = Private, education = Bachelors, Status = Married, Job = Exec-managerial, etc.

Of course, the values on the diagonal must be equal to 1, since an attribute correlates fully with itself. When we plot an attribute against itself in a scatter plot, the points lie on a perfect line, the diagonal, so that Pearson's correlation coefficient must be 1 in this case. The matrix with the coefficients must also be symmetric.

It is also not surprising that the Pearson's correlation coefficient between the length and the petal width is very high. This means more or less that the leaves roughly keep their shape. A short leaf will also not be very broad, and a longer leaf will be broader. It seems counterintuitive that there is more or less no correlation between the sepal length and width or even a very small negative correlation. When we take a look at the scatter plots in Fig. 4.11 on page 46, this can be explained easily. The negative correlation originates from the fact that we have the measurements from different species. Setosa has short but broad leaves compared to the other species. When we compute Pearson's correlation coefficient separately for the species for the sepal length and width, we obtain the values 0.743, 0.526, and 0.457 for Iris setosa, Iris versicolor, and Iris virginica, respectively. The correlation is not as high as for the petal length and width, but at least it is positive and not negative.

Pearson's correlation coefficient measures linear correlation. Even if there is a functional dependency between two attributes, but the function is nonlinear but monotone, Pearson's correlation coefficient will not be -1 or 1. It can even be far away from these values, depending on how much the function describing the functional relationship deviates from a line.

Rank correlation coefficients avoid this problem by ignoring the exact numerical values of the attributes and considering only the ordering of the values. Rank correlation coefficients intend to measure monotonous correlations between attributes where the monotonous function does not have to be linear.

Spearman's rank correlation coefficient or **Spearman's rho** is defined as

$$\rho = 1 - 6\frac{\sum_{i=1}^{n}(r(x_i) - r(y_i))^2}{n(n^2-1)}, \tag{4.15}$$

where $r(x_i)$ is the rank of value x_i when we sort the list $(x_1, \ldots, x_n)$; $r(y_i)$ is defined analogously.

Spearman's rho measures the sum of quadratic distances of ranks and scales this measure to the interval $[-1, 1]$. When the rankings of the x- and y-values are exactly in the same order, Spearman's rho will yield the value 1; if they are in reverse order, we will obtain the value -1.

Spearman's rho assumes that there are no ties, i.e., no two values of one attribute are equal. If two or more values coincide, their rank is not defined. When ties exist, the rank $r(x_i)$ is usually defined as the mean value of all ranks of consecutive coinciding values in the sorted list. So if we have the (already sorted) list of values 0.6, 1.2, 1.4, 1.4, 1.6 for an attribute, the corresponding ranks would be 1, 2, 3.5, 3.5, 4.

Kendall's tau rank correlation coefficient or simply **Kendall's tau** is not, like Spearman's rho, based on ranks, but rather on the comparison of the orders of pairs of values. Assuming that $x_i < x_j$, the two pairs (x_i, x_j) and (y_i, y_j) are called **concordant** if $y_i < y_j$, i.e., when the two pairs are in the same order. They are called **discordant** when they are in reverse order, which means that $y_i > y_j$.

Table 4.4 Spearman's rank correlation coefficients for the numerical attributesof the Iris data set

	Sepal length	Sepal width	Petal length	Petal width
Sepal length	1.000	−0.167	0.882	0.834
Sepal width	−0.167	1.000	−0.289	−0.289
Petal length	0.882	−0.289	1.000	0.938
Petal width	0.834	−0.289	0.938	1.000

Table 4.5 Kendall's tau for the numerical attributes of the Iris dataset

	Sepal length	Sepal width	Petal length	Petal width
Sepal length	1.000	−0.077	0.719	0.655
Sepal width	−0.077	1.000	−0.186	−0.157
Petal length	0.719	−0.186	1.000	0.807
Petal width	0.655	−0.157	0.807	1.000

Kendall's tau is computed as

$$\tau_a = \frac{C - D}{\frac{1}{2}n(n-1)},$$

where C and D denote the numbers of concordant and discordant pairs, respectively:

$$C = \left|\left\{(i,j) \;\middle|\; x_i < x_j \text{ and } y_i < y_j\right\}\right|, \tag{4.16}$$

$$D = \left|\left\{(i,j) \;\middle|\; x_i < x_j \text{ and } y_i > y_j\right\}\right|. \tag{4.17}$$

Tables 4.4 and 4.5 contain the values for Spearman's rho and Kendall's tau for the numerical attributes of the Iris data set. The values are not identical but very similar to those in Table 4.3 for Pearson's correlation coefficient.

Rank correlation coefficients like Spearman's rho and Kendall's tau depend only on the order (ranks) of the values and are therefore more robust against extreme outliers than Pearson's correlation coefficient.

For categorical attributes, these correlation coefficients are not applicable. Instead, one can carry out independence tests like the χ^2 test for independence, as it is described in the appendix in Sect. A.4.3.7. This test can also be used for numerical attributes when they are discretized by some binning strategy.

4.5 Outlier Detection

Since it has an intuitive, though imprecise meaning, we have used the term outlier already before without giving a precise definition. Actually, there is no formally precise definition of outliers. An **outlier** is simply a value or data object that is far away or very different from all or most of the other data.

Outliers can be a hint to data quality problems as they will be discussed in the next section. Outliers can correspond to erroneous data coming from wrong measurements or typing mistakes when data are entered manually. Erroneous data should be corrected, or, if this is not possible, they should be excluded from the data set before further analysis steps are carried out. However, outliers can also be correct data that differ from the rest of the data just by chance or for other reasons like special exceptional situations. Even if outliers are correct data, it might be worthwhile to exclude them from the data set for further analysis and to consider them separately. Some data analysis methods are robust against outliers and more or less not influenced by them, whereas other are extremely sensitive to outliers and might produce incoherent or nonsense results, just because of the presence of one or a few outliers. As a simple example, consider the median and the mean value. If one replaces the largest value in a sample by an extremely large value or even infinity, the median will not change. But the mean value will tend to infinity, even though only one value in the sample goes to infinity.

4.5.1 Outlier Detection for Single Attributes

For a categorical attribute, one can consider the finite set of values. An outlier is a value that occurs with a frequency extremely lower than the frequency of all other values. However, in some cases, this might be actually the target of our analysis. If we want to set up an automatic quality control system and want to train a classifier, classifying the parts as correct or with failures based on measurements of the produced parts, we will probably have so many correct parts in comparison to those with failures that we would consider them as outliers. However, removing these "outliers" from the data set would actually make it impossible to achieve our original goal to derive a classifier from the data set that can identify the parts with failures.

For numerical attributes, outlier detection is more difficult. We have already classified certain data points in a boxplot as outliers, which is one of the simplest and a nonparametric outlier detection method. Here outliers are identified by means of the interquartile range $IQR = Q_3 - Q_1$, which is derived from the first quartile Q_1 and the third quartile Q_3. An outlier is then a data point x_i that lies outside the interquartile range. That is,

$$x_i < Q_1 - k \cdot IQR \qquad \vee \qquad Q_3 + k \cdot IQR < x_i .$$

By using the interquartile multiplier value $k = 1.5$, the range limits are the typical upper and lower whiskers of a boxplot.

However, the definition of outliers in a boxplot does not take the number of data into account, so that for larger data sets, boxplots will usually contain points marked as outliers. We have seen this already in the boxplots in Fig. 4.6, both showing samples from a standard normal distribution. The left boxplot in Fig. 4.6 for a sample of size $n = 1000$ contains eight outliers corresponding to what we expect theoretically, namely seven points outside the whiskers. As mentioned before, for a normal distribution, we can expect roughly 0.7% points to be marked as outliers in a boxplot.

For asymmetric distributions, boxplots tend to contain more outliers. For example, in boxplots for samples from an exponential distribution with $\lambda = 1$ as they are shown in Fig. 4.8, we would expect roughly 4.8% points marked as outliers. Heavy-tailed distributions tend to show more outliers in a boxplot, whereas for a sample from a uniform distribution, we would expect no outliers at all, no matter how large the sample size is.

Even if we try to adjust the definition of outliers according to the sample size, the above examples show that what we consider as an outlier depends strongly on the underlying distribution from which the data are sampled. Therefore, statistical tests for outliers are usually based on assumptions about the underlying distribution, although we might not know from which distribution the data are sampled.

The standard assumption for outlier tests for continuous attributes is that the underlying distribution is a normal distribution. A simple and frequently applied approach in a one-dimensional feature space (but low-dimensional spaces as well) is to identify outliers as the data points in the tails of the distribution and far from the mean. How far depends on a threshold z_{thr} for the z-score standardized data points $z_i = \frac{x_i - \bar{x}}{s}$ where x_i is a data point, $\bar{x}$ is the mean and s is the standard deviation of all x_i. A point x_i is considered to be an outlier if it is z_{thr} times the standard deviation apart from the mean, that is, $\|z_i\| > z_{thr}$. Commonly used values for z_{thr} are 2.5, 3.0, and 3.5.

A somewhat stricter statistical approach would be the **Grubbs test**, which also assumes an underlying normal distribution and tests for the existence of (at least one) outlier in the data. An outlier, if detected, has to be removed, and the test is repeated on the remainder of the data set until no more outliers are found. The test is based on the statistics

$$G = \frac{\max\{|x_i - \bar{x}| \mid 1 \leq i \leq n\}}{s}, \tag{4.18}$$

where $x_1, \ldots, x_n$ is the sample, $\bar{x}$ is its mean value, and s is its empirical standard deviation. For a given significance level α, the null hypothesis that the sample does not contain outliers is rejected if

$$G > \frac{n-1}{\sqrt{n}} \sqrt{\frac{t^2_{1-\alpha/(2n),n-2}}{n-2+t^2_{1-\alpha/(2n),n-2}}}, \tag{4.19}$$

where $t_{1-\alpha/(2n),n-2}$ denotes the $(1 - \alpha/(2n))$-quantile of the t-distribution with $(n - 2)$ degrees of freedom.

For the Iris data set, Grubbs test yields the p-values[4] 0.92, 0.13, 1.0, and 1.0 for the sepal length and width and the petal length and width, respectively. Even the lowest p-value does not indicate that there is an outlier. Note that this is the

[4]For Grubbs test, the null hypothesis is that there are no outliers. Then the point in the sample with the largest distance to the mean is considered. In the case of Grubbs test, the p-value is the probability that in a sample of size n, such a large or even larger deviation from the mean would occur. For a more formal and general definition of p-values, see also Appendix A.

case although the assumption that the attributes follow normal distributions is not realistic at all as can be seen from the scatter plot in Fig. 4.11 on page 46. The attributes for each species separately might follow normal distributions, but not the attributes considered for all species together.

When Grubbs test with its assumption of normal distribution is applied to very skewed and heavy-tailed distributions like the exponential distribution, an outlier will almost always be indicated.

4.5.2 Outlier Detection for Multidimensional Data

Outlier detection in multidimensional data is usually not based on specific assumptions on the distribution of the data and is not carried out in the sense of statistical tests. Visualization techniques provide a simple method for outlier detection in multidimensional data. Scatter plots can be used when only two attributes are considered. In the right scatter plot in Fig. 4.12 on page 47, we had added two outliers. The one in the upper left corner would not have been detected when only single attributes are considered. Instead of using projections to two attributes, one can also use dimension-reduction methods like PCA or multidimensional scaling in order to identify outliers in the corresponding plots as in Figs. 4.16 and 4.18.

There are many approaches for finding outliers in multidimensional data based on clustering the data and defining those data objects as outliers that cannot be assigned reasonably to any cluster [19, 20]. There are also distance-based [13, 14], density-based [4], and projection-based methods [1] for outlier detection.

The underlying idea of clustering-based outlier detection is that isolated points, i.e., clusters of one or a few data points, far away from all other clusters can be interpreted as outliers. Among those, the clustering algorithm **DBScan** (cf. Sect. 7.4) turns out to be particularly suitable to discover outliers, since it does include the concept of noise points that are too far away from all other points to be part of any cluster. DBScan is a nonparametric, density-based clustering method in a one- or multidimensional feature space. In the DBScan clustering technique, all data points are defined either as Core Points, Border Points or Noise Points. Core Points and Border Points have neighbors or can be reached via neighbor points. Noise points are isolated points and can therefore be identified as outliers. Outlier detection with DBScan thus depends on the required number of neighbors to be a Core Point, the distance threshold ε to define the neighborhood size, and the selected distance measure, like Euclidean or Manhattan.

Isolation Forest is a nonparametric method for large data sets [15]. The idea is to measure how difficult it is to isolate a single point from all other points of the data set. An important concept here is the isolation number, that is, the number of splits needed in order to isolate a single point. This number of splits is ascertained by following these steps:

- A data point a to isolate is selected.
- A random data point b is selected that is between the minimum and maximum value and different from a.

- If the value of b is lower than the value of a, the value of b becomes the new lower limit.
- If the value of b is greater than the value of a, the value of b becomes the new upper limit.
- This procedure is repeated as long as there are data points other than a between the upper and the lower limit.

Now, we can observe that it requires fewer splits to isolate an outlier than it does to isolate a nonoutlier. That is, an outlier has a lower isolation number in comparison to a nonoutlier point. A data point is therefore defined as an outlier if its isolation number is lower than a given threshold. The threshold is generally defined based on the estimated percentage of outliers in the data.

4.6 Missing Values

In the ideal case, all attributes of all objects in the data table have well-defined values. However, in real data sets, one has to deal with **missing values** quite often. The occurrence of missing values can have different causes. A sensor might be broken leading to a missing value. People might have refused or forgotten to answer a question in a questionnaire, or an attribute might not be applicable for a certain object. For instance, the attribute *pregnant* with the two possible values *yes* and *no* does not make sense for men. Of course, one could always enter the value *no* for the attribute *pregnant*. But this might lead to a grouping of men with nonpregnant women.

Especially in a customer database, missing values will be quite common. Attributes like *age*, *sex*, and *email address* might be available for some customers, but probably not for all customers.

In the best case, missing values are indicated in a unique way. A symbol like "?" or null entry might be used for missing value. Sometimes a special numeric value is used to indicate missing values. For example, the value -1 could indicate a missing value for a counting attribute (like number of products bought by a customer in a certain period, number of children, etc.). Since the value -1 can never occur in a counting attribute, it cannot be misunderstood as meaningful (nonmissing) value. One must, however, take this into account when simple characteristics like the mean value or the median of an attribute are computed. If one simply adds up all values—including the -1's for the missing values—and divide it by the number of values to obtain the mean value, the result might even be negative if there are many missing values. If we know the number of children for only very few of our customers, we might find out in this way that the average number of children of our customers is -0.3, which is, of course, complete nonsense.

Sometimes, missing values are not even marked in a specific way. We have seen already an example in Fig. 2.2 on page 18 where a missing value of the customer's birth date was replaced by a default value that could also be just a usual value.

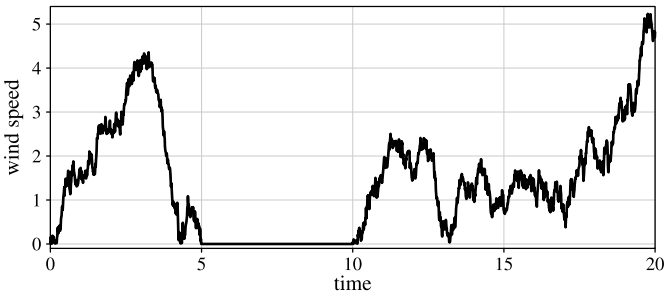


Fig. 4.26 Measured wind speeds with a period of a jammed sensor

As another example, consider an anemometer, a sensor for measuring the wind speed. Such an anemometer is equipped with a small wheel that is turned by the wind, and from the speed of the wheel the speed of the wind can be deduced. Sometimes, the wheel might be blocked for some reason. In this case, the sensor will constantly yield the value zero for the wind speed. This value zero is also a possible wind speed although it is very improbable in most geographical regions that the wind speed is absolutely zero over a longer time. When we see a diagram like in Fig. 4.26, it is very probable that during the period between 5 and 10, the wind speed was not zero, but that there are missing values in this period due to a jammed anemometer.

It is very important to identify such hidden missing values. Otherwise, the further analysis of the data can be completely mislead by such erroneous values.

When there are missing values, one should also take into account how missing values enter the data set. The simplest and most common assumption about missing values is that they are **missing completely at random** (**MCAR**) or are **observed at random** (**OAR**). This means that special circumstances or special values in the data lead to higher or lower chances for missing values. One can imagine that one has printed out the data table on a large sheet of paper with no missing values and then someone has dropped accidentally some random spots of ink on the paper so that some values cannot be read anymore.

In a more formal way, the situation missing completely at random can be described in the following way. We consider the random variable X for which we might have some missing entries in the data set. The random variable X itself does not have missing values. The corresponding value is just not available in our data set. The random variable X_{miss} can assume the value 0 for a missing values and 1 for a nonmissing value for the random variable X of interest. This means that for $X_{\text{miss}} = 1$, we see the true value of X in our data set, and for $X_{\text{miss}} = 0$, instead of the true value of X, we see a missing value. We also consider the random vector Y representing all attributes except X in our data set. The situation observed at random means that X_{miss} is independent of X and Y, i.e.,

$$P(X_{\text{miss}}) = P(X_{\text{miss}}|X, Y) \qquad \text{(MCAR)}.$$

Consider a sensor for the outside air temperature X whose battery might run out of energy once in a while, leading to missing values of the missing completely at random. This is the best case for missing values. It can, for instance, be concluded that the unknown missing values follow the same distribution as the known values of X.

The situation **missing at random** (**MAR**) is more complicated but might still be manageable with appropriate techniques. The probability for a missing value depends on the value of the other attributes Y but is conditionally independent[5] of the true value of X given Y:

$$P(X_{\text{miss}}|Y) = P(X_{\text{miss}}|X, Y) \qquad \text{(MAR)}.$$

To illustrate what missing at random means, consider the above example of a sensor for the outside air temperature with the battery running out of energy once in a while. But assume now that the staff responsible for changing the batteries does not change the batteries when it is raining. Therefore, missing values for the temperature are more probable when it is raining, so that the occurrence of missing values depends on other attributes, in this case the attribute for rain or amount of rain.

Note that the occurrence of a missing value is in this case not independent of the value of the variable X, it is only conditionally independent given the attribute for the rain. Usually, the outside temperatures are lower when it is raining, so that missing values correlate with lower temperatures. In contrast to missing completely at random, here the missing values of X do not follow the same distribution as the measured values of X. If we compute the average temperature based only on the measured values, we would obviously overestimate the average temperature. Therefore, special care has to be taken when missing values occur at random, but not completely at random.

The worst case are **nonignorable missing values**, where the occurrence of missing values directly depends on the true value, and the dependence cannot be resolved by other attributes. Consider a bad sensor for the outside temperature that always fails when there is frost. This would mean that we have no information about temperatures below 0 °C. In the case of missing at random, we might still be able to obtain information from the data set about the missing values. For the nonignorable values, we have no chance to make correct guesses, at least based on the available data.

How can we distinguish between the three different types of missing values? First of all, the knowledge and background information about why missing values occur should be taken into account. It is also possible to try to distinguish the two cases observed at random and missing at random based on the given data. If an attribute X has missing values and one wants to find out of which type the missing values are, one way would be to apply independence tests checking directly whether $P(X_{\text{miss}}) = P(X_{\text{miss}}|Y)$. Usually, it is not feasible to apply a test that checks the independence of X_{miss} against the multivariate attribute Y standing for all other attributes, but only against single attributes. Applying independence test of X_{miss}

[5] A formal definition of conditional independence is given in Sect. A.3.3.4 in the appendix.

against each other attribute Y will not uncover multiple dependencies, but if even these tests indicate a dependency already, then we can be quite sure that the values of X are not missing completely at random.

Another way to approach the problem of distinguishing between missing completely at random and missing at random is the following procedure:

1. Turn the considered attribute X into a binary attribute, replacing all measured values by the values *yes* and all missing values by the value *no*.
2. Build a classifier with now binary attribute X as the target attribute and use all other attributes for the prediction of the class values *yes* and *no*.
3. Determine the misclassification rate. The misclassification rate is the proportion of data objects that are not assigned to the correct class by the classifier.

In the case of missing values of the type observed at random, the other attributes should not provide any information, whether X has a missing value or not. Therefore, the misclassification rate of the classifier should not differ significantly from pure guessing, i.e., if there are 10% missing values for the attribute X, the misclassification rate of the classifier should not be much smaller than 10%. If, however, the misclassification rate of the classifier is significantly better than pure guessing, this is an indicator that there is a correlation between missing values for X and the values of the other attributes. Therefore, the missing values for X might not be of the type observed at random but of the type missing at random or, even worse, nonignorable. Note that it is in general not possible to distinguish the case nonignorable from the other two cases based on the data only.

4.7 A Checklist for Data Understanding

In this chapter, we have provided a selection of techniques for data understanding. It is not necessary to go through all of them in the phase of data understanding, but rather to keep all of them in mind.

- First of all, one should keep in mind that there are general and specific goals for data understanding. One important part of data understanding is to get an idea of the data quality. There are standard data quality problems like syntactic accuracy which are easy to check.
- Outliers are another problem, and there are various methods to support the identification of outliers. Apart from methods exclusively designed for outlier detection as they have been discussed in Sect. 4.5, there are especially visualization techniques like boxplots, histograms, scatter plots, projections based on PCA and MDS that can help find outliers but are also useful for other purposes.
- Missing values are another concern of data quality. When there are explicit missing values, i.e., entries that are directly marked as missing, then one should still try to find out of which type—OAR, MAR, or nonignorable—they are. This can sometimes be derived from domain knowledge, but also based on classification methods as described in the previous section. We should also be aware of the

possibility of hidden missing values that are not explicitly marked as missing. The simplest case might be hidden missing values that have a default value. Histograms might help to identify candidates for such hidden missing values when there are unusual peaks as in the example in Fig. 2.2 on page 18. However, there is no standard test or technique to identify possible hidden missing values. Therefore, whenever we see something unexpected in the data, hidden missing values of a specific type might be one explanation.
- Apart from these data quality issues, data understanding should also help to discover new or confirm expected dependencies or correlations between attributes. Techniques like those mentioned in Sect. 4.4 are one way to solve this task. Apart from this, scatter plots can show correlations between pairs of attributes.
- Specific application dependent assumptions—for instance, the assumption that a specific attribute follows a normal distribution—should also be checked during data understanding.
- Representativeness of the data cannot always be checked just based on the data, but we have to compare the statistics with our expectations. If we suspect that there is a change in a numerical attribute over time, we can compare histograms or boxplots for different time periods. We can do the same with bar charts for categorical attributes.

No matter what the specific application dependent purposes of data understanding are, here is a checklist what we consider as a must for the data understanding phase:

- Check the distributions for each attribute whether there are unusual or unexpected properties like outliers. Are the domains or ranges correct? Do the medians of numerical attributes look correct? This should be done based on
  - histograms and boxplots for continuous attributes and
  - bar charts for categorical attributes.
- Check correlations or dependencies between pairs of attributes with scatter plots which should be density-based for larger data sets. For small numbers of attributes, inspect scatter plots for all pairs of attributes. For higher numbers of attributes, do not generate scatter plots for all pairs, but only for those where independence or a specific dependency is expected. Generate in addition scatter plots for some randomly chosen pairs.

To cite Tukey [25] again, if problems in later phases of the data analysis process occur that could have been discovered or even avoided in an early plotting and looking at the data, there is absolutely no excuse for this failure.

Of course, one should also exploit the other methods described in this section for these and other more specific purposes.

4.8 Data Understanding in Practice

A key issue in data understanding (and actually the entire data science process) often causes considerable headaches: the loading of the data into the analysis tool

of choice. Data indeed can come in a variety of formats, types, and sources. We must be able to load data into the tool either from a database, a pure text file, a CSV file, an Excel file, and probably from many more sources and in many more different shapes.

In Sect. B we have seen the basics of KNIME Analytics Platform: what is a node, what is a workflow, and how the workbench is organized. We have seen how to create, configure, and execute a node, and how to inspect its results. We have also seen how to read a text file using the File Reader node. Please refer to Sect. B for more details about all of these concepts and tasks.

4.8.1 Visualizing the Iris Data

In this example workflow, named `01_Simple_Visualization`, we have artificially split the iris data set in three parts: 88 flowers in a table "iris1" in an SQLite database; 60 more flowers in an Excel file; and the class map (class x -> iris name) into a CSV file.

Reading the CSV File Let's start by reading the class map. We can use a File Reader node or a CSV Reader node. They both read CSV files. The File Reader node is just a bit more flexible in the kind of text files it can read. Since we have used the File Reader node in Sect. B, we will use here the CSV Reader node. Like all nodes, it can be created by a drag & drop from the Node Repository panel into the workflow editor or by a double-click on the node in the Node Repository. We know that the newly created node has a red light status.

In order to configure it, you can right-click the node and then select *Configure* or just double-click on the node. Then the configuration window opens where you can define the appropriate settings to read the file (Fig. 4.27): the file path; the character used as a delimiter between column values; the character used for quotes if needed; the character used for comments if any; whether the first row contains the column headers; whether the first column contains the row headers; and whether shorter lines with missing values should also be accepted. After setting has been completed, click the button *OK* at the bottom of the window. If the configuration has been successful, the status will turn to the yellow light.

Did you notice the *knime://* protocol in the file path? For better portability of the workflow, it is possible to incorporate the file with the data within the workflow itself. We have created a folder named "data" in the workflow folder inside the workspace folder and we have copied the data file in it. This file can then be accessed using the *knime://* protocol and starting the path from the current workflow folder with *knime.workflow*. In this way, when exporting the workflow, the data are exported with it.

Now right-click and select *Execute* or click on one of the two green arrow buttons in the tool bar to execute the node. If the execution is successful, the status will turn to the green light.

After a successful execution, resulting data can be inspected by right-clicking the node and selecting the last option in the context menu. This action gives a chance to inspect that the node task has been executed correctly.

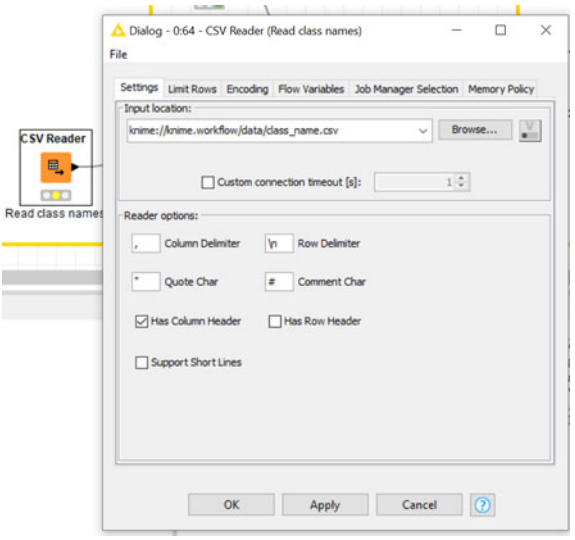


Fig. 4.27 Reading the CSV file `class_name.csv` with a CSV Reader node

Reading the Excel File Similarly, an Excel Reader (XLS) node reads an Excel file. Again, double-click the node or right-click the node and select *Configure*. The configuration of an Excel Reader (XLS) node is more complex than that of a CSV Reader node, since Excel allows for more flexibility in the Excel formatted table (Fig. 4.28).

Notice the data preview in the lower part of the configuration window, the *Refresh* button to refresh it every time a new configuration setting has changed, and the option *Table contains column names in row number:* to assign headers to the columns from the values in the selected row.

Again execute (right-click -> *Execute* or click one of the green arrowed buttons in the tool bar) the node and inspect the results by right-clicking on the node and then selecting the last option in the context menu.

Data Types KNIME Analytics Platform supports all basic data types, such as strings, integers, and numbers, but also date and time types and nominal values. In addition, various extensions add the ability to process images, molecular structures, sequences, and textual data. The repository of type extension is constantly growing as well.

Importing Data from the Database Table Finally, let's import the data from the SQLite database into KNIME Analytics Platform. KNIME Analytics Platform offers a wide range of connectors to the most commonly used databases, such as to H2, MySQL, Oracle, SQLite, and more. Each connector node requires different configuration settings depending on the database requirements.

KNIME Analytics Platform also offers the possibility to connect to less common databases or database versions using the generic *DB Connector* node together with the database JDBC driver.

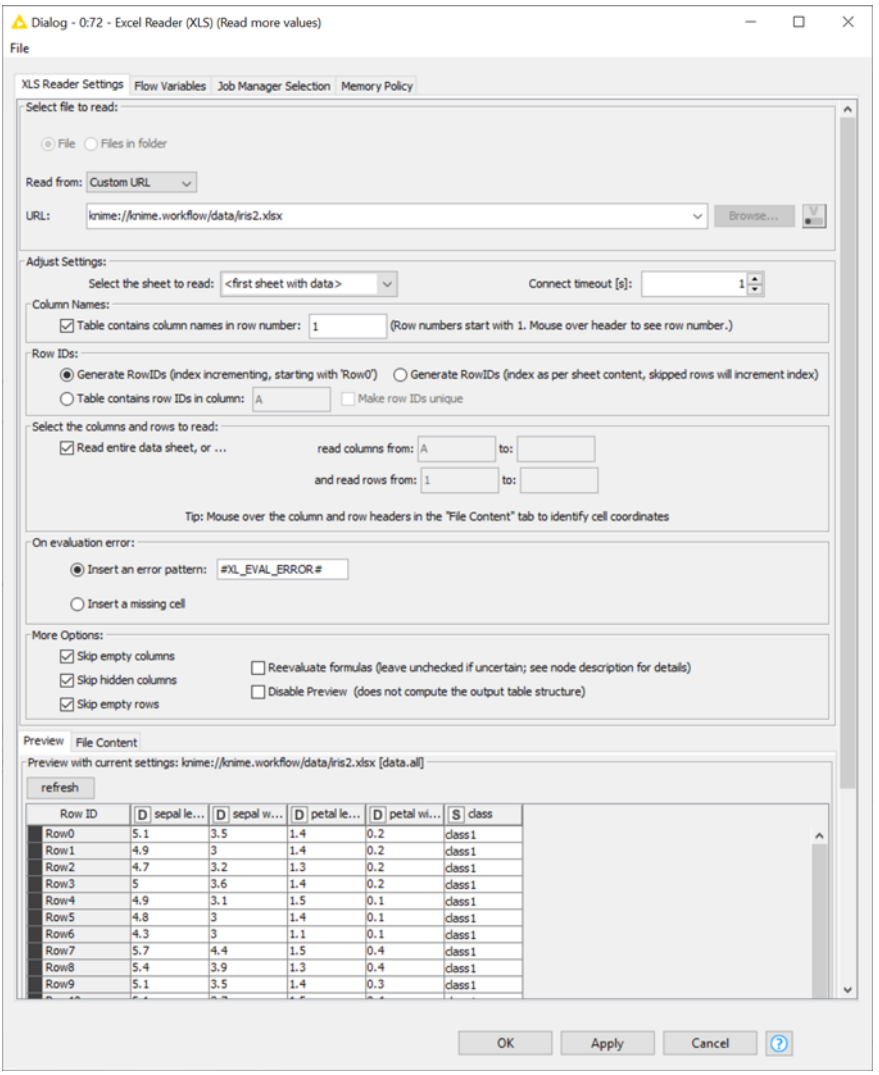


Fig. 4.28 Reading Excel file `iris2.xlsx` with the Excel Reader (XLS) node

Since we want to connect to an SQLite database, we need the *SQLite Connector* node. The *SQLite Connector* node requires just the path of the SQLite file (SQLite is a file based database). Notice the different port, a red square. The red square is the port exposing a connection to a database.

After the connector node, we use a *DB Table Selector* node. The *DB Table Selector* node receives a database connection as input and produces a SQL statement (brown square port) as output. The SQL statement is either the default SELECT statement for the table selected in the configuration window or a more accurate custom SQL statement typed into the configuration window.

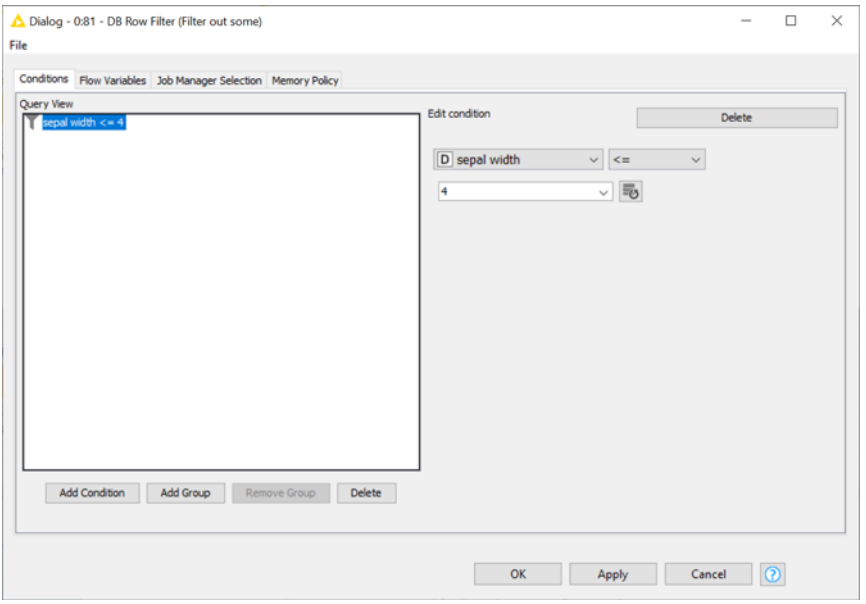


Fig. 4.29 Configuration window of the DB Row Filter node. Filtering database records via SQL statement and without writing SQL code

For basic data manipulation, specialized nodes are also available, which allow one to build complex SQL statements without coding but using the familiar Graphical User Interface of the KNIME configuration windows. It is the case of the *DB Row Filter* node that we use here to filter out all plants with sepal width smaller than 4 cm, just for demonstrational purposes (Fig. 4.29). This node adds one additional SQL query (the one filtering the rows) on top of the input SQL query and produces the more complex SQL query at the output port.

Finally, the *DB Reader* node collects and executes the SQL statement at its input port on the database and makes the resulting data available at its output port. No configuration settings are required, since the whole task is defined in the input SQL statement. This node is the one responsible for the final in-database processing.

This lazy loading of the data from the database (first we build the SQL query and only at the end we run it on the database) saves time for larger data sets if only a much smaller subset is to be analyzed. The top part of Fig. 4.30 shows the sequence of DB nodes used to import data from the SQLite database.

Putting It All Together: Concatenation and Join Now we have the data from the SQLite database (some of the iris plants), from the Excel file (more iris plants), and from the CSV file (the actual classes). Let's put it all together.

The Concatenate node puts data tables with same structure one on top of another. If the two input data tables differ in the number of columns, we can choose via the configuration window whether to keep only the intersection or the union of columns from the two tables. The Joiner node performs a join operation (inner join, left outer

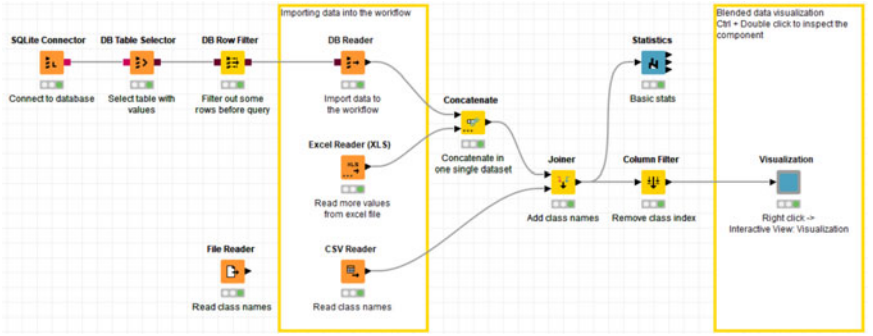
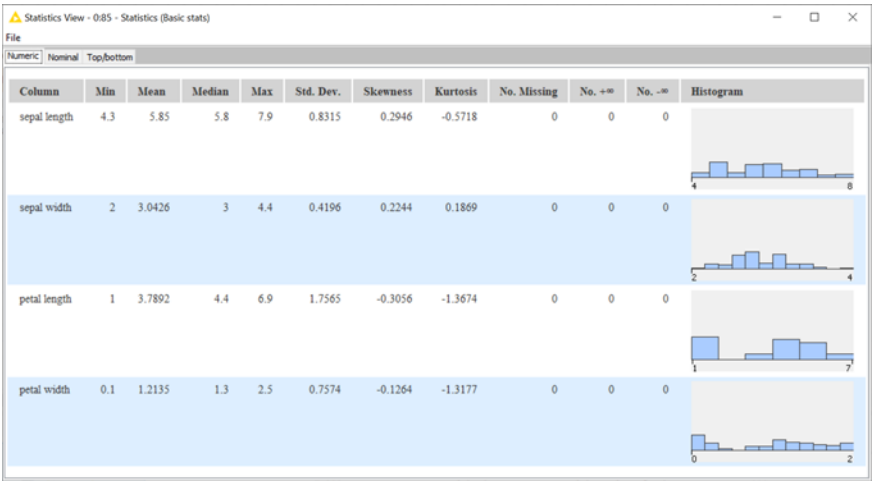


Fig. 4.30 The workflow named `01_Simple_Visualization` reads data from three different sources: CSV file, Excel file, SQLite database. Then it concatenates and joins the data together, calculates some basic statistics, and finally visualizes the data with some plots and charts. Notice the component background set to the light blue "Visualizer" color rather than to unknown component's gray color



Column	Min	Mean	Median	Max	Std. Dev.	Skewness	Kurtosis	No. Missing	No. +∞	No. -∞	Histogram
sepal length	4.3	5.85	5.8	7.9	0.8315	0.2946	-0.5718	0	0	0	
sepal width	2	3.0426	3	4.4	0.4196	0.2244	0.1869	0	0	0	
petal length	1	3.7892	4.4	6.9	1.7565	-0.3056	-1.3674	0	0	0	
petal width	0.1	1.2135	1.3	2.5	0.7574	-0.1264	-1.3177	0	0	0	

Fig. 4.31 Basic statistical measures for numeric columns in the iris data set, as shown in the View of the Statistics node

join, right outer join, and full outer join) on one or more key values from one or more key columns.

The Basic Statistics For a first glance at the data and in order to check domain and nominal value range, we added the *Statistics* node. The Statistics node computes basic statistical measures (minimum, maximum, mean, standard deviation, variance, overall sum), counts the number of missing values for numerical attributes, and lists the most and least frequent values for nominal values. All those measures are then visualized through the node View (right-click → View) shown in Fig. 4.31. The

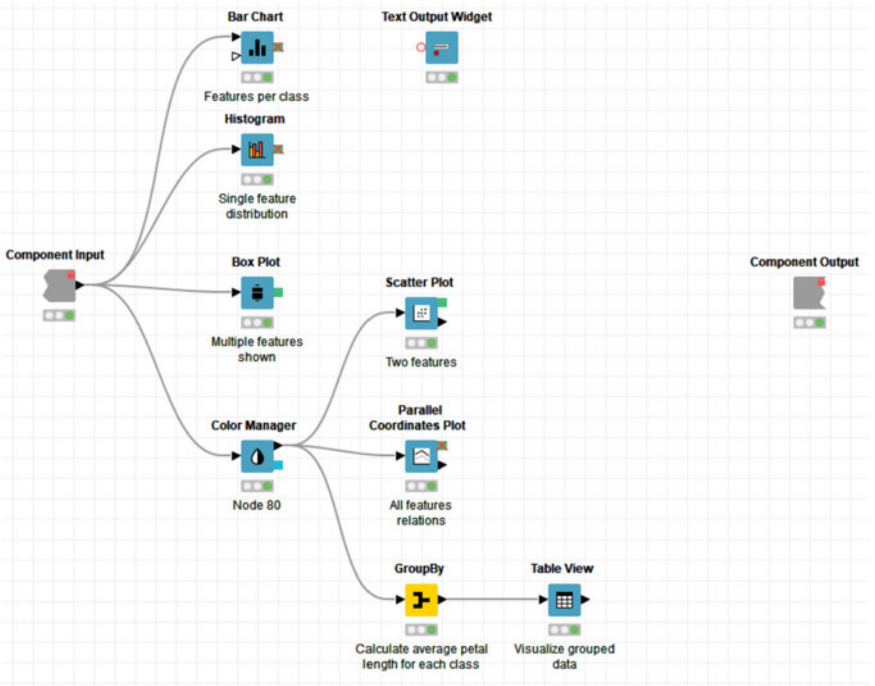


Fig. 4.32 Content of component named "Visualization"

View has two tabs, one for nominal columns and one for numeric columns. Stats for numeric columns are shown in Fig. 4.31.

Data Visualization Checking basic descriptors of your data is important, but, as pointed out earlier in this chapter, looking at visualizations often gives dramatically better insights into your data. KNIME Analytics Platform offers all of the visualizations described in Sect. 4.3: histograms, bar charts, boxplots (also conditional), scatter and scatter matrix plots, and parallel coordinates. Each one of these charts and plots has a dedicated node offering a graphical view where the chart or plot is reported.

All of these views make use of color, shape, and size information whenever possible. Such additional **visual dimensions** are added by using "Color/Shape/Size Manager" nodes, with the additional advantage that all subsequent nodes are using the same color/shape/size information.

Collecting some of these visualization nodes inside one component produces a composite view, formed by the composition of the different subviews inherited from each node. In Fig. 4.30, the last node on the right named "Visualization" produces one of these composite views, including a bar chart of the average sepal/petal length/width by iris class, a histogram of the sepal width across the whole data set (Fig. 4.33), a boxplot for all features (Fig. 4.34), a scatter plot of sepal width vs. sepal length, a parallel coordinates plot, and finally, as the last item at the bottom of

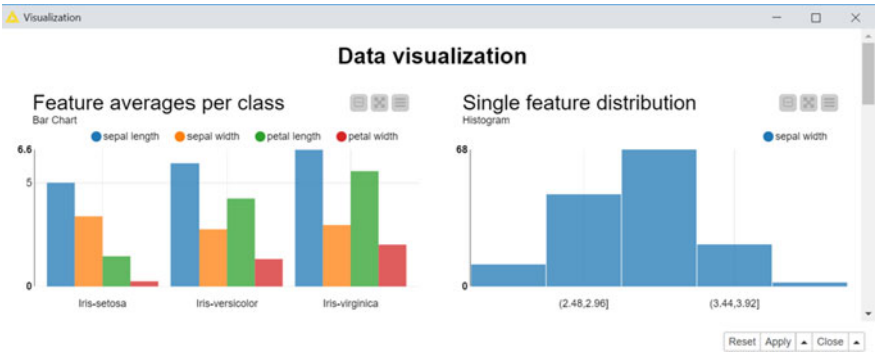


Fig. 4.33 Top part of the composite view of the "Visualization" component node with bar chart of the average sepal/petal length/width by iris class and a histogram of the sepal width across the whole data set

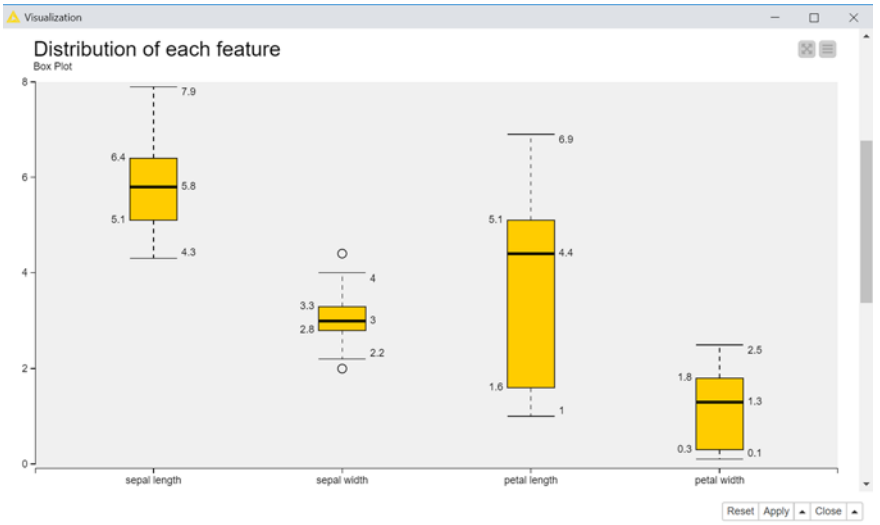


Fig. 4.34 Middle part of the composite view of the "Visualization" component node with a box-plot for all features

the composite view, a table with the average petal length (Fig. 4.35). The content of the component is shown in Fig. 4.32. There you can see all the nodes responsible for each subview in the composite view, plus a *Color Manager* node to set the colors in the upcoming plots.

KNIME Analytics Platform has been developed to also support explorative data analysis and therefore most of the views allow for interactivity and **visual brushing**. This means that points (or other visual objects) representing records in the underlying data set can be marked and will then be marked in other, related views as well. After enabling the selection option in the menu in the top right corner of the scatter

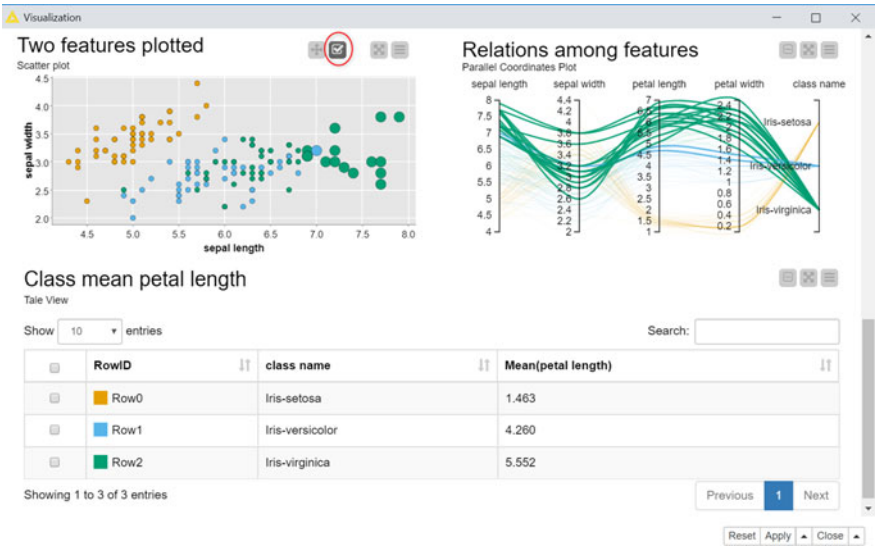


Fig. 4.35 Lower part of the composite view of the "Visualization" component node with a scatter plot of sepal width vs. sepal length, a parallel coordinates plot, and finally, as the last item at the bottom of the composite view, a table with the average petal length

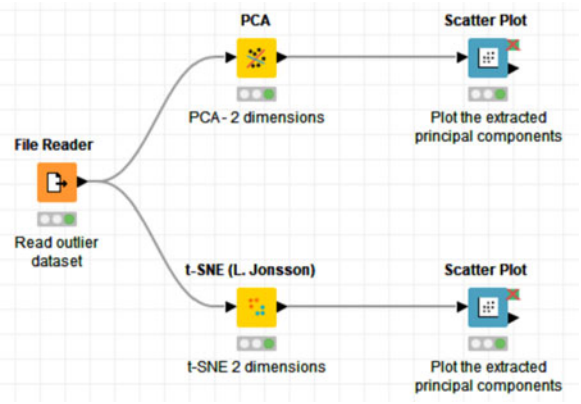


Fig. 4.36 Using PCA and t-SNE to reduce the dimensionality of the original data set to just two coordinates to use as x- and y-axis in a scatter plot

plot view (Fig. 4.35), you can select data points in the scatter plot, for example, by drawing a rectangle around them or clicking on a single point. Automatically these same data points are selected in other connected views, like, for example, the parallel coordinate view. Some views also allow seeing only the selected data points, by enabling the menu item *Show selected rows only* still from the buttons in the top right corner of the object view.

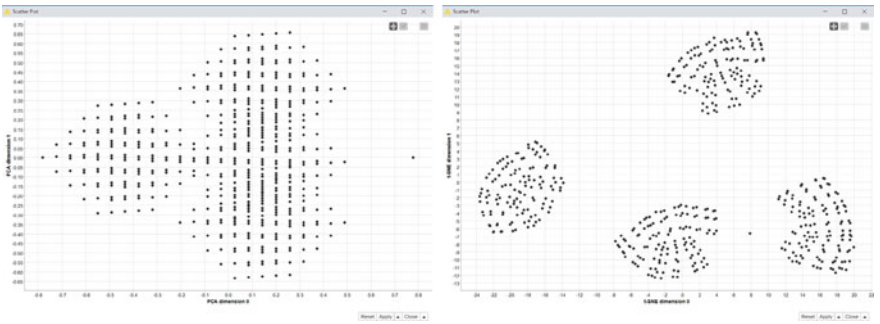


Fig. 4.37 Scatter plot of the data set in 3doutlier.data of the two top PCs after PCA (left) and of the two coordinates from t-SNE (right)

4.8.2 Visualizing a Three-Dimensional Data Set on a Two-Coordinate Plot

Principal Component Analysis, **t-SNE**, and **Multidimensional Scaling** methods are also available and can be used to calculate a new smaller set of coordinates for, e.g., scatter plot views. In this example, we use PCA and t-SNE to reduce the data set 3doutlier.data from its original three dimensions to the most important two to visualize in a scatter plot (Fig. 4.36 and Fig. 4.37).

References

1. Aggarwal, C., Yu, P.: Outlier detection for high dimensional data. In: Proc. ACM SIGMOD Int. Conf. on Management of Data (SIGMOD 2001, Santa Barbara, CA), pp. 37–46. ACM Press, New York (2001)
2. Anderson, E.: The irises of the Gaspe Penisula. Bull. Am. Iris Soc. **59**, 2–5 (1935)
3. Borg, I., Groenen, P.: Modern Multidimensional Scaling: Theory and Applications. Springer, Berlin (1997)
4. Breunig, M., Kriegel, H.-P., Ng, R., Sander, J.: LOF: identifying density-based local outliers. In: Proc. ACM SIGMOD Int. Conf. on Management of Data (SIGMOD 2000, Dallas, TX), pp. 93–104. ACM Press, New York (2000)
5. Cook, D., Buja, A., Cabrera, J.: Projection pursuit indices based on orthonormal function expansion. J. Comput. Graph. Stat. **2**, 225–250 (1993)
6. Diaconis, P., Freedman, D.: Asymptotics of graphical projection pursuit. Ann. Stat. **17**, 793–815 (1989)
7. Fisher, R.A.: The use of multiple measurements in taxonomic problems. Ann. Eugen. **7**(2), 179–188 (1936)
8. Freedman, D., Diaconis, P.: On the histogram as a density estimator: L_2 theory. Z. Wahrscheinlichkeitstheor. Verw. Geb. **57**, 453–476 (1981)
9. Friedman, J.: Exploratory projection pursuit. J. Am. Stat. Assoc. **82**, 249–266 (1987)
10. Friedman, J., Tukey, J.: A projection pursuit algorithm for exploratory data analysis. IEEE Trans. Comput. **C-23**, 881–890 (1974)
11. Hall, P.: On polynomial-based projection indices for exploratory projection pursuit. Ann. Stat. **17**, 589–605 (1989)
12. Huber, P.: Projection pursuit. Ann. Stat. **13**, 435–475 (1985)

13. Knorr, E., Ng, R.: Algorithms for mining distance-based outliers in large datasets. In: Proc. 24th Int. Conf. on Very Large Data Bases (VLDB 1998, New York, NY), pp. 392–403. Morgan Kaufmann, San Mateo (1998)
14. Knorr, E., Ng, R., Tucakov, V.: Distance-based outliers: algorithms and applications. VLDB J. **8**, 237–253 (2000)
15. Liu, F.T., Ting, K.M., Zhou, Z.-H.: Isolation Forest. In: Proc. 8th IEEE Int. Conf. Data Mining (Pisa, Italy) (2008)
16. Morrison, A., Ross, G., Chalmers, M.: Fast multidimensional scaling through sampling. Inf. Vis. **2**, 68–77 (2003)
17. Pekalska, E., Ridder, D., Duin, R., Kraaijveld, M.: A new method of generalizing Sammon mapping with application to algorithm speed-up. In: Proc. 5th Annual Conf. Advanced School for Computing and Imaging (Delft, Netherlands), pp. 221–228 (1999)
18. Rehm, F., Klawonn, F., Kruse, R.: MDS_{polar}: a new approach for dimension reduction to visualize high dimensional data. In: Advances in Intelligent Data Analysis, vol. VI, pp. 316–327. Springer, Berlin (2005)
19. Rehm, F., Klawonn, F., Kruse, R.: A novel approach to noise clustering for outlier detection. Soft Comput. **11**, 489–494 (2007)
20. Santos-Pereira, C., Pires, A.: Detection of outliers in multivariate data: a method based on clustering and robust estimators. In: Proc. 5th Annual Conference of the Advanced School for Computing and Imaging, pp. 291–296. Physica, Berlin (2002)
21. Scott, S.: On optimal and data-based histograms. Biometrika **66**, 605–610 (1979)
22. Scott, D.: Sturges' rule. In: Wiley Interdisciplinary Reviews: Computational Statistics, vol. 1, pp. 303–306. Wiley, Chichester (2009)
23. Sturges, H.: The choice of a class interval. J. Am. Stat. Assoc. **21**, 65–66 (1926)
24. Tschumitschew, K., Klawonn, F.: Veda: statistical tests for finding interesting visualisations. In: Knowledge-Based and Intelligent Information and Engineering Systems 2009, Part II, pp. 236–243. Springer, Berlin (2009)
25. Tukey, J.W.: Exploratory Data Analysis. Addison-Wesley, Reading (1977)
26. van der Maaten, L., Hinton, G.: Visualizing data using t-SNE. J. Mach. Learn. Res. **9**, 2579–2605 (2008)

Principles of Modeling

5

After we have gone through the phases of project and data understanding, we are either confident that modeling will be successful or return to the project understanding phase to revise objectives (or to stop the project). In the former case, we have to prepare the data set for subsequent modeling. However, as some of the data preparation steps are motivated by modeling itself, we first discuss the principles of modeling. Many modeling methods will be introduced in the following chapters, but this chapter is devoted to problems and aspects that are inherent in and common to all the methods for analyzing the data.

All we need for modeling, it might seem, is a collection of methods from which we have to choose the most suitable one for our purpose. By now, the project understanding has already ruled out a number of methods. For example, when we have to solve a regression problem, we do not consider clustering methods. But even within the class of regression problems, there are various methods designed for this task. Which one would be the best for our problem, and how do we find out about this? In order to solve this task, we need a better understanding of the underlying principles of our specific data analysis methods. Most of the data analysis methods can be viewed within the following four-step procedure:

- **Select the Model Class**
  First of all, we must specify the general structure of the analysis result. We call this the "architecture" or "model class." In a regression problem, one could decide to consider only linear functions; or instead, quadratic functions could be an alternative; or we could even admit polynomials of arbitrary degree. This, however, defines only the structure of the "model." Even for a simple linear function, we still would have to determine the coefficients (Sect. 5.1).
- **Select the Score Function**
  We need a score function that evaluates the possible "models," and we aim to find the best model with respect to our goals—which is formalized by the score function. In the case of the simple linear regression function, our score function will tell us which specific choice of the coefficients is better when we compare different linear functions (Sect. 5.2).

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_5

- **Apply the Algorithm**
  The score function enables us to compare models, but it does not tell us how to *find* the model that obtains the best evaluation from the score function. Therefore, an algorithm is required that solves this optimization problem (Sect. 5.3).
- **Validate the Results**
  Even if our optimization algorithm is so cleverly designed that it will find the best model in the model class we consider, we still do not know, whether this is the best model among very good models for our data or just the best one among even worse choices. Therefore, we still need other means than the score function to validate our model (Sect. 5.5).

Unfortunately, any of these steps can cause serious problems and can completely spoil the result of the data analysis process. If we are not satisfied with the result of the process, the reason might be an improper choice in one of these four steps. But it might also be the case that our data are too noisy or do not contain enough useful information to yield a satisfactory analysis result. Therefore, in order to better understand what can lead to unsatisfactory data analysis results, we need to take a closer look at what types of errors we have to deal with in the four-step procedure and the data (see Sect. 5.4).

We will not distinguish between the terms "model class" and "model" too pedantically, because usually the model class is characterized by writing down a model in which, say, the coefficients are replaced by formal parameters. As soon as we have decided which values these parameters will get, we obtain a concrete model. That is, by looking at a parameterized model we usually know the model class, and therefore both terms are used interchangeably.

5.1 Model Classes

In order to extract information from data, it is necessary to specify which general form the analysis result should have. We call the form or structure of the analysis result the **model class** or **architecture** to be derived from the data. This very general concept of a model should not be understood in a narrow sense that it must reflect a structure inherent in the data or represent the process of data generation.

(a) Examples for models in a more strict sense are linear models. The simplest case of a linear model is a regression line

$$y = ax + b, \tag{5.1}$$

describing the idealized relationship between attributes x and y. The parameters a and b still need to be determined. They should be chosen in such a way that the regression line fits best to the given data. A perfect fit, i.e., the data points lie exactly on the regression line, cannot be expected. In order to fit the regression line to the data, a criterion or error measure is required, defining how well a line with given parameters a and b fits the data. Such criteria or error measures are

discussed in more detail in Sect. 5.2. The error measure itself does not tell us how to find the regression line with the best fit. Depending on the error measure, an algorithm is needed that finds the best fit or at least a good fit.

(b) Another example of an even simpler model is a constant value as a representative or prototype of a numerical attribute (or an attribute vector) either for the full data set or for various subsets of it. The mean value or median are typical choices for such prototypes. Although the mean value and median can be viewed as purely descriptive concepts, they can also be derived as optimal fits in terms of suitable error measures, as we will see in Sect. 5.3.

(c) Multidimensional scaling as it was introduced in Sect. 4.3.2.3 can also be interpreted as a model. Here, the model is a set of points in the plane or 3D-space, representing the given data set. Again, a criterion or error measure is needed to evaluate whether a given set of points is a good or bad representation of the data set under consideration.

(d) All examples so far were numerical in nature, but there are, of course, also models for nominal data. A simple model class is that of propositional rules like *"If temperature = cold and precipitation = rain, then action = read book"* or *"If a customer buys product A, then he also buys product B with a probability of 30%"* can also be viewed as models. The latter rules are called association rules, because they *associate* features of arbitrary variables.

The four problems or models described in (a) the regression line, (b) the constant value, (c) multidimensional scaling, and (d) rules will serve as simple examples throughout this chapter. In most cases the models are parameterized as in the above first three examples. The number of parameters depends on the model. The constant value and the regression line have only one and two parameters, respectively, whereas $2n$ parameters are needed for multidimensional scaling for the representation of a data set with n records in the plane. The freedom of choice in the rule models are the variables and values that build the conditions in the rule's antecedent.

When we are looking for a regression function $f : \mathbb{R} \to \mathbb{R}$, we might decide that we want to restrict f to lines of the form $f(x) = a_1 x + a_0$, so that we have a fixed set of two parameters. But we might also not know, whether a line, a quadratic function, or even a polynomial of higher degree $f(x) = a_k x^k + \cdots + a_1 x + a_0$ might be the best choice. When we do not fix the degree of the polynomial for the regression function, we have not a fixed but a variable set of parameters. A propositional rule (example (d)) or sets of rules also belong to this case, as the number of conditions and rules is typically not fixed in advance. The same holds for decision trees (see Chap. 8), which can also not be represented in terms of a simple fixed set of parameters.

Another distinction can be made between, say, linear regression models and a propositional rule, by their applicability: a linear regression model can (in principle) be applied to all possible values of x and yields a resulting y. As they can be applied to all data from the data space, they are often called **global models**. This is different with rules: the consequent of the rule applies only in those cases where all conditions of the antecedent hold—there is no information returned otherwise. As such models can be applied to a somewhat limited fraction of the whole dataspace, they are called **local models**. We will use the term local model and **pattern** synonymously.

In any case, we need to define the class of possible models that we consider as possible candidates for solving our data analysis task. Project understanding aims at identifying the goals of the data analysis process and will therefore already restrict the choice of possible model classes. But even a well-defined goal will in most cases not imply a unique class of possible models. For instance, in the case of a regression function to be learned from the data, it is often not obvious what type of regression function should be chosen. Unless there is an underlying model—for instance, for a physical process—that provides the functional dependencies between the attributes, a polynomial of arbitrary degree or even other functions might be good candidates for a model.

Finding the "best" model for a given data set is not a trivial task at all, especially since the question what a good or the best model means is not easy to answer. How well the model fits the data is one criterion for a good model which can often be expressed in a more or less obvious manner, for instance, in the case of regression as the mean square or the mean absolute error of the values predicted by the model and the values provided in the data set. Such fitting criteria will be discussed in more detail in the next section.

Another important aspect is the simplicity of the model. Simpler models will be preferred for various reasons:

- They are usually easier to understand and to be interpreted.
- Their computational complexity is normally lower.
- A model can also be seen as a summary of the data. A complex model might not be a summary but, in the worst case, just more or less a one-to-one representation of the data. General structures in the data can only be discovered by a model, when it summarizes the data to a certain extent. The problem that too complex models often fail to reveal the general relations and structures in the data is called overfitting and will be discussed in more detail in Sect. 5.4.

Interpretability is often another requirement for the model derived from the data. In some cases, when only a classifier or regression function with a low error is the target, **black-box models** like neural networks might be accepted. But when the model should deliver an understandable description of the data or when the way how a classifier selects the predicted class needs an explanation, black-box models are not a proper choice. It is impossible to define a general measure for interpretability, because interpretability depends very much on the context. For instance, a quadratic regression function might be perfectly interpretable in a mechanical process where acceleration, velocity, and distance are involved, whereas in other cases a quadratic function would be nothing else than a black-box model, although a very simple one.

Computational aspect also plays a role in the choice of the model class. Finding the model from a model class that satisfies the desired criteria—data fitting and low model complexity and interpretability—best or at least reasonably well is also a question of computational effort. In some cases, there might be simple ways to derive a model based on the given data. For example, for the regression line example (a), it might be the goal to minimize sum of squared errors. In this case, an

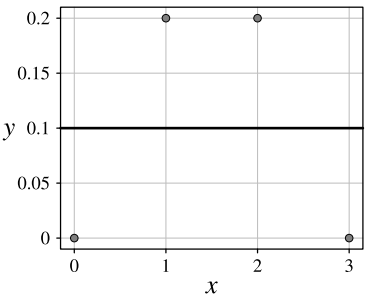


Fig. 5.1 A small data set to which a line is to be fitted

explicit analytical solution for the best choice of the parameters a and b can be provided as we will see in the next section. Although the search space in the association rule example (d) is finite in contrast to the search space $\mathbb{R}^2$ for the regression line, it is so large that highly efficient strategies are needed to explore all potentially interesting rules.

Although all these aspects, the fitting criterion or score function, the model complexity, interpretability, and the required computational complexity for finding the model, are important, the focus is very often put on the fitting criterion that can be usually defined in a more or less obvious way.

5.2 Fitting Criteria and Score Functions

The choice of a model class determines only the general structure of the model, very often in terms of a set of parameters. In order to find the best or at least a good model for the given data, a fitting criterion is needed, usually in the form of an **objective function**

$$f : \mathcal{M} \to \mathbb{R}, \tag{5.2}$$

where $\mathcal{M}$ is the set of considered models.

In the case of example (a), a regression line of the form (5.1), a model is characterized by two real parameters, so that the objective function simplifies to

$$f : \mathbb{R}^2 \to \mathbb{R}. \tag{5.3}$$

Figure 5.1 shows a small data set to which a line $y = ax + b$ should be fitted. The most common error measure is the mean squared error,[1] i.e.,

$$E(a, b) = \frac{1}{n} \sum_{i=1}^{n} (ax_i + b - y_i)^2. \tag{5.4}$$

where (x_i, y_i) $(i \in \{1, \ldots, n\})$ are the points to which the line should be fitted.

[1] Note that minimizing the mean squared error or the sum of squared errors leads to the same solution, since the parameters a and b do not depend on the constant factor $\frac{1}{n}$.

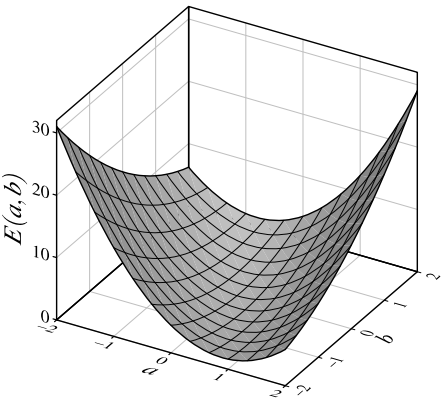


Fig. 5.2 The mean squared error for the data set in Fig. 5.1

The goal is to choose the parameters a and b in such a way that (5.4) is minimized. The error function (5.4) does not tell us directly how to find the best choice for the parameters a and b. It only provides a criterion telling us whether parameter combination (a_1, b_1) is considered better than parameter combination (a_2, b_2). This is the case where $E(a_1, b_1) < E(a_2, b_2)$. When we plot the error function (5.4), which is a function of the parameters a and b, we obtain the graph shown in Fig. 5.2.

The mean squared error is not the only reasonable choice to measure how well a model fits the data. Other obvious examples for alternative measures include the mean absolute error

$$E(a, b) = \frac{1}{n} \sum_{i=1}^{n} |ax_i + b - y_i| \tag{5.5}$$

or the mean Euclidean distance of the data points to the regression line. Instead of the mean or the sum of the errors, the maximum of the errors could also be considered.

All these error measures have in common that they only yield the value zero when the regression line fits perfectly to the data and that they increase with larger distance of the data points to the regression line. Properties of such error measures and their advantages and disadvantages are discussed in more detail in Sect. 8.3.3.3.

Error functions for the even simpler model (b) (on page 87) of a single value m representing a sample can be defined in a similar way. It can be shown that the minimization of mean squared errors

$$E(m) = \frac{1}{n} \sum_{i=1}^{n} (x_i - m)^2 \tag{5.6}$$

leads to the mean value

$$m = \bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i, \tag{5.7}$$

Table 5.1 A cost matrix for the tea cup classifier

True class	Predicted class	
	OK	broken
OK	0	c_1
broken	c_2	0

whereas the median minimizes the mean absolute error

$$E(m) = \frac{1}{n} \sum_{i=1}^{n} |x_i - m|. \tag{5.8}$$

For model (c), multidimensional scaling, three different error functions were already introduced in Sect. 4.3.2.3.

5.2.1 Error Functions for Classification Problems

A propositional rule may be regarded as a very simple classifier: if all conditions in the antecedent hold, it predicts the value of some target variable. If the rule does not apply, there is no prediction (which is usually wrong, of course). For classifiers, the most common error function is the **misclassification rate**, i.e., the proportion of records that are wrongly classified. However, the misclassification rate can be considered as a special form of a **cost function** or **cost matrix**. Such a cost matrix takes into account that misclassifications for one class can have stronger consequences or higher costs than another class. Consider the example of a tea cup producer. An automatic quality control device checks the produced cups before they are delivered to the shops. The quality control device classifies the tea cups into the two classes *broken* and *ok*. The device might not be absolutely correct, so that some broken cups are classified as *ok* and some intact cups are wrongly recognized as broken. Table 5.1 shows the four possible cases that can occur when a tea cup is classified. The diagonal of the table represents the situations where the tea cup was classified correctly. In these cases, the classifier does not cause any extra costs because of an erroneous classification.

When the device classifies an intact cup as broken, the cup will be wasted, and it must be produced again. Therefore, the costs c_1 caused by this error are equal to the production costs for one cup. The costs c_2 for a broken cup that was considered as intact are more difficult to estimate. The cup must be produced again, after the buyer's complaint. But these costs for reproduction are not caused by the erroneous classification, so that they are not included in c_2. The costs c_2 must cover the mailing costs for the reproduced cup and also the costs for the loss of reputation of the company for delivering broken cups. The costs for misclassifications differ significantly in certain cases. If instead of cups, safety critical parts like brake pads are produced, the costs for classifying a broken item as intact might be enormous, since they might lead to serious accidents.

Table 5.2 A general cost matrix

True class	Predicted class			
	c_1	c_2	$\ldots$	c_m
c_1	0	$c_{1,2}$	$\ldots$	$c_{1,m}$
c_2	$c_{2,1}$	0	$\ldots$	$c_{2,m}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_m	$c_{m,1}$	$c_{m,2}$	$\ldots$	0

When a cost matrix for a classification problem with m classes is provided as in Table 5.2, instead of the misclassification rate, the **expected loss** given by

$$\text{loss}(c_i|E) = \sum_{j=1}^{m} P(c_j|E)c_{ji} \tag{5.9}$$

should be minimized. Here E is the evidence, i.e., the observed values of the predictor attributes used for the classification, and $P(c_j|E)$ is the predicted probability that the true class is c_j given observation E.

Sometimes, classification can be considered as a special form of regression. A classification problem, where each instance belongs to one of two classes, can be reformulated as a regression problem by assigning the values 0 and 1 to the two classes. This means that the regression function must be learned from data where the values y_i are either 0 or 1. Since the regression function will only approximate the data, it will usually not yield the exact values 0 and 1. In order to interpret such a regression function as a classifier, it is necessary to assign arbitrary values to the classes 0 and 1. The obvious way to do this is to choose 0.5 as a threshold and consider values lower than 0.5 as class 0 and values greater than 0.5 as class 1.

When a classifier is learned as a regression function based on error measures as they are used for regression problems, this can lead to undesired results. The aim of regression is to minimize the approximation error which is not the same as the misclassification rate. In order to explain this effect, consider a classification problem with 1000 instances, half of them belonging to class 0 and the other half to class 1. Assume that there are two regression functions f and g as possible candidates to be used as a classifier.

- Regression function f yields 0.1 for all data from class 0 and 0.9 for all data from class 1.
- Regression function g always yields the exact and correct values 0 and 1, except for 9 data objects where it yields 1 instead of 0, and vice versa.

Although f does not yield the exact values 0 and 1 for the classes, it classifies all instances correctly when the threshold value 0.5 is used to make the classification decision. As a regression function, the mean squared error of f is 0.01. The regression function g has a smaller mean squared error of 0.009, but classifies 9 instances incorrectly. From the viewpoint of regression, g is better than f, and from the viewpoint of the misclassification rate, f should be preferred.

5.2.2 Measures of Interestingness

When we search for patterns, for instance, for single classification rules or association rules in the form *if* $A = a$ *then* $B = b$, statistical measures of interest are typically used to evaluate the rule which is then considered as the "model." Assume that $A = a$ for n_a records, $B = b$ for n_b records, and $A = a$ and $B = b$ at the same time holds for n_{ab} records, where we have n records altogether. If $\frac{n_b}{n} \approx \frac{n_{ab}}{n_a}$, the rule has no significance at all, since replacing the records with $A = a$ by a random sample of size n_A, we would expect roughly the same fraction of records with $B = b$ in the random sample. The rule can only be considered as relevant if $\frac{n_{ab}}{n_a} \gg \frac{n_b}{n}$. In order to measure the relevance of the rule, we can compute the probability that a random sample of size n_a contains at least n_{ab} records with $B = b$. This probability can be derived from a hypergeometric distribution:

$$\sum_{i=n_{ab}}^{\min\{n_a,n_b\}} \frac{\binom{n_b}{i} \cdot \binom{n-n_b}{n_a-i}}{\binom{n}{n_a}}. \tag{5.10}$$

This can be interpreted as the p-value for the statistical test[2] with the null hypothesis that the rule applies just by chance. The lower this p-value, the more relevant the rule can be considered.

A very simple measure of interestingness that is often used in the context of association rules and frequent item set mining is the support or frequency of the rule in the data set. There, a rule is considered "interesting" if the support exceeds a given lower bound. As we have seen above, the support alone is not a good advisor when looking for unexpected, interesting rules. The focus on support in the context of association rules/frequent patterns has merely technical/algorithmic reasons.

These are two examples for statistical measures of interest. There are many others depending on the type of pattern we are searching [7]. We will encounter some of them in the subsequent chapters.

5.3 Algorithms for Model Fitting

The objective functions, mostly error functions, as they were explained in the previous section do not tell us directly how to find the best or at least a good model with respect to the fitting criterion. For this purpose, methods from optimization are needed.

5.3.1 Closed-Form Solutions

In the best case, a closed-form solution for the optimization problem can be obtained directly. This is, however, not possible for most of the objective functions we consider. A positive example, for which we can find a closed-form solution, is our

[2] The test here is Fisher's exact test.

case (a) (see page 86), the linear regression function (see also Sect. 8.3 for more details). For a minimum of the error function of linear regression, it is necessary that the partial derivatives with respect to the parameters a and b of the error function (5.4) vanish. This leads to the system of two linear equations

$$\frac{\partial E}{\partial a} = \frac{2}{n}\sum_{i=1}^{n}(ax_i + b - y_i)x_i = 0,$$
$$\frac{\partial E}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(ax_i + b - y_i) = 0.$$

The solution of this system of equations is

$$a = \frac{n\sum_{i=1}^{n} x_i y_i - (\sum_{i=1}^{n} x_i)(\sum_{i=1}^{n} y_i)}{n\sum_{i=1}^{n} x_i^2 - (\sum_{i=1}^{n} x_i)^2},$$
$$b = \bar{y} - a\bar{x}.$$

For the least squares error, such an analytical solution can be derived as long as the regression function is a linear function in the model parameters (for details, see Sect. 8.3).

Sometimes, constraints are defined on the parameters to be optimized, for instance, that the parameters must be positive values. For continuous optimization problems with such constraints, so-called Lagrange functions or Kuhn–Tucker conditions can be used to incorporate the constraints in the optimization.

However, in many cases the simple technique of computing partial derivatives of the objective function—with or without constraints—and solving for the parameters is not applicable, since the resulting system of equations is nonlinear or the derivatives might not exist, for instance, when the absolute error instead of the squared error is considered.

5.3.2 Gradient Method

If the model to be learned from the data is described by k real-valued parameters, i.e., the objective function is of the form $f : \mathbb{R}^k \to \mathbb{R}$, the problem of optimizing the objective function might be as simple as in the case of least squares linear regression as illustrated in Fig. 5.2, but for other models and other objective functions, it could look more like in Fig. 5.3 with a number of local minima and maxima. Even worse, the landscape in Fig. 5.3 is three-dimensional because the model is assumed to be described by two parameters only. But when the model is described by k parameters, where k is greater than two, the objective function corresponds to a landscape in $(k+1)$ dimensions, so that is not even possible to plot it adequately. For instance, in the case of multidimensional scaling, the landscape defined by the objective function represents a $2n$-dimensional surface in a $(2n+1)$-dimensional space, where n is

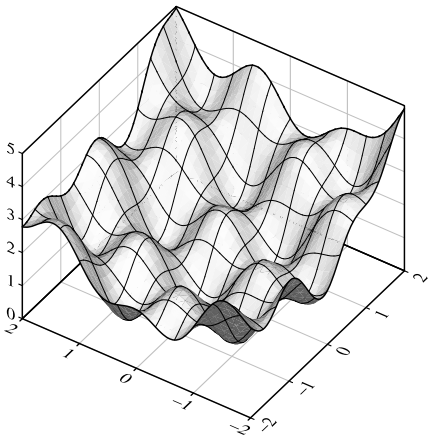


Fig. 5.3 A landscape with many local minima and maxima

the number of instances in the data set displayed in the plane by multidimensional scaling.

When the objective function is differentiable, a **gradient method** can be applied. The gradient, i.e., the vector of partial derivatives with respect to the model parameters, points in the direction of steepest ascend. The idea of optimization based on a gradient method is to start at a random point—an arbitrary choice of the parameters to be optimized—and then to go a certain step in the direction of the gradient, when the objective function should be maximized, and in the opposite direction of the gradient, when the objective function should be minimized, leading to a new point in the parameter space. If this point yields a better value for the objective function, the gradient in this point is computed, and the next point in the direction or, respectively, in the opposite direction of the new gradient is chosen. This procedure is continued until no more improvements can be achieved or a fixed number of gradient steps has been carried out.

The stepwidth can be chosen constant. However, the problem with constant stepwidth is that, with a large stepwidth, one might "jump" over or oscillate around a local optimum. On the other hand, a very small stepwidth can lead to extremely slow convergence or even to starving, which means that the algorithm converges before a local optimum is reached. Therefore, an adaptive stepwidth is usually preferred, however, for the price of higher computational costs.

Applying a gradient method to minimize an objective function, it can only find the local minimum in the same "valley" of the landscape where the starting point is located.[3] Therefore, it is recommended to run a gradient method repeatedly, starting with different initial points in order to increase the chance to find the global or at least a good local optimum.

[3]For maximization, the same holds, except that the gradient method will just climb the "mountain" or even small "hill" on which the starting point is located.

5.3.3 Combinatorial Optimization

When the model is determined by k real-valued parameters, the domain $\mathcal{M}$ of the objective function (5.2) is $\mathbb{R}^k$ or a subset of it. It is also very common in data analysis to consider discrete models, usually leading to finite domains of the objective function. Rule mining is an example where the domain of the objective function is finite, at least when we assume that we only consider categorical attributes. In such cases, taking derivatives is impossible, so that neither a least squares technique as in the case of linear regression nor a gradient method is applicable. In principle, an **exhaustive search** of the finite domain $\mathcal{M}$ is possible; however, in most cases it is not feasible, since $\mathcal{M}$ is much too large. As an example, consider the problem of finding the best possible association rules with an underlying set of 1000 items (products). Every combination of items, i.e., every nonempty subset is a possible candidate set from which several rules may be constructed. The number of subsets alone contains $2^{1000} - 1 > 10^{300}$ elements, making it impossible to apply an exhaustive search.

Fortunately, in certain cases there are algorithms that exploit the specific structure of the finite search space and avoid a complete search by suitable branch-and-bound techniques. Examples for such techniques will be provided in Sect. 7.6.

5.3.4 Random Search, Greedy Strategies, and Other Heuristics

No matter whether the domain of the objective function is a subspace $\mathbb{R}^k$, a finite set, or even a space composed of real and discrete model parameters, in many cases it is impossible to find the optimum by an analytical method. Therefore, it is very common to apply heuristic search strategies that cannot guarantee finding the global optimum but will—hopefully—find at least some good solutions. In most general terms, we want to find the optimum (minimum or maximum) of an objective function as in (5.2) where $\mathcal{M}$ is an arbitrary search space. We use the very simple function below as an example to demonstrate some optimization strategies:

$$f(x) = \sin\left(\frac{x}{2}\right) + 0.5\sin(2x) + 0.25\cos(4.5x).$$

The goal of the optimization procedure in this simplified use case is to find the one parameter x that maximizes the value of the function.

This simplified setup allows us to visualize the experimental values of the parameter x and the corresponding function values $f(x)$ on a simple x–y plot. On the x axis are the parameter values and on the y axis the function outputs. The (x, y) points are then colored according to a white–red gradient describing the point position in the generation of the parameter sequence. Whiter points correspond to hyperparameter values generated earlier in the process; redder points correspond to parameter values generated later on in the process.

Grid search is a basic brute-force strategy. If one does not know which values to try, one can try them all. In case of continuous values, one cannot try all values but at least all possible values within a range with a fixed step width for the function evaluation. For example, if the range is $[0, 10]$ and the step size is 0.1, then we

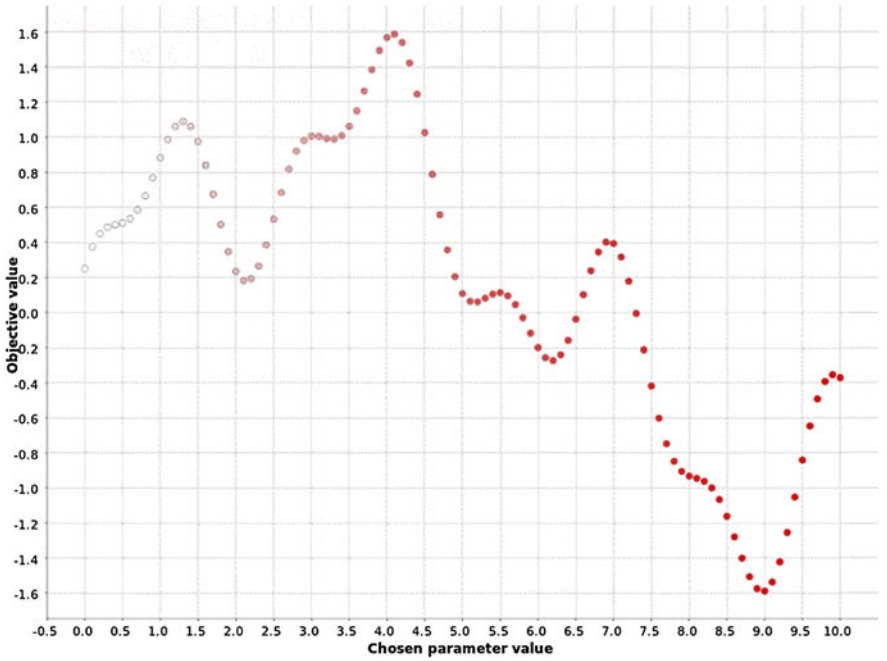


Fig. 5.4 Grid search of the parameter values on a [0, 10] range with step 0.1. The color gradient reflects the position in the generated sequence of parameter candidates. Whiter points correspond to parameter values generated earlier on in the process; redder points correspond to parameter values generated later on. The range of the parameter is scanned from small to large values

would get the sequence of parameter values (0, 0.1, 0.2, 0.3, . . . , 9.5, 9.6, 9.7, 9.8, 9.9, 10). In a grid search strategy, we calculate the function output for each and every one of these parameter values (Fig. 5.4). Therefore, the finer the grid, the closer we get to the optimum—but also the higher the required computation resources. The grid search strategy can work well in the case of a single parameter, but it becomes very inefficient or even infeasible when multiple parameters have to be optimized simultaneously.

For the **random search** strategy, the values of the parameters are selected randomly, as the name suggests. This strategy is typically preferred in the case of multiple parameters, and it is particularly efficient when some parameters affect the final metric more than others. Again, the parameter values are generated within a range [0, 10]. Then, a fixed number N of parameters is randomly generated (Fig. 5.5). The fixed number N of predefined parameters to experiment with allows to control the duration and speed of this optimization strategy. The larger N, the higher the probability to get to the optimum—but also the higher the required computational resources.

As long as we do not impose any assumptions on the search space and the objective function, random search or systematic search are the only ways to explore the search space. Usually, there is a neighborhood structure on the search space, and the

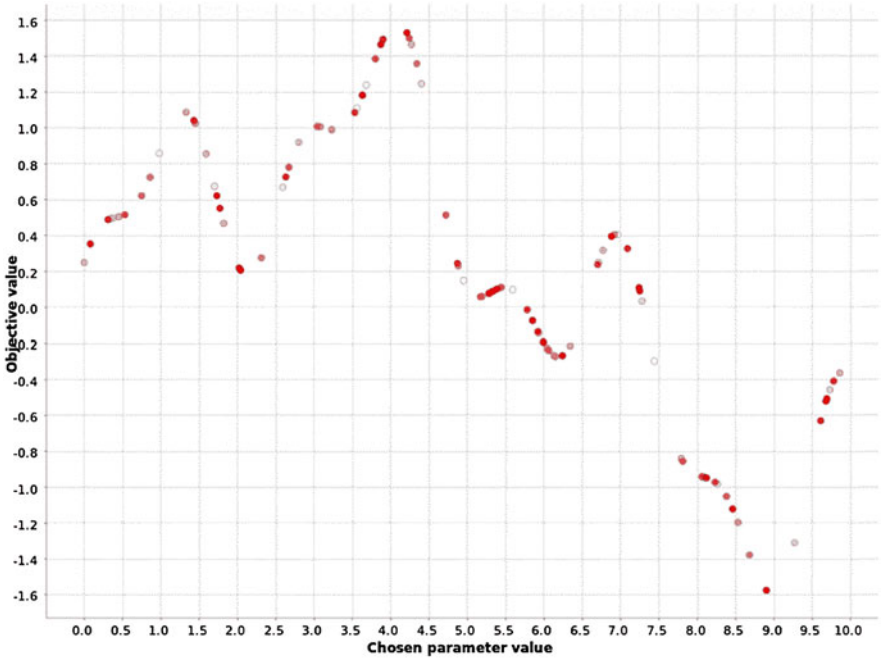


Fig. 5.5 Random search of the parameter values on a [0, 10] range. The color gradient reflects the position in the generated sequence of parameter candidates. Whiter points correspond to parameter values generated earlier on in the process; redder points correspond to parameter values generated later on. As expected, the parameter values from the generated sequence are used in no decreasing or increasing order: white and red dots mix randomly in the plot

objective function will often yield similar values for neighboring elements (models) in the search space. If the search space consists of real-valued parameters, then a very small change of the parameters will usually not lead to an extreme difference in the values of the objective function.

With such an assumption, it is normally possible to formulate a **greedy strategy** in order to find a good solution faster than by pure random search.

The **hill climbing** approach is a greedy optimization strategy. Instead of generating arbitrary random solutions, hill climbing starts from a random initial solution and then explores only the neighborhood of the so far best solution for better solutions. This means that hill climbing restricts the random search to the neighborhood of the best solution that was found so far. If no neighbor improves the objective function, the optimization loop stops (Fig. 5.6). Note that this procedure is different from the grid and random searches in one important aspect: selection of the next parameter value takes into account the outcomes of previous iterations.

This example illustrates a caveat related to this strategy: it can get stuck in a local optimum. From the other plots, we can see that the global optimum is located at $x = 4.0$ with corresponding objective function value of 1.6. This strategy does not find the global optimum but gets stuck in a local one. A good rule of thumb for

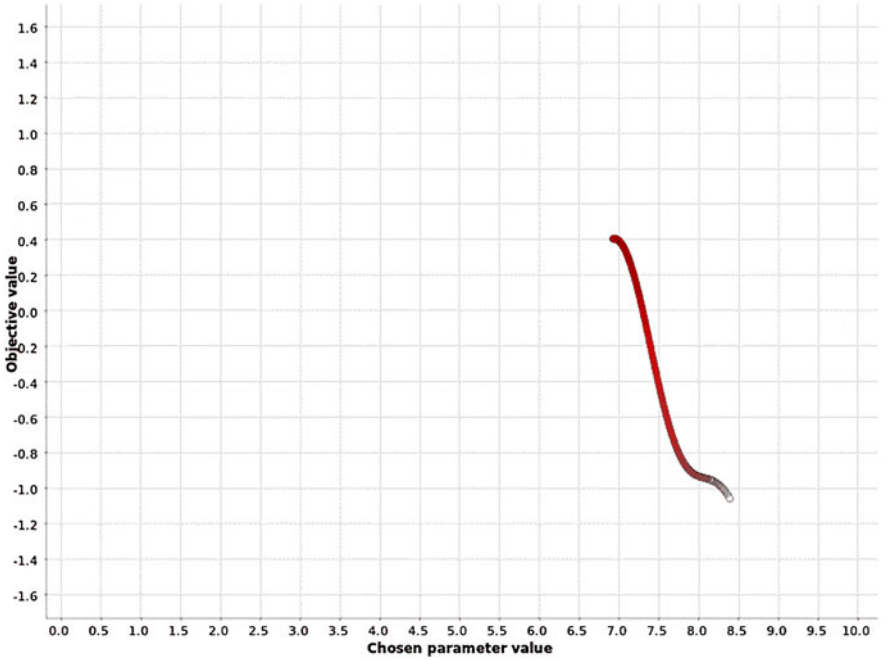


Fig. 5.6 Hill climbing search of the parameter values on a [0, 10] range. The color gradient reflects the position in the generated sequence of parameter candidates. Whiter points correspond to parameter values generated earlier on in the process; redder points correspond to parameter values generated later on. Hill climbing applied to our function started at the random parameter value $x = 8.4$ and then moved toward the function maximum $y = 0.4$ at $x = 6.9$. Once the maximum was reached, no further increase in the function was observed in the next neighbor, and the search procedure stopped

this method is to run it multiple times with different starting values and to check whether the algorithm converges to the same optimum. Gradient methods are also greedy strategies. In contrast to hill climbing, they do not explore the neighborhood of the so far best solution randomly, but in a more systematic manner, namely in the direction of the gradient. Other greedy strategies do not try to greedily improve a given solution but construct a solution step by step in a greedy manner. Decision trees are usually built in this way (see Sect. 8.1). In each step, the attribute is chosen as a node that seems to lead to the best split for the data.

Greedy strategies have the advantage that they have the ability to quickly improve a given solution. But they will only find a local optimum of the objective function. Good optimization heuristics need a greedy component in order to be efficient and to find good solutions quickly. But the greedy component alone is too limited since it will get lost in local optima. Therefore, a random component is also required to provide a chance to escape from local optima. A simple mixture of random search and a greedy strategy is to apply the greedy strategy repeatedly starting from different initial random solutions.

The **Bayesian optimization** strategy selects the next parameter value based on the function outputs in the previous iterations, similar to the hill climbing strategy. Unlike hill climbing, Bayesian optimization looks at past iterations globally and not only at the last one. There are typically two phases in this procedure:

During the first phase, called warm-up, parameter values are generated randomly. After a user-defined number N of such random generations of parameters, the second phase kicks in.

In the second phase, at each iteration, a "surrogate" model of type $P(\text{output} \mid \text{past parameters})$ is estimated to describe the conditional probability of the output values on the parameter values from past iterations. This surrogate model is much easier to optimize than the original function. Thus, the algorithm optimizes the surrogate and suggests the parameter values at the optimum of the surrogate model as the optimal values for the original function as well. A fraction of the iterations in the second phase is also used to probe areas outside of the optimal region. This is to avoid the problem of local optima. You can also see that intense red points are clustered closer to the optimum, while pale red and white points are scattered. This demonstrates that the definition of the optimal region is improved with each iteration of the second phase (Fig. 5.7).

Better optimization heuristics will mix the greedy and the random component in a more intelligent way by focusing on those parts of the search space which seem to have a higher chance to contain good solutions. **Genetic algorithms** and **evolution strategies** or, more generally, **evolutionary algorithms** are examples for such methods that combine random with greedy components, using a population of solutions in order to explore the search space in parallel and efficiently.

There are many other problem-dependent heuristics that are applied to model fitting. For instance, when the set of parameters can be split into disjoint subsets in such a way that, for each subset an analytical solution for the optimum can be provided, given that the parameters in the other subsets are considered to be fixed, then **alternating optimization** is a useful technique. Alternating optimization computes the analytical solution for the parameter subsets alternatingly and iterates this scheme until convergence.

No matter which kind of heuristic optimization strategy is applied to fit a model to data, one should be aware that, except when an analytical solution is known or exhaustive search is feasible, the result may only represent a local optimum that may have a much lower quality than the global optimum.

5.4 Types of Errors

When fitting a model to data based on an error measure, a perfect fit will be seldom possible. A perfect fit with zero error is suspicious in most cases. It is often an indication that the model fitted to the data may be too flexible and is able to represent the data exactly, but does not show the actual structures or dependencies in the data. Consider a simple regression problem as it is illustrated in Fig. 5.8. The data are approximated by a regression line which fits quite well the data with a small error.

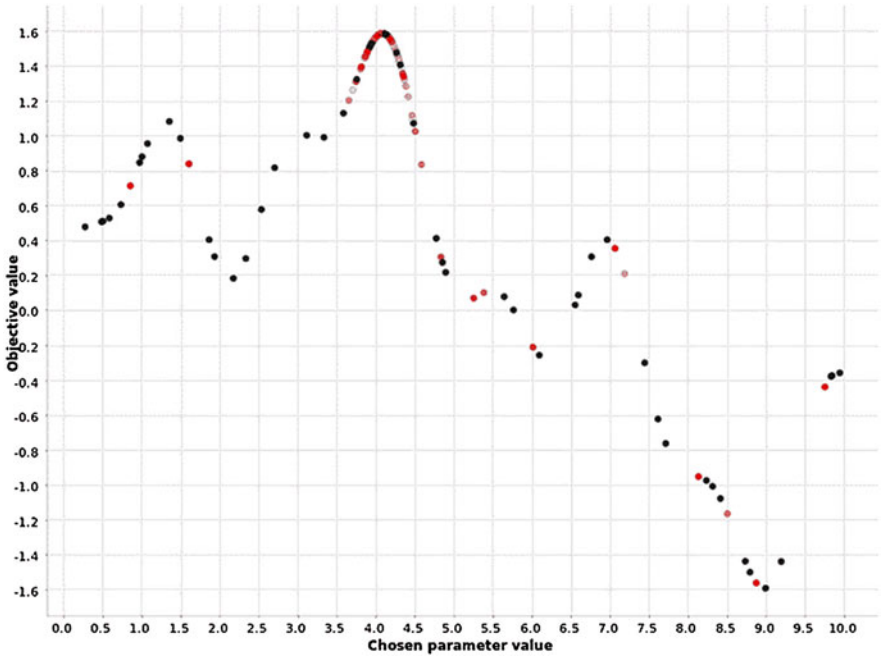
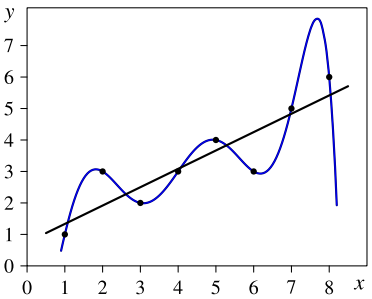


Fig. 5.7 Bayesian optimization of the parameter values on a [0, 10] range. The color gradient reflects the position in the generated sequence of parameter candidates. Whiter points correspond to parameter values generated earlier on in the process; redder points correspond to parameter values generated later on. The gray points are generated in the first random phase of the strategy. Bayesian optimization strategy uses the warm-up phase to define the most promising area and then selects the next values for the parameters in that area

Fig. 5.8 An example for overfitting



The figure also shows a polynomial—the blue line—that interpolates the data points with zero error. If we just consider the error, the polynomial is a better choice than the simple regression line.

However, if we assume that the data points are corrupted with noise, the polynomial may mainly fit the noise rather than capture the underlying relationship. When we obtain further data points, the regression line is likely even a better approximation for these points than the polynomial. Therefore, a model with a smaller error is

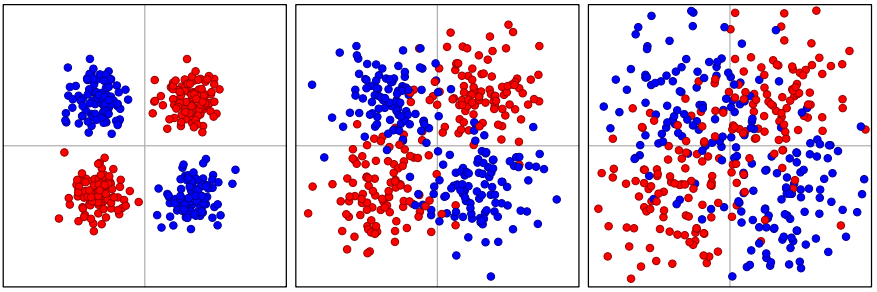


Fig. 5.9 A simple classification problem with perfect separation (*left*) and more difficult classification problems with slightly (*middle*) and strongly overlapping classes (*right*)

not necessarily a better fit for the data. In general, more complex models can fit the data better but have a higher tendency to show this bad effect called **overfitting**.

Once we have fitted a model to given data, the fitting error can be composed into four components:

- the pure or experimental error,
- the sample error,
- the lack of fit or model error, and
- the algorithmic error.

5.4.1 Experimental Error

The **pure error** or **experimental error** is inherent in the data and is due to noise, random variations, imprecise measurements, or the influence of hidden variables that cannot be observed. It is impossible to overcome this error by the choice of a suitable model. This error is inherent in the data. Therefore, it is also called **intrinsic error**. In the context of classification problems, it is also called **Bayes error**.

5.4.1.1 Bayes Error

Classification problems are often illustrated by very simplified examples as shown on the left in Fig. 5.9, where samples from two classes are shown. The samples from one class are depicted as red circles, and samples from the other class as blue circles. In this case, the two classes can be separated easily as indicated by the gray lines. The upper left and the lower right regions correspond to the blue class, whereas the upper right and the lower left regions correspond to the red class.

However, in real-world problems, the situation is usually more complicated. Classes tend to overlap. An example of such a case is shown in the middle of Fig. 5.9. Here the samples from both classes tend to scatter a little bit more, so that it is impossible to classify all objects correctly by a simple partition of the product space defined by the predictor variables. Nevertheless, the number of incorrectly

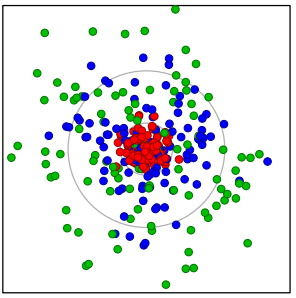


Fig. 5.10 Results of three darts players, one of which is a beginner (*green*), one a hobby player (*blue*), and one a professional (*red*)

classified samples will still be small, and the corresponding samples can be considered as exceptions. On the right in Fig. 5.9, however, we see an even more extreme example with a high overlap of the classes.

In all three cases, the upper left and the lower right regions are dominated by the blue class, and the upper right and the lower left regions by the red class. In the left chart of Fig. 5.9, the regions are exclusively occupied by the corresponding class, and in the middle chart, the regions are not exclusively occupied by one class, but at least highly dominated. Finally, in the right chart, although the domination of regions by classes is still there, incorrectly classified samples can no longer be considered as exceptions. Although there are two prototypical examples for each class—the centers of the four regions—the elements of the classes tend to deviate strongly from the ideal prototypes leading to the high overlap of the classes and making it impossible to achieve a high rate of correctly classified samples.

Of course, the data sets shown in Fig. 5.9 are artificial examples for illustration purposes. The samples from each class were drawn from two bivariate normal distributions, so that the data come from four normal distributions altogether. In all cases, the means of the distributions are the same, and only the variances differ, leading to a higher overlap of the classes for higher variance. Despite the overlap, the prototypical examples, i.e., the means of the normal distributions, differ significantly. However, there are applications where even the prototypical examples from different classes may be more or less identical. Consider, for instance, the example of three darts players who try to hit the middle of the dartboard (the so-called *bull's eye*). The three darts players correspond to the classes, and the goal is to predict who has thrown the dart, given the hitting point of the dart. Assume that one of the players is a professional, one is a hobby player, and one is a complete beginner.

The results from a number of trials of the three players may look like as shown in Fig. 5.10. The red dots correspond to the results of the professional player, the blue dots come from the hobby player, and the green dots were caused by the complete beginner. Figure 5.11 shows the results of the three player separately.

In all three cases, the results scatter around the center of the dartboard. The difference between the three players comes from *how much* their results scatter around the center of the dartboard. The hits of the professional player will all be close to the center, whereas the complete beginner sometimes almost hits the center but has the highest deviation.

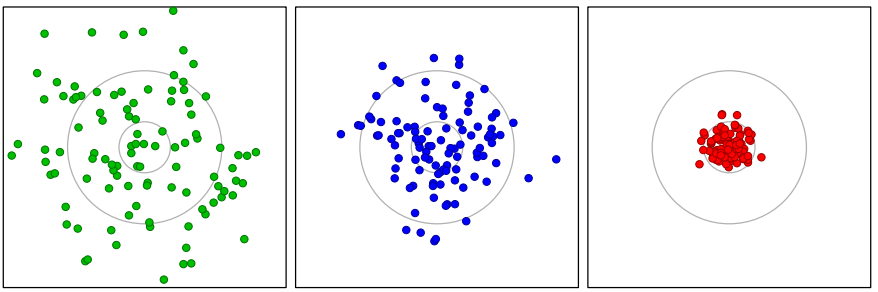


Fig. 5.11 Darts results for a beginner (*left*), a hobby player (*middle*), and a professional (*right*)

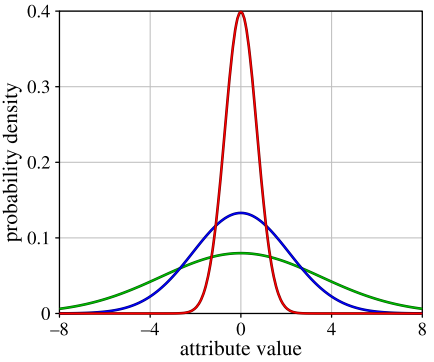


Fig. 5.12 Three normal distributions with identical means but different standard deviations (*green*, $\sigma = 5$; *blue*, $\sigma = 3$; *red*, $\sigma = 1$)

In terms of a classification problems, all three classes have the same prototype or center, and only the deviation differs. In such cases, a classifier can still be constructed, however, with a high misclassification rate.

We take an abstract simplified look at the darts player example. Assume that we draw samples from three univariate normal distributions, representing three different classes, with the same mean but with different variances, as shown in Fig. 5.12. This corresponds more or less to the classification problem of the darts players when we consider only the horizontal deviation to the center of the dartboard. Samples from the normal distribution with the smallest variance correspond to hits of the professional player, whereas the normal distribution with the largest variance represents the complete beginner. In this theoretical example, it is obvious how to find the classification decision. Assuming that the three classes have the same frequency, an object, i.e., a simple value in this case, should be assigned to the normal distribution (class) with the highest likelihood, in other words, to the normal distribution with the highest value of the corresponding probability density function at the position of the given value. In this way, the region in the middle would be assigned to the normal distribution with smallest variance, the left and right outer region would be assigned to the normal distribution with highest variance, and the region in-between would be assigned to the remaining normal distribution.

In this sense, it is obvious how to make best guesses for the classes, although these best guesses will still lead to a high misclassification rate. For the best guesses, there are clear boundaries for the classification decision. However, one should not mix up the classification boundaries with class boundaries. Classification boundaries refer to the boundaries drawn by a classifier by assigning objects to classes. These boundaries will always exist. But these classification boundaries do not necessarily correspond to class boundaries that separate the classes. In most cases, class boundaries do not even exist, since classes tend to overlap in real applications and cannot be clearly separated as in Fig. 5.9. This is due to the Bayes or the pure error.

5.4.1.2 ROC Curves, Accuracy Metrics and Confusion Matrices

For many classification problems, there are only two classes which the classifier is supposed to distinguish. Let us call the two classes *plus* and *minus*. The classifier can make two different kinds of mistake. Objects from the class *minus* can be wrongly assigned to the class *plus*. These objects are called **false positives**. And vice versa, objects from the class *plus* can be wrongly classified as *minus*. Such objects are called **false negatives**. The objects that are classified correctly are called **true positives** and **true negatives**, respectively.

There is always a trade-off between false positives and false negatives. One can easily ensure that there are no false positives by simply classifying all objects as *minus*. However, this means that all objects from the class *plus* become false negatives. The other extreme is to classify all objects as *plus*, in this way avoiding false negatives but accepting that all objects from the class *minus* are false positives. A classifier must find a compromise between these two extremes, trying to minimize both the number of false positives and false negatives. A classifier biased to the class *plus* will have fewer false negatives but more false positives, whereas a classifier biased to the class *minus* will have fewer false positives but more false negatives. Cost functions, as they were explained in Sect. 5.2.1, are one way to introduce such biases to false positives or false negatives.

Some classifiers also provide for each object a probability, whether it belongs to a class or not. The usual decision is then to assign the object to the class with the highest probability. So in the case of the two classes *plus* and *minus*, we would assign an object to the class *plus* if and only if the probability for this class is greater than 0.5. But we could also decide to be more careful and to assign objects to the class *plus* only when the corresponding probability is higher than $\tau = 0.8$, leading to fewer false positives but more false negatives. If we choose a threshold τ for assigning an object to the class *plus* lower than 0.5, we will reduce the number of false negatives for the price of having false positives. This concept can be generalized from probabilities to arbitrary scores with the property that the higher scores speak in favor of the positive class.

This trade-off between false positives and false negatives is illustrated by the **receiver operating characteristic** or **ROC curve** showing the false positive rate versus the true positive rate (in percent). Figure 5.13 shows examples for possible ROC curves. For various choices of τ, a new point is drawn at the respective coordinates of false positive rate and true positive rate. These dots are connected to a

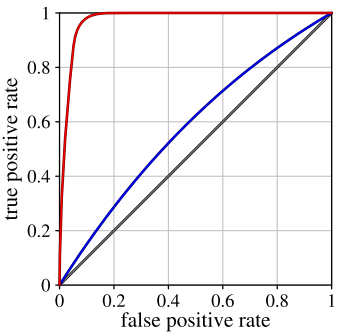


Fig. 5.13 Examples of ROC curves

curve. The ROC curves in Fig. 5.13 are idealized. Normally, the ROC curves based on sampled data look less smooth and more ragged.

The best case for an ROC curve would be to jump immediately from 0% to 100%, so that we could have a classifier with 100% true positives and no false positives. The red line shows a very good, but not perfect, ROC curve. We can have a very high true positive rate together with a low false positive rate. The diagonal, shown as a gray line, corresponds to pure random guessing, so that such a classifier has actually learned nothing from the data, or there is no connection between the classes and the attributes used for prediction. Therefore, the diagonal line is the worst case for a ROC curve.

The **area under curve** (**AUC**), i.e., the area under the ROC curve, is an indicator of how well the classifier solves the problem. The larger the area, the better the solution for the classification problem. The area is measured relative to the area of the square $[0, 100] \times [0, 100]$ in which the ROC curve is drawn. The lowest value for AUC is 0.5, corresponding to random guessing, and the highest value is 1 for a perfect classifier with no misclassifications. The blue line in Fig. 5.13 is an ROC curve with a lower performance. The reason for the low performance may be due to the Bayes error, but also because of other errors that will be discussed in the following sections.

The AUC value can also be interpreted as the probability that the score of a randomly chosen object from the positive class is higher than the score of a randomly chosen object from the negative class.

When there are more than two classes, it is not possible to draw an ROC curve as described above. One can only draw ROC curves with respect to one class against all others or for pairs of classes.

The **confusion matrix** is another way to describe the classification errors. A confusion matrix is a table where the rows represent the true classes and the columns the predicted classes. Each entry specifies how many objects from a given class are classified into the class of the corresponding column. An ideal classifier with no misclassifications would have only entries different from zero in the diagonal.

Table 5.3 shows a possible confusion matrix for the iris data set. From the confusion matrix we can see that all objects from the class setosa are classified correctly and no object from another class is wrongly classified as setosa. A few objects from

Table 5.3 A possible confusion matrix for the iris data set

True class	Predicted class		
	Iris setosa	Iris versicolor	Iris virginica
Iris setosa	50	0	0
Iris versicolor	0	47	3
Iris virginica	0	2	48

the other classes—three from versicolor and two from virginica—are wrongly classified.

Based on the confusion matrix and the false positive (FP), false negative (FN), true positive (TP), and true negative (TN) counts, a number of overall metrics have been defined for the evaluation of the overall quality of a classifier.

Sensitivity, defined as

$$\text{sensitivity} = \frac{TP}{TP + FN},$$

measures the model's capability to recognize the positive class correctly. If all instances of the positive class are recognized correctly, the result is 0 "false negatives" which means that no instance of the positive class is assigned to the negative class. Sensitivity is then 1.

Specificity, defined as

$$\text{specificity} = \frac{TN}{TN + FP},$$

measures the model's capability of recognizing what does not belong to the positive class. If the model recognizes what does not belong to that class, the result is 0 "false positives" which means no data objects are misclassified into the positive class. Specificity is then 1.

Precision—also called **positive predictive value**—and **recall** are two widely used statistical accuracy measures:

$$\text{precision} = \frac{TP}{TP + FP},$$
$$\text{recall} = \frac{TP}{TP + FN}.$$

Precision can be seen as a measure of exactness or fidelity, whereas recall is a measure of completeness. In a classification task, the precision for a class is the number of instances correctly labeled as belonging to that class with respect to the total number of elements labeled as belonging to that class. Recall is the number of correctly classified instances with respect to the total number of objects belonging to that class. Recall has the same definition as sensitivity.

Sensitivity, specificity, precision, and recall depend on the proportions of the classes whereas AUC does not take into account whether the classes are balanced or not.

The **F-measure** is built on precision and recall,

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

The F-measure can be interpreted as a weighted average of precision and recall, where the F-measure reaches its best value at 1 and its worst score at 0.

The **overall accuracy** is a measure of average correctness of the classifier,

$$\text{overall accuracy} = \frac{TP + TN}{TP + FP + TN + FN} = \frac{TP + TN}{n},$$

where n is the number of instances in the data set. This is an overall measure and is calculated across all classes. An accuracy of 1 means that the classified values are exactly the same as the original class values.

The overall accuracy works well for balanced data sets. However, this average accuracy might hide the performance on a minority class in case of an unbalanced data set. **Cohen's kappa** was introduced to take more into account the classifier's performance on a minority classes. Cohen's kappa is a measure of interrater agreement—where one rater corresponds to the true classes and the other rater to the classifier under consideration–defined as

$$\kappa = \frac{p_0 - p_e}{1 - p_e},$$

where p_0 describes the agreement between the two raters (the overall accuracy); p_e is the probability of the raters to agree by chance, i.e.,

$$p_e = p_{\text{pos}} + p_{\text{neg}},$$

where

$$\begin{aligned} p_{\text{pos}} &= \frac{TP + FN}{n} \cdot \frac{TP + FP}{n} \\ &= \frac{\#\text{ positive instances}}{n} \cdot \frac{\#\text{ instances classified as positive}}{n} \end{aligned}$$

and

$$\begin{aligned} p_{\text{neg}} &= \frac{TN + FP}{n} \cdot \frac{TN + FN}{n} \\ &= \frac{\#\text{ negative instances}}{n} \cdot \frac{\#\text{ instances classified as negative}}{n}. \end{aligned}$$

For perfect classification, i.e., $p_0 = 1$, Cohen's kappa is 1. If the classifier performance is just random, then $p_0 = p_e$ and Cohen's kappa becomes 0. Notice that p_e introduces the a priori probabilities of both the positive and negative classes and therefore Cohen's kappa gives a more balanced accuracy estimation in case of strong differences in the class distributions.

Table 5.4 Error metrics for regression problems

Error metric	Formula	Notes
R-squared	$1 - \frac{\sum_{i=1}^{n}(y_i - f(x_i))^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2}$	Universal range: the closer to 1 the better. $\overline{y}$ is the mean of the target value.
Mean absolute error (MAE)	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - f(x_i)\rvert$	Equal weights for all distances. Same unit as the target values.
Mean squared error (MSE)	$\frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))^2$	Commonly used loss function
Root mean squared error (RMSE)	$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))^2}$	Weighs large distances more. Same unit as the target values.
Mean signed difference	$\frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))$	Only useful for detecting a bias of the error.
Mean absolute percentage error (MAPE)	$\frac{100\%}{n}\sum_{i=1}^{n}\lvert y_i - f(x_i)\rvert$	Requires nonzero target values.

5.4.1.3 Error Metrics on Numerical Outputs

If the output is numerical, error metrics for classification cannot help. In this case, distance functions are used as error metrics. The most commonly used error metrics are reported in Table 5.4 **R-squared**, **Mean Absolute Error** (MAE), **Mean Squared Error** (MSE), **Root Mean Squared Error** (RMSE), **Mean Signed Difference**, and **Mean Absolute Percentage Error** (MAPE). We consider a numerical target attribute with y_i as the target value for the ith instance and $f(x_i)$ being the model's produced prediction, assuming we have altogether n instances. Some metrics maintain the measure unit as for the original data, making it easier to evaluate the absolute magnitude of the error; some others give information only about the direction of the error; some weigh large or small distances differently; while some others require a nonzero target value.

5.4.2 Sample Error

The **sample error** is caused by the fact that the data is only an imperfect representation of the underlying distribution of the data.

A finite sample, especially when its size is quite small, will seldom exactly reflect the true distribution of the probability distribution generating the data. According to the strong law of large numbers, the sample distribution converges with probability one to the true distribution when the sample size approaches infinity. However, a finite sample can deviate significantly from the true distribution, although the probability for such a deviation might be small. The bar chart in Fig. 5.14 shows the result for throwing a fair die 60 times. In the ideal case, one would expect each of

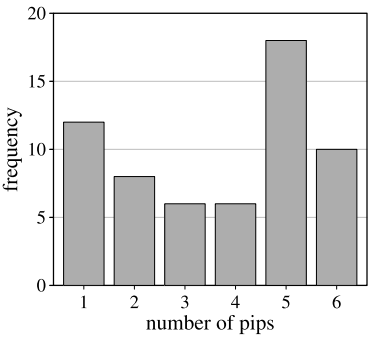


Fig. 5.14 A sample of 60 trials from a fair die

the numbers $1, \ldots, 6$ to occur 10 times each. But for this sample, the sample distribution does not look uniform. Another source for sample errors are measurements with limited precision and round-off errors.

Sometimes the sample is also biased. Consider a bank that supplies loans to customers. Based on the data available on the customers who have obtained loans, the bank wants to estimate the probability for paying back a loan for new customers. However, the collected data will be biased in the direction of better customers because customers with a more problematic financial status have not been granted loans, and therefore, no information is available for such customers whether they might have paid back the loan nevertheless. From the perspective of a representative sample of all applicants, we deal with a sample error when using the bank's database.

5.4.3 Model Error

A large error may be caused by a high pure error, but it may also be due to a **lack of fit**. When the set of considered models is too simple for the structure inherent in the data, no model will yield a small error. Such an error is also called **model error**. Figure 5.15 shows how a regression line is fitted to data with no pure error. But the data points originate from a quadratic and not from a linear function.

The line shown in the figure is that with the smaller mean squared error. Such a line can always be computed, no matter from which true function the data come. But the line that fits best such data does not reflect the structure inherent in the data.

Unfortunately, it is often difficult or even impossible to distinguish between the pure error and the error due to the lack of fit. Simple models tend to have a large error due to the lack of fit, whereas more complex models lead to small errors for the given data but tend to be overfitting and may lead to very large errors for new data.

A discussion of the problem of finding a compromise between too simple models with large and too complex models with overfitting is discussed in Sect. 5.5.

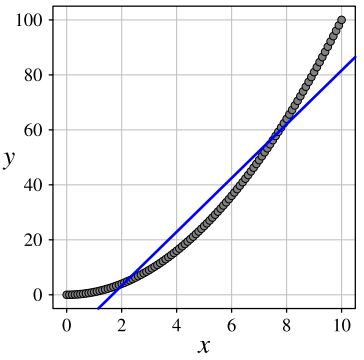


Fig. 5.15 A large error due to lack of fit

5.4.4 Algorithmic Error

There is the **algorithmic error** caused by a method that is used to fit the model or the model parameters. In the ideal case, when an analytical solution for the optimum of the objective function exists, the algorithmic error is zero or is only caused by numerical problems. But as we have seen in Sect. 5.3, in many cases an analytical solution cannot be provided, and heuristic strategies are needed to fit the model to the data.

Even if a model exists with a very good fit—the global optimum of the objective function—the heuristic optimization strategy might only be able to find a local optimum with a much larger error that is caused neither by the pure error nor by the error due to the lack of fit.

Most of the time, the algorithmic error will not be considered, and it is assumed that the heuristic optimization strategy is chosen well enough to find an optimum that is at least close to the global optimum.

5.4.5 Machine Learning Bias and Variance

The types of error mentioned in the previous four subsections can be grouped into two categories. The algorithmic and model errors can be controlled to a certain extent, since we are free to choose a suitable model and algorithm. These errors are also called **machine learning bias**. We have no influence on the pure or intrinsic errors. The same applies to the sample errors when the data to be analyzed have already been collected. The error caused by the intrinsic and sample error sometimes is also called **variance**.

It is also well known from statistics that the mean squared error (MSE) of an estimator θ^* for an unknown parameter θ can be decomposed in terms of the variance of the estimator and its bias:

$$\text{MSE} = \text{Var}(\theta^*) + (\text{Bias}(\theta^*))^2. \tag{5.11}$$

Note that this decomposition deviates from the classification into model bias and variance above as it is popular in machine learning. The variance in (5.11) depends on the intrinsic error, i.e., on the variance of the random variable from which the sample is generated, and also on the choice of the estimator θ^* which is considered as part of the model bias in machine learning.

A more detailed discussion on the different meanings and usages of the terms variance and bias can be found in [5].

5.4.6 Learning Without Bias?

The different types of errors or biases discussed in the previous section have an interesting additional impact on the ability to find a suitable model for a given data set: if we have no model or learning bias, we will not be able to generalize. Essentially this means that we need to constrain either the types of model that are available or the way we are searching for a suitable model (or both). Tom Mitchell demonstrates this very convincingly in his hypothesis learning model [11]—in this toy world he can actually prove that in the unrestricted case of a boolean classification problem, the fitting models we can possibly find predict "false" in exactly half of the cases and "true" for the other half. This means that without any constraint we always leave all choices open. The learner or model bias is essential to put some sort of a priori knowledge into the model learning process: we either limit what we can express, or we limit how we search for it.

5.5 Model Validation

In the previous section, algorithms were discussed that fit a model from a predefined model class to a given data set. Complex models can satisfy a simple fitting criterion better than simple models. However, the problem of overfitting increases with the complexity of the model. Especially, when the model is built for prediction purposes, the error of the model based on the data set from which it was computed is usually smaller than for data that have not been used for determining the model. For instance, in Fig. 5.8 on page 101, the polynomial fits the data perfectly. The error for the given data set is zero. Nevertheless, the simple line might be a better description of the data, at least when we assume that the data are corrupted by noise. Under this assumption, the polynomial would lead to larger errors for new data, especially in those regions where it tends to oscillate. How do we find out, which model is actually suited best to our problem?

5.5.1 Training and Test Data

The most common principle to estimate a *realistic performance* of the model for unknown or future data is separating the data set for training and testing purposes. In

the simplest case, the data set is split into two disjoint sets, the **training data** which are used for fitting the model and the **test data** which only serve for evaluating the trained model but not for fitting the model. Usually, the training set is chosen larger than the test data set, for instance, 2/3 of the data are used for training, and 1/3 for testing.

One way to split the data into training and test sets is a random assignment of the data objects to these two sets. This means that on average the distributions of the values in the original data set and in the training and test data sets should be roughly the same. However, by chance it can happen that the distributions may differ significantly. When a classification problem is considered, it is usually recommended to draw stratified samples for the training and test sets. **Stratification** means that the random assignments of the data to the test and training sets are carried out per class and not simply for the whole data set. In this way, it is ensured that the relative frequency in the original data set, the training, and the test set are the same.

Sometimes, it is not advisable to carry out a (stratified) random assignment of the data to the training and test sets. Consider again the example of the producer of tea cups from Sect. 5.2.1 who wants to classify the cups automatically into *ok* and *broken*. Assume that six different types of cup are produced at the moment and in the future new types of cup might be introduced. Dividing the data set randomly into training and test data would not reflect the classification problem to be encountered in the future. If the producer had no intention to change the types of the cup, it would be correct to draw a random sample for testing from all data. In this way, the classifier will be trained and tested with examples from all six types of cup. But since new models of cups might be introduced in the future, this would yield an overoptimistic estimation of the classification error for future cups. A better way would be to use the data from four types of cup for training the classifier and to test them on the remaining two types of cup. In this way, we can get an idea of how the classifier can cope with cups it has never seen before.

This example shows how important it is in prediction tasks to consider whether the given data are representative for future data. When predictions are made for future data for which given data are not representative, extrapolation is carried out with a higher risk of wrong predictions. In the case of high-dimensional data, it cannot be avoided to have scarce or no data in certain regions of the space of possible values as we have discussed already in Sect. 4.2, so that we always have to be aware of this problem.

Sometimes, the data set is split into three parts: In addition to the training and test data sets, a **validation set** is also used. If, for instance, a classifier should be learned from data, but we do not know which kind of model is the most appropriate one for the classifier, we could make use of a validation set. All classifier models are generated based on the training data only. Then the classifier with the best performance on the validation set is chosen. The prediction error of this classifier is then estimated based on the test set.

5.5.2 Cross-Validation

The estimation of the fitting error for new data based on a test data set that has not been used for learning the model depends on the splitting of the original data set into training and test data. By chance, we might just be lucky that the test set contains more easy examples leading to an overoptimistic evaluation of the model. Or we might be unlucky when the test set contains more difficult examples and the performance of the model is underestimated. **Cross-validation** does not rely on only one estimate of the model error, but rather on a number of estimates. For k-fold cross-validation, the data set is partitioned into k subsets of approximately equal size. Then the first of the k subsets is used as a test set, and the other $(k-1)$ sets are used as training data for the model. In this way, we get the first estimate for the model error. Then this procedure is repeated by using each of the other k subsets as test data and the remaining $(k-1)$ subsets as training data. Altogether, we obtain k estimates for the model error. The average of these values is taken as the estimate for the model error. Typically, $k = 10$ is chosen.

Small data sets might not contain enough examples for training when 10% are left out for testing. In this case, the **leave-one-out method**, also known as the **jackknife method**, can be applied which is simply n-fold cross-validation for a data set with n data objects, so that each time only one data object is used for evaluating the model error.

5.5.3 Bootstrapping

Bootstrapping is a resampling technique from statistics that does not directly evaluate the model error but aims at estimating the variance of the estimated model parameters. Therefore, bootstrapping is suitable for models with real-valued parameters. Like in cross-validation, the model is computed not only once but multiple times. For this purpose, k bootstrap samples, each of size n, are drawn randomly *with replacement* from the original data set with n records. The model is fitted to each of these bootstrap samples, so that we obtain k estimates for the model parameters. Based on these k estimates, the empirical standard deviation can be computed for each parameter to provide information on how reliable the estimate of the parameter is.

Figure 5.16 shows a data set with $n = 20$ data points from which $k = 10$ bootstrap samples were drawn. For each of the bootstrap samples, the corresponding regression line is shown in the figure. The resulting parameter estimates for the intercept and the slope of the regression line are listed in Table 5.5. The standard deviation for the slope is much lower than for the intercept, so that the estimation for the slope is more reliable. It is also possible to compute confidence intervals for the parameters based on bootstrapping [4].

The results from bootstrapping can be used to improve predictions as well by applying **bagging** (bootstrap aggregation). In the case of regression, one would use the average of the predicted values that the k models generated from the bootstrap

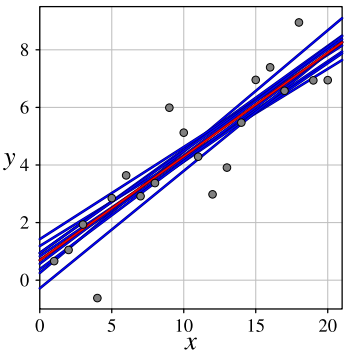


Fig. 5.16 10 lines fitted to a data set based on 10 bootstrap samples (*blue*) and their mean (*red*)

Table 5.5 The estimators for the 10 bootstrap samples in Fig. 5.16

Bootstrap sample	Intercept	Slope
1	0.3801791	0.3749113
2	0.5705601	0.3763055
3	−0.2840765	0.4078726
4	0.9466432	0.3532497
5	1.4240513	0.3201722
6	0.9386061	0.3596913
7	0.6992394	0.3417433
8	0.8300100	0.3385122
9	1.1859194	0.3075218
10	0.2496341	0.4213876
Mean	0.6940766	0.3601367
Standard deviation	0.4927206	0.0361004

samples yield. In the example in Fig. 5.16, for a given value x, all 10 lines would provide a prediction for y, and we would use the average of these predictions. For classification, one would generate k classifiers from the bootstrap samples, calculate the predicted class for all k classifiers, and use the most frequent class among the k predicted classes as the final prediction. Bagging will be introduced in more detail in Sect. 9.5.

5.5.4 Measures for Model Complexity

Complex models are more flexible and can usually yield a better fit for the training data. But how well a model fits the training data does not tell much about how well a model represents the inherent structure in the data. Complex models tend to overfitting as we have already seen in Fig. 5.8 on page 101.

Model selection—the choice of a suitable model—requires a trade-off between simplicity and fitting. Based on the principle of **Occam's razor**, one should choose the simplest model that "explains" the data. If a linear function fits the data well

Table 5.6 A classification task

ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	a_1	a_1	a_1	a_1	a_1	a_1	a_1	a_1	a_1	a_1	a_2	a_2	a_2	a_2	a_2	a_2	a_2	a_1	a_2	a_2
B	b_1	b_2	b_2	b_2	b_1	b_1	b_1	b_2	b_1	b_2	b_1	b_1	b_1	b_1	b_1	b_1	b_1	b_1	b_2	b_2
C	+	+	+	+	+	+	+	+	+	+	−	−	−	−	−	−	−	−	+	+

enough, one should prefer the linear function and not a quadratic or cubic function. However, it is not clear what is meant by "fitting the data well enough." There is a need for a trade-off between model simplicity and model fit. But the problem is that it is more or less impossible to measure these two aspects in the same unit. In order to combine the two aspects, **regularization** techniques are applied. Regularization is a general mathematical concept that introduces additional information in order to solve an otherwise ill-posed problem. A penalty term for more complex models can be incorporated into to the pure measure for model fit as a regularization technique for the avoidance of overfitting. Examples for regularization are provided in Sect. 8.3.5.

5.5.4.1 The Minimum Description Length Principle

The **minimum description length principle** (**MDL**) is one promising way to join measures for model fit and complexity into one measure. The basic idea behind MDL is to understand modeling as a technique for data compression. The aim of data compression is to minimize the memory—measured in bits—needed to store the information contained in the data in a file. In order to recover the original data, the compressed data and decompression rule are needed. Therefore, the overall size of the compressed data file is the sum of the bits needed for the compressed data and the bits needed to encode the decompression rule. In principle, any file can be compressed to the size of one bit, say the bit with value 1, by defining the decompression rule "if the first bit is 1, then the decompressed file is the original data." This implies that the decompression rule contains the original data set and is therefore as large as the original data, so that no compression is achieved at all when we consider the compressed data and the decompression rule together. The same applies when we do not compress the original data at all. Then we need no space at all for the decompression rule but we have not saved memory space at all, since we have not carried out any compression. The optimum lies somewhere in-between by using a simple decompression rule allowing a reasonable compression.

The application of these ideas to model selection requires an interpretation of the model as a compression or decompression scheme, the (binary) coding of the model, and the compressed (binary) coding of the data. We illustrate the minimum description length principle by two simplified examples.

The first example is a classification task based on the data set shown in Table 5.6. The attribute C with the two possible values $+$ and $-$ shall be predicted with a decision tree based on the two binary attributes A and B with domains $\{a_1, a_2\}$ and $\{b_1, b_2\}$.

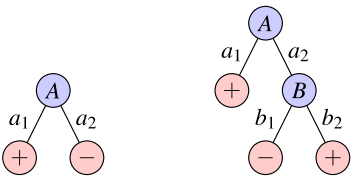


Fig. 5.17 Two decision trees for the data set in Table 5.6

We consider the two decision trees shown in Fig. 5.17 to solve this classification task. The decision tree on the left-hand side is simpler but leads to two misclassifications. The last two records in Table 5.6 are classified wrongly by this decision. The slightly larger decision tree on the right-hand side classifies all records in Table 5.6 correctly. If we were just concerned with misclassification rate for the training data, we would prefer the decision tree on the right-hand side. But the node with the attribute B is only required to correct two misclassifications—10% of the data set. This might just be an artificial improvement of the misclassification rate only for the training data set, and it might lead to overfitting.

In order to decide whether we should prefer the smaller or the larger decision tree, we interpret the two decision trees as compression schemes for the data set in the following way. Based on the corresponding decision tree, we can predict the attribute C, so that we do not have to store the value of the attribute C when we know the decision tree and the values of the attributes A and B for each record. However, this is only true for the larger decision tree. For the smaller decision tree, we have to correct the value of the attribute C for the two records that are misclassified by the smaller decision tree. The length of the compressed file for the data set based on any of the two decision trees is the sum of the lengths needed for coding

- the corresponding decision tree,
- the values of the records for the attributes A and B, and
- the corrected values for the attribute C only for the misclassified records.

The coding of the values of the records for the attributes A and B is needed for any decision tree and can be considered as a constant. The length needed for coding the decision tree depends on the size of the tree. The larger the tree, the more bits are needed to store the tree. However, with a larger tree, the misclassification rate can be reduced, and we need less bits for coding the corrections for attribute C for the misclassified records. When we want to minimize the overall length needed to store the compressed data, we need a compromise between a smaller decision tree with a higher misclassification rate and a larger decision tree with a lower misclassification rate. The smaller decision tree will need less space for its own coding but more for the corrections of the misclassified records, whereas the larger decision tree needs more space for its own coding but can save space due to a lower number of misclassifications. According to the minimum description length principle, we should choose the decision tree with the minimum number of bits needed to code the tree itself and the corrected values for the attribute C.

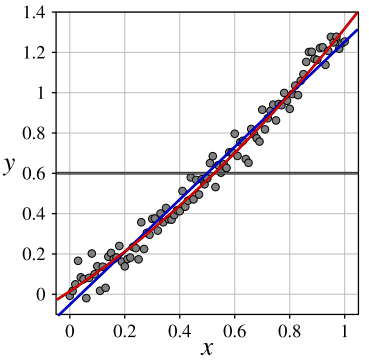


Fig. 5.18 A constant (*gray*), a line (*blue*), and a quadratic curve (*red*) fitted to a data set

Table 5.7 A regression problem

ID	1	2	3	4	5	6	7	8	9
X	0.00	1.00	2.00	3.00	4.00	5.00	6.00	7.00	8.00
Y	1.19	1.33	2.03	1.21	1.80	2.33	1.63	2.88	2.84

Of course, the number of bits needed for the coding depends on the binary coding scheme we use for encoding the decision tree and the corrections for the values of the attribute C. If we have a highly efficient coding scheme for decision trees but an inefficient one for the corrections for the values of the attribute C, larger trees would be preferred. The naive MDL approach will ignore this problem and simply try to find the most efficient binary coding for both parts. However, there are more general concepts of universal codes and models freeing the minimum description length principle from the dependency on the specific coding scheme. For a detailed introduction to universal codes and models that are out of the scope of this book, we refer to [6].

As a second example for the application of the minimum description length principle, we consider a regression problem. Figure 5.18 shows a simple data set to which a line and a quadratic curve are fitted. Which of these two models should we prefer? Of course, the quadratic curve will lead to a smaller error for the training data set than the simple line. We could also think of a polynomial of higher degree that would reduce the error even more. But then we have to face the problem of overfitting in the same as we have to take this for larger decision trees into account.

For illustration purposes, we consider an even simpler regression problem provided by the data set in Table 5.7.

What would be the best model for this data set? Should we describe the relation between X and Y by a simple constant function, by a linear, or even a quadratic function? Of course, the quadratic function would yield the smallest, and the constant function the largest error. But we want to avoid overfitting by applying the minimum description length principle. As in the example of the classification problem, it will suffice to consider the naive approach and not to bother about universal codings and models. Instead of the decision tree, our models are now functions with

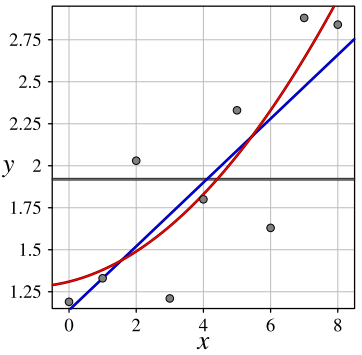


Fig. 5.19 A constant (*gray*), a straight line (*blue*), and a quadratic curve (*red*) fitted to the data set of Table 5.7

Table 5.8 The errors for the three regression functions

ID	1	2	3	4	5	6	7	8	9
$y = 1.92$	0.73	0.59	−0.11	0.71	0.12	−0.41	0.29	−0.96	−0.92
$y = 1.14 + 0.19x$	−0.05	0.00	−0.51	0.50	0.10	−0.24	0.65	−0.41	−0.18
$y = 1.31 + 0.05x + 0.02x^2$	0.12	0.05	−0.54	0.43	0.03	−0.27	0.70	−0.24	0.15

one, two, and three real-valued parameters. The errors are now also real numbers and not just binary values as in the case of the decision trees. If we insist on exact numbers, the coding of a single error value could need infinite memory, since a real number can have infinitely many digits. To avoid this problem, we restrict our precision to two digits right after the decimal point. For reasons of simplicity, we do not consider a binary coding of the numbers, but a decimal coding. A real number is coded backwards starting from the lowest digit, in our case the second digit after the decimal point. Therefore, the numbers 1.23, 2.05, 0.06, and 0.89 would be coded as 321, 502, 6, and 98, respectively. Note that smaller numbers require less memory because we do not code leading zero digits. We also would have to take the sign of each number into account for the coding. But we will neglect this single bit here in order not to mix the decimal coding for the numbers with a binary coding for the sign. If we use a binary coding for the numbers, the sign integrates naturally to the coding as an additional bit. Figure 5.19 shows a plot of the data set and least squares fit of constant, linear, and quadratic functions.

Table 5.8 lists the errors for the constant, linear, and quadratic functions that have been fitted to the data. How many decimal digits do we need for coding of the data set when we use a constant function? We need three digits for the constant 1.92 representing the function. We also need to encode the errors in order to recover the original values of the attribute Y from our constant function. For the errors of this constant function, we always need to encode to digits. Only the digit before the decimal point is always correct. Altogether, the coding of the data set with the constant function requires $3 + 9 \cdot 2 = 21$ decimal digits.

What about the linear function? The function itself requires the coding of the two coefficients, 1.14 and 0.19, for which we need 5 decimal digits. The coding errors require two decimal digits each time, except for the data points with the ID 1 and 2,

for which we need only one and zero decimal digits, respectively. This means that we have altogether $5 + 7 \cdot 2 + 1 + 0 = 19$ decimal digits. Similar considerations for the quadratic curve lead to $5 + 7 \cdot 2 + 2 \cdot 1 = 20$ decimal digits.

This means that, in terms of our extremely simplified MDL approach, the linear function leads to the most efficient coding for the data set, and we would therefore prefer the linear regression function over the others.

It should be emphasized again that there is a more rigorous theory for the minimum description length principle that avoids the problem of finding the most efficient coding and the restriction to a fixed precision for the representation of real numbers. But the naive approach we have described here often suffices to give an idea how complex the chosen model should be.

5.5.4.2 Akaike's and Bayesian Information Criteria

Akaike's information criterion [1] is based on entropy considerations in terms of how much additional information a more complex model provides with respect to the (unknown) stochastic model generating the data. For large sample sizes, Akaike could show that under these assumptions one should choose the model that minimizes Akaike's information criterion **AIC**, originally called "an information criterion,"

$$\mathrm{AIC} = 2k - 2\ln(L), \tag{5.12}$$

where k is the number of parameters of the model, and L is the likelihood of the corresponding model. The aim is to minimize the value in (5.12), which is achieved by a compromise between low model complexity—demanding a small value for k—and a good fit, i.e., a high likelihood which is achieved easier for complex models.

For a regression problem with the assumption that the underlying measurements of the "true" function are corrupted with noise coming from a normal distribution, the minimization problem in (5.12) can be simplified to minimizing

$$\mathrm{AIC}_{\mathrm{Gauss}} = 2k + n\ln(\mathrm{MSE}), \tag{5.13}$$

where MSE is the mean squared error

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(f(\mathbf{x}_i) - y_i)^2 \tag{5.14}$$

with regression function f and n data points $(\mathbf{x}_i, y_i)$.

Equation (5.13) applied to our simple regression example described in Table 5.7 yields the values -6.678, -12.130, -9.933 for the constant, linear, and quadratic regression functions, respectively, obtained from the least squares method. Since small values for Akaike's information criterion are better than larger, we would prefer the linear function that was also the best choice in the context of MDL.

The **Bayesian information criterion** (**BIC**) [12] is derived in the context of Bayesian inference for large sample sizes and under certain assumptions on the type of probability distribution that generates the data:

$$\mathrm{BIC} = k\ln(n) - 2\ln(L). \tag{5.15}$$

With the same assumptions as for (5.13), in the context of regression BIC becomes

$$\mathrm{BIC}_{\mathrm{Gauss}} = k \ln(n) + n(\mathrm{MSE})/\sigma^2, \tag{5.16}$$

where σ^2 is the (estimated) variance for the underlying normal distribution that causes the noise.

5.5.5 Coping with Unbalanced Data

If we are dealing with a, say, two-class classification problem and one class is drastically underrepresented, most of the algorithms will be biased towards the frequent class, just from its higher a priori probability. This may be a side-effect of the objective function used internally for model building or the model evaluation itself. If there is only little data from one class, an MDL approach, for instance, may consider the encoding costs for a complex condition not being worth the effort of increasing the accuracy only by a tiny margin.

In order to reduce the effect of a class **imbalance**, two roads are possible: reducing the number of samples of the majority class to be comparable to the number of samples in the minority class (undersampling) or, vice versa, increasing the number of samples in the minority class to become comparable to the number of samples in the majority class (oversampling).

- **Undersampling** Randomly remove samples from the majority class, with or without replacement. This is one of the earliest techniques used to alleviate imbalance in the dataset. However, it may potentially discard useful or important samples [10].
- **Oversampling** Historically, random oversampling supplemented the dataset with multiple copies of some of the minority classes [2] with or without replacement. Instead of using just copies of existing samples, some oversampling techniques create synthetic samples from the same statistical distribution of the minority class. The most common technique is called **SMOTE** (Synthetic Minority Oversampling Technique) [3]. SMOTE takes a sample from the dataset, and considers its k nearest neighbors. The vector between one of those neighbors, and the current data point is multiplied by a random number x which lies between 0 and 1. We then add the resulting vector to the current data point to create the new, synthetic data point.

5.6 Model Errors and Validation in Practice

Although we will introduce realizations of model classes and different optimization algorithms later in this book, we will already demonstrate how to measure error and estimate model accuracy in practice.

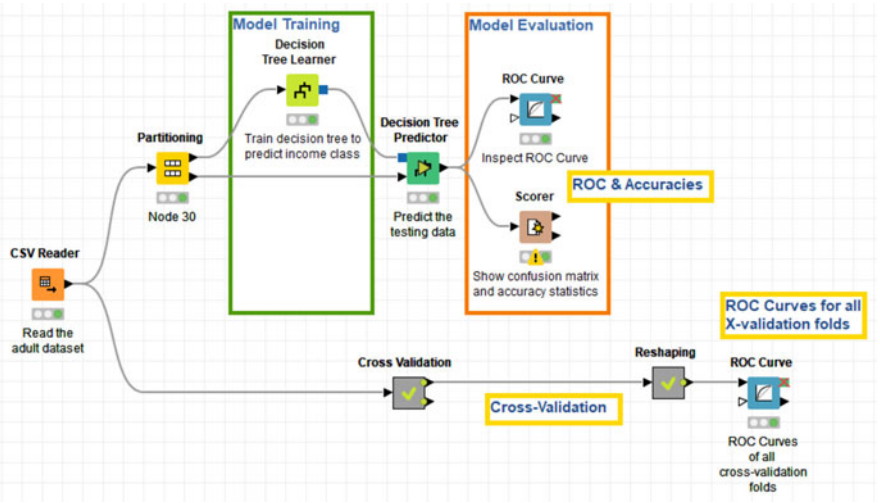
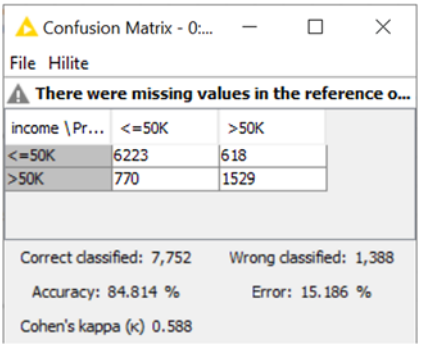


Fig. 5.20 This workflow in the upper part trains a decision tree on a training set extracted from the adult dataset to predict "income" based on the other demographic attributes; then tests it on a separate data subset, calculates the corresponding evaluation metrics, and draws the ROC curve. In the lower part, it applies the cross-validation loop using 10 iterations. The last ROC Curve node on the right plots the ROC curves for all iterations' results

Fig. 5.21 Confusion matrix and accuracy metrics calculated on the test set by the Scorer node



5.6.1 Scoring Models for Classification

KNIME Analytics Platform offers a number of modules to estimate errors. Most prominently, the *Scorer* node computes a confusion matrix and some accuracy metrics—given two columns with the actual and predicted class label—and the ROC curve draws the ROC curve—given the column with the actual class, the positive class, and the column(s) with the probability of the positive class.

Figure 5.20 shows the use of the *Scorer* and *ROC Curve* nodes in practice. The workflow reads the adult.csv data, a CENSUS dataset, and trains a decision tree model on a training set to predict the income of people given the other demographic attributes. The trained decision tree classifier is then applied to the second data set,

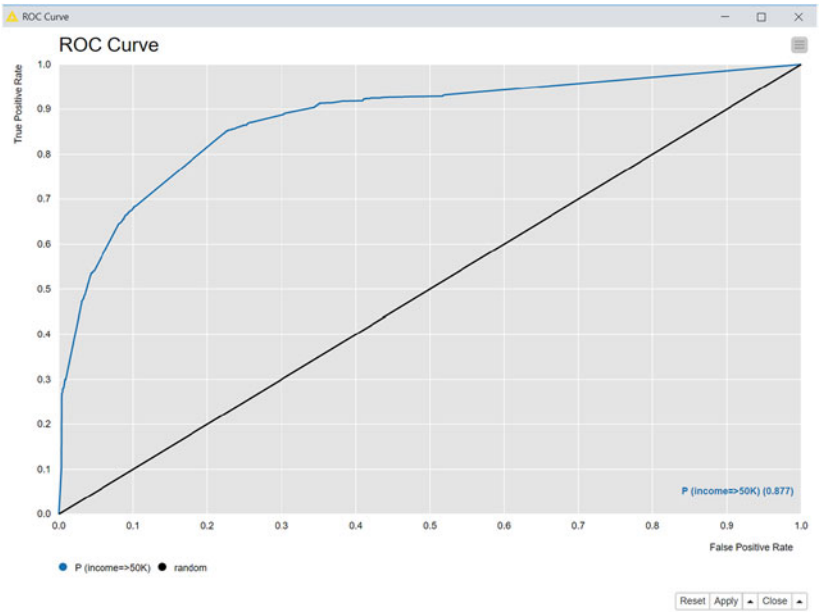


Fig. 5.22 ROC curve calculated over the test set and displayed by the ROC Curve node

the test set, to generate predictions and class probabilities for all records in the test set.

Class predictions are fed into the *Scorer* node, which compares them to the actual values in the target column ("income"). The output of this *Scorer* node is a confusion matrix (which is also available as node view) and a second data table listing some well-known evaluation metrics.

The income column has two classes: richer than 50K and poorer than 50K. The confusion matrix output by the *Scorer* node is shown in Fig. 5.21 together with an overall accuracy of 84.81% and a Cohen's kappa of 0.588. The corresponding ROC curve is shown in Fig. 5.22.

More interestingly, however, are methods to run cross- or other validation techniques. KNIME Analytics Platform offers those in the form of so-called meta nodes which encapsulate a series of other nodes. The metanode implementing the cross-validation loop is named *Cross Validation* and contains a loop that can be modified with the model of choice. Figure 5.23 shows the inside of such a node. Predictions, probabilities, and average errors for all iterations are reported at the output ports of the X-aggregator node.

In the lower part of the workflow used for this example, we see a cross-validation metanode. It was configured to implement a 10-fold cross-validation procedure. The last ROC Curve node on the right plots the ROC curves for all iterations' results. As a bonus to this example, after reshaping the output of the *Cross Validation* node, an ROC Curve node draws the ROC curves for all iterations from the cross-validation loop (Fig. 5.24).

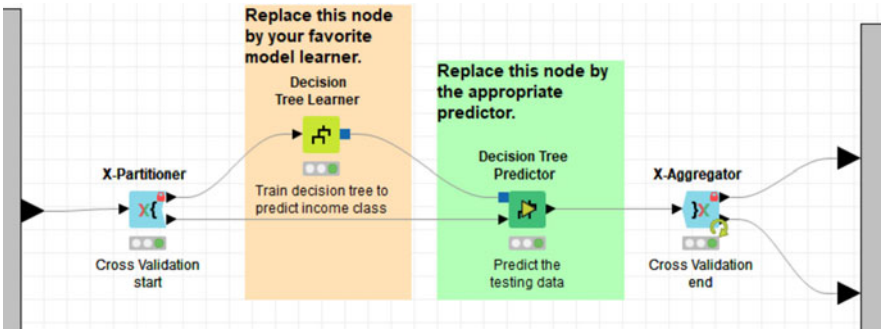


Fig. 5.23 The inside of the Cross Validation node contains the Cross-Validation loop (X-Partitioner node to start and X-Aggregator node to end). The body of the loop contains the Learner (training the model) and the Predictor (applying the model) node. The default model is a decision tree. It can be changed to any other model as long as Learner and Predictor nodes are consistent

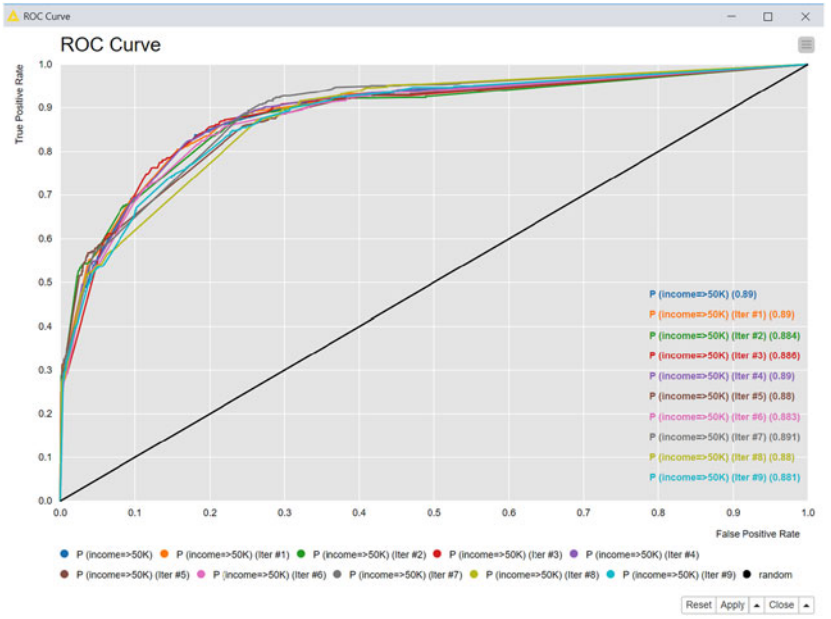


Fig. 5.24 ROC curves for the 10-fold outputs of the Cross Validation node

5.6.2 Scoring Models for Numeric Predictions

The last example workflow shows how to calculate the errors on a numeric prediction task. Again, the workflow reads a data set (the iris data set in this case) and trains a numeric prediction model (a linear regression) to predict a numeric target column (petal width) on a subset of the original data set (the training set). On a different subset (the test set) the linear regression model is applied to generate predictions about

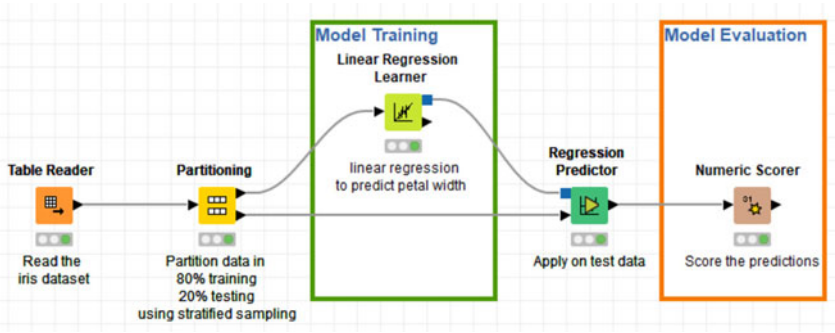


Fig. 5.25 Numeric Scorer node to calculate some common error measures on the test set of a numeric prediction task

Fig. 5.26 Common error measures available in the view of the Numeric Scorer node, shown in Fig. 5.25

Statist...

File

R^2:	0.942
Mean absolute error:	0.134
Mean squared error:	0.035
Root mean squared error:	0.187
Mean signed difference:	-0.011
Mean absolute percentage error:	0.166

the numeric values of petal width for each plant. At the end, the *Numeric Scorer* node calculates some common error measures for numeric predictions (Fig. 5.25). Such error measures are shown in Fig. 5.26.

5.7 Further Reading

The book [8] introduces data mining approaches in a similar view as in this chapter.

A discussion of the problems of bias and errors from the viewpoint of machine learning and statistical learning theory can be found in [11] and [9], respectively.

References

1. Akaike, H.: A new look at the statistical model identification. IEEE Trans. Autom. Control **19**, 716–723 (1974)
2. Chawla, N.V.: Data mining for imbalanced datasets: an overview. In: Data Mining and Knowledge Discovery Handbook, pp. 875–886. Springer, Berlin (2010)
3. Chawla, N.V., Herrera, F., Garcia, S., Fernandez, A.: SMOTE for learning from imbalanced data: progress and challenges, marking the 15-year anniversary. J. Artif. Intell. Res. **61**, 863–905 (2018)
4. Chernick, M.: Bootstrap Methods: A Practitioner's Guide. Wiley, New York (1999)
5. Dietterich, T., Kong, E.: Machine learning bias, statistical bias, and statistical variance of decision tree algorithms. Technical report, Oregon State University, USA (1995)

6. Grünwald, P.: The Minimum Description Length Principle. MIT Press, Cambridge (2007)
7. Hamilton, H.J., Hilderman, R.J.: Knowledge Discovery and Measures of Interest. Springer, New York (2001)
8. Hand, D., Mannila, H., Smyth, P.: Principles of Data Mining. MIT Press, Cambridge (2001)
9. Hastie, T., Tibshirani, R., Friedman, J.: The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd edn. Springer, New York (2009)
10. Ling, C. X., Li, C.: Data mining for direct marketing: problems and solutions. In: Proc. 4th Int. Conf. on Knowledge Discovery and Data Mining (KDD-98). AAAI Press, New York 1998
11. Mitchell, T.: Machine Learning. McGraw-Hill, New York (1997)
12. Schwarz, G.: Estimating the dimension of a model. Ann. Stat. **6**, 461–464 (1978)

Data Preparation

6

In the data understanding phase we have explored all available data and carefully checked if they satisfy our assumptions and correspond to our expectations. We intend to apply various modeling techniques to extract models from the data. Although we have not yet discussed any modeling technique in greater detail (see following chapters), we have already glimpsed at some fundamental techniques and potential pitfalls in the previous chapter. Before we start modeling, we have to prepare our data set appropriately, that is, we are going to modify our data set so that the modeling techniques are best supported but least biased.

The data preparation phase can be subdivided into at least four steps. The first step is data selection and will be discussed in Sect. 6.1. If multiple data sets are available, based on the results of the data understanding phase, we may select a subset of them as a compromise between accessibility and data quality. Within a selected data set, we may concentrate on a subset of records (data rows) and attributes (data columns). We support the subsequent modeling steps best if we remove all useless information, such as irrelevant or redundant data. The second step involves the correction of individual fields, which are conjectured to be noisy, apparently wrong or missing (Sect. 6.2). If something is known, new attributes may be constructed as hints for the modeling techniques, which then do not have to *rediscover* the usefulness of such transformations themselves. For some modeling techniques, it may even be necessary to construct new features from existing data to get them running. Such data transformations will be discussed in Sect. 6.3. Finally, most available implementations assume that the data is given in a single table, so if data from multiple tables have to be analyzed jointly, some integration work has to be done (Sect. 6.5).

6.1 Select Data

Sometimes there is a lot of data available, which sounds good in the first place, but only if the data is actually *relevant* for the given problem. By adding new columns with random values to a table of, say, customer transactions, the table may get an

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_6

impressive volume but does not carry any useful information. Even worse, these additional attributes are considered harmful as it is often difficult to identify them as being irrelevant (we will illustrate this point in a minute). This is the main reason why it is important to restrict the analysis process to potentially useful variables, but other reasons exist such as efficiency: the computational effort often depends greatly on the number of variables and the number of data records. The results of the data understanding phase greatly help to identify useless data sets and attributes, which have to be withheld from the analysis. But for other attributes, things may not be that clear, or the number of attributes that are classified as potentially useful is still quite large. In the following, we investigate how to select the most promising variables (Sect. 6.1.1) and, alternatively, how to construct a few variables that nevertheless carry most of the information contained in all given attributes (Sect. 6.1.2). Having selected the data columns, we finally discuss which data rows should enter our analysis in Sect. 6.1.3.

6.1.1 Feature Selection

Suppose that we have some target or response variable R that we want to predict. For the sake of simplicity, let us assume that we have no data that is relevant for the final outcome of R. What happens if we nevertheless try to build a model that explains or predicts the value of R given some other irrelevant variables? How will it perform? We are going to answer this question following a thought experiment.

Suppose that the probability for a positive response is p. We pick some binary variable A with $P(A = yes) = q$. In this experiment, A has nothing to do with the response R, but usually we do not know this in advance. As R and A are (statistically) independent, we expect the joint probability of observing, say, $A = yes$ and $R = yes$ to be $P(A = yes \wedge R = yes) = P(A = yes) \cdot P(R = yes) = p \cdot q$, as shown in Table 6.1a. Ignoring variable A and assuming $p = 0.4$, having no other information at hand, we would predict the more likely outcome $R = no$. In the long run, this prediction would be correct in 60% ($= p$) of the cases and wrong in 40% ($= 1 - p$).

Next we take A into account with $q = 0.7$. How would this additional knowledge of A influence our prediction of R? When looking at the expected number of records in Table 6.1b, the knowledge of A does not suggest a different decision: in both rows of the table ($A = yes$ and $A = no$) we have $P(R = no) > P(R = yes)$, so the chances of getting $R = no$ are higher, and this would be our prediction. Clearly, the irrelevant information did not help improve the prediction, but at least it did no harm—at the first glance. There is, however, one problem: The *true probabilities* p and q are not known in advance but have to be estimated from the data (introducing a sample error, see Sect. 5.4). Suppose that our data set consists of 50 records as shown in Table 6.1c. Note that, by pure chance, the sample from the whole population slightly differs in our database from the expected situation: only 2 records out of 50 are different (in $A = no$, $R = yes$ instead of $R = no$). Now there is a slight majority for $R = yes$ given $A = no$. If, based on this sample, the prediction of R is changed

Table 6.1 (a) Probability distribution of independent variables. (b) Expected number of cases for a total number of $n = 50$ records. (c) Observed number of cases

(a)

		R		
		yes	no	
A	yes	$p \cdot q$	$(1-p) \cdot q$	q
	no	$p \cdot (1-q)$	$(1-p) \cdot (1-q)$	$1-q$
		p	$1-p$	

(b)

$n = 50$		R		
		yes	no	
A	yes	14	21	35
	no	6	9	15
		20	30	

(c)

$n = 50$		R		
		yes	no	
A	yes	14	21	35
	no	8	7	15
		22	28	

to *yes* given $A = no$, the rate of accurate predictions goes down from 60% to 52%. Obviously, such situations occur easily if the numbers in the table are relatively small. If the data set consists of many more records, say 10,000, the estimates of the joint probabilities are much more robust—as long as the number of cells in our table remains constant. The number of cells, however, increases exponentially in the number of variables; therefore, the number of cases per cell decreases much faster than we can gather new records: To keep the number of cases per cell constant, we would have to double the number of records if we add a single binary variable.

Many methods we will discuss in the following chapters come along with their own weapons against the danger of overfitting, which generally grows with the number of (irrelevant) attributes. By removing irrelevant (and redundant) attributes in advance, the variance of the estimated model is reduced, because we have less parameters to estimate, and the chances of overfitting decrease. In some cases, the application of feature selection methods seems to be more important than the choice of a modeling technique [23].

The goal of **feature selection** is to select an *optimal* subset of the full set of available attributes $\mathcal{A}$ of size n. The more attributes there are, the wider is the range of possible subsets: the number of subsets increases exponentially in the size of $\mathcal{A}$. Feature selection typically implies two tasks: (1) the selection of some evaluation function that enables us to compare two subsets to decide which will perform better and (2) a strategy (often heuristic in nature) to select (some of) the possible feature subsets that will be compared against each other via this measure.

Selecting the *k* Top-Ranked Features If we assume that an *attribute subset* is only as good as the *individual attributes*, a straightforward approach is to use an *attribute evaluation* function to rank the attributes only rather than the full subsets. The selection strategy (step 2) could then be as simple as "*take the k top-ranked features*." This leads to an extremely fast selection process, because the evaluation function has to be evaluated only once per attribute (rather than evaluating all possible subsets once).

If a target variable is given (searching for explanations or predictions), the evaluation function may investigate into the correlation of the considered attribute and the target attribute. For categorical data, Pearson's χ^2-test (see Sect. A.4.3.5) indicates the goodness of fit for the observed contingency table (of target and considered variable) versus the table expected from the marginal observations. A large deviation from the expected distribution points towards a dependency among both

Table 6.2 The full data set consists of 9 repetitions of the four records on the left plus the four records in the middle, which differ only in the last record. To the right, there are contingency tables for all four variables vs. the target variable

	A	B	C	D	Target
	+	+	+	−	*no*
9×	+	−	+	−	*yes*
	−	+	+	−	*yes*
	−	−	−	+	*no*

,

	A	B	C	D	Target
	+	+	+	−	*no*
1×	+	−	+	−	*yes*
	−	+	+	−	*yes*
	−	−	+	+	*no*

A	Target		B	Target		C	Target		D	Target	
	no	*yes*		*no*	*yes*		*no*	*yes*		*no*	*yes*
+	10	10	+	10	10	+	11	20	+	10	0
−	10	10	−	10	10	−	9	0	−	10	20

attributes, which is desirable for predictive purposes. Another evaluation function is the information gain or its normalized variants *symmetric uncertainty* or *gain ratio*, which will be discussed in Sect. 8.1. The information gain itself is known to prefer attributes with a large number of values, which is somewhat compensated by the normalized variants (see also Sect. 6.3.2).

As an example, consider the data set shown in Table 6.2. As we can see from the contingency tables, the values +/− of the attributes A and B are equally distributed among the target values *no*/*yes* and thus appear useless at first glance. When observing $C = -$, the target value distribution *no* : *yes* is 9 : 0, which indicates a very clear preference for *no* and makes C valuable. When observing $D = +$, the target value distribution is 10 : 0, which is even slightly better because it covers one more case. All the mentioned evaluation measures provide the same ranking for this set of variables: D–C–A, B (with A and B having the same rank).

The interesting observation is that in this artificial example the attributes A and B together are sufficient to perfectly predict the target value, which is impossible with attributes C and D, each of them individually ranked higher than A and B. Both C and D are quite good in predicting the target value *no*, but they do not complement one another. By our initial assumption, the evaluation functions considered so far do not take the interaction among features into account but look at the performance of individual variables only. They do not recognize that C and D are almost redundant, nor do they realize that A and B jointly perform much better than individually. This holds for all evaluation functions that analyze the contingency table of individual attributes only.

However, we may arrive at different conclusions if we account for the values of other attributes, say B, C, D, while evaluating a variable, say A, as it is done by the Relief family of measures. We discuss only one member (Relief, see Table 6.3) and refer to the literature for the more robust extensions ReliefF and RReliefF [22]. An advantage of the Relief family is that it is easily implemented for numerical and categorical attributes (whereas many of the aforementioned evaluation measures are only available for categorical data in most data analysis toolboxes).

Table 6.3 Evaluation of attributes—Relief algorithm. The nearest hit or miss is selected upon the same diff function (sum over all attributes). Rather than applying the algorithm to all data, it may be applied to a sample only to reduce the high computational costs of handling big data sets (caused by the neighborhood search)

Algorithm Relief($\mathcal{D}, \mathcal{A}, C) \to w[\cdot]$

input: data set $\mathcal{D}$, $|\mathcal{D}| = n$,
attribute set $\mathcal{A}$, target variable $C \in \mathcal{A}$
output: attribute weights $w[A]$, $A \in \mathcal{A}$

1 set all weights $w[A] = 0$
2 **for all** records $\mathbf{x} \in \mathcal{D}$
3 find nearest hit $\mathbf{h}$ (same class label $\mathbf{x}_C = \mathbf{h}_C$)
4 find nearest miss $\mathbf{m}$ (different class label)
5 **for all** $A \in \mathcal{A}$:
6 $w[A] = w[A] - \frac{\text{diff}(A,\mathbf{x},\mathbf{h})}{n} + \frac{\text{diff}(A,\mathbf{x},\mathbf{m})}{n}$

where

$$\text{diff}(A, \mathbf{x}, \mathbf{y}) = \begin{cases} 0 & \text{if } \mathbf{x}_A = \mathbf{y}_A, \\ 1 & \text{otherwise} \end{cases}$$

for categorical attributes A and

$$\text{diff}(A, \mathbf{x}, \mathbf{y}) = \frac{|\mathbf{x}_A - \mathbf{y}_A|}{\max(A) - \min(A)}$$

for numerical attributes A.

Intuitively, the Relief measure estimates how much an attribute A may help distinguish between the target classes by means of a weight $w[A]$: if $w[A]$ is large, the attribute is useful. The weights are determined incrementally and respect the interaction between the attributes by the concept of "nearest" or "most similar" records, which takes all attributes (rather than just A) into account. Given some record $\mathbf{x}$, if there are very similar *records* $\mathbf{h}$ and $\mathbf{m}$ with the same or different target label, the features present in this record are apparently not of much help in predicting the target value (and the weights $w[A]$ for all attributes A remain unchanged because the positive and negative diff-terms cancel out in line 6). On the other hand, if the most similar record with the same target label is close (first diff-term is small), but the most similar record with a different target label is far away (second diff-term is large), then the weight will increase overall.

In our particular example, since we have duplicated the records in our data set, the nearest hit is almost always identical to the selected record $\mathbf{x}$ itself; therefore the contribution of the first diff-term is zero. If we consider the first two records in Table 6.2, they carry different labels but differ otherwise only in the value of B. This makes B attractive, because it is the only attribute that helps to distinguish between both cases. Overall, the ReliefF ranking delivers $A, B - D - C$.

Selecting the Top-Ranked Subset A computationally more expensive approach is to consider an evaluation function that ranks *attribute subsets* rather than single attributes. This helps to recognize the power of complementing attributes rather than their individual contribution only. The question is how such a function may evaluate the attribute subset. Possible choices include:

Cross-product As a naive solution, we may construct a new attribute as the cross-product of all attributes in the subset and evaluate it by the functions discussed earlier (information gain, gain ratio, etc.). Continuing our example, if the subset contains A and B, we construct an attribute AB with $\Omega_{AB} = \{++, +-, -+, --\}$, which would lead to a perfect prediction

of the target value in our example. This naive approach is only feasible as long as the number of cases per newly constructed feature remains relatively high, otherwise we risk overfitting.

Wrapper Once we have methods available to learn models automatically from data (see the following chapters), we may evaluate the subset by the performance of the derived model itself: For each subset, we build a new model, evaluate it, and consider the result as an evaluation of the attribute subset. This is known as the so-called **wrapper approach** to feature selection. In order to avoid overfitting of the training data, we must carry out the evaluation on hold out validation data.

Filter based on correlations As we have already mentioned, we may remove both irrelevant attributes (carrying no information) and redundant attributes (information is already carried by other attributes). So far we have focussed on the former by investigating how much information a given attribute contains with respect to the target class (which should be close to zero in case of irrelevant attributes). To identify redundant attributes, we may apply the same evaluation function on attribute pairs: if one attribute tells us everything about another, they are obviously redundant, and one of them may be skipped. This is the idea of the **correlation-based filter** (CFS) [12], where a set of features is evaluated by the following heuristic:

$$\frac{k\bar{r}_{ci}}{\sqrt{k + k(k-1)\bar{r}_{ii}}},$$

where $\bar{r}_{ci}$ is the average correlation of the target attribute with all other attributes in the set, whereas $\bar{r}_{ii}$ is the average attribute–attribute correlation between different attributes (except the target) and k denotes the number of attributes. The evaluation function becomes the larger, the better the attributes correlate with the target attribute and the less they correlate with each other.

Filter based on missing values Another simple measure for the information (or lack thereof) carried by an attribute of the data set is the **ratio of missing values**. Attributes with too many missing values are unlikely to carry much useful information. Thus, attributes with a ratio of missing values greater than a given threshold can be removed. The lower the threshold, the more aggressive the reduction.

Filter based on variance Attributes with little changes in the data carry little information (a constant attribute is completely useless). Thus, another strategy is to remove attributes with a variance lower than a given threshold (**Low Variance Filter**). Notice that the variance depends on the column range, and therefore normalization is required before applying this technique.

Once a subset evaluation function has been found, a strategy to select subsets from the full set of attributes has to be found. Two very simple standard procedures are **forward selection** and **backward elimination** [11]. In the former, one starts

with a subset that does not use any attributes (and a model must always predict the majority class) and then adds attributes in a greedy manner: in each step the attribute that most improves the subset performance is added. In backward elimination the process is reversed: one starts with a subset that uses all available attributes and then removes them in a greedy manner: in each step the attribute whose removal most improves the subset performance is eliminated. The process stops if there is no attribute that can be added (forward selection) or removed (backward elimination) without actually worsening the model performance. An exhaustive search over all possible attribute sets is usually prohibitive, but at least possible, if only a limited number of attributes is available. Even a random set generation is possible (Las Vegas Wrapper).

The data from Table 6.2 evaluated by a forward selection wrapper with a standard decision tree learner selects the attribute D only. Exhaustive search of all subsets identifies the subset A, B, D as the best. In both cases, attribute D is included because of the preference for attributes that correlate directly with the target variable, which is built in with most decision tree learners. On the one hand, having identified the optimal subset does not help if the learner is not capable of constructing the right model out of it (because of representational limitations); therefore it is advantageous to use the method that builds the model to evaluate the attribute subsets directly. On the other hand, if we intend to test different techniques later, the optimal subset of one algorithm is not necessarily the smallest or optimal subset for another. Using exhaustive search together with the subset evaluation function that considers the cross product, the *minimal optimal subset* A, B is identified.

6.1.2 Dimensionality Reduction

Principal component analysis (PCA) has already been introduced in Sect. 4.3.2.1 in the context of data understanding. PCA was used for generating scatter plots from higher-dimensional data and to get an idea of how many intrinsic dimensions there are in the data by looking at how much of the variance can be preserved by a projection to a lower dimension. Therefore, PCA can also be used as a dimension-reduction method for data preparation. In contrast to feature selection, PCA does not choose a subset of features or attributes, but rather a set of linear combinations of features. Although this can be very efficient in certain cases, the extracted features in the form of linear combinations of the attributes are often difficult to interpret, so that the later steps will automatically result in a black-box model.

Similar to PCA, **linear discriminant analysis** (**LDA**, cf. Sect. 8.2.3.2) can be used as a dimension reduction method for classification tasks. The idea behind LDA is to find a line (or hyperplane in higher dimensions) that separates two classes from each other. The signed distance to the separating line (or hyperplane) is then used for classification (negative sign for one class, positive sign for the other). The separating hyperplanes are called linear discriminant functions. When using LDA for dimensionality reduction, all numeric attributes in the data set are projected onto these linear discriminant functions; effectively moving the data set from the original

n-dimensional to a reduced m-dimensional, with $m \leq K-1$ where K is the number of classes. The maximum number of reduced dimensions m is the number of classes K in the target attribute minus one.

This signed distance may also be used as a new attribute, which is (just as with PCA) a linear combination of the original features (but optimized for a class separation task this time). Multiple features may be established in this way in case of more than two classes. Again the price to pay for an increased efficiency in the data analysis is the loss of interpretability for the new attribute set.

PCA and LDA belong to a more general class of techniques called **factor analysis**. Factor analysis aims at explaining observed attributes as a linear combination of unobserved attributes. In PCA, the (first) principal components are the corresponding factors. **Independent component analysis** (**ICA**) is another method to identify such unobserved variables that can explain the observed attributes. In contrast to factor analysis, ICA drops the restriction to linear combinations of the unobserved variables.

Dimension-reduction techniques that construct an explicit, but not necessarily linear mapping, from the high-dimensional space to the low-dimensional space are often called **nonlinear PCA**, even though they might not aim at preserving the variance like PCA, but rather the distances like MDS. Examples for such approaches can be found in [14, 15, 20, 21].

6.1.3 Record Selection

Except for very rare occasions, the available data is already a sample of the whole population. If we had started our analysis a few weeks earlier, the sample would have looked different (smaller, in particular) but probably still representative for the whole population. Likewise we may create a subsample from our data and use it for the analysis instead of the full sample, e.g., for the sake of faster computation, the need for a withhold test data set (see Sect. 5.5.1), or the need for a rebalanced class distribution (see Sect. 5.5.5). Other reasons for using only a subsample include:

- *Timeliness* If we want to draw conclusions to take actions in the future, it is advisable in a changing world to primarily use recent data rather than data that is already several years old. Explicit or implicit time stamps in databases are of great help in selecting the most recent data.
- *Representativeness* Sometimes the population of interest is different from the population available in our database (e.g., applicants that are rejected early are not recorded in the database). By choosing underrepresented cases more frequently than overrepresented, this imbalance may be compensated and lead to a better representativeness in the sample. On the other hand, if the data is already representative and the subsample is relatively small, chances rise that the probability distribution of some attribute of interest becomes quite different in the subsample compared to the full data set. Stratified sampling takes care of this problem (see Sect. 5.5.1).

- *Rare events* If we want to predict some event that occurs only rarely, a remarkably good performance is achieved by simply predicting that the event will not occur (which is true most of the time but obviously does not help very much). As many methods are sensitive to this kind of problem, the cost function has to be chosen appropriately, e.g., by adapting the misclassification costs (see Sect. 5.2.1). Alternatively, the number of rare cases may be increased artificially by sampling them with a higher probability. We have discussed problems with unbalanced class distributions already in Sect. 5.5.5.

How large does the subsample have to be? The smaller the subsample, the less likely it is representative for the population. The more complex a model is, the more parameters have to be estimated, and the more data is necessary to achieve robust estimates. But on the other hand, if things seem to converge after observing several thousand cases, considering more and more data does not add any further value. In some cases, when the parameters to be estimated can be represented as a sum (like the mean), a theoretical bound for the size of the subsample can be provided: In case of the mean value $\bar{x}$ of all x_i, $1 \leq i \leq n$, and a given (almost certain) variable range $[a, b]$, that is, $x_i \in [a, b]$, the probability that the mean estimate $\bar{x}$ deviates from the true value $E[x]$ only by ε is given by

$$P\left(\left|\bar{x} - E[x]\right| \geq \varepsilon\right) \leq 2\exp\left(-\frac{2n\varepsilon^2}{(b-a)^2}\right),$$

which is known as the Hoeffding inequality. For some desired probability, the number of required samples can be derived directly. However, this conservative estimate requires quite a large number of samples.

The choice of the modeling technique is always a compromise of interpretability, runtime, stability, etc. (see Sect. 3.3). If runtime becomes a problem, why not reduce the data set to those cases that are really important for the modeling technique? Why not remove the bulk of highly similar, uninformative records and just keep a few representatives instead? This **instance selection** works surprisingly well for many data mining tasks. If class boundaries exist in a classification task (see Sect. 5.4), the instances that influence the class boundaries most are usually those close to the boundaries—we get the same resulting classifier if some of the *interior* cases are removed. This is exploited by classifiers called support vector machines (Sect. 9.4). Instance selection for clustering tasks often involves some condensation or compression by choosing one record that represents a number of very similar records in its neighborhood. Often the number of records it represents is included as an additional *weight* attribute, which gets higher the more data it substitutes. Many clustering algorithms (such as k-means clustering, Sect. 7.3) are easily extended to process this additional weight information appropriately. The instance selection can be considered as some kind of preclustering as it identifies representative cases that may stand for a small group of data (a small cluster). The small clusters may then be aggregated further in subsequent steps.

6.2 Clean Data

6.2.1 Improve Data Quality

Data scrubbing involves the correction of simple errors, such as inconsistencies, spelling mistakes, spelling variants, abbreviations, varying formats (time, date, currency, phone). Such distorted values are frequently called **noise**. We have to correct these values in order to recognize differently spelled but semantically identical values as such (which includes text entries and numerical values). Appropriate steps include:

- Turning all characters into capital letters to level case sensitivity;
- Removing spaces and nonprinting characters;
- Fixing the format of numbers, date, and time (including decimal point, sign, currency signs). Statistical methods have been used in the data understanding phase to identify potential outliers in numerical fields (e.g., limit the deviation from mean value to 2–3 standard deviations), which may indicate a missed decimal point resulting in a much too high value. Such errors are easily corrected, and others may require careful inspection;
- Splitting fields that carry mixed information into two separate attributes, e.g., "*Chocolate, 100 g*" into "*Chocolate*" and "*100.0*". This is known as **field overloading**;
- Using a spell-checker or stemming[1] to normalize spelling in free text entries;
- Replacing abbreviations by their long form (with the help of a dictionary);
- Normalizing the writing of addresses and names, possibly ignoring the order of title, surname, forename, etc., to ease their reidentification;
- Converting numerical values into standard units, especially if data from different sources (and different countries) are used;
- Using dictionaries containing all possible values of an attribute, if available, to assure that all values comply with the domain knowledge.

While humans are usually very good in spotting and correcting noise, the problem is that noise may occur in many variants which makes an automatic recovery very difficult. It is therefore extremely helpful to define rules and patterns which the *correctly spelled values* have to match (e.g., regular expressions for text and valid ranges for numerical values), so that the data can be filtered and conversion rules may be defined case-by-case whenever a new rule violation occurs. If additional data will be considered later, these rules prevent us from repeating a manual analysis over and over again.

While the steps mentioned so far were concerned with single records only, other errors may be recognized only if multiple records are investigated. The most prominent example is the discovery of duplicate records, e.g., in the table of customers

[1] The reduction of inflected or derived words to their root (or stem) is called stemming. So-called *stemmers* are computer programs that try to automate this step.

when creating customer profiles. This problem is known as **record linkage** or **entity resolution**. Methods for detecting duplicates utilize some measure of similarity between records (which will be discussed in Sect. 7.2); two records that are highly similar are considered to be variants of the same entity and are merged into a single record. The greatest difficulty to avoid is when two different entities are mistakenly merged (false positive).

It is always a good idea to document changes, such as the correction of an outlier, because it may turn out later that the entry is not defective at all but points out some rare, special case. An artificial attribute *degree of trustworthiness* may be introduced, which is 1.0 by default but decreased whenever some correction is applied to the record that involves heuristic guesswork, be it outlier correction or missing value treatment, which will be discussed in the next section. Methods which are numerical in nature frequently offer the possibility to take an additional weight into account, such that suspicious, less trustworthy records get less influence on the resulting model (see also robust regression and M-estimation in Sect. 8.3.3.3).

Correcting all possible errors is a very time-consuming task; and if the data turns out to be useless for the task at hand, it is definitely not worth the effort. If potentially useful attributes have been identified early, efforts in data cleaning can be geared towards those attributes that pay back.

6.2.2 Missing Values

Why is it necessary to treat missing data, anyway? If the data is not in the database, this is because we do not know anything about the true value—so there is not much we can do about it—besides *guessing* the true value, but this seems to be even worse than keeping the missing data, because it almost certainly introduces errors.

The reason why we nevertheless may want to change the missing fields is that the implementations of some methods simply cannot deal with empty fields. Then the imputation of estimated values is a simple mean to get the program run and deliver some result, which is often, although affected by estimation errors, better than having no result at all. Another option would be to remove records with missing values completely, so that only complete records remain, and the program does not run into any problems either. In case of nominal attributes there is a third possibility: we may simply make the missing value explicit by adding another possible outcome, say `MISSING`, to its range. Let us consider all these options individually.

Ignorance/Deletion In **case deletion** all records that contain at least one missing value are removed completely and are thus not considered in subsequent steps. Although this procedure is easy to carry out, it also removes a lot of information (from the nonempty attributes), especially if the number of attributes is large. It may be safely applied in case of missing completely at random (see Sect. 4.6), as long as the remaining data set is still sufficiently large. As soon as we deal with missing at random (MAR), we may start to seriously distort the data. To continue the example from Sect. 4.6, where missing temperature values due to empty batteries are less likely to get fixed when it is raining, case deletion will remove more

rainy, cold days than sunny, warm days, which may lead to biased estimates elsewhere. A removal of records may therefore threaten the representativeness of the data set.

Imputation The missing values may be replaced by some estimate. As a trivial example consider an attribute "*number of pregnancies*" in a database, which is for men either 0 or missing (in the sense of "not applicable"). In such cases *we know* that the true value must be 0 and may impute it safely. For arbitrary attributes, perhaps the most simple (but not advisable) guess is the mean (for numerical data) or the mode (for categorical data). As such a guess clearly does not add any new information to the data, it should not do any harm either: it should leave the characteristics of the data set unaltered. Indeed, by replacing all missing values of one variable by the mean of the measured values, the mean of the modified set will remain unchanged—however, its variance will decrease. This replacement procedure affects the variance of the attributes to different degrees (depending on the portion of missing values). Of course, it is also possible to impute a value that preserves the variance (but affects the mean). In one way or the other, any imputed value will affect some characteristics of the known data set. To decide which imputation is better, it is useful to know which statistic will play an important role in the subsequent steps, and this statistic should be least distorted.

If the values are missing at random (but not completely at random), other attributes can provide hints towards the true value. If the horsepower value is missing in a car database, we would estimate quite different values for a compact car and an off-road vehicle. If an attribute such as fuel consumption or engine displacement are given, their correlation to horsepower can be exploited to achieve better estimates. How to develop models that help us to predict numerical values (regression) or categorical values (classification) will be discussed in Chaps. 8 and 9.

Replacing all missing values with the very same constant number always reduces the variability of the data set, which often influences derived values such as correlation coefficients or fitting errors. This is less pronounced with substitutions that come from a predictor, but the problem remains. Adding a random value to each imputed value is not a general solution but helps reduce the effects.

Explicit Value or Variable A very simple approach is to replace the missing values by some new value, say `MISSING`. This is only possible for nominal attributes, as any kind of distance to or computation with this value is undefined and meaningless. If the fact that the value is missing carries important information about the value itself (nonignorable missing values), the explicit introduction of a new value may actually be advisable, because it may express an intention that is not recoverable from the other attributes. If we suppose that the absence of the value and the intention correlate, we may luckily capture this situation by a new constant, but the problem is that we cannot assure this from the data itself. If the values are missing completely at random, there is no need to introduce a new constant. The cases that are marked by a constant do not exhibit any deviation from those cases that are not marked. If we introduce it nevertheless, the models will have to deal with

Table 6.4 Treatment of missing values

Treatment of missing values for ...	
Operational reasons	
Case deletion	Removal of all records where missing values occur, only MCAR
Imputation	Estimate the missing value (MAR, with help from other attributes)
Global constant	Turn all missing values into a special, newly introduced global constant
Data quality reasons	
Harmonize	Select a single representation for missing values and change all other representations to it
Separate	Use background knowledge and other attributes to distinguish between *there is a real value, but it is unknown* and *there is no value (yet)* (e.g., grade in exams: unknown grade because the test got lost vs. absence of grade because student was ill)
Lossless preparation	
Construct data	Construct new attributes that indicate imputation, as this may carry important information itself

an additional, useless value, which makes the models unnecessary complicated and estimates less robust.

A better approach is to introduce a new (binary) variable that simply indicates that the field was missing in the original data set (and then impute a value). In those cases where neither the measured nor the estimated values really help, but the fact that the data was missing represents the most important bit of information, this attribute preserves all chances of discovering this relationship: the newly introduced attribute will turn out to be informative and useful during modeling. On the other hand, if no such missing value pattern is present, the original variable (with imputed values) can be used without any perturbing `MISSING` entries. A summary of the described techniques to deal with missing values can be found in Table 6.4.

6.2.3 Remove Outliers

Outliers can be a serious issue when training machine learning algorithms or applying statistical techniques. They are often the result of errors in measurements or exceptional system conditions and therefore do not describe the common functioning of the underlying system. Depending on their number, they can negatively influence the machine learning model till force it to learn useless or even wrong information. Indeed, it is best practice to implement an outlier removal phase before proceeding with further analysis.

There are many techniques to detect and optionally remove outliers from a data set. We have discussed outlier detection in Sect. 4.5 already. As outliers are supposed to be rare, a removal should influence the result much less than a wrong value does. So we may drop the case from the data set completely or remove the suspicious values only (but keep the case—and optionally impute a new value, cf. page 138).

This choice also depends on the envisaged modeling techniques—some can handle missing values particularly well (then there is no need to remove the full case), while others require complete cases. If the data understanding has generated insights why the outliers occurred, some other problem-specific solution may be found to recover the original value.

6.3 Construct Data

By now we have selected and cleaned the data; next we consider the transformation of existing and construction of new attributes that may replace the original attributes. There are at least three reasons for this step: (1) the tool implementations require a transformation because they will not work otherwise, (2) in absence of any background knowledge, we want to make sure that every attribute will play the same role in the model, and no particular attribute is preferred over or has larger influence than another, (3) given some domain knowledge, we want to give helpful hints to the modeling techniques to improve the expected results.

6.3.1 Provide Operability

Scale Conversion If the modeling algorithm handles only certain types of data, we have to transform the respective attributes such that they fit the needs of the modeling tool. Some techniques assume that all attributes are numerical (e.g., regression in Sect. 8.3, neural networks in Sect. 9.2), so we either ignore the categorical attributes or have to apply some transformation first. Simply assigning a numerical value to each of the possible (categorical) values is not an option because typical operations on numerical values (such as averaging or comparison) are not necessarily meaningful for the assigned numerical values. A typical solution is to convert nominal and ordinal attributes into a set of binary attributes, as it will be discussed in Sect. 7.2.

Other methods prefer categorical attributes (like Bayesian classifiers[2] in Sect. 8.2) or perform superior when a discretization is carried out beforehand [5, 24]. **Discretization** means that the range of possible numerical values is subdivided into several contiguous, nonoverlapping intervals and a label of the respective interval replaces the original value. There are many (more or less intelligent) ways to define the boundaries of the intervals. An **equi-width** discretization selects the boundaries such that all intervals are of the same width (see page 41 for selecting a reasonable number of bins). As we can see from Fig. 6.1(a) for the sepal length attribute of the iris data set, it may happen that some intervals contain almost no data. An **equi-frequency** discretization tries to assure that all intervals contain the same number of data objects. Of course, an equally distributed frequency is not always achievable. For example, if the data set with 100 cases contains only 4 distinct numerical values

[2] Bayesian classifiers can handle numerical data directly by imposing some assumptions on their distribution. If such assumptions cannot be justified, discretization may be a better alternative.

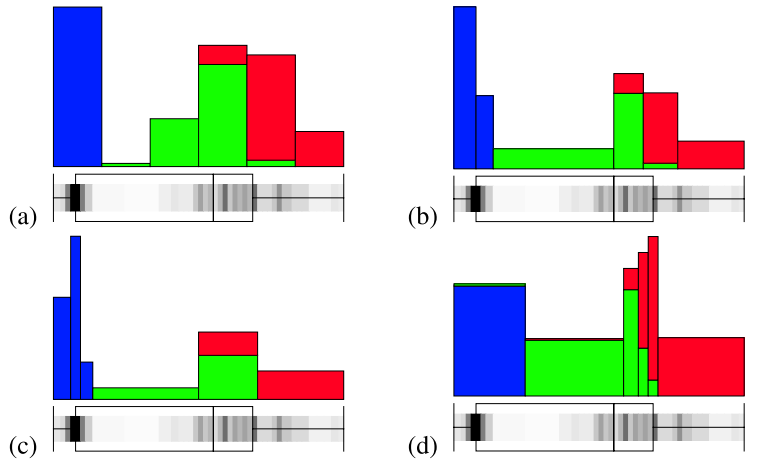


Fig. 6.1 Results of different discretization strategies of the sepal length attribute of the iris data set. The class distribution of the three iris flowers is shown by the *stacked bars*: (**a**) equi-width partition, (**b**) equi-frequency partition, (**c**) V-optimal partition, and (**d**) minimal entropy partition

and a partition into 5 intervals is desired, there are not enough split points. Again, such a partition does not necessarily represent the underlying data distribution appropriately: in the partition of the sepal length attribute in Fig. 6.1(b), the compact group of similar data (left) is separated into two intervals; in particular, the region of highest density is split into two intervals.

The discretization problem is essentially the same as the histogram problem (mentioned already in Sect. 4.3.1). A so-called **V-optimal** histogram minimizes the quantity $\sum_i n_i V_i$, where n_i is the number of data objects in the ith interval, and V_i is the variance of the data in this interval. Minimization of variances leads to intervals with a roughly uniform data density, as can be seen in Fig. 6.1(c), which makes it quite similar to clustering (Sect. 7.3) where we also search for areas of higher data density to define compact groups of similar data. Clustering algorithms may therefore also be used to derive partitions for discretization.

If additional information is available (such as the labels of some target variable as in classification tasks), it can provide valuable hints for finding a discretization that supports the classification task best. Note that the plots in Fig. 6.1(a)–(c) already show this class label information but that it has not been used by the discretization so far. We will discuss the idea of discretization via entropy minimization in the context of binary splitting in Sect. 8.1, which can be extended to an arbitrary number of splits (or intervals) [6, 7]. The underlying ideas are similar to those of the V-optimal histogram, but now we evaluate a partition by the (im)purity of its intervals: If an interval contains only cases with the same target label, such an interval appears informative with respect to the target variable. On the other hand, if the distribution of the target variable within one interval is identical to the overall distribution, such an interval is useless for the prediction. Just as in feature selection, we may now select the partition that utilizes the classification task most, that is, leads to

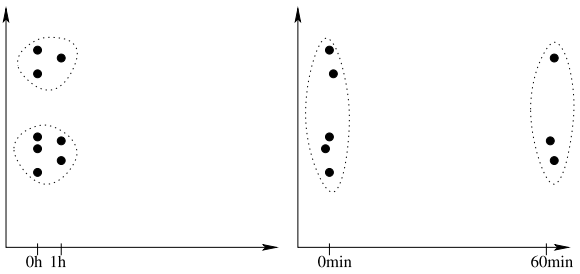


Fig. 6.2 Distance measures are influenced by scaling factors (here, hours vs. minutes)

the purest distribution of target values. This is the same objective we had in feature selection, so that we can reuse the evaluation measures mentioned there (such as information gain, gain ratio, Gini index, etc.). The partition that evaluates best among all possible partitions with k intervals can be found quite efficiently (see, e.g., [6]). An example for the sepal length attribute is shown in Fig. 6.1(d). Having fixed the number of intervals to 6, note that most of the data are already covered by three pure bins, whereas the remaining three bins are located where the class distribution is ambiguous.

Dynamic Domains We have already tackled the problem of dynamic domains with nominal attributes in Chap. 4. In order to apply our models to future data with dynamic domains, we must be prepared to handle values unseen before. One solution is to introduce a new variable, which is an abstraction of the original one. By mapping the original values to more abstract or general values, we can hope that newly occurring entries can also be mapped to the generalized terms so that the model remains applicable. In a data warehouse such a granularization scheme may already be installed, otherwise we have to introduce it, and install means that newly arriving values will be mapped into the hierarchy first.

Problem Reformulation Transformations also help reformulate a problem so that the range of applicable techniques is enlarged. The logit-transformation maps the unit interval to $\mathbb{R}$, enabling standard linear regression techniques to be used for (binary) classification problems (see Sect. 8.3.3.1). Some classification techniques handle binary class variables only, so that multiclass attributes have to be transformed first (e.g., one class against all others).

6.3.2 Assure Impartiality

Another reason for transforming attributes is to ensure that the influence or importance of every attribute is a priori the same. This is not automatically the case, as the next examples will show.

When looking for two natural groups in the two data sets of Fig. 6.2 (which is the task of clustering, see Chap. 7), most people would probably suggest the groups indicated by dotted lines. The rationale for both groupings is that the elements within

each group appear to be more similar than elements of other groups. However, any notion of similarity or distance strongly depends on the used *scaling*: Both graphs in Fig. 6.2 actually display the same data set, only the scaling of the x-axis has been changed from hour to minute. If the distance between the elements is taken as an indication of their similarity, a different scaling may change the whole grouping. In practical applications with considerable background knowledge, one may be in the comfortable position to conclude that a difference of 1.0 in attribute A is as important or relevant as a difference of 2.5 in attribute B. In such a case, all attributes should be rescaled accordingly so that in the rescaled variables a distance of 1.0 is of equal importance in any attribute. In absence of such knowledge, typical transformations include the min–max **normalization** and the **z-score standardization**.

- **min–max normalization** For a numerical attribute $X \in \mathcal{A}$ with $\min_X$ and $\max_X$ being the minimum and maximum value in the sample, respectively, the min–max normalization is defined as

$$n : \operatorname{dom} X \to [0, 1], \qquad x \mapsto \frac{x - \min_X}{\max_X - \min_X}.$$

- **z-score standardization** For a numerical attribute X with sample mean $\hat{\mu}_X$ and empirical standard deviation $\hat{\sigma}_X$, the z-score standardization is defined as

$$s : \operatorname{dom} X \to \mathbb{R}, \qquad x \mapsto \frac{x - \hat{\mu}_X}{\hat{\sigma}_X}.$$

- **robust z-score standardization** As the sample mean and empirical standard deviation are easily affected by outliers, a more robust alternative is to replace the mean by the median $\tilde{x}$ and the standard deviation by the interquartile range IQR_X (difference between third and first quartile, see also box-plots in Sect. 4.3.1):

$$s : \operatorname{dom} X \to \mathbb{R}, \qquad x \mapsto \frac{x - \tilde{x}}{IQR_X}.$$

- **decimal scaling** For a numerical attribute X and the smallest integer value s larger than $\log_{10}(\max_X)$, the decimal scaling is defined as

$$d : \operatorname{dom} X \to [0, 1], \qquad x \mapsto \frac{x}{10^s}.$$

While it may be considered as advantageous if the resulting range is the unit interval $[0, 1]$, the min–max normalization and the decimal scaling are very sensitive to outliers: even a single outlier can force the majority of the data to concentrate in a very small interval (and thus being very similar in comparison to other cases). This effect is less prominent in z-score normalization, and the robust z-score is unaffected by a moderate number of outliers.

Just as distance measures are used to capture the similarity of data, other measures are used for different purposes, such as the evaluation of attributes (see Sect. 6.1.1 on feature selection). Those measures try to identify how much an attribute contributes to the prediction of a given binary target attribute. Suppose that

we have 100 cases in the database and three attributes with 2, 10, and 50 values, respectively. For the sake of simplicity, let us assume that none of these attributes can help in predicting the target attribute and that the values are equally likely. It is probably easy to see that the 2-valued attribute cannot help very much in predicting the target value, but what about the 50-valued attribute? On average, we have only two cases in our database where the same value will occur. If (by chance) these two cases carry the same target value, some algorithm may find it easy to predict the target value (at least in this case). As we have discussed in Sect. 5.4, as the number of cases gets smaller and smaller, the danger of memorizing (overfitting) rises. Of course, this effect takes place gradually, and it will already be recognizable with the 10-valued attribute but to a lesser extent. Some measures (such as information gain) are sensitive to such effects and thus implicitly prefer attributes with a larger number of values (just because memorizing is then much easier). A model, such as a decision tree, is very likely to evaluate poorly if such a variable is chosen. Rather than removing such attributes completely, their range of values should be reduced to a moderate size. This corresponds to selecting a coarser **granularity** of the data. For instance, the very large number of individual (full) zip-codes should be reduced to some regional or area information. The website access timestamp (which even includes seconds) may be reduced to day-of-week and a.m./p.m. The binning techniques mentioned earlier for discretizing numerical attributes may also be helpful. If there is useful information contained in these attributes, it is very likely that it is retained in the coarsed representation, but the risk of overfitting is greatly reduced.

Another typical assumption is that some variables obey a certain probability distribution, which is not necessarily the case in practice. If the assumption of a Gaussian distribution is not met, we may transform the variable to better suit this requirement. This should be a last resort, because the interpretability of the transformed variable suffers. Typical transformations include the application of the square root, logarithm, or inverse when there is moderate, substantial, or extreme skewness. (If the variable's domain includes the zero, it has to be shifted first by adding an appropriate constant before applying the inverse or logarithm.) Another choice is the **power transform**: Given the mean value $\bar{y}$, the power transform

$$y \mapsto \begin{cases} \frac{y^\lambda - 1}{\lambda \bar{y}^{\lambda-1}} & \text{if } \lambda \neq 0, \\ \bar{y} \log y & \text{if } \lambda = 0, \end{cases}$$

transforms the data monotonically to better approximate a normal distribution. The parameter λ is optimized for the problem at hand, e.g., such that the sum of squares of residuals in regression problems is minimized.

6.3.3 Maximize Efficiency

As already mentioned in Sect. 6.1.1, the performance of many standard learning algorithms, like those we will discuss in the following chapters, degrades if irrelevant or redundant attributes are supplied. The purpose of feature selection is the identification and removal of such variables to prevent a performance decline. Sometimes

the attributes do contain all necessary information, but it is hard for the chosen modeling technique to extract this information from the data. As an example, suppose that the chances for some diagnosis are especially high if at least n out of m symptoms can be observed. In case of 4 available observations a, b, c, and d, this would mean that any combination of two of these symptoms can be taken as an indication of this diagnosis (typical case in the medical domain [17]). Is this relationship difficult to capture for a given modeling technique? Let us consider two possible models, a hyperplane model and a rule-based model. If the four symptoms are modeled by binary 0–1 values (absence–presence), the diagnosis corresponds to the following condition:

$$a + b + c + d \geq 2,$$

which is already the definition of the respective hyperplane: the hyperplane $a + b + c + d = 2$ divides the space into two half-spaces, on one side the sum is less, and on the other side the sum is larger than 2. If a new case lies on the hyperplane or the latter side, it satisfies the "*at least two symptoms*" condition. However, a rule-based learner delivers rules such as

if (a and b) or (a and c) or (a and d) or ... **then** diagnosis $= \cdots$.

Apparently, the rule-based model requires to include all possible combinations of 2 out of 4 symptoms, which makes the rule quite lengthy and more laborious to discover. Regardless of the chosen model and thus the model representation, it will always be well-suited for *some* real-world problems and perform poorly on others.

Accepting this fact, the idea of feature construction is to overcome the limits of the representation (representational bias) by offering new features that were otherwise difficult to represent by the original model. Such features include:

- **Derived features** Given some background knowledge, new attributes may be defined from existing attributes, such as *average speed* from *travel time* and *travel distance*. If the true relationship in the data depends on some threshold on the average travel speed but the model requires to condition on the individual attributes, the exact relationship may only be approximated by various pairwise conditions on the individual variables (see Fig. 6.3). Such models will perform worse and are difficult to interpret.
- **Change the baseline** For an important variable to stand out clearly, selecting the right baseline is often crucial. For instance, to estimate the spending power "*income per household*" (with a thoughtful definition of the size of a household) may outperform "*income per head*."
- **Grouping** If it is conjectured that the data form some natural groups or clusters, then several attributes may strongly correlate within the same group, but not necessarily across groups. Knowing to which group a particular case belongs to can be helpful to find a concise model. In such cases, a cluster analysis may be carried out first, and the resulting cluster labels are added as a new attribute. (Make

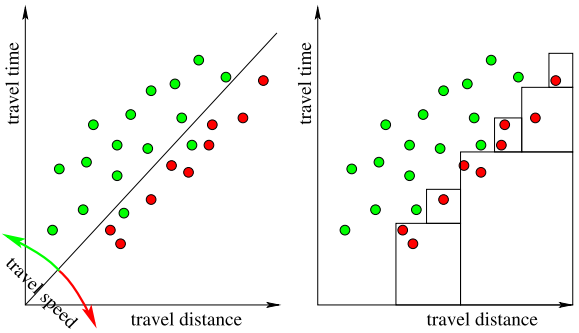


Fig. 6.3 Simple example for the usefulness of derived features: for a number of journeys, the travel time and distance are shown; the color indicates whether the driver was ticketed or not. Discriminating both classes with axis-parallel rectangles is laborious, but easy with a new attribute for travel speed

sure that the cluster analysis uses only attributes that will also be available for new data when applying the model.)

- **Hyperplanes** A linear combination of various attributes may define a new numerical attribute (where a threshold has still to be found by the learner) or, if a threshold is already supplied (cf. the example above), a binary attribute. A useful example is a binary attribute that becomes true if at least (or exactly) n out of m preselected features hold. Such conditions are hard to identify for models that work on the symbolic level (such as rules).
- **Level of granularity** When looking for interdependencies in the data, it is a priori not clear on which level of granularity they may unveil. Grouping the values of a variable to meaningful aggregates or even a hierarchy or taxonomy would allow us to test for dependencies at various levels of detail.
- **Nonlinearities** A linear model $y = \alpha x + \beta y + \gamma$ is linear in its parameters α, β, and γ, which is where the name *linear model* comes from, and also linear in x and y. If it is known or conjectured that the true dependencies *are not linear in x or* y, the input data may be transformed beforehand: new attributes x^2 and y^2 may be constructed, and a more complex linear model $y = \alpha x + \beta y + \gamma x^2 + \delta y^2 + \varepsilon$ can be built.
- **Add metadata** Attributes that indicate where we have altered the data (either because it was missing or because we supposed it was an outlier) can be helpful in situations where our assumptions were wrong (the data is not missing completely at random).

New attributes are often constructed by applying arithmetic or logic operators to existing attributes. Instead of deriving them manually with the help of background knowledge, it is in principle also possible to automatically *search* for helpful attributes by constructing them randomly or heuristically and identifying the best generated features. This idea has been successfully applied in the context of decision tree learning [16].

We have seen in Sect. 5.4.5 that the model error can be decomposed into machine learning bias and variance. The variance is caused by sample errors and causes overfitting. In principle, the sample error can be reduced by acquiring a larger data set (or by feature selection, as discussed before). The bias is a systematic error that expresses a lack of fit between the model and the real-world relationship. In this sense, feature construction can help to reduce the limitations of the learning bias by giving it some new direction. However, we must be aware that the introduction of newly constructed attributes may also simultaneously increase the sample error again (see [23] for a case study). If attributes are constructed that take a large number of different values, an increase of the sample error is inevitable. But it is also possible to construct features that reduce the sample error: whenever we condense information (from multiple attributes) to get a coarser view, the robustness of our estimates increases. Examples for such transformations are aggregations (one attribute *total number (or price) of equipment* rather than many individual attributes for each type of equipment), introducing hierarchies (one attribute *fruit juice* rather than individual attributes for *apple juice*, *orange juice*, etc.), or grouping. Although we have discussed feature selection first, this does not mean that we have to apply the same principles and requirements to our newly constructed attributes.

6.4 Complex Data Types

In this book we mainly focus on simple data types such as numerical and nominal attributes. However, the majority of data collected has much more structure: there are archives of documents, images, and videos. There are databases containing molecular structure and gene sequences. And there are, of course, the increasing number of network-style information repositories. It would be outside of the scope of this book to discuss the analysis of such complex data types in detail, but in almost all cases one of two approaches is used: either the complex data is somehow transformed into a simpler representation which can be used to apply standard data analysis methods to, or existing methods are adopted to deal with specific types of more complex data types. The following brief overview does not even attempt to be a complete list of complex data types but rather tries to show a few prominent examples of these two approaches.

Text Data Analysis Textual data is somewhat difficult to represent in a data table. Among the possible representations, the vector model is most frequently used. The individual words are reduced to their stem and very frequent and very rare (stemmed) words are removed. The remaining words are then represented by a number of boolean attributes, indicating that the word occurred in the text or not. This approach is also known as bag-of-words representation because it is usually easier to simply enumerate the words contained in the vector instead of storing the entire (usually very sparse) vector itself. Once we have reduced texts/documents to such a vector-based representation, we can apply the methods presented later in the book without change. A good summary can be found in [8]. Under the umbrella of information retrieval, a lot of work has been undertaken as well.

Graph Data Analysis The analysis of large graphs and also the analysis of many collections of graphs poses interesting challenges: from finding common patterns of similar structure in social networks that can help identify groups of similar interest to the identification of reoccurring elements in molecular databases that could potentially lead to the discovery of a new molecular theme for a particular medication. Network analysis is often performed on abstractions, describing degrees of connectivity, and then proceeds by using standard techniques. However, a number of approaches also exist that operate on the network structure directly. A prominent example is frequent subgraph mining approaches which identify graph patterns that occur sufficiently often or are sufficiently discriminative. These methods follow the frequent itemset mining approaches described later in this book (see Chap. 7) but adopt those methods to directly operate on the underlying graph or graph database. In [4] a good introduction to the analysis of graph data is given. We also discuss one example of a molecular graph algorithm in more detail in Sect. 7.6.3.4 when we introduce item mining algorithms.

Image Data Analysis Analyzing large numbers of images often requires a mix of both methods. In order to find objects or regions of interest, one needs to apply sophisticated image segmentation approaches. Often already here analysis steps are involved to semiautomatically find the best segmentation technique. Once this is done, various types of image feature computations can be applied to describe brightness, shape, texture, or other properties of the extracted regions. On these features classical data analysis routines are then applied. The analysis of these types of data emphasizes an interesting additional problem in data analysis, which is often ignored: each object can be described in many different ways, and it is not always clear from the beginning which of these representations is best suited for the analysis at hand. Finding out automatically which descriptors to use goes further than feature selection discussed earlier in this chapter (Sect. 6.1.1) as we cannot select from various subsets of features accompanied by different semantics. Petrushin and Khan [18] offer a good summary over the different aspects of image mining.

Other Data Types Of course, plenty other types of data are around: temporal data from stock indices, process control applications, etc. Lots of work has been done in the analysis of movies and other multimedia data such as music and speech. But complex data is also generated by various ways to record data such as weblogs and other types of user interactions recordings. Complex data also arise during observations of various types of populations (e.g., cattle herds). From an analysis point of view, the next big challenge lies in the combined analysis of separate information sources and types.

6.5 Data Integration

Almost all of the discussions in the remainder of this book focus on the modeling or analysis of data that arrives in one nice, well-defined table. In the previous sections of this chapter we have discussed how we can prepare the structure of this table by

selecting, cleaning, or constructing data, but what if our data is spread out over numerous repositories? In reality, the data recording has begun over time, and various departments have chosen their own recording mechanism. Worse, a merger in the life time of our corporate database may have thrown two completely independently developed database setups together that contain similar, but not quite the same, data. If we now want to analyze the joint customer base, we may want to merge the two (or more) customer databases into a uniform one first. This type of data integration is called vertical data integration since we are really interested in concatenating two tables holding essentially the same information. But already in this simple case we will run into many annoying problems as we will see in the following section. The other type of integration asks for combining different types of information and is called horizontal data integration. Here we aim to enrich an existing table of, e.g., customers with information from another database such as purchase behavior which was recorded independently. Essentially we are aiming to concatenate the entries from one database with the entries of the second one, somehow smartly matching which entries should be appended to each other.

6.5.1 Vertical Data Integration

An example of vertical data integration could be the combination of two customer databases from one company which recently acquired a second one. There usually are two main obstacles to overcome:

First we need to make sure that the structures of our databases are unified. One database may have stored last and first name in one field, separated by a comma, whereas the second database uses two separate fields for this. So we need to define a joint format which we can then transform the entries of both tables into. This often generates missing values, since very often not all fields will be obtainable from both tables. If one table holds gender and the other does not, we can either drop this information altogether or replace it with missing value identifiers for the entries of the second table. This structure unification will also often involve the definition of a derived identifier because very likely the two databases used two different types of identifiers.

Secondly, we need to deal with duplicates. This is a problem which often also occurs within one single database which has grown over the years. But joining two formerly completely independent databases will almost certainly create duplicate entries if we do not implement counter measures. The problem is that this is usually not just a problem of detecting duplicate identifiers. If both of our databases contain exactly the same, correct information duplicate detection is simple. But if our identifier is, for example, generated based on the last and first name, we may sometimes have a middle initial and sometimes not, and then two entries are left in our database. Slight spelling mistakes (Mayer vs. Meyer) or abbreviations (Sesame St. vs. Sesame Street) can make this a formidable task. In fact, for postal addresses, commercial companies exist whose only task is to help clean and merge duplicates in a corporate database. And for those of us who receive lots of advertisements, we can easily see how well such systems work.

Table 6.5 The two data sets on top contain information about customers and product purchases. The joint data set at the bottom combines these two tables. Note how we lose information about individual customers and how a lot of duplicate information is introduced. In reality this effect is, of course, far more dramatic

id	Last name	First name	Gender
p2	Mayer	Susan	F
p5	Smith	Walter	M
p7	Brown	Jane	F
…	…	…	…

+

Shopper id	Item id	Price
p2	i254	12.50
p5	i4245	1.99
p5	i32123	1.29
p5	i254	12.50
p5	i21435	5.99
p7	i254	12.50
…	…	…

Item id	Price	Last name	First name	Gender
i254	12.50	Mayer	Susan	F
i4245	1.99	Smith	Walter	M
i32123	1.29	Smith	Walter	M
i254	12.50	Smith	Walter	M
i21435	5.59	Smith	Walter	M
i254	12.50	Brown	Jane	F
…	…	…	…	…

Vertical data integration therefore comes back to data cleaning. We need to define a proper structure of our new, combined database, and we need to make sure to unify the representation in such a way that we can eliminate duplicate entries and clean incorrect ones.

6.5.2 Horizontal Data Integration

An example of horizontal data integration was discussed before, using our example customer/shopping data. If we have separate customer and purchase databases but are, for instance, interested in which purchases were actually done by females, we need to pull information from these data sets together. In order to do such a join, we will require some identifier which allows us to identify rows in both tables that "belong together."

Table 6.5 illustrates how this works. Here the identifier used for the join is the customer ID. Note that it is actually named differently in the two tables, so we will need to inform the joint operator which two columns to use to join the tables. There are, of course, other ways to join tables which we will discuss in a minute. However, let us first discuss the two most problematic issues related to joins in general: overrepresentation and data explosion.

- Overrepresentation of items is (although only mildly) already visible in our example table above. If we were to use the resulting table to determine the gender distribution of our shoppers, we would have a substantially higher degree of male shoppers that is evident from our table of shoppers. So here we would really answer the question "what is the number of items purchased by male (female) shoppers" and not a question related to shoppers.
- Data explosion is also visible above: we already see a few duplicate entries which are clearly not necessary to keep all essential information. Finding a more compact database setup, i.e., splitting a big database into several smaller ones, to avoid such redundancies is an important aspect in database normalization where the goal is to ensure dependency of database entries onto the key only.

Joins, as illustrated above, are only a very simple case of what is possible in real systems. Most database systems allow one to use predicates to define which rows are to be joined and more generic data analysis systems often allow one to at least choose one or more attributes as join identifiers and select the four main join types: inner, left, right, and outer joins:

- *Inner join* creates a row in the output table only if at least one entry in the left and right tables can be found with matching identifiers.
- *Left join* creates at least one row in the output table for each row of the left input table. If no matching entry in the right table can be found, the output table is filled with missing or null values. So we could make sure that each customer appears at least once, even if the customer has not made any purchase.
- *Right join* is similar to the left join, but at least one row is created for each row in the *right* table.
- *Outer join* creates at least one row for every row in the left *and* right table. If no matching entries can be found in the other tables, the corresponding entries are filled with null or missing values.

Implementing join operations is not an easy task, and various efficient algorithms have been developed. Especially on large tables when all data does not fit into main memory anymore, smart methods need to be used to allow for such out-of-memory joining. Databases are usually optimized to pick the right algorithm for the tables at hand so the data analyst does not need to worry about this. However, being aware of the risk of complex SQL statements, launching hidden joins does not hurt.

Integrating data is, of course, a brute force way to transforming a problem into one that we can then deal with using methods developed for single tables. Mining relational databases, which should actually be called "mining multirelational databases," is a research area which deals with finding information directly in such distributed tables. However, the methods developed so far are rather specialized, and in the following we will continue to assume the existence of one nice, well-formed table holding all the relevant data. This is, of course, a drastic simplification, and quite often, the integration and cleaning of data can easily require much more time and effort than the analysis itself.

6.6 Data Preparation in Practice

Data preparation is the most ignored aspect in data science, and this is also visible in many data science software packages. They often assume the existence of a nice file representation or a flat table in database. Visual data-flow-oriented tools, such as KNIME Analytics Platform, offer the ability to prepare the data intuitively, whereas command line or script-based tools are often a lot harder to use for data preparation.

Easy repeatability or reproducibility is another huge advantage. If a data analysis process is to be used in a production environment, it is often necessary to duplicate the preparation phase in a deployment workflow and to rerun the analysis periodically to check for model quality. Having a clear and intuitive way to transform and document the data in the preparation phase, in conjunction with the actual analysis, is then a serious benefit.

6.6.1 Removing Empty or Almost Empty Attributes and Records in a Data Set

KNIME Analytics Platform offers a vast array of nodes for all kinds of data preparation. Most of them can be found in the category *Data Manipulation* where there are nodes for binning, converting/replacing, filtering, and transforming columns, rows, and entire tables.

Data selection tasks can be implemented by filtering out columns using the *Column Filter* node. The Column Filter node allows removing attributes (columns) from the input data table manually, by type, or by regex pattern in their name. Similarly, the *Row Filter (Labs)* node filters rows based on certain criteria applied to individual cells. These criteria can be regular expressions or matching pattern for string cells or range or other constraints on numerical cells. These criteria can also be combined to form more complex filtering rules via AND and OR logical operations.

In the workflow used in this section (Fig. 6.4) we read the adult.csv data set and we start the whole data preparation phase by filtering out all records of people who are 40 year old and for whom the workclass is missing, because we know, for example, that these records have had problems during recording. Next we filter out the column named "fnlwgt" because of its unclear and unknown origin. Those—and more—data preparation functions are all contained within individual nodes. The corresponding very simple examples can be found on the KNIME Hub by searching for "Data Science Guide".

The subsequent nodes perform an automatic column (attribute) or row (record) removal. All columns with more than 80% missing values are automatically excluded. All columns with the same value across all rows are also automatically removed. Next we check the missing values in each record. If we want to be strict, we can remove all records where at least one attribute has a missing value. We can also adopt less strict solutions, either by keeping the record and replacing the missing value with a fictitious but likely value, or by removing the record only if the missing

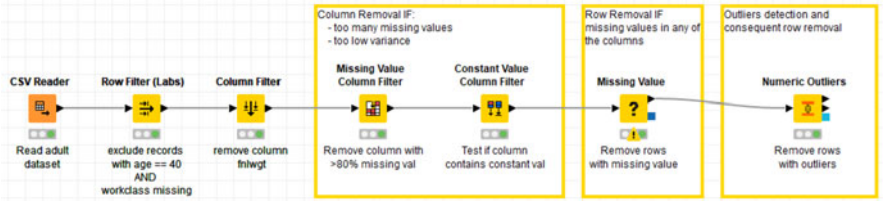


Fig. 6.4 Filtering columns and rows by manually inserted rules; then columns by missing value percentage and by constant value check; rows by outlier detection or missing values in any column

value is in a selected key attribute. Finally, the *Numeric Outliers* node removes all rows that are identified as outliers based on the InterQuartile Range (IQR) strategy on selected numeric columns.

Modeling data integration, transformation, and aggregation in such a graphical way has two advantages. Firstly, this process can be documented and communicated to others. Secondly, and even more importantly, this process can be easily executed again whenever the underlying data repositories change. The documentation aspect is often a problem with script-based platforms. In contrast, reproducibility is an issue with table-based tools, which do allow for an intuitive way to integrate and transform data but do not offer the ability to rerun this process reliably over and over again in the future.

6.6.2 Normalization and Denormalization

Nodes for constructing and converting data are available in KNIME Analytics Platform as well. Different types of normalization are available via the *Normalizer* node. Note that this node has a second outport which carries the normalization model. This allows one to easily apply the same normalization to a second data pipeline. An often overlooked problem in data analysis is that users tend to normalize their data using, e.g., min–max normalization, and then normalize their test data using this same technique! The resulting errors are hard to find since the min/max ranges of the training and test data can, but are not guaranteed to, be exactly the same. Especially, if the ranges are almost equal, the resulting small deviations can cause a lot of confusion.

Figure 6.5 shows an example of such a flow. Here a different type of model (support vector machine, see Chap. 9) is trained on normalized training data, and the resulting model is applied to testing data which underwent the same normalization procedure. In order to represent the test data in their original values, a denormalization procedure is applied as calculated from the normalization model.

KNIME Analytics Platform also offers nodes for different types of binning or discretization of data which are available in the *Data Manipulation/Column/Binning* category. Many other functionalities are available in special nodes.

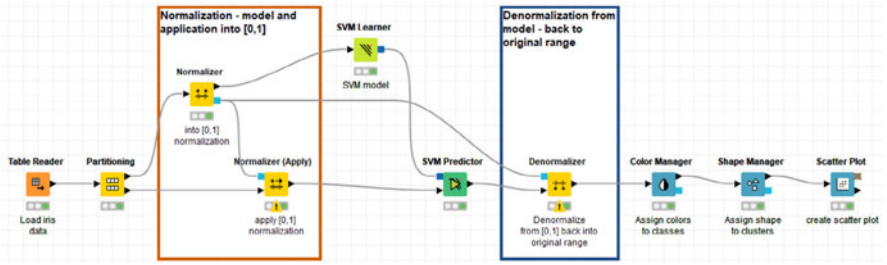


Fig. 6.5 This is how to apply normalization correctly in a data science problem. The normalization model is built on the training set and only applied on the test set! The same normalization model is used to denormalize the numerical attributes back into their original ranges

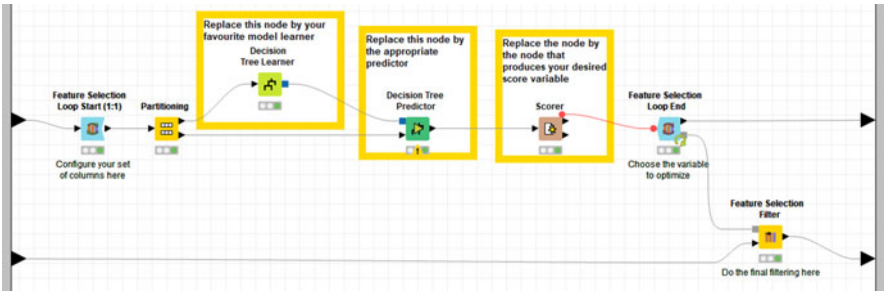


Fig. 6.6 Content of the Backward Feature Elimination metanode implementing the backward feature elimination procedure

6.6.3 Backward Feature Elimination

However, automated versions of column or feature selection such as those described in Sect. 6.1.1 are worth being discussed in a bit more detail as they demonstrate the interesting and very powerful loop-feature. KNIME Analytics Platform brings along the framework to run wrapper approaches for feature selection using arbitrary models.

Figure 6.6 shows a workflow performing backward feature elimination using a decision tree as the underlying model. It uses the prepackaged metanode named *Backward Feature Eleimination*, which contains a loop to create different subsets of input features by filtering out all other columns. The reduced data is then fed into a Partitioning node, which splits it into training and validation data. Afterwards we train a model (here a decision tree) on the training data and apply it to the validation data. The loop end node (*Backward Feature Elimination End*) calculates the accuracy of the resulting prediction and, together with the feature set, remembers it for all iterations to come. The loop is now executed several times (the number of runs and number of features per run can be adjusted in the dialog of the loop-start node), and finally a column filter model is produced, which can be used to filter all

but the top k features/columns from the second table using the *Backward Feature Elimination Filter* node.

6.7 Further Reading

A very good read and complete reference for the data preparation stage is [19], despite of its age. More recent works include [10, 13]. Various books concentrate on specific aspects, such as assessing and improving data quality [1], the identification of matching data records [3], feature selection for high-dimensional data [2], etc. In an effort to unify feature selection criteria, [25] is guided by the idea of preserving the sample similarity while controlling feature redundancy at the same time. Among the problems we did not tackle in this section is, e.g., the case of noise in the class label itself, which may have negative impact on subsequent modeling and evaluation steps (see [9] for a survey).

References

1. Batini, C., Scannapieco, M.: Data and Information Quality: Dimensions, Principles and Techniques. Springer, Berlin (2016)
2. Bolon-Canedo, V., Sanchez-Marono, N., Alonso-Betanzos, A.: Feature Selection for High-Dimensional Data. Springer, Berlin (2016)
3. Christen, P.: Data Matching: Concept and Techniques for Record Linkage, Entity Resolution and Duplicate Detection. Springer, Berlin (2014)
4. Cook, D.J., Holder, L.B.: Mining Graph Data. Wiley, Chichester (2006)
5. Dougherty, J., Kohavi, R., Sahami, M.: Supervised and unsupervised discretization of continuous features. In: Proc. 12th Int. Conf. on Machine Learning (ICML 95, Lake Tahoe, CA), pp. 115–123. Morgan Kaufmann, San Mateo (1995)
6. Elomaa, T., Rousu, J.: Efficient multisplitting revisited: optima-preserving elimination of partition candidates. Data Min. Knowl. Discov. **8**(2), 97–126 (2004)
7. Fayyad, U., Irani, K.: Multi-interval discretization of continuous-valued attributes for classification learning. In: Proc. 10th Int. Conf. on Artificial Intelligence (ICML'93, Amherst, MA), pp. 1022–1027. Morgan Kaufmann, San Mateo (1993)
8. Feldman, R., Sanger, J.: The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data. Cambridge University Press, Cambridge (2007)
9. Frenay, B., Verleysen, M.: Classification in the presence of label noise: a survey. IEEE Trans. Neural Netw. Learn. Syst. **25**(5), 845–869 (2014)
10. Garcia, S., Luengo, J., Herrera, F.: Data Preprocessing in Data Mining. Springer, Berlin (2015)
11. Guyon, I., Elisseeff, A.: An introduction to variable and feature selection. J. Mach. Learn. Res. **3**, 1157–1182 (2003)
12. Hall, M.A., Smith, L.A.: Feature subset selection: a correlation based filter approach. In: Proc. Int. Conf. on Neural Information Processing and Intelligent Information Systems, pp. 855–858. Springer, Berlin (1997)
13. Ilyas, I.F., Chu, X.: Data Cleaning. ACM Books, New York (2019)
14. Kolodyazhniy, V., Klawonn, F., Tschumitschew, K.: A neuro-fuzzy model for dimensionality reduction and its application. Int. J. Uncertain. Fuzziness Knowl.-Based Syst. **15**, 571–593 (2007)
15. Lowe, D., Tipping, M.E.: Feed-forward neural networks topographic mapping for exploratory data analysis. Neural Comput. Appl. **4**, 83–95 (1996)

16. Markovitch, S., Rosenstein, S.: Feature generation using general constructor functions. Mach. Learn. **49**(1), 59–98 (2002)
17. Murphy, P., Pazani, M.: ID2-of-3: constructive induction of *m*-of-*n* concepts for discriminators in decision trees. In: Proc. 8th Int. Conf. on Machine Learning (ICML'91, Chicago, IL), pp. 183–188. Morgan Kaufmann, San Mateo (1991)
18. Petrushin, V.A., Khan, L. (eds.): Multimedia Data Mining and Knowledge Discovery. Springer, New York (2006)
19. Pyle, D.: Data Preparation for Data Mining. Morgan Kaufmann, San Mateo (1999)
20. Rehm, F., Klawonn, F.: Improving angle based mappings. In: Advanced Data Mining and Applications, pp. 3–14. Springer, Berlin (2008)
21. Rehm, F., Klawonn, F., Kruse, R.: POLARMAP—efficient visualisation of high dimensional data. In: Information Visualization, pp. 731–740. IEEE Press, Piscataway (2006)
22. Robnik-Sikonja, M., Kononenko, I.: Theoretical and empirical analysis of ReliefF and RReliefF. Mach. Learn. **53**(1–2), 23–69 (2004)
23. van der Putten, P., van Someren, M.: A bias–variance analysis of a real world learning problem: the COIL challenge 2000. Mach. Learn. **57**, 177–195 (2004)
24. Yang, Y., Webb, G.I.: Discretization for naive Bayes learning: managing discretization bias and variance. Mach. Learn. **74**(1), 39–74 (2009)
25. Zhao, Z., Wang, L., Liu, H., Ye, J.: On similarity preserving feature selection. IEEE Trans. Knowl. Data Eng. **25**(3), 619–632 (2013)

Finding Patterns

7

In this chapter we are concerned with summarizing, describing, or exploring the data set as a whole. We do not (yet) concentrate on a particular target attribute, which will be the focus of Chaps. 8 and 9. Compact and informative representations of (possibly only parts of) the data set stimulate the data understanding phase and are extremely helpful when exploring the data. While a table with mean and standard deviation for all fields also summarizes the data somehow, such a representation would miss any interaction between the variables. Investigating two fields in a scatter plot and examining their correlation coefficients could reveal linear dependencies among two variables, but what if more variables are involved, and the dependency is restricted to a certain part of the data only? We will review several techniques that try to group or organize the data intelligently, so that the individual parts are meaningful or interesting by themselves. For the purpose of getting a quick impression of the methods we will address in this section, consider a data set of car offers from an internet platform. A number of interactions and dependencies, well known to everyone who has actively searched for a car, can be found in such a data set.

Clustering For instance, we may want to *group* all cars according to their similarities. Rather than investigating the offers car by car, we could then look at the groups as a whole and take a closer look only at the group of cars we are particularly interested in. This aggregation into similar groups is called clustering. (From our background knowledge we expect to recover the well-known classes, such as luxury class, upper and lower middle-sized class, compact cars, etc.) **Cluster analysis** (or clustering) looks for groups of similar data objects that can naturally be separated from other, dissimilar data objects. As a formal definition of clusters turns out to be quite challenging, depending on the application at hand or the algorithmic approach used to find the structure (e.g., search, mathematical optimization, etc., see Chap. 5), algorithms with quite different notions of a cluster have been proposed. Their strengths (and weaknesses) lie in different aspects of the analysis: some focus on the data constellation (such as compactness and separation of clusters), others concentrate on the detection of changes in data density, and yet others aim at an easily understandable and interpretable summary.

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_7

We have selected the following clustering methods for this chapter:

Hierarchical Clustering As the name suggests, hierarchical clustering constructs a full-fledged hierarchy for all data that can be inspected visually. As every datum is represented by its own leaf of the tree, this technique is only feasible for small data sets but provides a good overview (Sect. 7.1).

Prototype-Based Clustering Alternatively, the data may be condensed to a small, fixed number of *prototypical records*, each of them standing for a full subgroup of the data set (Sect. 7.3). This technique is efficient, and the results are interpretable even for much larger data sets. Such techniques assume, however, that the groups roughly follow some regular shape.

Density-Based Clustering Clusters may also be defined implicitly by the regions of higher data density, separated from each other by regions of lower density (Sect. 7.4). Most techniques from this category of density-based clustering allow for arbitrary shaped groups—but the price for this flexibility is a difficult interpretation of the obtained clusters.

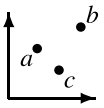
Self-organizing Maps We could even arrange all cars into a two-dimensional *map* where similar car offers are placed close together so that we explore cars similar to some selected car by examining its neighborhood in the map. Such an overview is generated by so-called self-organizing maps (Sect. 7.5). By means of color coding it becomes immediately clear where many similar records reside and where we have a sparsely populated area. Self-organizing maps were not intended as clustering techniques but can deliver similar insights.

While clustering methods seek for similarities among cases or records in our database, we may also be interested in relationships between different variables or in subgroups that behave differently with respect to some target variable.

Association Rules Rather than grouping or organizing the cars, we may be interested in interdependencies among the individual variables. Several automobile manufacturers, for instance, offer certain packages that contain a number of additional features for a special price. If the car equipment is listed completely, it is possible to recover those features that frequently occur together. The existence of some features increases the probability of others, either because both features are offered in a package or simply because people frequently select a certain set of features in combination. One technique to find associations of this kind are **association rules** (Sect. 7.6): For every feature that can be predicted confidently from the occurrence of some other features, we obtain a rule that describes this relationship.

Deviation Analysis The last method we are going to investigate in this chapter is deviation analysis. Usually expensive cars offer luxury equipment and consume significantly more petrol than standard cars. However, new cars with new fuel-saving engines, hybrid technology, etc., may represent exceptions from this general rule. Typically, there is a negative correlation with price and mileage, however, the group of old-timer cars represents an exception. Typically, a domain expert is already familiar with the more global dependencies and is more interested in deviations from the standard case or exceptional rules. The discovery of deviating subgroups of the population is discussed in Sect. 7.7.

id	X	Y
a	1	2
b	3	3
c	2	1



$[d_{i,j}]$	a	b	c
a	0.00	2.23	1.41
b	2.23	0.00	2.23
c	1.41	2.23	0.00

Fig. 7.1 A small two-dimensional data table, its graphical representation, and distance matrix

7.1 Hierarchical Clustering

We start the discussion of clustering algorithms with one of the oldest and most frequently used clustering techniques: **agglomerative hierarchical clustering**. The term "agglomerative" indicates that this type of clustering operates bottom-up. Consider a data set as shown in Fig. 7.1. Given a database consisting of many records, we are interested in a descriptive summary of the database, organizing all cases according to their properties and mutual similarity. Before comparing and arguing about *groups of records*, we must first solve a much simpler problem, namely the comparison of single cases. Given the first two rows in Fig. 7.1, how can their similarity be expressed by a single number of proximity or (dis)similarity? We do not just want to know whether two entries are identical, but we want to express *how different they are* in terms of a single number. Without loss of generality, let us concentrate on dissimilarity or distance (the smaller the value, the more similar the data) rather than similarity or proximity (the higher the value, the more similar the data).

In the case of a two-dimensional data set (only two relevant attributes X and Y are given), we can visualize the data set in a scatter plot and immediately see that, for instance, a and c are more similar than a and b—simply because a and c are *closer* together. Thus, the length of a straight line segment between two records appears to be a meaningful measure of dissimilarity (or distance). For the remainder of this section, we will use this notion of Euclidean distance but will discuss other options in detail in Sect. 7.2.

Given n records in our data set, we need all pairwise distances, that is, a distance matrix $[d_{i,j}] \in \mathbb{R}_{\geq 0}^{n \times n}$, where $d_{i,j}$ denotes the dissimilarity between record $\#i$ and $\#j$ (or $\#j$ and $\#i$ for reasons of symmetry, that is, $d_{i,j} = d_{j,i}$), as shown on the right in Fig. 7.1. For very small data sets, such a matrix could be acquired manually from experts (e.g., wine tasting), but for larger data sets, it is more practical to obtain the values directly from a distance metric $d(\mathbf{x}, \mathbf{y})$ that can be applied to arbitrary records $\mathbf{x}, \mathbf{y} \in \mathcal{D}$ and defines the distance matrix by $d_{i,j} := d(\mathbf{x}_i, \mathbf{x}_j)$.

Hierarchical clustering requires either the provision of a distance metric $d(\mathbf{x}, \mathbf{y})$ or a distance matrix $[d_{i,j}]$ and builds a series of partitions to organize the data. Formally, a **partition** $\mathcal{P}$ is defined as a set of clusters $\{\mathcal{C}_1, \ldots, \mathcal{C}_c\}$, where each cluster $\mathcal{C}_i \subseteq \mathcal{D}$ contains a subset of the data. Any partition $\mathcal{P}$ must contain all the data from $\mathcal{D}$ (i.e., $\bigcup_{\mathcal{C} \in \mathcal{P}} \mathcal{C} = \mathcal{D}$) and assign it exclusively to a single cluster (i.e., $\forall \mathcal{C}, \mathcal{C}' \in \mathcal{P} : \mathcal{C} \cap \mathcal{C}' = \emptyset$). For practical reasons, it is usually required that none of the clusters is empty (i.e., $\forall \mathcal{C} \in \mathcal{P} : \mathcal{C} \neq \emptyset$).

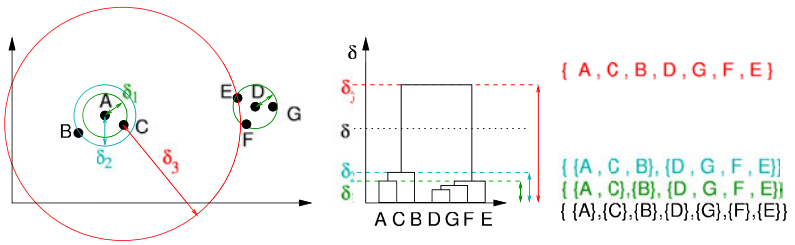


Fig. 7.2 Hierarchical clustering (single linkage) applied to a small example data set

7.1.1 Overview

Suppose that we have $\mathbf{x}, \mathbf{y} \in \mathcal{D}$ with $\delta = d(\mathbf{x}, \mathbf{y})$ and that from our background knowledge we conclude that both records should belong to the same cluster. As distance is our only measure to decide upon belongingness to a cluster, to be consistent, any other $\mathbf{z} \in \mathcal{D}$ with $d(\mathbf{x}, \mathbf{z}) \leq \delta$ should *also* belong to the same cluster, as $\mathbf{z}$ is even more similar to $\mathbf{x}$ than $\mathbf{y}$ is. Given some distance threshold δ, we may define clusters *implicitly* by requiring that any data $\mathbf{x}, \mathbf{y}$ must belong to the same cluster $\mathcal{C}$ if they are closer together than δ:

$$\forall \mathbf{x}, \mathbf{y} \in \mathcal{D}: \qquad \big(d(\mathbf{x}, \mathbf{y}) \leq \delta \Rightarrow \exists \mathcal{C} \in \mathcal{P} : \mathbf{x}, \mathbf{y} \in \mathcal{C}\big). \tag{7.1}$$

Apparently, for different values of δ, we will obtain different partitions. Deferring how to find the partition for the moment, one difficult question is then how to come up with the *right* distance threshold δ. The choice must not be arbitrary, because the resulting clusters should be *stable* and *robust* in the sense that a slightly changed threshold should not lead to completely different clusters (which would render our clusters arbitrary and useless). The idea is to overcome this problem by exploring the full range of thresholds and deciding on the best threshold on the basis of the obtained results.

Exploring the full range of thresholds sounds like a costly task delivering many different partitions. Comparing all these partitions against each other appears to be even more expensive. The key observation is, however, that the solutions form a hierarchy, where clusters obtained from a threshold δ_1 are always completely contained in clusters obtained from δ_2 if $\delta_1 < \delta_2$. This follows directly from (7.1): Suppose $\mathbf{x}, \mathbf{y} \in \mathcal{C}$, where cluster $\mathcal{C}$ has been obtained using δ_1. Then, $\mathrm{d}(\mathbf{x}, \mathbf{y}) \leq \delta_1$, and thus $\mathrm{d}(\mathbf{x} - \mathbf{y}) \leq \delta_2$ (as $\delta_1 \leq \delta_2$), so $\mathbf{x}$ must be contained in some cluster $\mathcal{C}'$ obtained from δ_2 that contains $\mathbf{y}$.

The evolution of the partitions (as δ increases) can therefore be summarized in a single hierarchy, called **dendrogram**. On the horizontal axis, all data objects are listed in a suitable order, and the vertical axis shows δ. Consider the example in Fig. 7.2: For the smallest possible value of $\delta = 0$, we obtain 7 clusters (each of size 1, represented by leaves in the hierarchy). Records d and g are closest, so at the level $\delta = d(\mathsf{d}, \mathsf{g})$, these two points unite to a cluster, while all others still remain singletons. For $\delta_1 = d(\mathsf{a}, \mathsf{c})$ (green color), all four points on the right side can be

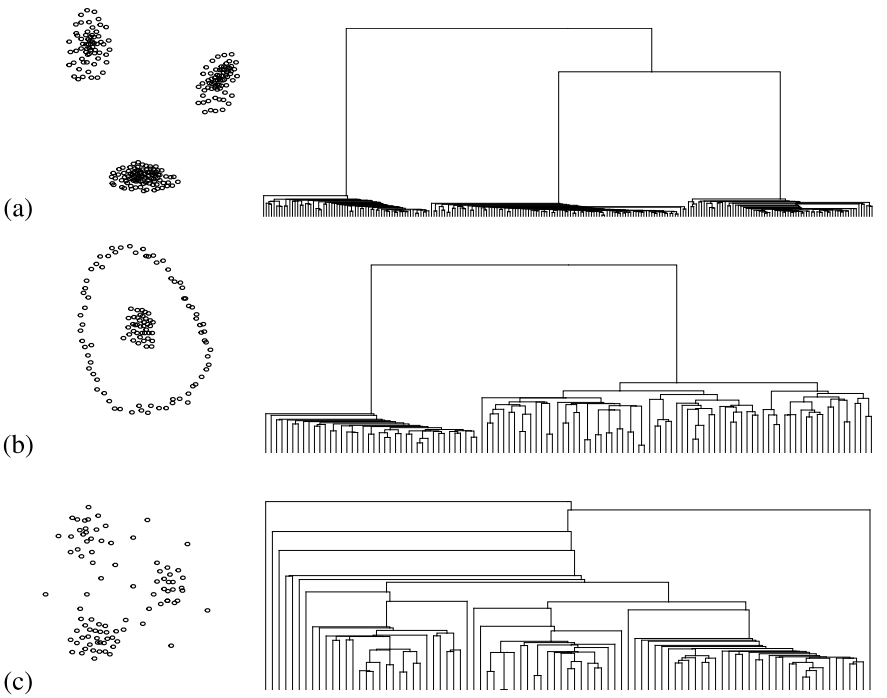


Fig. 7.3 Hierarchical clustering (single linkage) for three example data sets

reached within this distance from d, so that all points belong to the same cluster already. The three points on the left are still separated into two groups ({a, c} and {b}), because the distance $d(\mathsf{a}, \mathsf{b}) = \delta_2$ (blue color) is larger than δ_1. If we *cut the dendrogram* just above δ_1, the set of leaves of the remaining subtrees represent the currently obtained clusters: {a, c}, {b}, and {d, e, f, g}. At δ_2 we obtain two clusters {a, c, b} and {d, e, f, g}, and at a very large distance $\delta_3 = d(\mathsf{c}, \mathsf{e})$, the two clusters become connected, and all data belong to a single cluster.

Coming back to the initially stated problem of finding the best distance threshold δ, the dendrogram provides useful information: As already mentioned, for robust clusters, a slightly changed δ should not lead to completely different results. If we select δ_1, it makes a difference whether we cut the tree slightly above or slightly below δ_1, because we obtain different clusters in both cases. However, for δ_*, we obtain clusters that are stable in the sense that we get the same clusters if we choose δ somewhat smaller or larger. By using $\delta_* = \frac{1}{2}(\delta_2 + \delta_3)$ this *stability* is maximized, and we end up with two clusters. Looking for the largest difference between δ_i and δ_{i+1}, which corresponds to the largest gap between in the dendrogram, helps us in finding the most stable partition.

Figure 7.3 shows two examples of performance (so-called single linkage clustering, see the next section) to illustrate the interpretation of a dendrogram. Example (a) consists of three well-separated clusters, which are easily discovered by the algo-

rithm. The data belonging to each of the clusters unites at relatively low values of δ already, whereas quite a large value of δ is required to merge two of the three clusters into one. Therefore, the vertical gap is large, and the existence of three clusters is easily recognized by visual inspection. From hierarchy (b) we immediately see that we have two clusters, one being a bit larger (left cluster in tree occupies more horizontal space and thus has more leaves) and less dense than the other (higher δ-values compared to the right cluster). The fact that the shape of the clusters is very different (cf. scatter plot) does not make a difference for the algorithm (and is likewise not reflected in the dendrogram).

Although example (c) is somewhat similar to (a), just some random noise has been added, the dendrogram is very different. To belong to a cluster, it is sufficient to have a high similarity to just one of the cluster members, noise points get easily *chained*, thereby building bridges between the clusters that cause cluster merging at a relatively low δ-value. These *chaining* capabilities were advantageous for case (b) but prevent us from observing a clear cluster structure in case (c). Hierarchical (single-linkage) clustering is extremely sensitive to noise—adding just a few noise points to data set (a) quickly destroys the clear structure of the dendrogram. We will discuss solutions to this problem in Sect. 7.1.3.

In addition to the dendrogram, a **heatmap** can also help to illustrate the clustering result. For heatmap, hierarchical clustering is carried out for the data records, i.e., the rows of a data table, and also for the attributes by transposing the data table, so that clustering is applied to the columns of the data table. Then the records and the attributes are reordered according to the clustering result. Instead of showing the numerical entries in the data table, a color scale is used to represent the numerical values.

Figure 7.4 shows a heatmap for clustering the numerical attributes of the iris data set (after z-score standardization). The dendrogram for the record clustering is shown on the left, and the dendrogram for the attribute clustering on top. One can easily see that in the lower part of the diagram, records are clustered based on lower values for sepal width, the petal length, and the petal width (red color) and on higher values for the sepal length (yellow or orange color).

7.1.2 Construction

Table 7.1 shows an algorithm to perform hierarchical clustering. Starting from $\delta_0 = 0$, we have $n = |\mathcal{D}|$ clusters of size 1. In line 4 we find the two closest clusters in the current partition $\mathcal{P}_t$. These two clusters are removed from the current partition, and their union is reinserted as a new cluster (line 6). In this way, the number of clusters reduces by 1 for each iteration, and after n iterations the algorithm terminates.

Note that the distance measure d' used in Table 7.1 measures the distance between clusters $\mathcal{C}$ and $\mathcal{C}'$ and not between data objects. While the Euclidean distance can be used to measure pointwise distances, it has to be extended to measure the distance between sets of points (clusters). According to our initial idea in (7.1), the

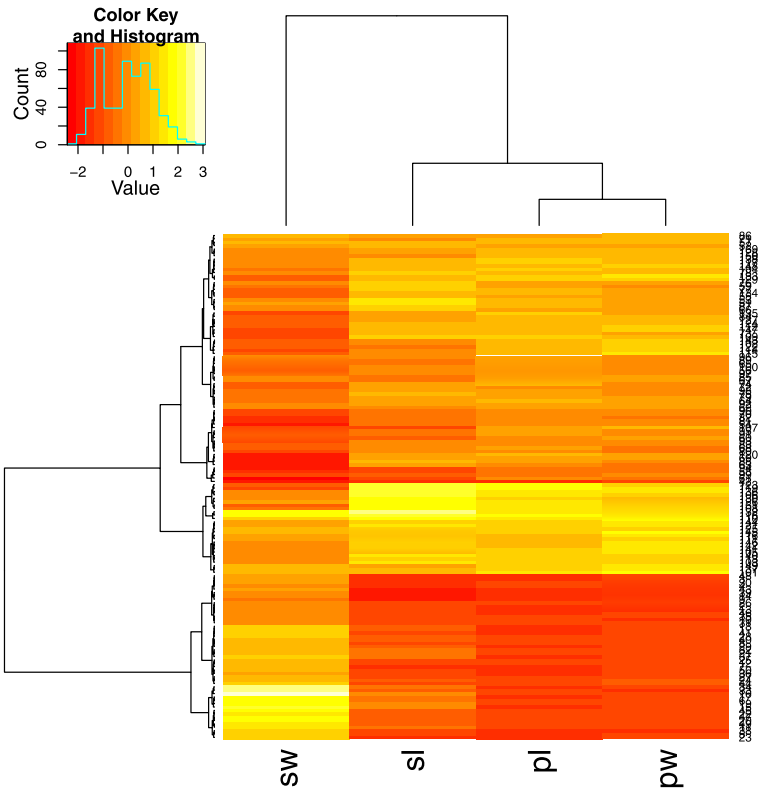


Fig. 7.4 A heatmap for the iris data set

Table 7.1 Hierarchical clustering

Algorithm HC($\mathcal{D}$) : $(\mathcal{P}_t)_{t=0..n-1}$, $(\delta_t)_{t=0..n-1}$

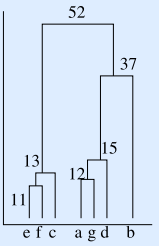
input: data set $\mathcal{D}$, $|\mathcal{D}| = n$
output: series of hierarchically nested partitions $(\mathcal{P}_t)_{t=0..n-1}$
series of hierarchy levels $(\delta_t)_{t=0..n-1}$

$\mathcal{P}_0 = \{\{\mathbf{x}\} \mid \mathbf{x} \in \mathcal{D}\}$
$t = 0, \delta_t = 0$
while current partition $\mathcal{P}_t$ has more than one cluster
  find pair of clusters $(\mathcal{C}_1, \mathcal{C}_2)$ with minimal distance $d'(\mathcal{C}_1, \mathcal{C}_2)$
  $\delta_{t+1} = d'(\mathcal{C}_1, \mathcal{C}_2)$
  construct $\mathcal{P}_{t+1}$ from $\mathcal{P}_t$ by removing $\mathcal{C}_1$ and $\mathcal{C}_2$ and inserting $\mathcal{C}_1 \cup \mathcal{C}_2$
  $t = t + 1$
end while

Table 7.2 First five iterations of hierarchical clustering on records a–g with given distance matrix. The cluster pairs $(\mathcal{C}_1, \mathcal{C}_2)$ are selected by minimal distances (bold font)

	a	b	c	d	e	f	g
a	0	50	63	17	72	81	12
b		0	49	41	42	54	37
c			0	52	13	16	61
d				0	56	66	15
e					0	**11**	64
f						0	74
g							0

	a	b	c	d	ef	g
a	0	50	63	17	72	**12**
b		0	49	41	42	37
c			0	52	13	61
d				0	56	15
ef					0	74
g						0



	ag	b	c	d	ef
ag	0	37	61	15	72
b		0	49	41	42
c			0	52	**13**
d				0	56
ef					0

	ag	b	d	efc
ag	0	37	**15**	61
b		0	41	42
d			0	52
efc				0

	agd	b	efc
agd	0	**37**	52
b		0	54
efc			0

distance to a cluster is equivalent to the distance of the nearest point of the cluster; therefore,

$$d'(\mathcal{C}, \mathcal{C}') := \min\{d(x, y) \mid x \in \mathcal{C}, y \in \mathcal{C}'\}. \tag{7.2}$$

This definition eases the recalculation of distances of merged clusters, because $d'(\{\mathcal{C} \cup \mathcal{C}'\}, \mathcal{C}'')$ can be obtained directly from the previously used distances: $d'(\{\mathcal{C} \cup \mathcal{C}'\}, \mathcal{C}'') = \min\{d'(\mathcal{C}, \mathcal{C}''), d'(\mathcal{C}', \mathcal{C}'')\}$. This recursive formulation is helpful if the distances are given by a distance matrix, because then the distances to the merged cluster are obtained by simply looking up two distances from the old distance matrix and taking their minimum.

As an example, consider the data set from Table 7.2, which shows the distance matrix for seven objects a–g. In each iteration of the algorithm, the smallest distance in the matrix is identified, and the corresponding objects (clusters, resp.) are merged. Afterwards, a new distance matrix is constructed, where the new cluster replaces the two merged clusters, and the entries in the matrix are determined according to (7.2). The process eventually stops at a 1×1 matrix.

7.1.3 Variations and Issues

As long as the data set size is moderate, the dendrogram provides useful insights, but a tree with several thousand leaves is quite difficult to manage. Furthermore, at least in the single linkage clustering, one cannot tell anything about how the data distributes in the data space, since the cluster may be shaped arbitrarily. A brief characterization of each cluster is thus difficult to provide. To get an impression of the cluster, one has to scan through all of its members. On the

Table 7.3 Alternative cluster distance aggregation functions

$$d'(\{\mathcal{C} \cup \mathcal{C}'\}, \mathcal{C}'') = \cdots$$

$$\text{single linkage} = \min\{d'(\mathcal{C}, \mathcal{C}''), d'(\mathcal{C}', \mathcal{C}'')\} \tag{7.3}$$

$$\text{complete linkage} = \max\{d'(\mathcal{C}, \mathcal{C}''), d'(\mathcal{C}', \mathcal{C}'')\} \tag{7.4}$$

$$\text{average linkage} = \frac{|\mathcal{C}|d'(\mathcal{C}, \mathcal{C}'') + |\mathcal{C}'|d'(\mathcal{C}', \mathcal{C}'')}{|\mathcal{C}| + |\mathcal{C}'|} \tag{7.5}$$

$$\text{Ward} = \frac{(|\mathcal{C}| + |\mathcal{C}''|)d'(\mathcal{C}, \mathcal{C}'') + (|\mathcal{C}'| + |\mathcal{C}''|)d'(\mathcal{C}', \mathcal{C}'') - |\mathcal{C}''|d'(\mathcal{C}, \mathcal{C}')}{|\mathcal{C}| + |\mathcal{C}'| + |\mathcal{C}''|} \tag{7.6}$$

$$\text{centroid (metric)} = \frac{1}{|\mathcal{C} \cup \mathcal{C}'||\mathcal{C}''|} \sum_{\mathbf{x} \in \mathcal{C} \cup \mathcal{C}'} \sum_{\mathbf{y} \in \mathcal{C}''} d(\mathbf{x}, \mathbf{y}) \tag{7.7}$$

other hand, hierarchical clustering may be carried out even if only a distance matrix is given, whereas the approaches in the next two sections require an explicitly given distance function (often even a specific function, such as the Euclidean distance).

The runtime and space complexity of the algorithm is quite demanding: In a naïve implementation we need to store the full distance matrix and have a cubic runtime complexity (n times find the minimal element in an $n \times n$ matrix). At least the runtime complexity may be reduced by employing data structures that support querying for neighboring data. Further possibilities to reduce time and space requirements are subsampling (using only part of the data set), omission of entries in the distance matrix (transformation into a graph, application of, e.g., minimum spanning trees), and data compression techniques (use representatives for a bunch of similar data, e.g., [68]).

7.1.3.1 Cluster-to-Cluster Distance

The so-called **single-linkage** distance (7.2) is responsible for the high sensitivity to noise we have observed already in Fig. 7.3(c). Several alternative distance aggregation heuristics have been proposed in the literature (cf. Table 7.3). The **complete-linkage** uses the maximum rather than the minimum and thus measures the diameter of the united cluster rather than the smallest distance to reach one of the cluster points. It introduces a bias towards compact cloud-like clusters, which enables hierarchical clustering to better recover the three clusters in Fig. 7.3(c) but fails in case of Fig. 7.3(b) because any cluster consisting of the ring points would also include all data from the second (interior) cluster.

The single-linkage and complete-linkage approaches represent the extremes in measuring distances between clusters. Calculating the exact average distance between data from different clusters via (7.7) is computationally more expensive than using the heuristics (7.5) or (7.6). If the data stems from a vector space, the cluster means may be utilized for distance computation. The cluster means can be stored together with the clusters and easily updated at the merging step, making approach (7.7) computationally attractive. For hyperspherical clusters, all these measures perform comparable but otherwise may lead to quite different dendrograms.

full data set

car	u	v	w	x	y	z
a	1	1	1	0	0	0
b	1	1	0	1	0	0
c	1	1	0	0	1	0
d	1	1	0	0	0	0
e	0	0	0	1	1	1
f	0	0	1	0	1	1
g	0	1	0	0	1	1
h	0	0	0	0	1	1

Jaccard distance

	a	b	c	d	e	f	g	h
a	0	.50	.50	.33	1	.80	.80	1
b	.50	0	.50	.33	.80	1	.80	1
c	.50	.50	0	.33	.80	.80	.50	.80
d	.33	.33	.33	0	1	1	.75	1
e	1	.80	.80	1	0	.50	.50	.33
f	.80	1	.80	1	.50	0	.50	.33
g	.80	.80	.50	.75	.50	.50	0	.33
h	1	1	.80	1	.33	.33	.33	0

common neighbors

	a	b	c	d	e	f	g	h
a		2	2	2	0	0	1	0
b	2		2	2	0	0	1	0
c	2	2		2	0	1	0	1
d	2	2	2		0	0	1	0
e	0	0	0	0		2	2	2
f	0	0	1	0	2		2	2
g	1	1	0	1	2	2		2
h	0	0	1	0	2	2	2	

Fig. 7.5 Grouping of categorical data: A Jaccard distance of 0.5 is achieved within the groups (e.g., $d(\mathsf{a}, \mathsf{b})$, $d(\mathsf{e}, \mathsf{g})$) and also across groups ($d(\mathsf{c}, \mathsf{g})$)

7.1.3.2 Divisive Clustering

Rather than agglomerating small clusters to large ones, we may start with one single cluster (full data set) and subsequently subdivide clusters to find the best result. Such methods are called **divisive hierarchical clustering**. In agglomerative clustering we have merged two clusters, whereas in divisive clustering typically a single cluster is subdivided into two subclusters (**bisecting strategy**), which leads us again to a binary tree of clusters (a hierarchy). Two critical questions have to be answered in every iteration: (1) which cluster has to be subdivided, and (2) how to divide a cluster into subclusters. For the first problem, validity measures try to evaluate the quality of a given cluster, and the one with the poorest quality may be the best candidate for further subdivision. Secondly, if the number of subclusters is known or assumed to be fixed (e.g., 2 in a bisecting strategy), clustering algorithms from Sect. 7.3 may be used for this step, as these algorithms subdivide the data into a partition with a fixed number of clusters. In the bottom-up approach of agglomerative clustering, the decisions are based on local information (neighborhood distances), which give poor advice in case of diffuse cluster boundaries and noisy data. In such situations, top-down divisive clustering can provide better results, as the global data distribution is considered from the beginning. The computational effort is higher for divisive clustering, though, which can be alleviated by stopping the construction of the hierarchy after a few steps, as the desired number of clusters is typically rather small.

7.1.3.3 Categorical Data

Providing distances for categorical data leads to a coarse view on the data, for instance, the number of distinct distance values is limited (as we will see in the next section). For the moment, just consider six binary features and let us compare the cases: Visual inspection of the cases a–h in Fig. 7.5 reveals two clusters, one group of cars (a–d) equipped with features u and v (plus one optional feature) and the second group (e–h) equipped with y and z (plus one optional features).

Applying the Jaccard measure (a well-suited distance measure for binary data, which will be discussed in the next section) to this data set leads to the distance matrix in Fig. 7.5. A distance of 0.5 occurs both within groups (e.g., $d(\mathsf{a}, \mathsf{b})$, $d(\mathsf{e}, \mathsf{g})$)

and across groups ($d(\mathsf{c}, \mathsf{g})$). The Jaccard distance is therefore not helpful to discriminate between these two groups.

Distance measures usually compare only two records from the data set, but they do not take any further information into account. We have seen that single-linkage clustering reacts sensitive to single outliers, which may drastically reduce the distance between clusters. Knowing the outliers can thus help to improve the clustering results we obtain from distances alone. One approach in this direction is to use link information between data objects: The number of links that exist between data $\mathbf{x}_1$ and $\mathbf{x}_2$ is the number of records $\mathbf{x} \in \mathcal{D}$ that are close to both, $\mathbf{x}_1$ and $\mathbf{x}_2$. Based on some threshold ϑ, we define the neighborhood $N_\vartheta(\mathbf{x})$ of $\mathbf{x}$ as $\{\mathbf{y} \in \mathcal{D} \mid d(\mathbf{x}, \mathbf{y}) \leq \vartheta\}$. In our example, $N_{0.5}(\mathsf{c}) = \{\mathsf{a}, \mathsf{b}, \mathsf{c}, \mathsf{d}, \mathsf{g}\}$. The number of links or common neighbors of $\mathbf{x}$ and $\mathbf{y}$ is then $(N_\vartheta(\mathbf{x}) \cap N_\vartheta(\mathbf{y}))\backslash\{\mathbf{x}, \mathbf{y}\}$. In the example we obtain a single common neighbor for a and g: $(\{\mathsf{a}, \mathsf{b}, \mathsf{c}, \mathsf{d}\} \cap \{\mathsf{c}, \mathsf{e}, \mathsf{f}, \mathsf{g}, \mathsf{h}\})\backslash\{\mathsf{a}, \mathsf{g}\} = \{\mathsf{c}\}$. The matrix on the right of Fig. 7.5 shows the number of links for all pairs of data objects. Using the number of links as a measure of similarity rather than the distance now allows us to discriminate both clusters, and we have no longer the same degree of similarity within and across groups. This approach is used in the **ROCK** algorithm (robust clustering using links) [29].

7.2 Notion of (Dis-)Similarity

In the previous section we have simply taken the length of the line segment between two data points as the distance or dissimilarity between records. This section is solely dedicated to distance measures, as they are crucial for clustering applications. We have already seen in Sect. 6.3.2 that by a different scaling of individual attributes, the clustering result may be completely different. If we have clearly pronounced clusters in two dimensions (as in Fig. 7.3(a)), but the distance measure takes also a third variable into account, which is just random noise, this may mess up the clear structure completely, and in the end, all groups contain data from the original two-dimensional clusters. We have discussed the negative influence of irrelevant and redundant variables already in Sect. 6.1.1 but assumed a predictive task such that we had some reference variable to measure a variable's informativeness—we have no such target variable in explorative clustering tasks. Instead, we can only observe how good the data gets clustered in two and three dimensions. A clear grouping with respect to variables A and B means that both attributes correlate somehow (certain combinations of attribute values occur more frequently than others). There is no reason why a similar co-occurrence pattern should be observable if we add irrelevant or random variables. Thus, whether we will observe clusters or not strongly depends on the choice of the distance measure—and that includes the choice of the variable subset.

For this section, we return to the example of car offers as those shown in Table 7.4. This time we have a mixed data set with binary, categorical, and numerical attributes. How can we define the similarity between two cars?

Table 7.4 Small excerpt from a database of cars offered on an internet platform

id	Manufacturer	Name	Type	yofr	cap	fu	fxu	used	air	abs	esp	hs	ls
1	Volkswagen	Golf Variant	Estate Car	2004	1.9	6.6	4.3	Y	Y	Y	Y	Y	N
2	Dacia	Logan	Limousine	2006	1.4	9.4	5.5	Y	N	Y	N	N	N
3	Ford	Galaxy	Van	2004	1.9	12.2	8.3	Y	Y	Y	N	N	N

Legend: yofr = year of first registration, cap = engine capacity, fu = fuel consumption (urban), fxu = fuel consumption (extra urban), used = used car, air = air conditioned, abs = anti blocking system, esp = electronic stability control, hs = heated seats, ls = leather seats

Numerical Attributes At the first glance, turning attributes such as manufacturer, car type, or optional equipment into a numerical value seems a bit more challenging; therefore we start with numerical attributes, such as horsepower, fuel consumption, or year of first registration. Given that the differences of the numerical values are meaningful (that is, the data is at least on an interval scale), we can simply take the absolute difference to measure the dissimilarity in terms of, say, fuel consumption. Let $\mathbf{x}_1$ be the first, and $\mathbf{x}_2$ the second row in Table 7.4; then based on fuel consumption (fu) alone, we have $d(\mathbf{x}_1, \mathbf{x}_2) = |\mathbf{x}_{1,\text{fu}} - \mathbf{x}_{2,\text{fu}}| = |6.6 - 9.4| = 2.8$. If we want to include the extraurban fuel consumption into our comparison, we actually compare tuple $(\mathbf{x}_{1,\text{fu}}, \mathbf{x}_{1,\text{fxu}})$ against $(\mathbf{x}_{2,\text{fu}}, \mathbf{x}_{2,\text{fxu}})$ rather than just numbers and have the additional freedom of how to combine the dissimilarity of the individual attributes (see Table 7.5): we could use the sum of the individual distances (Manhattan distance)

$$d(\mathbf{x}_1, \mathbf{x}_2) = |\mathbf{x}_{1,\text{fu}} - \mathbf{x}_{2,\text{fu}}| + |\mathbf{x}_{1,\text{fxu}} - \mathbf{x}_{2,\text{fxu}}| = |6.6 - 9.4| + |4.3 - 5.5| = 4.0,$$

the length of the shortest line between $\mathbf{x}_1$ and $\mathbf{x}_2$ (Euclidean distance)

$$d(\mathbf{x}_1, \mathbf{x}_2) = \sqrt{(\mathbf{x}_{1,\text{fu}} - \mathbf{x}_{2,\text{fu}})^2 + (\mathbf{x}_{1,\text{fxu}} - \mathbf{x}_{2,\text{fxu}})^2} = \sqrt{7.84 + 1.44} \approx 3.04,$$

or even the maximum of all distances

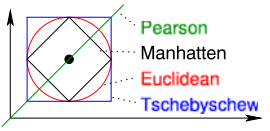
$$d(\mathbf{x}_1, \mathbf{x}_2) = \max\{|\mathbf{x}_{1,\text{fu}} - \mathbf{x}_{2,\text{fu}}|, |\mathbf{x}_{1,\text{fxu}} - \mathbf{x}_{2,\text{fxu}}|\} = \max\{2.8, 1.2\} = 2.8.$$

Under the maximum norm, the Logan (car #2) is equally dissimilar to the Golf (car #1) and the Galaxy (car #3), whereas for the other measures, the dissimilarity of the Galaxy is 1.4 times higher for the Manhattan distance and 1.3 times higher for the Euclidean distance. We have already seen that these distance values directly influence the order in which the clusters get merged, so we should carefully check that our choice corresponds to our intended notion of dissimilarity.

Nonisotropic Distances The term **isotropic** means that the distance grows in all directions equally fast, which is obviously true for, e.g., the Euclidean distance. All points with distance 1.0 from some reference point $\mathbf{x}$ form a perfect, symmetric surface of a sphere. We use the distance to capture the dissimilarity between records,

Table 7.5 Dissimilarity of numeric vectors. The graph on the right shows (for selected measures) all two-dimensional points that have a distance of 1.0 from the small bullet in the center

Minkowksi	L_p	$d_p(x,y) = \sqrt[p]{\sum_{i=1}^{n} \lvert x_i - y_i \rvert^p}$
Euclidean	L_2	$d_E(x,y) = \sqrt{(x_1-y_1)^2 + \ldots + (x_n-y_n)^2}$
Manhattan	L_1	$d_M(x,y) = \lvert x_1 - y_1 \rvert + \ldots + \lvert x_n - y_n \rvert$
Tschebyschew	L_∞	$d_\infty(x,y) = \max\{\lvert x_1 - y_1 \rvert, \ldots, \lvert x_n - y_n \rvert\}$
Cosine		$d_C(x,y) = 1 - \frac{x^\top y}{\lVert x \rVert \lVert y \rVert}$
Tanimoto		$d_T(x,y) = \frac{x^\top y}{\lVert x \rVert^2 + \lVert y \rVert^2 - x^\top y}$
Pearson		Euclidean of z-score transformed $\vec{x}$, $\vec{y}$



and a large distance may even be used as an indication for an outlier. (In particular, if we have z-score transformed data, which can be interpreted as "*how many standard deviations is this value away from the mean?*", see Sect. 6.3.2.) If we consider a pair of correlated variables, say urban and extraurban fuel consumption, we may perceive isotropic distances and counterintuitive: The differences of the fuel consumptions $\mathbf{x}_1 = (8.1, 5.3)$ and $\mathbf{x}_2 = (5.1, 5.3)$ from the fuel consumption $\mathbf{x} = (6.6, 4.3)$ is $\Delta_1 = (1.5, 1.0)$ and $\Delta_2 = (-1.5, 1.0)$. The lengths of the difference vectors are identical, so are the distances when using an isotropic distance function. However, we would be very surprised if the difference in the urban fuel consumption is negative while at the same time the difference in the extraurban fuel consumption is positive—this deviates vastly from our expectations.

Nonisotropic distance functions help us to respect such cases. We have already seen that the joint distribution of multiple variables is captured by the covariance matrix C (see also Sect. 4.3.2.1). The nonisotropic **Mahalanobis distance** is defined as

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top C^{-1} (\mathbf{x} - \mathbf{y})}$$

and considers the variance across all variables. Similar to the z-score transformed data, the distance may be interpreted as the number of standard deviations a vector deviates from the mean, but this time the deviation is not measured independently for every variable but jointly for all variables at the same time. Now, all points with distance 1.0 from some reference points $\mathbf{x}$ do no longer form the surface of a symmetric sphere but the surface of an arbitrary hyperellipsoid.

Text Data and Time Series Sometimes not the absolute difference is important but the fact that the data *varies in the same way*. This is often the case with embedded time series ($\mathbf{x}$ contains the values of a variable at consecutive time points) or a bag-of-words text representation (the attributes indicate the number of word occurrences, see Sect. 6.4). When comparing two texts in a bag-of-words representation, they are usually considered similar if the same words occur in comparable frequency. However, if one of the texts is, say, three times longer than the other, the absolute frequencies will be three times higher, so both texts appear dissimilar. By normalizing both word vectors (to unit length), the absolute frequencies become relative frequencies, and both vectors become similar. The cosine measure yields

the cosine of the angle between the normalized vectors $\mathbf{x}$ and $\mathbf{y}$ and is well suited for bag-of-words comparison due to the built-in normalization. If we apply a z-score transformation to the vectors $\mathbf{x}$ and $\mathbf{y}$, the Euclidean metric becomes the Pearson metric (up to a constant factor) and is also invariant to shifting (adding a constant). The latter is useful when comparing embedded time series that have different offsets (and scaling).

Binary Attributes Next, we consider binary attributes, e.g., whether the car was offered by a dealer or not (private offer). If we encode the Boolean values (yes/no) into numbers (1/0), we may apply the same differencing we used for numeric values before. In this case, we get a zero distance if both cars are used cars or both cars are new cars (and a distance of one otherwise).

However, this approach may be inappropriate for other binary attributes. Suppose that we have a feature "offer by local dealer," where "local" refers to the vicinity of your own home. Then, the presence of this feature is much more informative than its absence: The cars may be offered by private persons in the vicinity and by private persons or dealers far away. The presence of this feature tells us much more than the absence; the probability of absence is typically much higher, and therefore the absence should not have the same effect in the dissimilarity measure as the presence. A similar argument applies to the optional car equipment, such as ABS, EPS, heated seats, leather seats, etc. We could easily invent hundred such attributes, probably only a handful of cars having more than ten of these properties. But if we consider the absence of a feature as a *sharing a common property*, most cars will be very similar, simply because they share that 90% of the equipment is not present. In such cases, the measure by Russel and Rao could be used that accounts only for shared *present* features (see Table 7.6). Another possibility is the Jaccard measure, which takes its motivation from set theory, where the dissimilarity of two sets $A, B \subseteq \Omega$ is well reflected by $J = 1 - \frac{|A \cap B|}{|A \cup B|}$. Apparently, this definition is independent of the size of the domain $|\Omega|$. For the cars example restricted to the last six binary attributes, we obtain, for the Jaccard measure, $d_J(\mathbf{x}_1, \mathbf{x}_2) = 1 - \frac{2}{5} = \frac{3}{5}$ and $d_J(\mathbf{x}_2, \mathbf{x}_3) = 1 - \frac{2}{3} = \frac{1}{3}$ versus $d_S(\mathbf{x}_1, \mathbf{x}_2) = 1 - \frac{3}{6} = \frac{1}{2}$ and $d_S(\mathbf{x}_2, \mathbf{x}_3) = 1 - \frac{5}{6} = \frac{1}{6}$ for the simple match.

Nominal Attributes Nominal attributes may be transformed into a set of binary attributes, each of them indicating one particular feature of the attribute (see example in Table 7.7). Only one of the introduced binary attributes may be active at a time. The measures from Table 7.6 could then be reused for comparison. Clearly, as soon as $n > 2$, we *always* have cooccurring zero fields, and therefore the Jaccard or Dice measure should be used. Equivalently, a new attribute might be introduced that becomes true if and only if both records share the same value. Finally, a third alternative is the provision of a look-up table for individual dissimilarity values, where Porsche and BMW are more similar than, say, Mercedes and Dacia.

Ordinal Attributes Ordinal attributes can also be transformed into binary attributes in the same fashion as nominal attributes, but then any two different values appear equally (dis)similar, which does not reflect the additional information contained in an ordinal attribute. A better transformation is shown in Table 7.8, where the Jaccard dissimilarity is proportional to the number of ranks lying between the

Table 7.6 Measuring the dissimilarity in case of a vector of predicates (binary variables) **x** and **y**, which may also be interpreted as a set of properties $X, Y \subseteq \Omega$. The Tanimoto measure from Table 7.5 becomes identical to the Jaccard measure when applied to binary variables

	Binary attributes	Sets of properties
simple match	$d_S = 1 - \frac{b+n}{b+n+x}$	
Russel & Rao	$d_R = 1 - \frac{b}{b+n+x}$	$1 - \frac{\lvert X \cap Y \rvert}{\lvert \Omega \rvert}$
Jaccard	$d_J = 1 - \frac{b}{b+x}$	$1 - \frac{\lvert X \cap Y \rvert}{\lvert X \cup Y \rvert}$
Dice	$d_D = 1 - \frac{2b}{2b+x}$	$1 - \frac{2\lvert X \cap Y \rvert}{\lvert X \rvert + \lvert Y \rvert}$

no. of predicates that...
$b = \ldots$ hold in both records
$n = \ldots$ do not hold in both records
$x = \ldots$ hold in only one of both records

Example:

x	**y**	set X	set Y	b	n	x	d_M	d_R	d_J	d_D
101000	111000	$\{A, C\}$	$\{A, B, C\}$	2	3	1	$0.1\bar{6}$	$0.6\bar{6}$	$0.3\bar{3}$	0.20

Table 7.7 Converting a nominal attribute

Manufacturer	...		Binary vector
Volkswagen	...	→	00001
Dacia	...		01000
Ford	...		00100

Table 7.8 Converting an ordinal attribute

Number of previous owners	...		Binary vector	/	Rank
1 previous owner	...	→	100	/	1
2 previous owner	...		110		2
more than 2 previous owners	...		111		3

compared values. This is equivalent to introducing an integer-valued rank attribute and apply, say, the Manhattan distance to measure the dissimilarity of two values. The provision of a dissimilarity matrix remains as a third option.

In a mixed scale situation, such as the comparison of car offerings in Table 7.4, there are two possibilities. First, even interval- and ratio-scaled attributes could be discretized and then transformed into binary attributes, so that only the Jaccard measure needs to be applied on the final set of (only) binary attributes. A second option is the combination of various distance measures (sum, mean, etc.), where each of the measures considers a different subset of attributes.

Attribute Weighting Although we have mentioned this point earlier, we want to emphasize once more that it is important to take care about the relative weight an individual attribute gains. This holds for numeric and nominal-scaled attributes. Suppose that we have compared two cars based on fuel consumption (urban and extra-

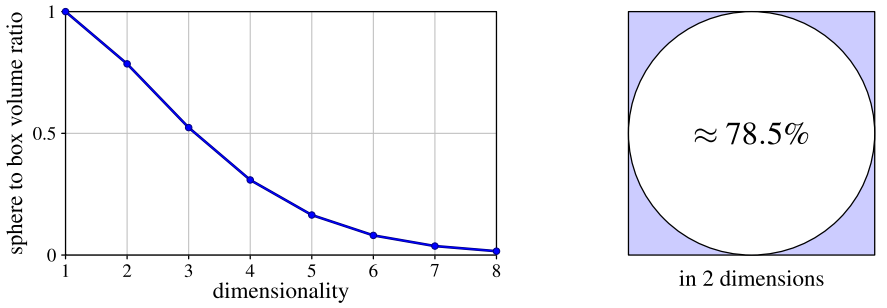


Fig. 7.6 Curse of dimensionality: The higher the dimensionality, the smaller the fraction of the hyperbox that is covered by the embedded hypersphere

urban)—now we want to take mileage into consideration. What will be the effect? The huge differences in the additional attribute will completely dominate the small differences obtained earlier. The results would be almost identical to those obtained from mileage only. The variances of the attributes differ far too much. As a second example, consider the attributes capturing the car equipment: If two cars share an additional property, they become more similar. Should they become more similar to the same degree, regardless of the property they share? Compare a drink-holder to an automatic air conditioning (a feature that much more contributes to the value of the car). For the first example (mileage), the standard solution is to standardize the data so that all numeric attributes have unit variance. The second problem is only solved by using additional weights for the aggregation of dissimilarity values when calculating the overall car-to-car distance.

The Curse of Dimensionality While we are used to work with distances in two- or three-dimensional spaces, our intuition does not help us in high-dimensional spaces. By considering more and more attributes in a data set, we mathematically construct a high-dimensional space. While the generalization of, e.g., the Euclidean distance to an arbitrary number of dimensions is straightforward, the interpretation of the distances may be somewhat counterintuitive.

Suppose that all variables have been normalized to the unit interval $[-1, 1]$. If we select two attributes, the space in which our data resides corresponds to the box of edge length 2 around the origin $\mathbf{0}$ of the coordinate system. Most of its area ($\approx$78%) is contained in the circle of radius 1 around the origin; only the four corners are omitted. If we add another attribute, the data space corresponds to a three-dimensional cube. Now, we have already eight corners that are not covered by the sphere with radius 1 around the origin, and the sphere contains less than 55% of the cube's volume. Figure 7.6 shows the percentage of the data space's hypervolume that is covered by a hypersphere of radius 1: The percentage quickly approximates zero.

If one data object is located at the origin of a two-dimensional space and we take a second data object from $[-1, 1]^2$, there is a chance of only $\approx$22% that its distance to the first object is greater than 1 (assuming the data space is uniformly populated). In case of an eight-dimensional data space, the chances of being further away than a

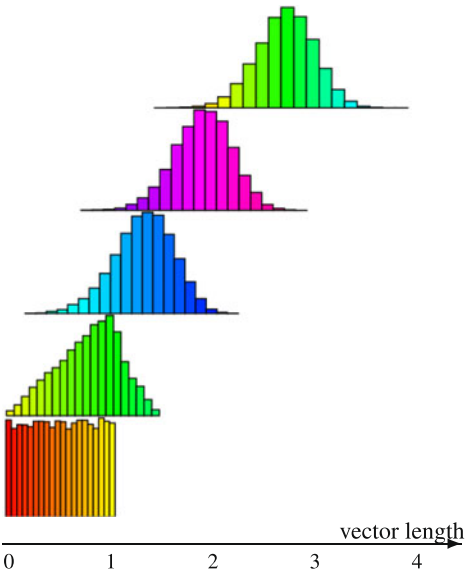


Fig. 7.7 The distribution of the length of random vectors in the unit hypercube (Euclidean norm) in 1, 2, 5, 10, and 20 dimensions (from bottom to top). While in 1D any vector length in [0, 1] is equally likely, there were no vectors of length ≤ 1 in the 20-dimensional sample of size 20,000

distance of 1 rises to more than 95%. The distances among the data objects increase, and most of the data is in the corners of the hypercube. If we want to identify data similar to some reference object (here, the origin), the higher the dimensionality, the less data we will find. This phenomenon is known as the **curse of dimensionality**, a term coined by Bellmann [9].

This effect is also shown in Fig. 7.7, which shows the distribution of the vector lengths of 20,000 random vectors in the m-dimensional unit hypercube $[0, 1]^m$. As expected, in the one-dimensional case (bottom row) the unit interval [0,1] is almost uniformly covered. To obtain a Euclidean distance of zero in an m-dimensional space, all m components must be zero at the same time, so the chances of low overall distances (sum of all components) decreases rapidly with the dimensionality. In the 10-dimensional sample there are almost no vectors of length ≤ 1, that is, within a range of 1.0 from the origin.

7.3 Prototype- and Model-Based Clustering

Clusters are represented by subtrees in a dendrogram, and all cluster members are represented individually by leaves. For large data sets, such dendrograms are too detailed, and the clusters themselves somewhat difficult to grasp, because the only way to inspect the cluster is by looking at the members one by one. A condensed representation by, say, the mean and variance of the members may be misleading, because the cluster's shape is arbitrary. The alternative perspective on clustering in this section is that we explicitly search for clusters that can be well represented by simple statistics such as mean and variance.

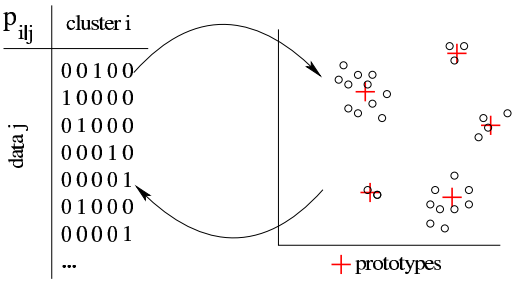


Fig. 7.8 Schema of prototype-based clustering: The membership matrix and the cluster representation (prototype) are optimized alternatingly until convergence

7.3.1 Overview

In this section, we consider approaches that make (directly or indirectly) assumptions on the shape of the clusters, either because there is actually some evidence about their shape or simply because clusters of a simple shape are easy to grasp. From the summarization point of view, a roughly hyperspherical cluster is well characterized by its mean (where we would expect the bulk of the data), standard deviation (compact or widely spread data), and its size. A table of these values per cluster eases the interpretation of the results (given that the clusters roughly follow this model of hyperspherical data clouds). The question is how to determine the location (or more generally, the parameters) of each cluster. Several approaches in this direction exist—although their motivation is different, the resulting algorithms are quite similar.

As the section title indicates, a cluster is represented by a **model**, which may be as simple as a prototypical data object (in which case it is usually called a **prototype**). For the moment, we assume that the space of models/prototypes $\mathcal{M}$ is identical to the dataspace $\mathcal{X} = \mathbb{R}^m$. We require that some distance measure

$$d : \mathcal{M} \times \mathcal{X} \to \mathbb{R}_{\geq 0}$$

allows us to compare clusters (represented by prototypes) and data directly. As long as $\mathcal{M} = \mathcal{X}$, we may use the Euclidean distance here.

From our assumptions on the cluster shape we can derive the correct cluster representative (e.g., by calculating the mean of the associated data) if we just know which data belongs to the cluster. Here we use a different encoding of cluster membership than in Sect. 7.1: A **membership matrix** $[p_{i|j}] \in [0,1]^{c \times n}$ contains the degree of belongingness of data $\mathbf{x}_j \in \mathcal{D}$ to cluster $\mathcal{C}_i$ by $p_{i|j} \in [0,1]$. A one-to-one mapping of a **partition** onto a binary membership matrix $p_{i|j} \in \{0,1\}$ is obtained from $p_{i|j} = 1 \Leftrightarrow \mathbf{x}_j \in \mathcal{C}_i$. Again, none of the clusters shall be empty ($\forall i : \sum_j p_{i|j} > 0$), and all data must participate at equal shares in the clustering ($\sum_i p_{i|j} = 1$), so that the membership degrees of the full matrix add up to the size of the data set ($\sum_i \sum_j p_{i|j} = n$). The exclusiveness of the partition ($p_{i|j} \in \{0,1\}$) is required only in some variants of prototype-based clustering.

As shown in Fig. 7.8, prototype-based clustering starts with some initial guesses of the prototypes and usually alternates between the adjustment of the membership matrix (using the currently given prototypes) and the adjustment of the pro-

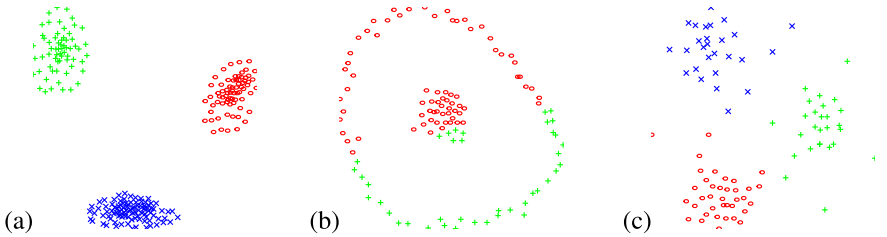


Fig. 7.9 Fuzzy c-Means clustering for three example data sets (cf. Fig. 7.3)

totypes (using the currently given membership degrees). It is assumed that the number of clusters c is known beforehand. Actually, all prototype-based clustering procedures optimize an objective-function during this iterative refinement (which could not be formulated without knowing the number of clusters in advance). Although an inspection of the membership matrix might be useful in some cases, the prototypes are considered as the primary output, since they concisely characterize the data associated with them. New data may be associated with the established clusters easily by assigning it to the nearest prototype (according to the distance $d(\mathbf{p}_i, \mathbf{x})$).

If the cluster is represented by a point prototype (assuming hyperspherical clusters), results for the data sets already presented in Fig. 7.3 are shown in Fig. 7.9. Where the model assumption was correct (cases (a) and (c)), the clusters are quickly and robustly recovered. In case (b), however, the ring-shaped outer cluster cannot be discriminated from the inner cluster: a typical prototype for the outer ring would have roughly the same distance to all its members, and thus would have to be placed in the center of the inner cluster. But then, both prototypes compete for the central data, and eventually both clusters are split among the prototypes.

7.3.2 Construction

The general sketch of the algorithm has already been discussed and is depicted in Table 7.9. However, there are still placeholders for the two main steps, the membership update and the prototype update. We will briefly review three common algorithms for hyperspherical clusters, namely the k-Means, Fuzzy c-Means, and Gaussian mixture decomposition.

7.3.2.1 Minimization of Cluster Variance (k-Means Model)

The k-Means model corresponds to our initially stated situation where the prototypes stem from the same space as the data and the membership degrees are binary ($p_{i|j} \in \{0, 1\}$). The objective function of k-Means is given by

$$J_{kM} = \sum_{i=1}^{c} \sum_{j=1}^{n} p_{i|j} \|\mathbf{x}_j - \mathbf{p}_i\|^2 = \sum_{i=1}^{c} \underbrace{\sum_{\mathbf{x} \in C_i} \|\mathbf{x}_j - \mathbf{p}_i\|^2}_{\text{variance within cluster } C_i} \tag{7.8}$$

Table 7.9 Algorithm for prototype-based clustering: k-Means (kM), Fuzzy c-Means (FCM), and Gaussian mixture decomposition (GMD)

Algorithm CentroidBasedClustering($\mathcal{D}$, $\mathbf{p}_1, \ldots, \mathbf{p}_c$) $\rightarrow$ $\mathbf{p}_1, \ldots, \mathbf{p}_c$

input: data set $\mathcal{D}$, $|\mathcal{D}| = n$,
initial models/prototypes $\mathbf{p}_1, \ldots, \mathbf{p}_c$,
output: optimized models/prototypes $\mathbf{p}_1, \ldots, \mathbf{p}_c$

		kM	FCM	GMD
1	**repeat**			
2	**for all** $1 \leq j \leq n$:			
3	update $p_{i\|j}$ according to...	(7.10)	(7.11)	(7.14)
4	**for all** $1 \leq i \leq c$:			
5	update prototype/model $\mathbf{p}_i$ according to...	(7.9)	(7.13)	(7.15)–(7.17)
6	**until** maximum number of iterations reached			
7	or $\mathbf{p}_1, \ldots, \mathbf{p}_c$ converged			

subject to $\sum_{i=1}^{c} p_{i|j} = 1$ for $1 \leq i \leq c$. If we think of a prototype $\mathbf{p}_i$ as the center of a cluster, the term $\sum_{j=1}^{n} p_{i|j} \|\mathbf{x}_j - \mathbf{p}_i\|^2$ calculates (up to a constant) the variance within the cluster (only data associated with the cluster is considered because $p_{i|j}$ is zero otherwise). By minimizing J_{kM} the prototypes will be chosen such that the variance of all clusters is minimized, thereby seeking for compact clusters. From the necessary condition for a minimum of J_{kM} ($\nabla_{\mathbf{p}_i} J_{kM} = 0$) the optimal prototype location is derived:

$$\mathbf{p}_i = \frac{\sum_{j=1}^{n} p_{i|j} \mathbf{x}_j}{\sum_{j=1}^{n} p_{i|j}}, \tag{7.9}$$

which corresponds to the mean of the data associated with the prototype. All prototypes are updated using (7.9) in the prototype-update step of Algorithm 7.9. As to the membership update step, optimization of J_{kM} is simple: for data $\mathbf{x}_j$, we have to select a single cluster i with $p_{i|j} = 1$ (and $p_{k|j} = 0$ for all $k \neq i$). Since we seek to minimize the objective function, we select the prototype where $\|\mathbf{x}_j - \mathbf{p}_i\|^2$ becomes minimal (which constitutes the membership update step):

$$p_{i|j} = \begin{cases} 1 & \text{if } \|\mathbf{p}_i - \mathbf{x}_j\| \text{ becomes minimal for } i\text{; ties are broken arbitrarily;} \\ 0 & \text{otherwise.} \end{cases} \tag{7.10}$$

7.3.2.2 Minimization of Cluster Variance (Fuzzy c-Means Model)

The k-Means model can be extended to a membership matrix that contains continuous membership degrees within [0, 1] rather than only binary memberships from {0, 1} (the cluster representation remains unchanged). A gradual membership allows one to distinguish data close to the prototype (very typical for the cluster) from incidentally associated data (far away from prototype). To guarantee a meaningful comparison of membership values, they should be normalized ($\sum_{i=1}^{c} p_{i|j} = 1$) and proportional to the distance to the prototype (if $\mathbf{x}_1$ has twice the distance to the prototype than $\mathbf{x}_2$, $\mathbf{x}_1$ should receive only one half of the membership that $\mathbf{x}_2$ receives).

This determines the membership degrees up to a constant [37]:

$$p_{i|j} = \frac{1}{\sum_{k=1}^{c} \frac{\|\mathbf{x}_j - \mathbf{p}_i\|^2}{\|\mathbf{x}_j - \mathbf{p}_k\|^2}}. \tag{7.11}$$

These membership degrees can also be obtained by minimizing the objective function of Fuzzy c-Means

$$J_{FcM} = \sum_{i=1}^{c} \sum_{j=1}^{n} p_{i|j}^2 \|\mathbf{x}_j - \mathbf{p}_i\|^2, \tag{7.12}$$

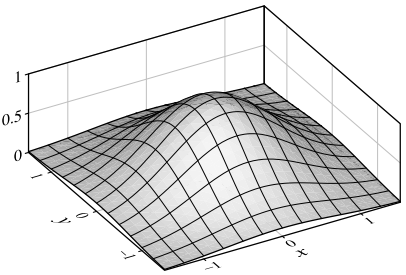
which includes an exponent of 2 and is usually called **fuzzifier**. Without the introduction of a fuzzifier, the minimization of J_{FcM} would still lead to binary membership degrees. Other choices than 2 are possible but are less frequently used in practice. The cluster centroid update is very similar to k-Means; the only difference is due to the introduction of the fuzzifier:

$$\mathbf{p}_i = \frac{\sum_{j=1}^{n} p_{i|j}^2 \mathbf{x}_j}{\sum_{j=1}^{n} p_{i|j}^2}. \tag{7.13}$$

7.3.2.3 Gaussian Mixture Decomposition (GMD)

In Gaussian mixture decomposition a probabilistic model is estimated that may explain the observed data. It is assumed that a single cluster roughly follows an isotropic[1] Gaussian distribution, which is parameterized by its mean μ and variance σ:

$$g(\mathbf{x}; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\frac{\|\mathbf{x}-\mu\|}{\sigma}^2}.$$



Since such a distribution is **unimodal** (i.e., there is only a single maximum) but each cluster represents its own peak in the density itself, we assume the overall density being a **mixture of Gaussians**. Given that the number of clusters c in the data is a priori known, the **generative model** of a data set of size n is repeating the following two steps n times:

[1] Isotropic means that the density is symmetric around the mean, that is, the cluster shapes are roughly hyperspheres. Using the Mahalanobis distance instead would allow for ellipsoidal shape, but we restrict ourselves to the isotropic case here.

- First, draw a random integer between 1 and c with probability p_i of drawing i. The data generated next will then belong to cluster #i.
- Second, draw a random sample from the Gaussian distribution $g(\mathbf{x}; \mu_i, \sigma_i)$ with the parameters μ_i, σ_i taken from model #i.

The overall density function is thus given by

$$f(\mathbf{x}; \theta) = \sum_{i=1}^{c} p_i g(\mathbf{x}; \mu_i, \sigma_i),$$

where $\theta = (\theta_1, \ldots, \theta_c)$ and $\theta_i = (p_i, \mu_i, \sigma_i)$. For some given data $\mathbf{x}_j$, we interpret the probability of being generated by model #i as the cluster membership

$$p_{i|j} = p_i g(\mathbf{x}_j; \mu_i, \sigma_i). \tag{7.14}$$

Given a mixture θ, the likelihood of the full data set being generated by this model is

$$L = \prod_{j=1}^{n} f(\mathbf{x}_j; \theta).$$

To find the most likely cluster configuration θ, the following **maximum likelihood estimate** can be derived:

$$\mu_i = \frac{\sum_{j=1}^{n} p_{i|j} \mathbf{x}_j}{\sum_{j=1}^{n} p_{i|j}}, \tag{7.15}$$

$$\sigma_i = \sqrt{\frac{1}{m} \frac{\sum_{j=1}^{n} p_{i|j} \|x_j - \mu_i\|^2}{\sum_{j=1}^{n} p_{i|j}}}, \tag{7.16}$$

$$p_i = \frac{1}{n} \sum_{i=1}^{n} p_{i|j}. \tag{7.17}$$

(Note the similarity of (7.15) with (7.9) and (7.13).) The so-called EM-algorithm performs a gradient descent search for the maximum likelihood estimate and as such is prone to local minima. If the model is poorly initialized and the clusters interact, convergence may be very slow.

7.3.3 Variations and Issues

If a good initialization is provided, the model- or prototype-based clustering algorithms are faster than hierarchical clustering, because only a few iterations are necessary. They are therefore better suited for larger data sets, also because the resulting models are easier to grasp than a complex dendrogram.

7.3.3.1 How Many Clusters?

One drawback of all prototype-based methods is that the number of clusters needs to be known (or suspected) in advance. If nothing is known whatsoever about the correct number of clusters, additional effort is necessary to apply the methods from this section. One may think of at least three different approaches: (1) top-down, divisive clustering: start with a relatively small number of clusters and split the cluster in case it does not fit the associated data well (e.g., bisecting k-means; see, e.g., [69]); (2) bottom-up, agglomerative clustering: overestimate the number of clusters and merge similar clusters; and (3) run the algorithm for a full range of possible numbers of cluster and evaluate each partition w.r.t. the overall goodness of fit. From the plot of obtained results the *optimal number of clusters* is determined by finding an extremum.

To find the *optimal* result, all these methods make use of so-called **validity measures** (see, e.g., [19, 30]). Approaches (1) and (2) employ a local validity measure that evaluate a single cluster only, such as the data density within the cluster or the distribution of membership degrees (unambiguous memberships are preferable as they indicate a successful separation of clusters). Another example is the **silhouette coefficient** [52] of a cluster $\mathcal{C}$, which is the average of silhouette coefficients $s(\mathbf{x})$ of its members $\mathbf{x} \in \mathcal{C}$:

$$s(\mathbf{x}) = \frac{b(\mathbf{x}) - a(\mathbf{x})}{\max\{a(\mathbf{x}), b(\mathbf{x})\}} \in [-1, 1],$$

where $a(\mathbf{x}) = d(\mathbf{x}, \mathcal{C})$ is the average distance of $\mathbf{x}$ to members of the same cluster $\mathcal{C}$, and $b(\mathbf{x})$ is the average distance to the members of the nearest cluster $\mathcal{C}'$ other than $\mathcal{C}$ ($b(\mathbf{x}) = \min_{\mathcal{C}' \neq \mathcal{C}} d(\mathbf{x}, \mathcal{C}')$). Well-clustered data $\mathbf{x}$ is close to members of its own cluster (small $a(\mathbf{x})$) but far away from members of other clusters (large $b(\mathbf{x})$) and thus receives a high silhouette coefficient near 1. A good cluster receives a high average silhouette coefficient from its members.

In contrast, approach (3) requires a global validity measure that evaluates all clusters at the same time. Regarding the global validity measures, we could employ the objective functions themselves to determine the optimal number of clusters. However, all the objective functions tend to become better and better the larger the number of cluster is (and at the end, there would be n singleton clusters). Validity measures such as the Aikake information criterion compensate this effect by reducing the obtained likelihood by the number of necessary parameters. Other typical measures, such as the **separation index** [21], identify *compact and well-separated clusters*:

$$D = \min_{i=1..c} \min_{j=i+1..c} \frac{\min_{x \in \mathcal{C}_i, y \in \mathcal{C}_j} d(\mathbf{x}_i, \mathbf{x}_j)}{\max_{k=1..c} \operatorname{diam}_k},$$

where the numerator represents the distances between clusters $\mathcal{C}_i$ and $\mathcal{C}_j$ (which should be large), and the denominator $\operatorname{diam}_k = \max_{x,y \in \mathcal{C}_k} \|x - y\|$ expresses the extension of cluster $\mathcal{C}_k$ (which should be small for compact clusters). Since the separation index considers neither fuzzy membership nor probabilities, it is well applicable to k-Means. There are also fuzzified versions of the separation index [63].

Admittedly these heuristic approaches fail to find the number of clusters if the model assumptions do not hold (e.g., cluster shape is not hyperspherical, cluster sizes are very different, cluster density is very different) or there is too much noise in the data. Other clustering algorithms may be better suited in this case, but unfortunately most validity measures cannot be applied *across different approaches*, which renders the selection of the best result difficult. A holistic evaluation has been suggested in [36], which includes three key criteria for cluster evaluation: the robustness of the result (confidence of the clustering algorithm in its result), the amount of reliably discovered structure and the consistency of the discovered clusters with the notion of a cluster.

Although finding the correct number of clusters using model-based clustering may be difficult, sometimes it is very handy to specify the number of groups in advance. Clustering algorithms may also be used as a tool for other tasks, such as discretization or allocation. If customers have to be assigned to k account managers, k-Means clustering could help to assign a homogeneous group of customers to each manager. In such settings, the number of groups is usually known beforehand.

7.3.3.2 Cluster Shape

We have only considered hyperspherical, **isotropic** clusters, that is, clusters that extend uniformly from the center into the dataspace. With real data this assumption is seldom satisfied. The Euclidean distance used in GMD and FcM can be generalized to the Mahalanobis distance, which adapts to the correlations between dimensions and accounts for ellipsoidal clusters. There are also many variants of Fuzzy c-Means that consider somewhat exotic cluster shapes such as circles, ellipses, rectangles, or quadrics [38] (shell clustering).

Integrating flexible cluster shapes into the used distance measure greatly increases the number of parameters; typically, the resulting algorithms react very sensitively to poor initializations. Another approach is to subdivide the data into a pretty large number of small clusters (using, e.g., k-Means) and then agglomeratively join these prototypes to clusters. In a sense, k-Means clustering is then used like a preprocessing phase to compress the data volume before then agglomerative hierarchical clustering is applied. (On the other hand, such a condensed representation can help to speed up k-Means and its variants significantly [34, 68].)

7.3.3.3 Noise Clustering

A very simple extension of fuzzy c-means clustering to cope with outliers is noise clustering [18]. We have already discussed that, to consider a different cluster shape, we only have to define an appropriate distance function. The idea of a noise cluster is that it has a fixed (usually large) distance d_{noise} to any data point. As soon as the distance of some data $\mathbf{x} \in \mathcal{X}$ to the nearest prototype $\mathbf{p}_i$ comes close to d_{noise}, the noise cluster gains a considerable fraction of the total membership degree, thereby reducing the influence of $\mathbf{x}$ with respect to $\mathbf{p}_i$. Noise clustering simply requires exchanging the membership update (7.11) by

$$p_{i|j} = \frac{1}{\left(\frac{\|\mathbf{x}_j - \mathbf{p}_i\|^2}{d_{\text{noise}}^2}\right) + \sum_{k=1}^{c}\left(\frac{\|\mathbf{x}_j - \mathbf{p}_i\|^2}{\|\mathbf{x}_j - \mathbf{p}_k\|^2}\right)} \tag{7.18}$$

and represents an effective means to reduce the influence of noise and extract cluster prototypes more clearly.

The noise cluster allows for a better analysis of the clustering results. What typically happens when no noise cluster is used is that the memberships of an outlier tend to $\approx \frac{1}{k}$ for the k closest clusters. This situation may also occur for data that is well within the usual data range (and thus not an outlier) but halfway between these k clusters. By inspection of membership degrees these two cases are undistinguishable. When a noise cluster is present, the former case is easily detected, as it has gained most of the membership degree (and only a small remainder is equally distributed among the closest prototypes).

7.4 Density-Based Clustering

We have seen in Sect. 7.1 that single-linkage clustering is flexible in terms of recognizable cluster shapes, but sensitive to outliers, which may easily be misused as *bridges* between clusters. These bridges are particularly counterintuitive if the core of the clusters consists of a considerable amount of data, but the bridge is established by a few data objects only. To establish a substantial connection between the clusters, the data density of the bridge should be comparable to that of the cluster itself. Density-based clustering goes into this direction by requiring a minimum data density within the whole cluster.

7.4.1 Overview

The underlying idea of density-based clustering is to measure the data density at a certain spot or region of the data space and then declare those regions as being clusters, whose density exceeds a certain density threshold. The final clusters are then obtained by *connecting* neighboring dense spots or regions. This is easy to illustrate for so-called grid-clustering algorithms, where an artificial (multidimensional) grid is laid over the data space and the number of data objects falling into a particular cell is counted. Figure 7.10(a) shows an illustrative example for the two-dimensional case. While the two larger groups are recognized as clusters, there is no bridge between both clusters as the data density is too low.

However, with this grid two other small data agglomerations are not recognized as a cluster (blue regions) but would have been if the grid was slightly shifted. Another problem is that in a naïve implementation the total number of cells increases exponentially with the number of attributes (but not the number of cells with high data density). A simple solution to both problems is to center the cells directly on the data rather than defining the grid independently from the data. Figure 7.10(b) shows a spherical area centered on four data points where each of these *neighborhoods* contains at least 3 data objects, similarly to the approach of the **DBScan** algorithm [22]. If the threshold for establishing a cluster (that is, a dense sphere) is four, we have thereby found the first four (distinct) clusters in Fig. 7.10(b). All data

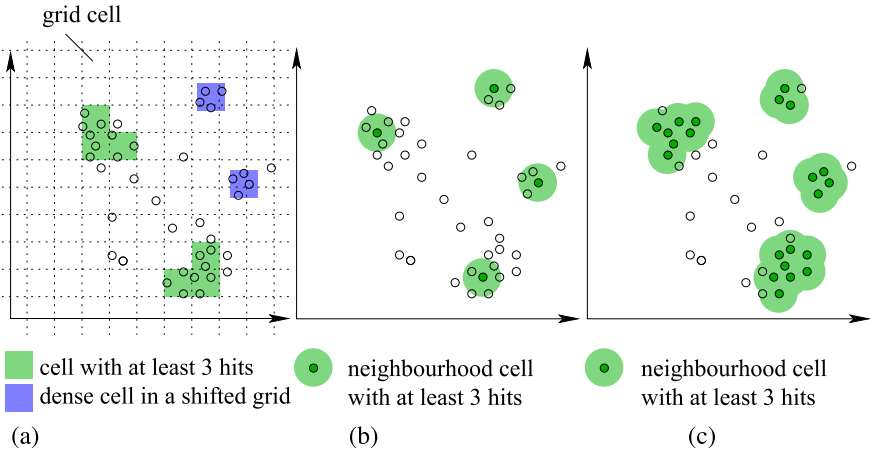


Fig. 7.10 Density-based clustering using (**a**) grid cells or (**b**)/(**c**) data neighborhoods to estimate the density

within the neighborhoods is considered as belonging to the same cluster. Using this data as starting points, the area occupied by the cluster is expanded as long as the minimal density threshold is satisfied. At the end, we arrive at the clusters shown in Fig. 7.10(c). Note that at the cluster border some data may belong to a cluster although their density does not satisfy the threshold.

The result of density-based clustering algorithms is a set of clusters, together with all data belonging to each cluster (a partition). The data within the cluster may be summarized by statistical measures, but—just as with hierarchical clustering—has to be interpreted with care as the shape of the clusters may be arbitrary.

7.4.2 Construction

We consider the DBScan algorithm given in Table 7.10. The density threshold is encoded by two parameters, the radius of the (hyperspherical) neighborhood and the number *MinPts* of data objects that is needed in the neighborhood to consider it as *dense*. The actual density at some location $\mathbf{x}$ is measured within the ε-**neighborhood** $N_\varepsilon(\mathbf{x}) = \{\mathbf{y} \in \mathcal{D} \mid \|\mathbf{x} - \mathbf{y}\| \leq \varepsilon\}$ of $\mathbf{x}$. If this neighborhood contains at least *MinPts* elements, $\mathbf{x}$ is located *within* a cluster and is called a **core object**. All data in the ε-neighborhood of a core object also belong to the same cluster as the core object. Other core objects within $N_\varepsilon(\mathbf{x})$ may likewise be core objects (they are **density-reachable**) such that those neighboring elements are also included in the same cluster. This expansion strategy is repeated until no further expansion is possible. Then, eventually, the full cluster has been determined.

The main routine *dbscan* passes once through the database and evokes the subroutine *expand* for every data x unlabeled so far (line 5). Whenever this subroutine returns true, a new cluster has successfully been extracted from the seed point $\mathbf{x}$.

Table 7.10 The DBScan algorithm

Algorithm dbscan($\mathcal{D}$, ε, *MinPts*)

input: data set $\mathcal{D}$, neighborhood radius ε, density threshold *MinPts*
output: labels the data with cluster id (or NOISE)

```
1.    label all data x ∈ D as UNCLASSIFIED
2.    initialize cluster-counter cid = 0
3.    for all x ∈ D
4.      if x is labeled as UNCLASSIFIED
5.        if expand(D, x, cid, ε, MinPts)
6.          increment cluster counter cid = cid + 1
```

Algorithm expand($\mathcal{D}$, **x**, **cid**, ε, **MinPts**) : bool

input: data set $\mathcal{D}$, $x \in D$, currently unused cluster-id *cid*, neighborhood radius ε, density threshold *MinPts*
output: returns true iff a new cluster has been found

```
1.    let S = {y ∈ D | ||x − y|| ≤ ε} (range query)
2.    if not enough data in neighborhood of x (|S| < MinPts)
3.      relabel x as NOISE and return false
4.    forall x′ ∈ S : re-label x′ with current cluster-ID cid
5.    remove x from S
6.    forall x′ ∈ S
7.      T = {y ∈ D | ||x′ − y|| ≤ ε} (range query)
8.      if enough data in neighborhood of x′ (|T| ≥ MinPts)
9.        forall y ∈ T
10.         if y does not belong to a cluster (labeled as NOISE or UNCLASSIFIED)
11.           if y is labeled UNCLASSIFIED: insert y into S
12.           re-label y with cluster-counter cid
13.     remove x′ from S
14.   return true
```

The *expand* subroutine firstly identifies the data in the neighborhood of **x** (set S in line 1). In case the desired density is not reached ($|S| < MinPts$), **x** is relabeled as NOISE, and no cluster has been found at **x**. Otherwise, a (potentially new) cluster has been found, and all data from the neighborhood are assigned to it (relabeled with cluster-ID *cid*, line 4). At this point, the processing of **x** has been completed, and the function *expand* tries to extend the cluster further. For all the data in the neighborhood, it is consecutively checked if they also satisfy the density threshold (line 8). If that is the case, they also belong to the *core* of the cluster (and are called core-points). All data in the neighborhood of a core-point are then also added to the cluster, that is, they are relabeled and inserted into the set S of data that has to be tested for further cluster expansion. If the data are organized in a data structure that supports neighborhood queries (as called in line 1), the runtime complexity is low ($O(n \log n)$). However, the performance of such data structures (e.g., R^*-trees) degrades as the data dimensionality increases.

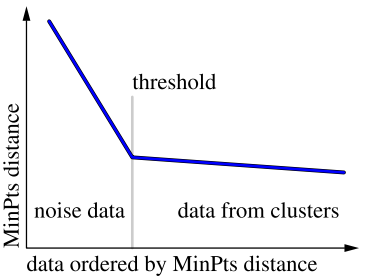


Fig. 7.11 Determination of the ε-parameter by visual inspection (identification of knee point)

To obtain optimal results, two parameters need to be adjusted, *MinPts* and ε. An alternative set of parameters is the desired *density threshold* and the *resolution* of the analysis: Any choice of *MinPts* and ε corresponds to the selection of a lower bound on the data density. But for some given density threshold ϱ, we have the freedom to choose *MinPts* twice as large and double at the same time the volume of N_ε without affecting the density threshold. The larger the volume, the more robust the density estimation, but at the same time we lose the capability of recognizing clusters smaller than this volume. Besides that, a single best choice may not exist for a given data set if clusters of different densities are present. As a rule of thumb, *MinPts* may be set to $2 * \mathit{dimension} - 1$, and then ε is determined from visually inspecting a sorted *MinPts*-distance plot: For all data $\mathbf{x}_i$, the radius r_i is determined for which $N_{r_i}(\mathbf{x}) = \mathit{MinPts}$. These radii r_i are then plotted in descending order (cf. Fig. 7.11). Under the assumption that the clusters have roughly comparable densities, the r_i-values of data belonging to clusters should roughly be the same (in the ordered plot this corresponds to a marginal negative slope). In contrast, outside the clusters the distance to the *MinPts*th-nearest neighbor strongly depends on the surrounding noise points, and thus the *MinPts*-distances will vary much stronger (leading to a deeper descent in the plot). A reasonable choice for ε is then given by the knee point in the *MinPts*-distance plot.

7.4.3 Variations and Issues

7.4.3.1 Relaxing the Density Threshold

The **OPTICS** algorithm [5] is an extension of DBScan that solves the problem of determining *MinPts* and ε differently. It can also be considered as a hierarchical clustering algorithm, as it provides the resulting partition for a full range of possible values ε. For each data object, OPTICS determines the so-called reachability- and core-distance. The core-distance is basically the smallest value of ε under which the data object becomes a core point. The reachability distance is the smallest distance under which the data object becomes a cluster member, that is, belongs to the neighborhood of the closest core object. The reachability plot of all data objects in a specific order determined during clustering can be interpreted as a cluster hierarchy, as shown in Fig. 7.12. Valleys in this plot indicate clusters in the data set: the

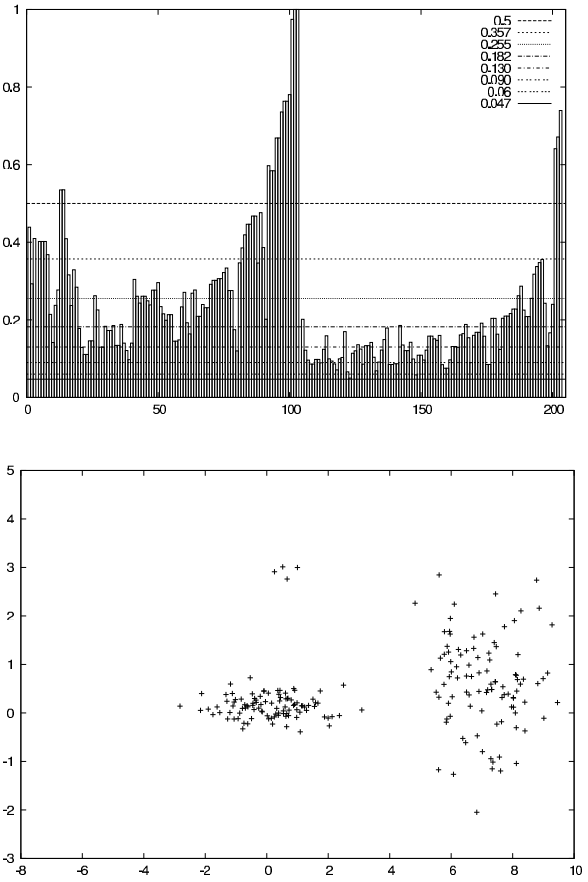


Fig. 7.12 The reachability plot of the data set on the bottom. The data objects have been ordered, and for each data object, its reachability value is shown. Contiguous areas on the *horizontal axis*, where the reachability drops below a *horizontal line*, represent a cluster. The *horizontal lines* represent certain data density levels; the lower the line, the higher the density

broader the valley, the larger the cluster, and the deeper the valley, the higher the density.

Another possibility is to define the data density over the full data space $\mathcal{X}$ by the superposition of influence functions centered at every data object:

$$\varrho(\mathbf{x}) = \sum_{\mathbf{y}\in\mathcal{D}} f_{\mathbf{y}}(\mathbf{x}),$$

where $f_{\mathbf{y}}(\cdot)$ tells us how well $\mathbf{x}$ is represented (or influenced) by $\mathbf{y}$. In the case of DBScan, we may define $f_{\mathbf{y}}(\mathbf{x}) = 1 \Leftrightarrow \|\mathbf{x}-\mathbf{y}\| \leq \varepsilon$ and 0 otherwise. For a core object of a DBScan-cluster, we have $\varrho(\mathbf{x}) \geq \mathit{MinPts}$, because at least *MinPts* data objects exist around $\mathbf{x}$ within the ε-neighborhood. The more data exist around $\mathbf{x}$, the higher the density function $\varrho(\mathbf{x})$. The **DENCLUE** algorithm [32] uses a Gaussian distribution for $f_{\mathbf{y}}(\mathbf{x})$ instead and concentrates on local maxima of ϱ (called density attractors) via a hill-climbing technique. Such local maxima can be considered as point-like clusters as in prototype-based clustering. Alternatively, several points with densities above some threshold can be considered as representing jointly a cluster of arbitrary

shape. The DENCLUE algorithm preprocesses the data and organizes it into grid cells. The summation of ϱ is restricted to some neighboring cells, thereby keeping the overall runtime requirements low. Since the neighborhood relationship between grid cells is known, no expensive neighborhood queries are necessary as with DBScan or OPTICS. The regular grid of DENCLUE is adapted to the data at hand in the OptiGrid approach [33].

7.4.3.2 Subspace Clustering

We have emphasized in Sect. 7.2 that the choice of the (dis)similarity measure has major impact on the kind and number of clusters the respective algorithm will find. A certain distance function may ignore some attributes; therefore we may consider the feature selection problem being part of the distance function design. All the clustering algorithms we have discussed so far assume that the user is capable of providing the right distance function—and thus, to select the right features or, in other words, the right subspace that contains the clusters.

The term **subspace clustering** refers to those methods that try to solve the problem of subspace selection simultaneously while clustering [45]. (This short section on subspace clustering has been located in the section on density-based clustering, because most subspace clustering techniques base their measurements on a density-based clustering notion.) As in feature selection, the problem with a naïve approach, where we simply enumerate all possible subspaces (i.e., subsets of the attribute set) and run the clustering algorithm of our choice, is the exponential complexity: the number of potential subspaces increases exponentially with the dimensionality of the dataspace. There are at least three ways to attack this problem.

In a bottom-up approach, we first look for clusters in the individual one-dimensional spaces (1D). From those we can construct two-dimensional spaces via the cross-product of dense one-dimensional clusters (blue shaded regions in Fig. 7.13, right). We thereby save ourselves from investigating those 2D-areas whose 1D-projections are not dense (white areas in Fig. 7.13, right), as they have no chance of becoming dense in higher dimensions anyway. The algorithm **CLIQUE** [4] follows this approach and takes the basic idea of dimensionwise cluster construction from frequent pattern mining, which will be discussed in Sect. 7.6.

In a top-down solution we seek for a few data points with high data density in the full data set (anchor points) and then identify those variables that lead to the highest data density. This can be done clusterwise, such that every cluster may live in a different subspace. A member of this family of algorithms is **PROCLUS** (projected clustering) [1]. Each dimension gets a weight that indicates whether the dimension belongs to the cluster's subspace, and this weight is iteratively adjusted to increase the cluster validity measure.

The third option is to rank subspaces according to their suspected suitability for clustering and use the top-ranked subspace(s) for a *conventional* clustering algorithm (similar to feature selection). The basic idea of the **SURFING** (subspaces relevant for clustering) approach [7] is to look at the distribution of distances to the kth nearest neighbor, as it has already been used to determine ε in the DBScan algorithm, cf. Fig. 7.11. If the data is distributed on a regular grid, this rather uninteresting data set with respect to clustering is recognized by an almost straight

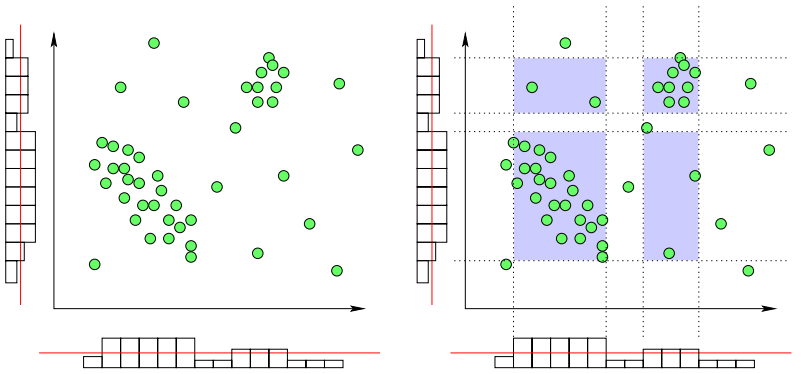


Fig. 7.13 The stylized histograms along the axes indicate density estimates. The *red line* corresponds to a threshold for dense intervals. By analyzing the density along the individual attributes, areas in the 2D-space can be excluded from further cluster search (areas *left white*)

line in this graph. If clusters are present, we suspect varying distances to the kth neighbor. The more the distances deviate from a constant line, the more promising is the subspace. Similar to CLIQUE, the potential subspaces are then explored levelwise to identify those subspaces that appear most promising. However, in high-dimensional spaces with nonuniform marginal distributions we expect locally dense areas in many subspaces, but they are not necessarily worth a closer inspection because the high density was expected from the marginal distributions. In [35] a feature selection algorithm is proposed that efficiently searches for subspaces that exhibit an unexpected high data density that cannot be explained from the marginal distributions alone.

7.5 Self-organizing Maps

Maps of all kinds are used to distill a large amount of information into a handy, small, and comprehensible visualization. People are used to maps; being able to transform the (potentially high-dimensional) data set into a two-dimensional representation may therefore help get an intuitive understanding of the data.

7.5.1 Overview

A map consists of many tiny pictograms, letters, and numbers, each of them meaningful to the reader, but the most important property when using a map is the neighborhood relationship: If two points are close to each other on the map, so they are in the real world. The **self-organizing maps** or **Kohonen maps** [43] start from a mesh of map nodes[2] for which the neighborhood relationship is fixed. One may think

[2]The node are usually called neurons, since self-organizing maps are a special form of neural network.

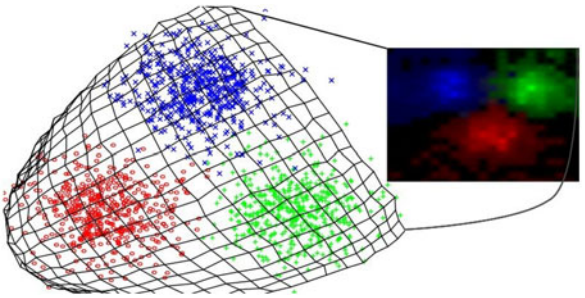


Fig. 7.14
The self-organizing map (*middle*) covers the occupied data space (*left*) over time

of a fishing net, where the knots represent the nodes, and all neighbors are connected by strings. The structure of the fishing net is two-dimensional, but in a three-dimensional world it may lay flat on the ground, forming a regular grid of mesh nodes, may be packed tightly in a wooden box, or may even float freely in the ocean. While the node positions in the higher-dimensional space varies, the neighborhood relationship remains fixed by the a priori defined connections of the mesh (knots). The idea of **self-organizing maps** is to define the mesh first (e.g., two-dimensional mesh)—which serves as the basic layout for the pictorial representation—and then let it float in the dataspace such that the data gets well covered. Every node $\mathbf{p}_i$ in the pictorial representation is associated with a corresponding reference node $\mathbf{r}_i$ in the high-dimensional space. At the end, the properties of the map at node $\mathbf{p}_i$, such as color or saturation, are taken directly from the properties of the data space at $\mathbf{r}_i$. The elastic network represents a nonlinear mapping of reference vectors (high-dimensional positions in the data space) to the mesh nodes (two-dimensional positions in the mesh).

Figure 7.14 shows an example for a two-dimensional data set. In such a low-dimensional case the self-organizing map provides no advantages over a scatter plot, it is used for illustrating purposes only. The occupied dataspace is covered by the reference vectors of the self-organizing map (overlaid on the scatter plot). The color and data density information is taken from the respective locations and gets mapped to the two-dimensional display coordinates (right subfigure).

7.5.2 Construction

Learning a self organizing map is done iteratively (see Table 7.11): for any data vector $\mathbf{x}$ from the data set, the closest reference vector $\mathbf{r}_w$ (in the high-dimensional dataspace) is identified. The node w is usually called the **winner neuron**. From the low-dimensional map topology the neighboring nodes of node w are identified. Suppose that node i is one of these neighbors, then the associated reference vector $\mathbf{r}_i$ is moved slightly towards $\mathbf{x}$ so that it better represents this data vector. How much it is shifted is controlled by a heuristic learning rate η, which slowly approaches zero to ensure the convergence of the reference vectors. Thus, any new data vector leads to a local modification of the map around the winner neuron. After a fixed number

Table 7.11 Algorithm for mapping a data set onto a two-dimensional map

Algorithm SOM($\mathcal{D}, \mathbf{p}_1, \ldots, \mathbf{p}_m, h$) $\rightarrow \mathbf{r}_1, \ldots, \mathbf{r}_m$

input: data set $\mathcal{D}$, $|\mathcal{D}| = n$,
two-dimensional mesh nodes $\mathbf{p}_1, \ldots, \mathbf{p}_m \in \mathbb{R}^{2 \times m}$,
neighborhood function $h : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow [0, 1]$
output: reference vectors $\mathbf{r}_1, \ldots, \mathbf{r}_m$

```
1  initialize reference vectors r_1, ..., r_m
2  repeat
3    forall x ∈ X
4      find reference vector r_w closest to x with respect to distance function d(x, r_i)
5      update all reference vectors r_i:
6        r_i = r_i + η · h(p_w, p_i) · (x − r_i)
7      slightly decrease η and δ
8  until maximum number of iterations reached or r_1, ..., r_m converged
```

of iterations or when the changes drop below some threshold, the map adjustment is finished.

To determine the winner node any distance function might be used (see the measures discussed in Sect. 7.2), but the Euclidean and the cosine distances are most frequently employed. As not only the winning reference vector $\mathbf{r}_w$ is shifted towards the new sample, but also neighboring vectors, we need a way to specify which nodes will be affected. This is done by a neighborhood function $h : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow [0, 1]$: For node i, this function provides a factor $h(\mathbf{p}_w, \mathbf{p}_i)$ that directly influences the reference vector modification. Note that the neighborhood information is taken from the two-dimensional node locations $\mathbf{p}_i$, while the update concerns the high-dimensional reference vectors $\mathbf{r}_i$. The smaller the factor, the smaller the effect of the modification; a factor of zero means that the reference vector is left unaltered. The following, typically used functions share a common parameter, the influence range δ, which controls the size of the neighborhood, see Fig. 7.15.

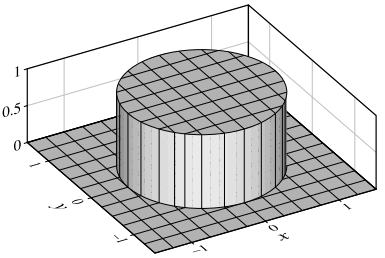
Initially, the neighborhood should be relatively large as the map needs to be first unfolded. The random initialization leads to an unorganized initial state, and neighboring nodes i and j in the low-dimensional representation are not associated with neighboring reference vectors. To avoid distorted or twisted maps, the so-called stiffness parameter δ must be relatively large at the beginning and must not decrease too quickly. Figure 7.16 shows some intermediate steps in the evolution of a self-organizing map.

7.6 Frequent Pattern Mining and Association Rules

Frequent pattern mining tackles the problem of finding common properties (patterns) that are shared by all cases in certain sufficiently large subgroups of a given data set. The general approach to find such patterns is to (efficiently) search a space of potential patterns. Based on the type of data to be mined, frequent pattern mining

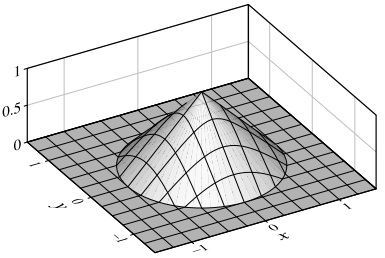
cylinder:

$$h_1(\mathbf{p}_w, \mathbf{p}_i) = \begin{cases} 1 & \text{if } d(\mathbf{p}_w, \mathbf{p}_i) < \delta \\ 0 & \text{otherwise} \end{cases}$$



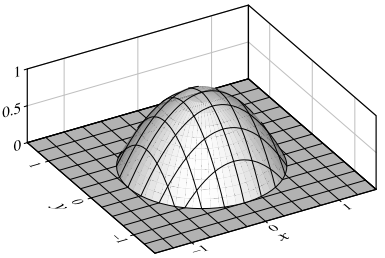
cone:

$$h_2(\mathbf{p}_w, \mathbf{p}_i) = \max\{0, 1 - d(\mathbf{p}_w, \mathbf{p}_i)/\delta\}$$



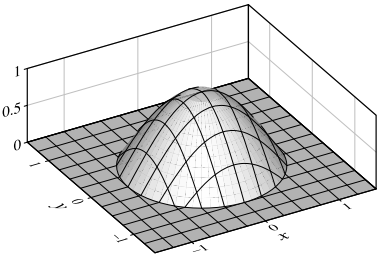
parabola:

$$h_4(\mathbf{p}_w, \mathbf{p}_i) = \max\{0, 1 - (d(\mathbf{p}_w, \mathbf{p}_i)/\delta)^2)\}$$



cosine:

$$h_3(\mathbf{p}_w, \mathbf{p}_i) = \max\{0, \cos(d(\mathbf{p}_w, \mathbf{p}_i)/\delta \cdot \tfrac{\pi}{2})\}$$



Gauss:

$$h_5(\mathbf{p}_w, \mathbf{p}_i) = \exp(-d(\mathbf{p}_w, \mathbf{p}_i)^2/\delta^2)$$

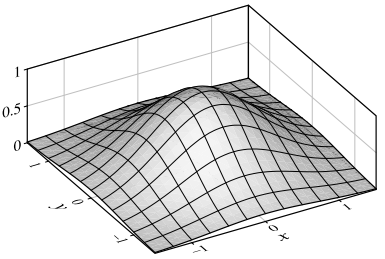


Fig. 7.15 Examples of neighborhood functions that may be used in self-organizing maps

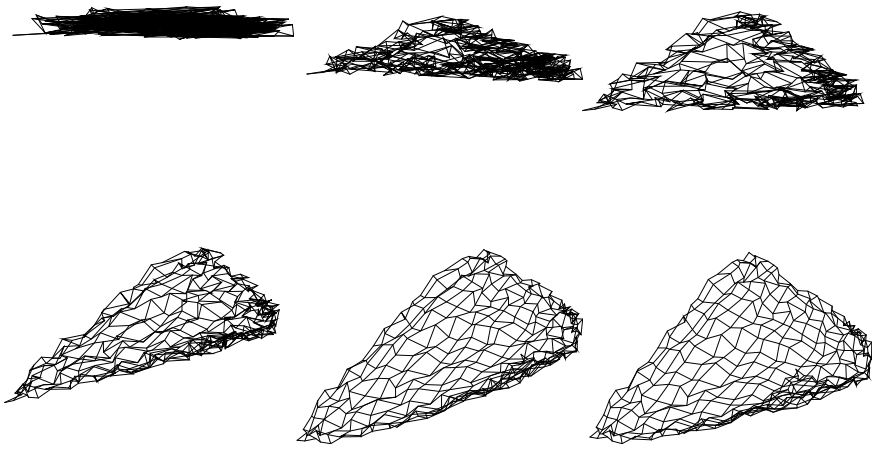


Fig. 7.16 Evolution of a self-organizing map

is divided into (1) frequent item set mining and association rule induction, (2) frequent sequence mining, and (3) frequent (sub)graph mining with the special subarea of frequent (sub)tree mining.

7.6.1 Overview

Frequent pattern mining was originally developed—in the special form of **frequent item set mining**—for market basket analysis, which aims at finding regularities in the shopping behavior of the customers of supermarkets, mail-order companies, and online shops. In particular, it tries to identify sets of products that are frequently bought together. Once identified, such sets of associated products may be exploited to optimize the organization of the offered products on the shelves of a supermarket or on the pages of a mail-order catalog or web shop, may be used to suggest other products a customer could be interested in (so-called cross-selling), or may provide hints which products may conveniently be bundled. In order to find sets of associated products, one analyzes recordings of actual purchases and selections made by customers in the past, that is, basically lists of items that were in their shopping carts. Due to the widespread use of scanner cashiers (with bar code readers), such recordings are nowadays readily available in basically every supermarket.

Generally, the challenge of frequent pattern mining consists in the fact that the number of potential patterns is usually huge. For example, a typical supermarket has thousands of products on offer, giving rise to astronomical numbers of sets of potentially associated products, even if the size of these sets is limited to few items: there are about 8 quadrillion ($8 \cdot 10^{12}$) different sets of 5 items that can be selected from a set of 1000 items. As a consequence, a brute force approach, which simply enumerates and checks all possible patterns, quickly turns out to be infeasible.

More sophisticated approaches exploit the structure of the pattern space to guide the search and the simple fact that a pattern cannot occur more frequently than any of its subpatterns to avoid useless search, which cannot yield any output. In addition, clever representations of the data, which allow for efficient counting of the number of cases satisfying a pattern, are employed. In the area of frequent item set mining, research efforts in this direction led to well-known algorithms like Apriori [2, 3], Eclat [67], and FP-growth [31], but there are also several variants and extensions [8, 27].

It is often convenient to express found patterns as rules, for example:
If a customer buys bread and wine, she will probably also buy cheese.

Such rules are customarily called **association rules**, because they describe an association of the item(s) in the consequent (then-part) with the item(s) in the antecedent (if-part). Such rules are particularly useful for cross-selling purposes, because they indicate what other products may be suggested to a customer. They are generated from the found frequent item sets by relating item sets one of which is a subset of the other, using the smaller item set as the antecedent and the additional items in the other as the consequent.

With the algorithms mentioned above, the basic task of frequent item set mining can be considered satisfactorily solved, as all of them are fast enough for most practical purposes. Nevertheless, there is still room for improvement. Recent advances include filtering the found frequent item sets and association rules (see, e.g., [60, 61]), identifying temporal changes in discovered patterns (see, e.g., [13, 14]), and mining fault-tolerant or approximate frequent item sets (see, e.g., [16, 50, 59]).

Furthermore, **sequences**, **trees**, and generally **graphs** have been considered as patterns, thus vastly expanding the possible applications areas and also introducing new problems. In particular, for patterns other than item sets—and especially for general graphs—avoiding redundant search is much more difficult. In addition, the fact that with these types of data a pattern may occur more than once in a single sample case (for example, a specific subsequence may occur at several locations in a longer sequence) allows for different definitions of what counts as frequent and thus makes it possible to find frequent patterns for single instances (for example, mining frequent subgraphs of a single large graph).

The application areas of frequent pattern mining in its different specific forms include market basket analysis, quality control and improvement, customer management, fraud detection, click stream analysis, web link analysis, genome analysis, drug design, and many more.

7.6.2 Construction

Formally, the task of **frequent item set mining** can be described as follows: we are given a set B of **items**, called the **item base**, and a database T of **transactions**. Each item may represent a product, a special equipment item, a service option, etc.,

transaction database
1: $\{a, d, e\}$
2: $\{b, c, d\}$
3: $\{a, c, e\}$
4: $\{a, c, d, e\}$
5: $\{a, e\}$
6: $\{a, c, d\}$
7: $\{b, c\}$
8: $\{a, c, d, e\}$
9: $\{b, c, e\}$
10: $\{a, d, e\}$

Frequent item sets (with support)
(minimum support $s_{\min} = 3$)

0 items	1 item	2 items	3 items
$\emptyset$: 10	$\{a\}$: 7	$\{a, c\}$: 4	$\{a, c, d\}$: 3
	$\{b\}$: 3	$\{a, d\}$: 5	$\{a, c, e\}$: 3
	$\{c\}$: 7	$\{a, e\}$: 6	$\{a, d, e\}$: 4
	$\{d\}$: 6	$\{b, c\}$: 3	
	$\{e\}$: 7	$\{c, d\}$: 4	
		$\{c, e\}$: 4	
		$\{d, e\}$: 4	

Fig. 7.17 A simple example database with 10 transactions (market baskets, shopping carts) over the item base $B = \{a, b, c, d, e\}$ (*left*) and the frequent item sets that can be discovered in this database if the minimum support is chosen to be $s_{\min} = 3$ (*right*; the numbers state the support of these item sets)

and the item base represents the set of all products, etc., that are offered. The term **item set** refers to any subset of the item base B. Each transaction is an item set and represents a set of products that has been bought by a customer. Since two or even more customers may have bought the exact same set of products, the total of all transactions must be represented as a vector, a bag, or a multiset, since in a simple set each transaction could occur at most once. (Alternatively, each transaction may be enhanced by a unique *transaction identifier*, and these enhanced transactions may then be combined in a simple set.) Note that the item base B is usually not given explicitly but only implicitly as the union of all transactions. An example transaction database over the item base $B = \{a, b, c, d, e\}$ is shown in Fig. 7.17.

The **support** $s_T(I)$ of an item set $I \subseteq B$ is the number of transactions in the database T it is contained in. Given a user-specified **minimum support** $s_{\min} \in \mathbb{N}$, an item set I is called **frequent** in T iff $s_T(I) \geq s_{\min}$. The goal of frequent item set mining is to identify all item sets $I \subseteq B$ that are frequent in a given transaction database T. In the transaction database shown in Fig. 7.17, 16 frequent item sets with 0 to 3 items can be discovered if $s_{\min} = 3$ is chosen. Note that more frequent item sets are found than there are transactions, which is a typical situation.

In order to design an algorithm to find frequent item sets, it is beneficial to first consider the properties of the support of an item set. Obviously, we have

$$\forall I : \forall J \supseteq I : \quad s_T(J) \leq s_T(I).$$

That is, *if an item set is extended, its support cannot increase*. This is immediately clear from the fact that each added item is like an additional constraint a transaction has to satisfy. One also says that support is **antimonotone** or **downward closed**. From this property it immediately follows what is known as the **a priori property**:

$$\forall s_{\min} : \forall I : \forall J \supseteq I : \quad s_T(I) < s_{\min} \rightarrow s_T(J) < s_{\min}.$$

That is, *no superset of an infrequent item set can be frequent*. As a consequence, we need not explore any supersets of infrequent item sets—an insight which can nicely be exploited to prune the search for frequent item sets.

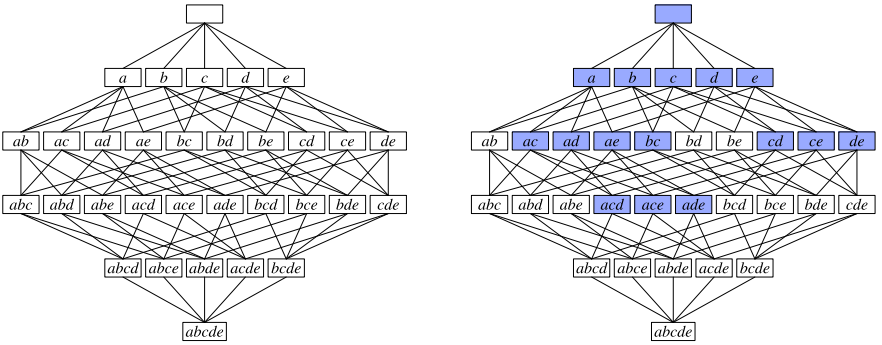


Fig. 7.18 A subset lattice for five items (*left*) and the frequent item sets (*right*, frequent item sets in *blue*) for the transaction database shown in Fig. 7.17

In order to structure the actual search, it is advantageous to consider the **subset lattice** of the item base B. For the simple item base $B = \{a, b, c, d, e\}$, which underlies the transaction database shown in Fig. 7.17, a Hasse diagram of this subset lattice is shown in Fig. 7.18 on the left: two item sets are connected if they differ in size by only one item and if one item set is a subset of the other.

Because of the a priori property, the frequent item sets must be located at the top of this lattice, thus suggesting that we should search the subset lattice top-down. As an illustration, in the right part of Fig. 7.18 the frequent item sets of Fig. 7.17 are highlighted. In this diagram the a priori property is exhibited by the fact that there cannot be a link between a white node (infrequent item set) in a higher level to a blue node (frequent item set) in a lower level. Links leading from a blue node to a white node mark the boundary between frequent and infrequent item sets.

A core problem of searching the subset lattice is that all item sets with at least two items can be explored on multiple paths. For example, the item set $\{a, c, e\}$ may be reached through $\{a\}$ and $\{a, c\}$, or through $\{c\}$ and $\{c, e\}$, or through $\{e\}$ and $\{a, e\}$, etc. As a consequence, a lot of redundant search would ensue if we always extended any frequent item set by all possible other items until we reach the boundary between the frequent and infrequent item sets. In order to avoid this redundancy, a **unique parent** is assigned to each item set, thus turning the subset lattice into a **subset tree**. This is illustrated in Fig. 7.19, where the parents are assigned by the following simple scheme: we choose an order of the items (which is arbitrary but fixed; here we simply use the alphabetically order) and then apply the rule

$$\forall I \neq \emptyset: \quad \text{parent}(I) = I - \left\{ \max_{i \in I} i \right\}.$$

This scheme has the important advantage that the permissible extensions of an item set, which only generate proper children, are easy to find: an item set may be extended only by items succeeding all already present items in the chosen item order. Since with this scheme of assigning unique parents, all sibling item sets share the same **prefix** (with respect to the chosen item order), namely the item set that is their

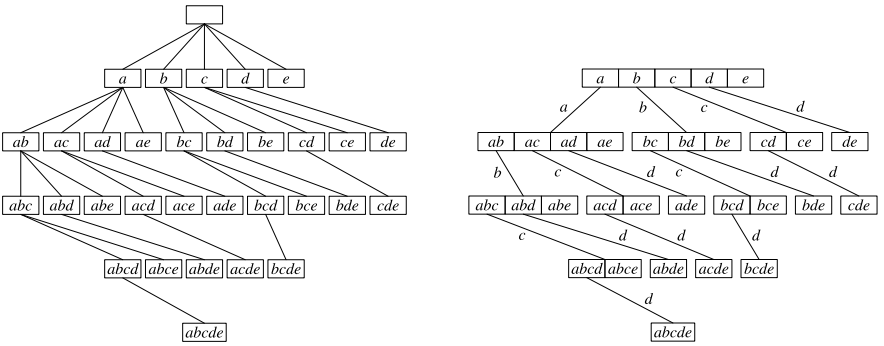


Fig. 7.19 A subset tree that results from assigning a unique parent to each item set (*left*) and a corresponding prefix tree in which sibling nodes with the same prefix are merged (*right*)

parent, it is convenient to structure the search as a prefix tree, as shown in Fig. 7.19. In this prefix tree the concatenation of the edge labels on the path from the root to a node is the common prefix of all item sets explored in this node.

A standard approach to find all frequent item sets with respect to a given database T and given support threshold $s_{\min}$, which is adopted by basically all frequent item set mining algorithms (except those of the Apriori family), is a **depth-first search** in the subset tree of the item base B. Viewed properly, this approach can be interpreted as a simple **divide-and-conquer** scheme. For the first item i in the chosen item order, the problem to find all frequent item sets is split into two subproblems: (1) find all frequent item sets containing the item i, and (2) find all frequent item sets *not* containing the item i. Each subproblem is then further divided based on the next item j in the chosen item order: find all frequent item sets containing (1.1) both items i and j, (1.2) item i but not j, (2.1) item j but not i, (2.2) neither item i nor j, and so on, always splitting with the next item (see Fig. 7.20 for an illustration).

All subproblems that occur in this divide-and-conquer recursion can be defined by a **conditional transaction database** and a **prefix**. The prefix is a set of items that has to be added to all frequent item sets that are discovered in the conditional database. Formally, all subproblems are tuples $S = (C, P)$, where C is a conditional database, and $P \subseteq B$ is a prefix. The initial problem, with which the recursion is started, is $S = (T, \emptyset)$, where T is the given transaction database, and the prefix is empty. The problem S is, of course, to find all item sets that are frequent in T.

A subproblem $S_0 = (T_0, P_0)$ is processed as follows: Choose an item $i \in B_0$, where B_0 is the set of items occurring in T_0. Note that, in principle, this choice is arbitrary but usually respects a predefined order of the items, which then also defines the structure of the subset tree as discussed above.

If $s_{T_0}(i) \geq s_{\min}$ (that is, if the item i is frequent in T_0), report the item set $P_0 \cup \{i\}$ as frequent with the support $s_{T_0}(i)$ and form the subproblem $S_1 = (T_1, P_1)$ with $P_1 = P_0 \cup \{i\}$. The conditional database T_1 comprises all transactions in T_0 that contain the item i, but with the item i removed. This also implies that transactions

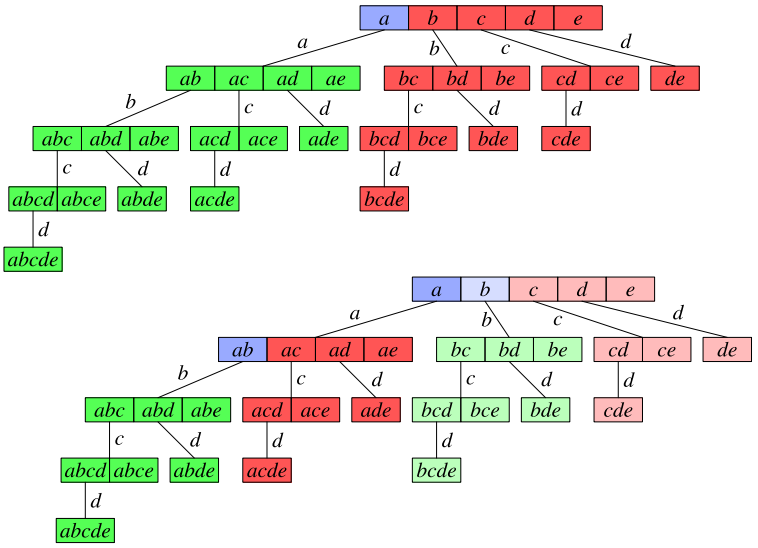


Fig. 7.20 Illustration of the divide-and-conquer approach to find frequent item sets: (*top*) first split; (*bottom*) second split; (*blue*) split item/prefix; (*green*) first subproblem; and (*red*) second subproblem

that contain no other item than i are entirely removed: no empty transactions are ever kept in the search. If T_1 is not empty, S_1 is processed recursively.

In any case (that is, regardless of whether $s_{T_0}(i) \geq s_{\min}$ or not), form the subproblem $S_2 = (T_2, P_2)$, where $P_2 = P_0$. The conditional database T_2 comprises all transactions in T_0 (including those that do not contain the item i), but again with the item i removed (and, as before, transactions containing no other item than i discarded). If T_2 is not empty, S_2 is processed recursively.

Eclat, FP-growth, and several other frequent item set mining algorithms all rely on the described basic recursive processing scheme. They differ mainly in how they represent the conditional transaction databases. There are two basic approaches: in a **horizontal representation**, the database is stored as a list (or array) of transactions, each of which is a list (or array) of the items contained in it. In a **vertical representation**, on the other hand, a database is represented by first referring with a list (or array) to the different items. For each item, a list (or array) of transaction identifiers is stored, which indicate the transactions that contain the item.

However, this distinction is not pure, since there are many algorithms that use a combination of the two forms of representing a database. For example, while Eclat uses a purely vertical representation and the SaM algorithm presented in the next section uses a purely horizontal representation, FP-growth combines in its FP-tree structure (basically a prefix tree of the transaction database, with links between the branches that connect equal items) a vertical representation (links between branches) and a (compressed) horizontal representation (prefix tree of transactions).

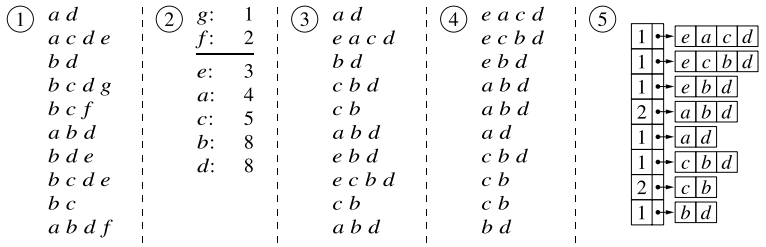


Fig. 7.21 The example database: original form (1), item frequencies (2), transactions with sorted items (3), lexicographically sorted transactions (4), and the used data structure (5)

Apriori also uses a purely horizontal representation but relies on a different processing scheme, because it traverses the subset tree levelwise rather than depth-first.

In order to give a more concrete idea of the search process, we discuss the particularly simple SaM (Split-and-Merge) algorithm [12] for frequent item set mining. In analogy to basically all other frequent item set mining algorithms, the SaM algorithm first preprocesses the transaction database with the aim to find a good item order and to set up the representation of the initial transaction database. The steps are illustrated in Fig. 7.21 for a simple example transaction database. Step 1 shows a transaction database in its original form. In step 2 the frequencies of individual items are determined from this input in order to be able to discard infrequent items immediately. If we assume a minimum support of three transactions for this example, there are two infrequent items, namely f and g, which are discarded. In step 3 the (frequent) items in each transaction are sorted according to their frequency in the transaction database, since experience (also with other algorithms) has shown that processing the items in the order of increasing frequency usually leads to the shortest execution times. In step 4 the transactions are sorted lexicographically into descending order, with item comparisons again being decided by the item frequencies, although here the item with the higher frequency precedes the item with the lower frequency. In step 5 the data structure on which SaM operates is built by combining equal transactions and setting up an array, in which each element consists of two fields: an occurrence counter and a pointer to the sorted transaction. This data structure is then processed recursively, according to the divide-and-conquer scheme discussed in the preceding section, to find the frequent item sets.

The basic operations used in the recursive processing are illustrated in Fig. 7.22. In the **split step** (see the left part of Fig. 7.22) the given array is split with respect to the leading item of the first transaction (the *split item*; item e in our example): all array elements referring to transactions starting with this item are transferred to a new array. In this process the pointer (in)to the transaction is advanced by one item, so that the common leading item is removed from all transactions, and empty transactions are discarded. Obviously, the new array represents the conditional database of the first subproblem (see page 195), which is then processed recursively to find all frequent item sets containing the split item (provided that this item is frequent).

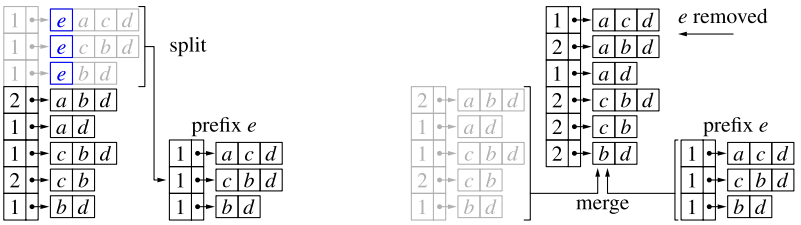


Fig. 7.22 The basic operations of the SaM algorithm: split (*left*) and merge (*right*)

The conditional database for frequent item sets *not* containing this item (needed for the second subproblem, see page 196) is obtained with a simple **merge step** (see the right part of Fig. 7.22). The created new array and the rest of the original array (which refers to all transactions starting with a different item) are combined with a procedure that is almost identical to one phase of the well-known *mergesort* algorithm. Since both arrays are obviously lexicographically sorted, one merging traversal suffices to create a lexicographically sorted merged array. The only difference to a *mergesort* phase is that equal transactions (or transaction suffixes) are combined. That is, there is always just one instance of each transaction (suffix), while its number of occurrences is kept in the occurrence counter. In our example this results in the merged array having two elements less than the input arrays together: the transaction (suffixes) *cbd* and *bd*, which occur in both arrays, are combined, and, consequently, their occurrence counters are increased to 2.

As association rules, we formally consider rules $A \rightarrow C$, with $A \subseteq B$, $C \subseteq B$, and $A \cap C = \emptyset$. Here B is the underlying item base, and the symbols A and C have been chosen to represent the antecedent and the consequent of the rule. For simplicity, we confine ourselves in the following to rules with $|C| = 1$, that is, to rules with only a single item in the consequent. Even though rules with several items in the consequent have also been studied, they are usually a lot less useful and only increase the size of the created rule set unnecessarily.

While (frequent) item sets are associated with only one measure, namely their support, association rules are evaluated with two measures:

- **support of an association rule**
  The support of an association rule $A \rightarrow C$ can be defined in two ways:
  - *support of all items appearing in the rule*: $r_T(A \rightarrow C) = \frac{s_T(A \cup C)}{s_T(\emptyset)}$.
    This is the more common definition:
    the support of a rule is the fraction of cases in which it is correct.
  - *support of the antecent of the rule*: $r_T(A \rightarrow C) = \frac{s_T(A)}{s_T(\emptyset)}$.
    This definition is actually more plausible:
    the support of a rule is the fraction of cases in which it is applicable.

  (Note that $s_T(\emptyset)$ is simply the number of transactions in the database T to mine, because the empty item set is contained in all transactions.)

association rule	support of all items	support of antecedent	confidence
$b \rightarrow c$	$3 \hat{=} 30\%$	$3 \hat{=} 30\%$	100.0%
$d \rightarrow a$	$5 \hat{=} 50\%$	$6 \hat{=} 60\%$	83.3%
$e \rightarrow a$	$6 \hat{=} 60\%$	$7 \hat{=} 70\%$	85.7%
$a \rightarrow e$	$6 \hat{=} 60\%$	$7 \hat{=} 70\%$	85.7%
$d, e \rightarrow a$	$4 \hat{=} 40\%$	$4 \hat{=} 40\%$	100.0%
$a, d \rightarrow e$	$4 \hat{=} 40\%$	$5 \hat{=} 50\%$	80.0%

Fig. 7.23 The association rules that can be discovered with a minimum support $s_{\min} = 3$ and a minimum confidence $c_{\min} = 80\%$ in the transaction database shown in Fig. 7.17. Note that in this example it is irrelevant which rule support definition is used

- **confidence of an association rule**
  The confidence of an association rule is the number of cases in which it is correct relative to the number of cases in which it is applicable: $c_T(A \rightarrow C) = \frac{s_T(A \cup C)}{s_T(A)}$. The confidence can be seen as an estimate of the conditional probability $P(C \mid A)$.

Given a transaction database T over some item base B, a **minimum support** $r_{\min} \in [0, 1]$, and a **minimum confidence** $c_{\min} \in [0, 1]$ (both to be specified by a user), the task of **association rule induction** is to find all association rules $A \rightarrow C$ with $A, C \subset B$, $r_T(A \rightarrow C) \geq r_{\min}$, and $c_T(A \rightarrow C) \geq c_{\min}$.

The usual approach to this task consists in finding, in a first step, the frequent item sets of T (as described above). For this step, it is important which rule support definition is used. If the support of an association rule is the support of all items appearing in the rule, then the minimum support to be used for the frequent item set mining step is simply $s_{\min} = \lceil s_T(\emptyset) r_{\min} \rceil$ (which explains why this is the more common definition). If, however, the support of an association rule is only the support of its antecendent, then $s_{\min} = \lceil s_T(\emptyset) r_{\min} c_{\min} \rceil$ has to be used. In a second step the found frequent item sets are traversed, and all possible rules (with one item in the consequent, see above) are generated from them and then filtered with respect to $r_{\min}$ and $c_{\min}$.

As an example, consider again the transaction database shown in Fig. 7.17 and the frequent item sets that can be discovered in it (for a minimum support, $r_{\min} = 30\%$, that is, $s_{\min} = 3$). The association rules that can be constructed from them with a minimum confidence $c_{\min} = 0.8 = 80\%$ are shown in Fig. 7.23. Due to the fairly high minimum confidence requirement, the number of found association rules is relatively small compared to the number of transactions.

7.6.3 Variations and Issues

A general scheme to enhance the efficiency of frequent pattern is perfect extension pruning (Sect. 7.6.3.1). Since usually many frequent patterns can be found in a data set, reducing the output is an important task (Sect. 7.6.3.2). The same holds for association rules (Sect. 7.6.3.3), which are usually filtered based on evaluation measures. Data types other than item sets need special methods to eliminate redundant search (Sect. 7.6.3.4). With this last section we go, in one special instance, beyond

the usual scope of this book, because we consider data that has a more complex structure than a record in a data set. The idea is to convey at least some idea of how data analysis of such data can look like, while we cannot provide a comprehensive treatment of all more complex data types.

7.6.3.1 Perfect Extension Pruning

The basic recursive processing scheme for finding frequent item sets described in Sect. 7.6.2 can easily be improved with so-called **perfect extension pruning** (also called **parent equivalence pruning**), which relies on the following simple idea: given an item set I, an item $i \notin I$ is called a **perfect extension** of I iff I and $I \cup \{i\}$ have the same support, that is, if i is contained in all transactions that contain I. Perfect extensions have the following properties: (1) if the item i is a perfect extension of an item set I, then it is also a perfect extension of any item set $J \supseteq I$ as long as $i \notin J$, and (2) if I is a frequent item set and K is the set of all perfect extensions of I, then all sets $I \cup J$ with $J \in 2^K$ (where 2^K denotes the power set of K) are also frequent and have the same support as I. These properties can be exploited by collecting in the recursion not only prefix items but also, in a third element of a subproblem description, perfect extension items. Once identified, perfect extension items are no longer processed in the recursion but are only used to generate all supersets of the prefix that have the same support. Depending on the data set, this can lead to a considerable acceleration of the search.

7.6.3.2 Reducing the Output

Often three types of frequent item sets are distinguished by additional constraints:

- **frequent item set**
  A frequent item set is merely frequent:
  I is a frequent item set $\Leftrightarrow s_T(I) \geq s_{\min}$.
- **closed item set**
  A frequent item set is called *closed* if no proper superset has the same support:
  I is a closed item set $\Leftrightarrow s_T(I) \geq s_{\min} \wedge \forall J \supset I : s_T(J) < s_T(I)$.
- **maximal item set**
  A frequent item set is called *maximal* if no proper superset is frequent:
  I is a maximal item set $\Leftrightarrow s_T(I) \geq s_{\min} \wedge \forall J \supset I : s_T(J) < s_{\min}$.

Obviously, all maximal item sets are closed, and all closed item sets are frequent. The reason for distinguishing these item set types is that with them one can achieve a compressed representation of all frequent item sets: the set of all frequent item sets can be recovered from the maximal (or the closed) item sets by simply forming all subsets of maximal (or closed) item sets, because each frequent item set has a maximal superset. With closed item sets, one even preserves the knowledge of their support, because any frequent item set has a closed superset *with the same support*. Hence the support of a nonclosed frequent item set I can be reconstructed with

$$\forall s_{\min} : \forall I : \quad s_T(I) \geq s_{\min} \quad \Rightarrow \quad s_T(I) = \max_{J \in C_T(s_{\min}), J \supseteq I} s_T(J),$$

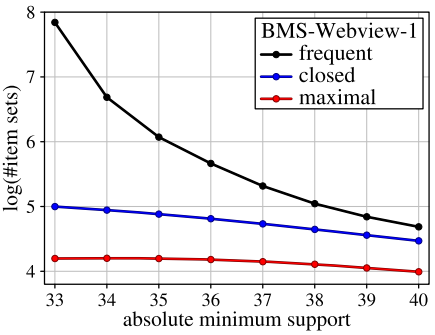


Fig. 7.24 The decimal logarithms of the numbers of frequent, closed, and maximal item sets (*vertical axis*) for the BMS-Webview-1 database and different minimum support values (*horizontal axis*)

where $C_T(s_{\min})$ is the set of all closed item sets that are discovered in a given transaction database T if the minimum support $s_{\min}$ is chosen.

Note that closed item sets are directly related to perfect extension pruning, since a closed item set can also be defined as an item set that does not possess a perfect extension. As a consequence, identifying perfect extensions is particularly useful and effective if the output is restricted to closed item sets.

For the simple example database shown in Fig. 7.17, all frequent item sets are closed, with the exception of $\{b\}$ (since $s_T(\{b\}) = s_T(\{b, c\}) = 3$) and $\{d, e\}$ (since $s_T(\{d, e\}) = s_T(\{a, d, e\}) = 4$). The maximal item sets are $\{b, c\}$, $\{a, c, d\}$, $\{a, c, e\}$, and $\{a, d, e\}$. Note that indeed any frequent item set is a subset of at least one of these four maximal item sets, demonstrating that they suffice to represent all frequent item sets. In order to give an impression of the relative number of frequent, closed, and maximal item sets in practice, Fig. 7.24 displays the decimal logarithms of the numbers of these item sets that are found for different minimum support values for a common benchmark data set, namely the BMS-Webview-1 database. Clearly, restricting the output to closed or even to maximal item sets can reduce the size of the output by orders of magnitude in this case.

Naturally, the notions of a closed and a maximal item set can easily be transfered to other pattern types, leading to closed and maximal sequences, trees, and (sub)graphs (with analogous definitions). Their purpose is the same in these cases—to reduce the output to a more manageable size. Nevertheless, however, the size of the output remains a serious problem in frequent pattern mining, despite several research efforts, for example, to define relevance or interestingness measures by which the best patterns can be singled out for user inspection.

7.6.3.3 Assessing Association Rules

In practice the number of found association rules is usually huge, often exceeding (by far) the number of transactions. Therefore considerable efforts have been devoted to filter them in order to single out the interesting ones, or at least to rank them. Here we study only a few simple methods, though often effective. More sophisticated approaches are discussed, for instance, in [60, 61].

The basic idea underlying all approaches described in the following is that a rule is interesting only if the presence of its antecedent has a sufficient effect on the presence of its consequent. In order to assess this, the confidence of a rule

	$A \not\subseteq t$	$A \subseteq t$	
$C \not\subseteq t$	n_{00}	n_{01}	n_{0*}
$C \subseteq t$	n_{10}	n_{11}	n_{1*}
	n_{*0}	n_{*1}	n_{**}

Fig. 7.25 A contingency table for the antecedent A and the consequent C of an association rule $A \to C$ with respect to the transactions t of a given transaction database T. Obviously, $n_{11} = s_T(A \cup C)$, $n_{1*} = s_T(C)$, $n_{*1} = s_T(A)$, and $n_{**} = s_T(\emptyset)$

is compared to its expected confidence under the assumption that antecedent and consequent are independent. If antecedent and consequent are independent, then $c_T(A \to C) = \hat{P}(C \mid A)$ and $c_T(\emptyset \to C) = \hat{P}(C)$ should not differ much (the true probabilities should actually be the same, but their estimates may differ slightly due to noise in the data), and thus we can assess association rules by comparing them.

The most straightforward measure based on this idea is the absolute difference of the two confidences, that is, $d_T(A \to C) = |c_T(A \to C) - c_T(\emptyset \to C)|$. However, it is more common to form their ratio, that is,

$$l_T(A \to C) = \frac{c_T(A \to C)}{c_T(\emptyset \to C)}.$$

This measure is known as the **lift value**, because it describes by what factor our expectation to observe C is lifted if we learn that A is present. Clearly, the higher the lift value, the more influence A has on C, and thus the more interesting the rule.

More sophisticated approaches evaluate the dependence of A and C based on the contingency table shown in Fig. 7.25, which lists the numbers of transactions which contain or do not contain A and C, alone and in conjunction. (Note that a star as a first index means that the value is a sum over a column, and a star in the second index means that the value is a sum over a row. Two stars mean a sum over both rows and columns.) Measures based on such a table are the well-known statistical χ^2 measure

$$\chi^2(A \to C) = \sum_{i=0}^{1} \sum_{j=0}^{1} \frac{(p_{i*}p_{*j} - p_{ij})^2}{p_{i*}p_{*j}} = \frac{n_{**}(n_{1*}n_{*1} - n_{**}n_{11})^2}{n_{1*}(n_{**} - n_{1*})n_{*1}(n_{**} - n_{*1})}$$

and **mutual information** or **information gain**

$$I_{\text{mut}}(A \to C) = \sum_{i=0}^{1} \sum_{j=0}^{1} p_{ij} \log_2 \frac{p_{ij}}{p_{i*}p_{*j}} = \sum_{i=0}^{1} \sum_{j=0}^{1} \frac{n_{ij}}{n_{**}} \log_2 \frac{n_{ij}n_{**}}{n_{i*}n_{*j}},$$

where $p_{kl} = n_{kl}/n_{**}$ for $k, l \in \{0, 1, *\}$. For both of these measures, a higher value means a stronger dependence of C on A and thus a higher interestingness of the rule.

It should be noted that such measures are only a simple aid, as they do not detect and properly assess situations like the following: suppose that $c_T(\emptyset \rightarrow C) = 20\%$ but $c_T(A \rightarrow C) = 80\%$. According to all of the above measures, the rule $A \rightarrow C$ is highly interesting. However, if we know that $c_T(A - \{i\} \rightarrow C) = 76\%$ for some item $i \in A$, the rule $A \rightarrow C$ does not look so interesting anymore: the change in confidence that is brought about by the item i is fairly small, and thus we should rather consider a rule with a smaller antecedent.

While this problem can at least be attacked by comparing a rule not only to its counterpart with an empty antecedent but to all rules with a simpler antecedent (even though this is more difficult than it may appear at first sight), semantic aspects of interestingness are completely neglected. For example, the rule "pregnant → female" has a confidence of 100%, while "$\emptyset$ → female" has a confidence of only 50%. Nevertheless the rule "pregnant → female" is obviously completely uninteresting, because it is part of our background knowledge (or can at least be derived from it).

7.6.3.4 Frequent Subgraph Mining

In analogy to frequent item set mining, with which item sets are found that are contained in a sufficiently large number of transactions of a given database (as specified by the minimum support), **frequent subgraph mining** tries to find (sub)graphs that are contained in a sufficiently large number of (attributed or labeled) graphs of a given graph database. Since the advent of this research area around the turn of the century, several clever algorithms for frequent subgraph mining have been developed. Some of them rely on principles from inductive logic programming and describe the graph structure by logical expressions [24]. However, the vast majority transfers techniques developed originally for frequent item set mining (see Sect. 7.6.2). Examples include MolFea [44], FSG [46], MoSS/MoFa [11], gSpan [65], CloseGraph [66], FFSM [39], and Gaston [48]. A related, but slightly different approach, is used in Subdue [17], which is geared toward graph compression with common subgraphs rather than frequent subgraph mining.

Here we confine ourselves to a brief outline of the core principles of frequent subgraph mining, because frequent sequence and tree mining can obviously be seen as special cases of frequent subgraph mining. Nevertheless, however, it should be noted that specific optimizations are possible due to the restricted structure of trees and those graphs to which different types of sequences can be mapped. Note also that this is the only place in this book where we consider the analysis of data that does not have a tabular structure. While we cannot provide a comprehensive coverage of all complex data types, we strive in this section to convey at least some idea of the special requirements and problem encountered when analyzing such data.

Formally, frequent subgraph mining works on a database of labeled graphs (also called *attributed graphs*). A **labeled graph** is a triple $G = (V, E, l)$, where V is the set of vertices, $E \subseteq V \times V - \{(v, v) \mid v \in V\}$ is the set of edges, and $l : V \cup E \rightarrow L$ assigns labels from some label set L to vertices and edges. The graphs we consider are **undirected** and **simple** (that is, there is at most one edge between two given

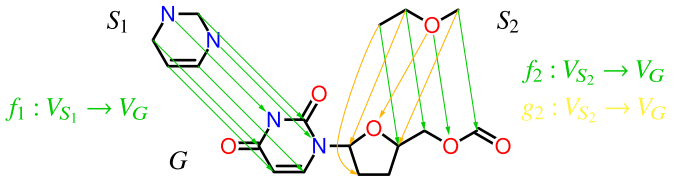


Fig. 7.26 Examples of subgraph isomorphisms with two molecular fragments and a molecule containing them in different ways

vertices) and contain **no loops** (that is, there are no edges connecting a vertex to itself). However, graphs without these restrictions (that is, directed graphs, graphs with loops and/or multiple edges) can be handled as well with properly adapted methods. Note also that several vertices and edges may carry the same label.

The **support** $s_{\mathcal{G}}(S)$ of a (sub)graph S with respect to a given graph database $\mathcal{G}$ is the number of graphs $G \in \mathcal{G}$ it is contained in. What is meant by a graph being contained in another is made formally precise by the notion of a *subgraph isomorphism*. Given two labeled graphs $G = (V_G, E_G, l_G)$ and $S = (V_S, E_S, l_S)$, a **subgraph isomorphism** of S to G is an injective function $f : V_S \to V_G$ satisfying (1) $\forall v \in V_S : l_S(v) = l_G(f(v))$ and (2) $\forall (u, v) \in E_S : (f(u), f(v)) \in E_G \wedge l_S((u, v)) = l_G((f(u), f(v)))$. That is, the mapping f preserves the connections and the labels of both vertices and edges.

Note that there may be several ways to map a labeled graph S to a labeled graph G so that the connection structure and the labels are preserved. For example, the graph G may possess several subgraphs that are isomorphic to S. It may even be that the graph S can be mapped in several different ways to the same subgraph of G. This is the case if there exists a subgraph isomorphism of S to itself (a so-called *graph automorphism*) that is not the identity. Examples of subgraph isomorphism with respect to a molecule G and two molecular fragments S_1 and S_2, all three of which are modeled as labeled graphs, are shown in Fig. 7.26. The fragment S_2 is contained in several different ways in the molecule, two of which are indicated in the figure.

Given a database $\mathcal{G}$ of labeled graphs and a user-specified **minimum support** $s_{\min} \in \mathbb{N}$, a (sub)graph S is called **frequent** in $\mathcal{G}$ if $s_{\mathcal{G}}(S) \geq s_{\min}$. The task of frequent subgraph mining is to identify all subgraphs that are frequent in the given graph database $\mathcal{G}$. However, usually the output is restricted to connected subgraphs for two reasons: in the first place, connected subgraphs suffice for most applications. Secondly, restricting the result to connected subgraphs considerably reduces the search space, thus rendering the search actually feasible.

Also in analogy to frequent item set mining, we consider the **partial order** of subgraphs of the graphs of a given database that is induced by the subgraph relationship. A Hasse diagram of an example partial order, for a simple database of three molecule-like graphs (no chemical meaning attached, to be seen at the bottom) is shown in Fig. 7.27 on the top: two subgraphs are connected if one possesses an additional edge (and an additional vertex, since there are no cycles) compared to

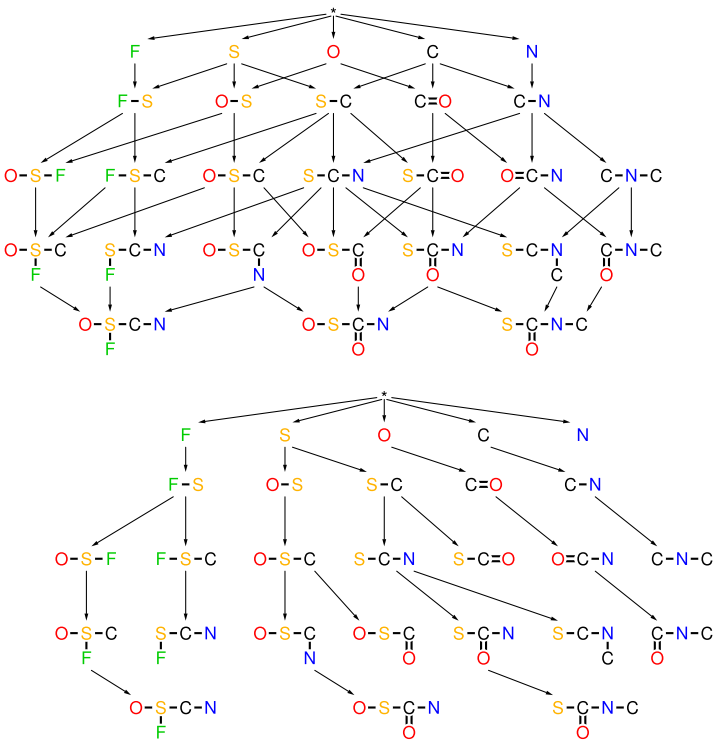


Fig. 7.27 A Hasse diagram of subgraphs of three molecule-like graphs, to be seen at the bottom (*top*) and an assignment of unique parents, which turns the Hasse diagram into a tree (*bottom*)

the other. The frequent subgraphs are located at the top of this Hasse diagram, thus suggesting a top-down search, just as for frequent item set mining.

Like for frequent item set mining (see Sect. 7.6.2), a core problem is that the same graph can be reached on different paths, as is easily visible from the Hasse diagram of subgraphs shown in Fig. 7.27. Hence we also face the task to prevent redundant search. The solution principle is the same as for frequent item sets: to each subgraph, a **unique parent** is assigned, which turns the subgraph partial order into a **subgraph tree**. This is illustrated in Fig. 7.27 on the bottom (the principle underlying this assignment of unique parents is discussed below).

With a unique parent for each subgraph, we can carry out the search for frequent subgraphs according to the following simple recursive scheme, which parallels the divide-and-conquer scheme described in Sect. 7.6.2: in a base loop, all possible vertex labels are traversed (their unique parent is the empty graph). All vertex labels (and thus all single vertex subgraphs) that are frequent are processed recursively. A given frequent subgraph S is processed recursively by forming all possible extensions R of S by a single edge and a vertex for which S is the chosen unique parent. All such extensions R that are frequent (that is, for which $s_{\mathcal{G}}(R) \geq s_{\min}$) are processed recursively, while infrequent extensions are discarded.

Whereas for frequent item sets, it was trivial to assign unique parents, namely by simply defining an (arbitrary but fixed) order of the items, ordering the labels in the set L, though also necessary, is not enough. The reason is mainly that several vertices (and several edges) may carry the same label. Hence the labels do not uniquely identify a vertex, thus rendering it impossible to describe the graph structure solely with these labels. We rather have to endow each vertex with a unique identifier (usually a number), so that we can unambiguously specify the edges of the graph.

Given an assignment of unique identifiers, we can describe a graph with a **code word** which specifies the vertex and edge labels and the connection structure, and from which the graph can be reconstructed. Of course, the form of this code word depends on the chosen numbering of the vertices: each numbering leads to a different code word. In order to single out one of these code words, the **canonical code word**, we simply select the lexicographically smallest code word.

With canonical code words, we can easily define unique parents: the canonical code word of a (sub)graph S is obtained with a specific numbering of its vertices and thus also fixes (maybe with some additional stipulation) an order of its edges. By removing the last edge according to this order, which is not a bridge or is incident to at least one vertex with degree 1 (which is then also removed), we obtain a graph that is exactly one level up in the partial order of subgraphs and thus may be chosen as the unique parent of S. Details about code words, together with a discussion of the important **prefix property** of code words, can be found, for example, in [10].

7.7 Deviation Analysis

The search for new and interesting association rules can be seen as a discovery of subgroups of the data set that share common properties, e.g., buy a certain product much more often than the average customer. Deviation analysis is another form of subgroup discovery. In contrast to association analysis, there is usually some target property given. The goal is to find subgroups of the populations that are statistically most interesting, that is, they are as large as possible and deviate from the whole population with respect to the property of interest as much as possible [42, 62]. For example, in a data set consisting of pricy car offers, most of them will have powerful engines with respectable cylinder capacity, so their fuel consumption will be quite high. If we look for a subgroup that deviates from the high-average consumption as much as possible, we may find a (relatively small) subgroup of pricy cars with significantly less fuel consumption having hybrid engine, aluminium body, etc.

7.7.1 Overview

In deviation analysis a pattern describes a divergence of some target measure in a subgroup of the full data set. The target measure is usually specified by the analyst, and subgroups with the largest deviations are sought by the respective method.

Cluster analysis may be considered as deviation analysis in the sense that the cluster represents a subgroup for which the data density deviates from the data density of the whole data set. Other possibilities are deviations in the distribution of a nominal target variable, the mean of a target variable, or more complex targets such as regression functions. While the methods described in Chaps. 8 and 9 also require a target attribute, their primary goal is the prediction of the target attribute for new data, which requires to build a model that covers the whole dataspace (rather than just subgroups). Furthermore, the successful discovery of a deviation in the distribution of some target attribute does not necessarily imply that we can easily predict the target value now (cf. a change in the distribution of 10 : 50 : 40 to 40 : 45 : 15).

To gain insights from the discovery, the respective subgroup should be easy to grasp by the analyst and is therefore best characterized by a conjunction of features (conditions on attributes like *fuel* = *liquid petrol gas*, *offered by* ≠ *dealer*, *mileage* > 100000, etc.). All possible combinations of constraints define jointly the set of possible subgroups. A search method is then applied to systematically explore this subgroup space. The target measure of each candidate subgroup is compared against the value obtained for, say, the whole data set, and a statistical test decides about the significance of a discovered deviation (under the null hypothesis that the subgroup does not differ from the whole data set). Only significant deviations are considered as interesting and reported to the analyst. The found subgroups are ranked to allow the search method to focus on the most interesting patterns and to point the user to the most promising findings. This ranking involves a quality function, which is usually a trade-off between size of the subgroup and its unusualness.

One of the major practical benefits of deviation analysis is that we do not even try to find a model for the full data set, because in practice such a model is either coarse and well known to the expert or very large and thus highly complex. Concentrating on the (potentially most interesting) highest deviation from the expected average and deriving short, understandable rules delivers invaluable insights.

7.7.2 Construction

The ingredients of deviation analysis are (1) the target measure and a verification test serving as a filter for irrelevant or incidental patterns, (2) a quality measure to rank subgroups, and (3) a search method that enumerates candidates subgroups systematically.

Any attribute of the data set may serve as the target measure, and any combination or derived variable, and even models themselves, may be considered (e.g., regression models [20]). Depending on the scale of the target attribute, an appropriate test has to be selected (see Fig. 7.28). For instance, for some binary target flag, we have a probability of $p_0 = 0.7$ in the full population of observing the flag F. Now, in a subgroup of size $n = 100$ we observe $m = 80$ cases where the flag is set, so $p = P(F) = 0.8$. Before we claim that we have found a deviation (and thus an interesting subgroup), we want to get confident that 80 cases or more are unlikely to occur by chance (say, only in 1%). We state the null hypothesis that $p = p_0$ and

data type/scale	target measure	possible test
binary	single share, e.g., percentage of females	binomial test, χ^2 test
nominal	vector of percentages, e.g. (0.3, 0.2, 0.5)	χ^2 goodness-of-fit
ordinal	median or probability of better value in subgroup	ridit analysis
ratio	mean, median, variance	median test, t-test

Fig. 7.28 Type of target measure

apply a one-sample z-test[3] with a significance level of 0.01. We have to compute the z-score

$$z = \frac{m - \mu}{\sigma} = \frac{80 - 70}{0.0458} \approx 2.18,$$

where $\mu = np_0$ is the mean, and σ is the standard deviation in the whole population, which is given by $\sigma = \sqrt{n\, p_0(1 - p_0)}$. A table of the standard normal distribution tells us that the probability of observing a z-value of 2.18 or above is approximately 0.0146. Thus, in $1 - 0.0146 = 98.54\%$ there will be less than 80 observed target flags. A significance level of 0.01 means that we reject the null-hypothesis if this situation occurs only in 1 out of 100 times—actually, it may occur in ≈1.5 out of 100 times, so the null-hypothesis cannot be rejected (at this significance level), and the subgroup is not reported. As we will perform quite a large number of tests, the significance level must be adjusted appropriately. If we test 1000 subgroups for being interesting and use a significance level of 0.01, we have to expect about 10 subgroups that pass this test (even if there is no substantial deviation).

All interesting subgroups are ranked according to their interestingness. One possibility is to reuse the outcome of the statistical test itself, as it expresses how unusual the finding is. In the above case, this could be the z-score itself:

$$z = \frac{m - n\, p_0}{\sqrt{n\, p_0(1 - p_0)}} = \frac{\sqrt{n}(p - p_0)}{\sqrt{p_0(1 - p_0)}}.$$

The z-score value balances the size of subgroup (factor of $\sqrt{n}$), the deviation (factor of $p - p_0$) and the level of the target share (p_0), which is important because an absolute deviation of $p_0 - p = 0.1$ may represent both, a small increase (from 0.9 to 1.0) or a doubled share (from 0.1 to 0.2). Another aspect that might be considered by an additional factor of $\sqrt{\frac{N}{N-n}}$ is the relative size of the subgroup with respect to the total population size. A frequently used quality function is the **weighted relative accuracy** (**WRAcc**), which is typically used for predictive tasks:

$$WRAcc = (p - p_0) \cdot \frac{n}{N}.$$

[3] As n is relatively large, we approximate the binomial distribution by a normal distribution. We use the z-test for reasons of simplicity—typically, p_0 is not known but has to be estimated from the sample, and then Student's t-test is used.

Table 7.12 Beam search for deviation analysis

Algorithm SubgrpDisc($\mathcal{D}$, F)

input: data set $\mathcal{D}$, $|\mathcal{D}| = n$, feature set F
parameter: beam width b
output: set of b interesting subgroups

```
1   initialize b subgroups with condition C_i and quality value q_i, 1 ≤ i ≤ b
2   do
3     for all subgroups (1 ≤ i ≤ b)
4       for all features f ∈ F
5         create new candidate subgroup with condition C_i ∧ f
6         evaluate subgroup and test for significant deviation
7         compute quality q of this new subgroup
8         if candidate is evaluated to be better than worst subgroup in beam
9           replace worst subgroup in beam by candidate
10  while beam has improved
```

However, the user may want to focus on other (application dependent) aspects such as practical usefulness, novelty, and simplicity and employ other quality function be used to rank the subgroups.

Since subgroups are to be described by a conjunction of features (such as *variable* = *value*), the number of possible subgroups on the syntactical level grows exponentially with the number of attributes and the number of possible values. Given an attribute A with $\Omega_A = \{high, medium, low, absent\}$, we may form four conditions on identity (e.g., $A = high$), four on imparity (e.g., $A \neq medium$), and as A has ordinal scale, even more features regarding the order (e.g., $A \leq low$ or $A \geq medium$). When constructing 11 features from A alone, another 9 attributes with 4 values would allow us to define $11^{10} \approx 25 \cdot 10^9$ syntactically different subgroups. Examining and testing each of them individually is apparently not an option.

A commonly applied search technique for deviation analysis is **beam search**. The algorithm keeps track of a fixed number b of subgroups (b is called **beam width**) and tries to further specialize all these subgroups, that is, make the subgroups more specific by including another feature in the subgroup description. If these candidate subgroups turn out to be interesting (according to the statistical test), they are evaluated with respect to their quality. In case they evaluate better than one of the b subgroups in the beam, they replace the worst subgroup. This process is iterated until the beam does no longer improve. The algorithm is shown in Table 7.12.

There are other search algorithms besides beam search. For instance, for a highly ranked subgroup, it is not necessarily the case that all subgroups obtained by taking prefixes from the condition list must also be interesting. If that is not the case, the beam search algorithm will not find such a subgroup, because it stops any further specialization of subgroups that were not successfully verified by the significance test. A broad view search would still keep such unverified subgroups for further expansion. Some quality measures, such as the J-measure [56], provide an upper bound for the quality that may be achieved in subsequent specializations. If such information is given, this optimistic estimate might be used to eliminate subgroups

only in case there is no chance of becoming one of the k top-ranked subgroups (as in [56]). Various heuristics may be implemented to direct the search to the most promising subgroups and to avoid delving into a large number of very similar subgroups [26, 41].

7.7.3 Variations and Issues

If a high quality subgroup is present, it is very likely that several variants of this subgroup exist, which are also evaluated as good subgroups (e.g., by adding a constraint $A \neq v$ for some attribute A and a rare feature v). Thus, it may happen that many of the subgroups in the beam are variations of a single scheme, which prevents the beam search from focussing on other parts of the dataspace—a small number of diverse subgroups would be preferable. A subset of diverse subsets can be extracted from the beam by selecting successively those subgroups that cover most of the data. Once a subgroup has been selected, the covered data is excluded from the subsequent selection steps [26]. If most of the subgroups in the beam are variations of one core subgroup, only a few diverse subgroups will be selected. A better approach is a sequential covering search, where good subgroups are discovered one after the other. Over several runs, only a few or a single best subgroup is identified, and the data covered by this subgroup is then excluded from subsequent runs. Thereby subgroups cannot be rediscovered, but the method has to focus on different parts of the dataspace. Similar techniques are applied for learning sets of classification rules (see Chap. 8). It is also possible to generate a new sample from the original data set that no longer exhibits the unusualness that has been discovered by a given subgroup [54]. If this subsampling is applied before any subsequent run, new subgroups rather than known subgroups will be discovered.

Another issue is the efficiency of search, the **scalability** to large data sets. Exhaustive searching is prohibitive unless intelligent pruning techniques are applied that prevent us from losing too much time with redundant, uninteresting subgroups. On the other hand, any kind of heuristic search (like beam search) bears the risk of missing the most interesting subgroups. There are multiple directions how to attack this problem.

For nominal target variables, efficient algorithms from association rule mining can be utilized. If it is possible to transform the database such that association mining (e.g., via the Apriori algorithm) can be applied, the validation and ranking of the patterns found are merely a postprocessing step [15]. Missing values require special care in this approach, as the case of missing data is usually not considered in market basket analysis. Furthermore, deviation analysis typically focusses on a target variable rather than associations between any attributes, which offers some optimization potential [6].

If the data set size becomes an issue, one may use a subsample to test and rank subgroups rather than the full data set. For a broad range of quality measures, one can derive upper bounds for the size of the sample with guaranteed upper bounds on the subgroup quality estimation error [53]. This speeds up the discovery process considerably, because there is no need for a full database scan.

7.8 Finding Patterns in Practice

As we have seen, finding patterns in data often consists of finding some sort of partitioning of the data: those may be hierarchical clusterings, typical patterns, embeddings of lower dimensional maps or subgroups. Any reasonably modern data analysis tool offers the classic clustering algorithm *k*-Means and provides association rule mining methods. All the different variants discussed in this chapter will usually only be available in certain flavors, however.

7.8.1 Hierarchical Clustering

KNIME Analytics Platform offers a number of nodes for pattern finding. Hierarchical clustering is, of course, among them with two possible implementations: a simple one in the *Hierarchical Clustering* node and a more complex one in the *Hierarchical Clustering (DistMatrix)* node.

As we have discussed in Sect. 7.2 the underlying notion of (dis)similarity plays a crucial role when using clustering algorithms. KNIME Analytics Platform therefore also allows one to use arbitrary distance matrices for some of the clustering methods.

In particular, KNIME Analytics Platform provides nodes to read in distance matrices but also various nodes to compute distance matrices on other than just numerical feature vectors. Using this functionality, one can use (dis)similarities metrics between documents, molecular structures, images, and other objects and the cluster, those using some of the distance-matrix clustering nodes. The *Hierarchical Clustering (DistMatrix)* node is one of those nodes.

The usage of the *Hierarchical Clustering* node is relatively straightforward. The user selects the (numerical[4]) columns to be used along with the distance function and the type of linkage. The node will provide the cluster assignment of the input on its outport, and for this, the user can specify the cutoff, i.e., the number of clusters to be created.

Figure 7.29 shows the workflow implementing hierarchical clustering with the simple Hierarchical Clustering node and with the more complex sequence of nodes, including the Distance Matrix Calculate node.

Figure 7.30 shows a dendrogram for the iris data obtained with Euclidean distance and complete linkage. Notice the branch that was selected in the interactive view, therefore in orange, and the threshold line in red. Moving the threshold line changes the number of clusters and the assignment for the input rows.

7.8.2 *k*-Means and DBSCAN

The standard method for prototype based clustering is provided via the *k-Means* node. The options here are again the selection of numerical columns to be used and additionally the number of clusters to build and the maximum number of iterations.

[4]We will discuss the use of other types of distance metrics in KNIME Analytics Platform later.

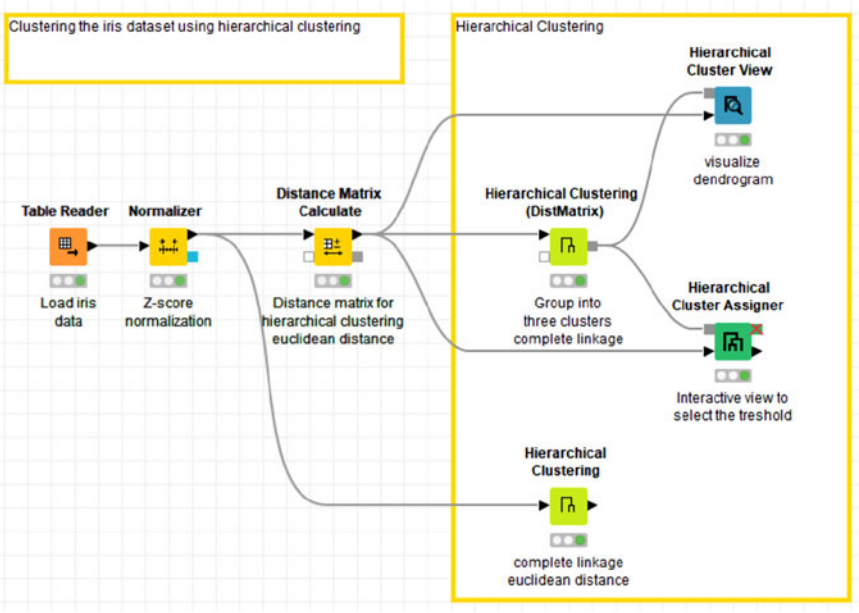


Fig. 7.29 Workflow implementing hierarchical clustering with the simple Hierarchical Clustering node and with the more complex sequence of nodes, including the Distance Matrix Calculate node

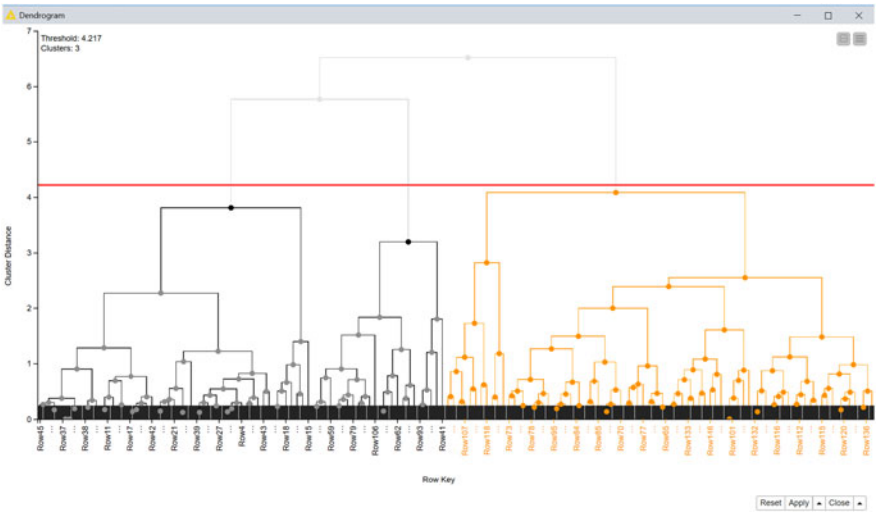


Fig. 7.30 A dendrogram for the iris data obtained with Euclidean distance and complete linkage. Notice the branch that was selected in the interactive view, therefore in orange, and the threshold line in red. Moving the threshold line changes the number of clusters and the assignment for the input rows

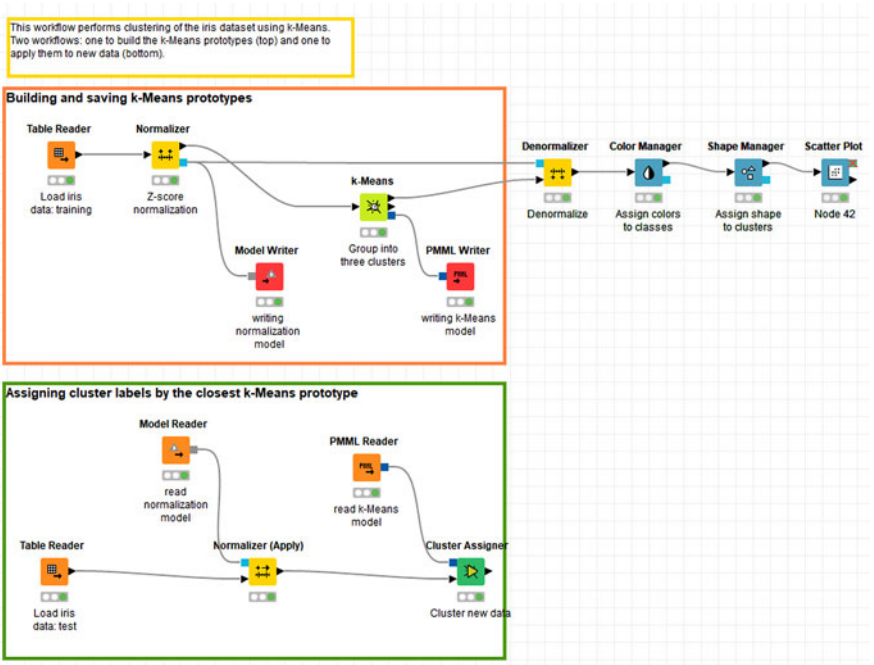


Fig. 7.31 Building of *k*-Means prototypes *(top)* and cluster assignment *(bottom)*

The latter is usually not required, but in the (rare) case of oscillations and hence nontermination of the underlying algorithm, KNIME Analytics Platform will force termination after reaching this number of iterations. The node *k-Means* has three outports: one carrying the input data together with the cluster labels; a second port carrying the prototypes; and a third port holding the cluster model. This model can be exported for use in other tools (using, e.g., the Model Writer or the PMML Writer node), or it can be used to apply the clustering model to new data. The *Cluster Assigner* node will compute the closest cluster prototype for each pattern and assign the corresponding label. Figure 7.31 shows two small workflows to demonstrate this.

The part on the top reads a file, normalizes the data, runs the *k*-Means algorithm, and writes the normalization formula in a model file and the resulting cluster model into a PMML file. It also de-normalizes the data and produces some visualization. The flow on the bottom reads the normalization model, the *k*-Means model, and a second data file and applies the cluster model to this new data.

PMML (Predictive Model Markup Language) is a standard XML based text format to store models and data transformations. .PMML files store models and data transformations in PMML format. On the opposite, .model files store models and data transformations according to a KNIME format, optimized for size and speed.

Of course, in this little example we could have simply connected the clustering node to the node assigning the cluster model, but this demonstrates how we can use one analysis application (or tool) to create a clustering and then use a second

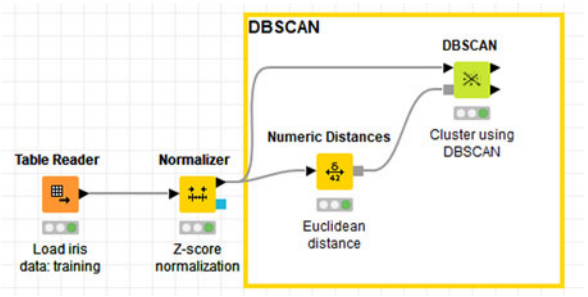


Fig. 7.32 This workflow implements a DBSCAN clustering on the iris data set. Notice the possibility to choose the distance metric in the Numeric Distances node

application (or tool), maybe a nightly batch job, for the cluster assignment. We will discuss this aspect of model deployment in more detail in Chap. 10. KNIME Analytics Platform also provides nodes which allow one to apply the fuzzy version of k-Means, fuzzy c-Means and the k-Medoids.

KNIME Analytics Platform also offers a DBSCAN node. An example of the DBSCAN implementation is shown in Fig. 7.32. Here the workflow performs a clustering of the iris data set using the DBSCAN algorithm. Notice the *Numeric Distances* node to feed the DBSCAN node with the matrix of the data to data distances. Distance type can be selected within the Numeric Distances node.

7.8.3 Association Rule Mining

KNIME Analytics Platform also offers nodes for association rule mining. An example of the implementation of the A-priori algorithm can be seen in Fig. 7.33 via the *Association Rule Learner (Borgelt)* node. The node *Item Set Finder (Borgelt)* is then used to match the current item set with any of the rule antecedents in the rule set.

Through various different extensions, KNIME Analytics Platform can be extended to deal with other types of data such as documents, images, and also molecular structures. Once the appropriate extension is installed (which are available online as well), KNIME Analytics Platform can read in different molecular file formats and a set of new nodes is available which can deal with such structures. One of them implements the MoFa algorithm discussed in Sect. 7.6.3.4. The algorithm can be applied very similarly to all other nodes. One or more columns in the input table should hold molecules, which are then also shown in a two-dimensional representation since the chemistry plugin for KNIME also brings along an open-source renderer via the Chemistry Development Kit (CDK [57]).

7.9 Further Reading

There are quite a number of books about cluster analysis. A good reference to start is [23]. A review of subspace clustering methods can be found in [49]. A review on clustering of streaming data can be found in [28, 55]. The deep learning paradigm

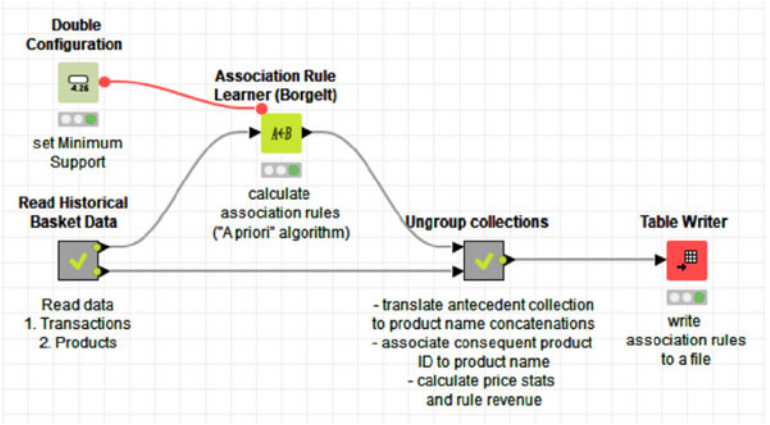


Fig. 7.33 Building association rules for Market Basket Analysis

(cf. Sect. 9.3) allows tackling the problem of data representation and clustering at the same time [47, 64]. Self-organizing maps are mentioned in almost every book about neural networks, but only a few books are more or less dedicated to them [40, 51]. The usefulness of SOMs for visualization is shown in [58]. When unsure about the best clustering approach, the **validity measures** in, e.g., [36] allow a comparison across different clustering paradigms by considering the robustness and the consistency of the results.

The website http://fimi.cs.helsinki.fi/ provides a comparison and the source code of many different frequent itemset mining implementations (and online proceedings of two workshops). There is also an open source pattern mining library with many Java implementations [25].

References

1. Aggarwal, C.C., Wolf, J.L., Yu, P.S., Procopiuc, C., Park, J.S.: Fast algorithms for projected clustering. In: Proc. 1999 ACM SIGMOD Int. Conf. on Management of Data, pp. 61–72. ACM Press, New York (1999)
2. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules. In: Proc. 20th Int. Conf. on very Large Databases (VLDB 1994, Santiago de Chile), pp. 487–499. Morgan Kaufmann, San Mateo (1994)
3. Agrawal, R., Mannila, H., Srikant, R., Toivonen, H., Verkamo, A.: Fast discovery of association rules. In: Fayyad, U.M., Piatetsky-Shapiro, G., Smyth, P., Uthurusamy, R. (eds.): Advances in Knowledge Discovery and Data Mining, pp. 307–328. AAAI Press/MIT Press, Cambridge (1996)
4. Agrawal, R., Gehrke, J., Gunopulos, D., Raghavan, P.: Automatic subspace clustering of high dimensional data. Data Min. Knowl. Discov. **11**, 5–33 (2005)
5. Ankerst, M., Breunig, M.M., Kriegel, H.-P., Sander, J.: OPTICS: ordering points to identify the clustering structure. In: ICMD (Philadelphia), pp. 49–60 (1999)
6. Atzmueller, M., Puppe, F.: Sd-map: a fast algorithm for exhaustive subgroup discovery. In: Proc. Int. Conf. Knowledge Discovery in Databases (PKDD). Lecture Notes in Computer Science, vol. 4213. Springer, Berlin (2006)

7. Baumgartner, C., Plant, C., Kailing, K., Kriegel, H.-P., Kröger, P.: Subspace selection for clustering high-dimensional data. In: Proc. IEEE Int. Conf. on Data Mining, pp. 11–18. IEEE Press, Piscataway (2003)
8. Bayardo, R., Goethals, B., Zaki, M.J. (eds.): Proc. Workshop Frequent Item Set Mining Implementations (FIMI 2004, Brighton, UK), CEUR Workshop Proceedings 126, Aachen, Germany (2004). http://www.ceur-ws.org/Vol-126/
9. Bellman, R.: Adaptive Control Processes. Princeton University Press, Princeton (1961)
10. Borgelt, C.: On canonical forms for frequent graph mining. In: Proc. 3rd Int. Workshop on Mining Graphs, Trees and Sequences (MGTS'05, Porto, Portugal), pp. 1–12. ECML/PKDD 2005 Organization Committee, Porto (2005)
11. Borgelt, C., Berthold, M.R.: Mining molecular fragments: finding relevant substructures of molecules. In: Proc. IEEE Int. Conf. on Data Mining (ICDM 2002, Maebashi, Japan), pp. 51–58. IEEE Press, Piscataway (2002)
12. Borgelt, C., Wang, X.: SaM: a split and merge algorithm for fuzzy frequent item set mining. In: Proc. Fuzzy Systems Association World Congress and Conf. European Society for Fuzzy Logic and Technology (IFSA/EUSFLAT'09, Lisbon, Portugal), pp. 968–973 (2009)
13. Böttcher, M., Spott, M., Nauck, D.: Detecting temporally redundant association rules. In: Proc. 4th Int. Conf. on Machine Learning and Applications (ICMLA 2005, Los Angeles, CA), pp. 397–403. IEEE Press, Piscataway (2005)
14. Böttcher, M., Spott, M., Nauck, D.: A framework for discovering and analyzing changing customer segments. In: Advances in Data Mining—Theoretical Aspects and Applications. Lecture Notes in Computer Science, vol. 4597, pp. 255–268. Springer, Berlin (2007)
15. Branko, K., Lavrac, N.: APRIORI-SD: adapting association rule learning to subgroup discovery. Appl. Artif. Intell. **20**(7), 543–583 (2006)
16. Cheng, Y., Fayyad, U., Bradley, P.S.: Efficient discovery of error-tolerant frequent itemsets in high dimensions. In: Proc. 7th Int. Conf. on Knowledge Discovery and Data Mining (KDD'01, San Francisco, CA), pp. 194–203. ACM Press, New York (2001)
17. Cook, D.J., Holder, L.B.: Graph-based data mining. IEEE Trans. Intell. Syst. **15**(2), 32–41 (2000)
18. Davé, R.N.: Characterization and detection of noise in clustering. Pattern Recognit. Lett. **12**, 657–664 (1991)
19. Ding, C., He, X.: Cluster merging and splitting in hierarchical clustering algorithms. In: Proc. IEEE Int. Conference on Data Mining, p. 139. IEEE Press, Piscataway (2002)
20. Duivestein, W., Feelders, A., Knobbe, A.: Exceptional model mining. Data Min. Knowl. Discov.. **30**(1), 47–98 (2016)
21. Dunn, J.: Well separated clusters and optimal fuzzy partitions. J. Cybern. **4**, 95–104 (1974)
22. Ester, M., Kriegel, H.-P., Sander, J., Xiaowei, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD 96, Portland, Oregon), pp. 226–231. AAAI Press, Menlo Park (1996)
23. Everitt, B.S., Landau, S., Leese, M.: Cluster Analysis. Wiley, Chichester (2001)
24. Finn, P.W., Muggleton, S., Page, D., Srinivasan, A.: Pharmacore discovery using the inductive logic programming system PROGOL. Mach. Learn. **30**(2–3), 241–270 (1998)
25. Fournier-Viger, P., Gomariz, A., Gueniche, T., Soltani, A., Wu, C.W., Tseng, V.S.: SPMF: a Java open-source pattern mining library. J. Mach. Learn. Res. **15**(1), 3389–3393 (2014)
26. Gamberger, D., Lavrac, N.: Expert-guided subgroup discovery: methodology and application. J. Artif. Intell. Res. **17**, 501–527 (2007)
27. Goethals, B., Zaki, M.J. (eds.): Proc. Workshop Frequent Item Set Mining Implementations (FIMI 2003, Melbourne, FL, USA), CEUR Workshop Proceedings 90, Aachen, Germany (2003). http://www.ceur-ws.org/Vol-90/
28. Guha, S., Mishra, N.: Clustering data streams. In: Data Stream Management, pp. 169–187. Springer, Berlin (2016)
29. Guha, S., Rastogi, R., Shim, K.: ROCK: a robust clustering algorithm for categorical attributes. Inf. Syst. **25**(5), 345–366 (2000)

30. Halkidi, M., Batistakis, Y., Vazirgiannis, M.: On clustering validation techniques. J. Intell. Inf. Syst. **17**(2–3), 107–145 (2001)
31. Han, J., Pei, H., Yin, Y.: Mining frequent patterns without candidate generation. In: Proc. Conf. on the Management of Data (SIGMOD'00, Dallas, TX), pp. 1–12. ACM Press, New York (2000)
32. Hinneburg, A., Keim, D.A.: An efficient approach to clustering in large multimedia databases with noise. In: Proc. 4th Int. Conf. on Knowledge Discovery and Data Mining (KDD), pp. 224–228. AAAI Press, Menlo Park (1998)
33. Hinneburg, A., Keim, D.A.: Optimal grid-clustering: towards breaking the curse of dimensionality in high-dimensional clustering. In: Proc. 25th Int. Conf. on Very Large Databases, pp. 506–517. Morgan Kaufmann, San Mateo (1999)
34. Höppner, F.: Speeding up Fuzzy C-means: using a hierarchical data organisation to control the precision of membership calculation. Fuzzy Sets Syst. **128**(3), 365–378 (2002)
35. Höppner, F.: A subspace filter supporting the discovery of small clusters in very noisy datasets. In: Proc. 26th Int. Conf. Scientific and Statistical Database Management. ACM, New York (2014)
36. Höppner, F., Jahnke, M.: Holistic assessment of structure discovery capabilities of clustering algorithms. In: Proc. Europ. Conf. Machine Learning and Knowledge Discovery in Databases. Lecture Notes in Computer Science. Springer, Berlin (2019)
37. Höppner, F., Klawonn, F.: A contribution to convergence theory of fuzzy C-means and derivatives. IEEE Trans. Fuzzy Syst. **11**(5), 682–694 (2003)
38. Höppner, F., Klawonn, F., Kruse, R., Runkler, T.A.: Fuzzy Cluster Analysis. Wiley, Chichester (1999)
39. Huan, J., Wang, W., Prins, J.: Efficient mining of frequent subgraphs in the presence of isomorphism. In: Proc. 3rd IEEE Int. Conf. on Data Mining (ICDM 2003, Melbourne, FL), pp. 549–552. IEEE Press, Piscataway (2003)
40. Kaski, S., Oja, E., Oja, E.: Kohonen Maps. Elsevier, Amsterdam (1999)
41. Klösgen, W.: Efficient discovery of interesting statements in databases. J. Intell. Inf. Syst. **4**, 53–69 (1995)
42. Klösgen, W.: Explora: a multipattern and multistrategy discovery assistant. In: Advances in Knowledge Discovery and Data Mining. MIT Press, Cambridge (1996). Chapter 10
43. Kohonen, T.: The self-organizing map. Proc. IEEE **78**, 1464–1480 (1990)
44. Kramer, S., de Raedt, L., Helma, C.: Molecular feature mining in HIV data. In: Proc. 7th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD 2001, San Francisco, CA), pp. 136–143. ACM Press, New York (2001)
45. Kriegel, H.P., Kröger, P., Zimek, A.: Subspace clustering. Data Min. Knowl. Discov. **2**, 351–364 (2012)
46. Kuramochi, M., Karypis, G.: Frequent subgraph discovery. In: Proc. 1st IEEE Int. Conf. on Data Mining (ICDM 2001, San Jose, CA), pp. 313–320. IEEE Press, Piscataway (2001)
47. Min, E., Guo, X., Qiang, L.: Zhang, G., Cui, J., Long, J.: A survey of clustering with deep learning: from the perspective of network architecture. IEEE Access **6**, 39501–39514 (2018)
48. Nijssen, S., Kok, J.N.: A quickstart in frequent structure mining can make a difference. In: Proc. 10th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD2004, Seattle, WA), pp. 647–652. ACM Press, New York (2004)
49. Parsons, L., Haque, E., Liu, H.: Subspace clustering for high dimensional data: a review. SIGKDD Explor. Newsl. **6**(1), 90–105 (2004)
50. Pei, J., Tung, A.K.H., Han, J.: Fault-tolerant frequent pattern mining: problems and challenges. In: Proc. ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery (DMK'01, Santa Babara, CA). ACM Press, New York (2001)
51. Ritter, H., Martinez, T., Schulten, K.: Neural Computation and Self-organizing Maps: An Introduction. Addison-Wesley, Reading (1992)
52. Rousseeuw, P.J.: Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. J. Comput. Appl. Math. **20**, 53–65 (1987)

53. Scheffer, T., Wrobel, S.: Finding the most interesting patterns in a database quickly by using sequential sampling. J. Mach. Learn. Res. **3**, 833–862 (2003)
54. Scholz, M.: Sampling-based sequential subgroup mining. In: Proc. 11th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, pp. 265–274. AAAI Press, Menlo Park (2005)
55. Silva, J.A., Faria, E.R, Barros, R.C. et al.: Data stream clustering: a survey. ACM Comput. Surv. **46**(1), 13 (2013)
56. Smyth, P., Goodman, R.M.: An information theoretic approach to rule induction from databases. IEEE Trans. Knowl. Discov. Eng. **4**(4), 301–316 (1992)
57. Steinbeck, C., Han, Y., Kuhn, S., Horlacher, O., Luttmann, E., Willighagen, E.: The chemistry development kit (CDK): an open-source Java library for chemo- and bioinformatics. J. Chem. Inf. Comput. Sci. **43**(2), 493–500 (2003)
58. Vesanto, J.: SOM-based data visualization methods. Intell. Data Anal. **3**(2), 111–126 (1999)
59. Wang, X., Borgelt, C., Kruse, R.: Mining fuzzy frequent item sets. In: Proc. 11th Int. Fuzzy Systems Association World Congress (IFSA'05, Beijing, China), pp. 528–533. Tsinghua University Press/Springer, Beijing/Heidelberg (2005)
60. Webb, G.I.: Discovering significant patterns. Mach. Learn. **68**(1), 1–33 (2007)
61. Webb, G.I., Zhang, S.: *k*-Optimal-rule-discovery. Data Min. Knowl. Discov. **10**(1), 39–79 (2005)
62. Wrobel, S.: An algorithm for multi-relational discovery of subgroups. In: Proc. 1st Europ. Symp. on Principles of Data Mining and Knowledge Discovery. Lecture Notes in Computer Science, vol. 1263, pp. 78–87. Springer, London (1997)
63. Xie, X.L., Beni, G.A.: Validity measure for fuzzy clustering. IEEE Trans. Pattern Anal. Mach. Intell. (PAMI) **3**(8), 841–846 (1991)
64. Xie, J., Girshick, R., Farhadi, A.: Unsupervised deep embedding for clustering analysis. In: Int. Conf. on Machine Learning, pp. 478–487 (2016)
65. Yan, X., Han, J.: gSpan: graph-based substructure pattern mining. In: Proc. 2nd IEEE Int. Conf. on Data Mining (ICDM 2003, Maebashi, Japan), pp. 721–724. IEEE Press, Piscataway (2002)
66. Yan, X., Han, J.: Close-graph: mining closed frequent graph patterns. In: Proc. 9th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD 2003, Washington, DC), pp. 286–295. ACM Press, New York (2003)
67. Zaki, M., Parthasarathy, S., Ogihara, M., Li, W.: New algorithms for fast discovery of association rules. In: Proc. 3rd Int. Conf. on Knowledge Discovery and Data Mining (KDD'97, Newport Beach, CA), pp. 283–296. AAAI Press, Menlo Park (1997)
68. Zhang, T., Ramakrishnan, R., Livny, M.: BIRCH: a new data clustering algorithm and its applications. Data Min. Knowl. Discov. **1**(2), 141–182 (1997)
69. Zhao, Y., Karypis, G., Fayyad, U.: Hierarchical clustering algorithms for document datasets. Data Min. Knowl. Discov. **10**, 141–168 (2005)

Finding Explanations

8

In the previous chapter we discussed methods that find patterns of different shapes in data sets. All these methods needed measures of similarity in order to group similar objects. In this chapter we will discuss methods that address a very different setup: instead of finding structure in a data set, we are now focusing on methods that find explanations for an unknown dependency within the data. Such a search for a dependency usually focuses on a so-called target attribute, which means we are particularly interested in why one specific attribute has a certain value. In case of the target attribute being a nominal variable, we are talking about a **classification problem**; in case of a numerical value we are referring to a **regression problem**. Examples for such problems would be understanding why a customer belongs to the category of people who cancel their account (e.g., classifying her into a yes/no category) or better understanding the risk factors of customers in general.

We are therefore assuming that, in addition to the object description $\mathbf{x}$, we have access to a value for the target attribute Y. In contrast to the methods described in previous chapters, where no target value was available, we now aim to model a dependency towards one particular attribute. Since this can be compared to a teacher model, where the teacher gives us the desired output, this is often called **supervised learning**. Our data set in this setup consists of tuples $\mathcal{D} = \{(\mathbf{x}_i, Y_i)|i = 1, \ldots, n\}$. Based on this data, we are interested in finding an explanation in the form of some type of interpretable model, which allows us to understand the dependency of the target variable to the input vectors. The focus of these models lies on their interpretability. In a subsequent chapter we will focus on the quality of the forecasting or predictive ability. Here we are interested in supervised (because we know the desired outcome) and descriptive (because we care about the explanation) data analysis. It is important to emphasize immediately that this model will not necessarily express a causal relationship, that is, the true underlying cause of how the outcome depends on the inputs. All we can extract (automatically, at least) from the given data are relationships expressed by some type of numerical correlations.

This chapter will cover the main representatives of model-based explanation methods: decision trees, Bayes classifiers, regression models, and rule extraction methods. These four types of explanation finding methods nicely cover four rather different flavors.

© Springer Nature Switzerland AG 2020

M.R. Berthold et al., *Guide to Intelligent Data Science*,
https://doi.org/10.1007/978-3-030-45574-3_8

Decision trees Decision trees aim to find a hierarchical structure to explain how different areas in the input space correspond to different outcomes. The hierarchical way to partition this space is particularly useful for applications where we drown in a series of attributes of unknown importance: the final decision tree often only uses a small subset of the available set of attributes. Decision trees are often thrown first at classification problems because they tend to be insensitive to normalization issues and tolerant toward many correlated or noisy attributes. In addition, the structure of the tree also allows for data clean-up. Quite often, the first attempt at generating a decision tree reveals unexpected dependencies in the data which would otherwise be hidden in a more complex model.

Bayes classifiers Bayes classifiers form a solid baseline for achievable classification accuracy—any other model should perform at least as well as the naive Bayes classifier. In addition they allow for quick inspection of possible correlations between any given input attribute and the target value. Before trying to apply more complex models, a quick look at a Bayes classifier can be helpful to get a feeling for realistic accuracy expectations and simple dependencies in the data. Bayes classifiers express their model in terms of simple probabilities (or parameters of simple probability distributions, such as mean and standard deviation) and hence explain nicely how the classifier works.

Regression models Regression models are the counterpart for numerical approximation problems. Instead of finding a classifier and minimizing the classification error, regression models try to minimize the approximation error, that is, some measure for the average deviation between the expected and predicted numerical output value. Again, many more complex models exist, but the regression coefficients allow for easy access to the internals of the model. Each coefficient shows the dependency between any input attribute and the target value.

Rule models Rule models are the most intuitive, interpretable representation. However, not many efficient or usable algorithms exist to date for complex real-world data sets. As a result, rule methods are generally not the first choice when it comes to finding explanations in complex data sets. We still discuss some of the most prominent methods in this chapter because the knowledge representation is best suited to offer insights into the data set and some of those methods deserve a bit more attention in data analysis setups. Generally, one would apply rule extraction algorithms only to data sets with a reasonably well-understood structure. Many of the algorithms tend to be rather sensitive toward useless or highly correlated attributes and excessive noise in the data.

8.1 Decision Trees

We start this chapter with what are likely to be the most prominent and heavily used methods for interpretable model extraction from large data sets, namely Decision Trees. They not only allow one to model complex relationships in an easy-to-understand manner, but additionally efficient algorithms exist to build such models

from large data sets. Ross Quinlan's c4.5 [21] and CART [5] started the boom, and nowadays practically every data mining toolkit offers at least one algorithm to construct decision trees from data sets.

We will start by explaining what forms of a decision tree exist, then cover the most prominent training algorithm ID3 (which, with various extensions, ends up forming c4.5), continue by outlining some extensions and issues of decision trees, and end this section by showing how decision trees can be built in practice.

8.1.1 Overview

Decision trees come in two flavors, classification and regression trees. We will first concentrate on the former, since classification problems are more common and most training algorithms and other issues can easily be generalized to regression problems as well. We will discuss issues particularly related to regression trees at the end of this section. Figure 8.1 shows a typical example of a classification tree. The simple tree can be used to classify animals, based on a number of attributes (the ability to swim and fly, in this case). As usual in computer science related areas, trees grow from top to bottom. The tree then builds a hierarchical decision structure which helps understand the classification process by traversing the tree from the root node until a leaf is reached. At each intermediate node, the relevant attribute is investigated and the branch matching the attribute's value is followed. The leaves then hold the classifications. Obviously, the tree in our example does not work correctly for all types of animals—we have already discussed this issue of generalization and performance on unseen cases in Chap. 5. However, it is a reasonable compact representation of a generic classification mechanism for animals based on easy to observe attributes and—more importantly—it summarizes (most of) our data.

Note how the tree in Fig. 8.1 uses different types of splits. In practice there are really three options:

1. Boolean splits consider boolean attributes and have two child nodes. An example for such a split would be an attribute "married" with two leaves "yes" and "no."
2. Nominal splits are based on nominal attributes and can be binary, that is, they split based on two disjoint subsets of the attribute values. Sometimes splits with more children are also considered—in the extreme for each nominal value one.

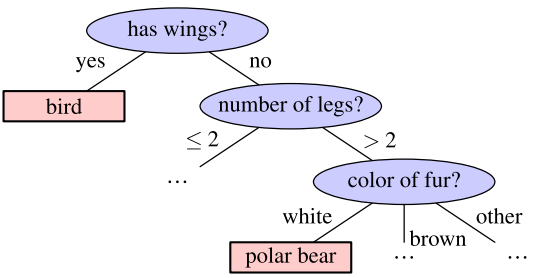


Fig. 8.1 A simple example for a decision tree to classify animals

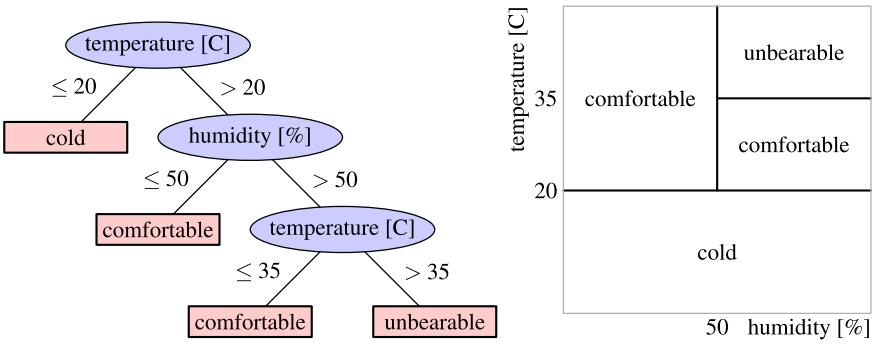


Fig. 8.2 A simple example for a decision tree using two numerical attributes (*left*) and the corresponding partitioning of the underlying feature space (*right*)

An example for such a split would be connected to the boolean split "married=no" above and could split into "never", "divorced", and "widow(er)". Note that one can also model such a split with a sequence of nodes splitting the same attribute into increasingly smaller subsets. We will see in the section on decision tree construction how such potentially very wide splits can lead to rather useless trees.

3. Splits on continuous attributes use a numerical variable and can either split based on one particular value (e.g., "temperature" with two splits ≤80 F and >80 F) or on a series of values defining bins for each child.

Decision tree algorithms are also known as **recursive partitioning** methods since each split of the tree essentially divides the remaining space into two (or more in case of nonbinary splits) disjoint subpartitions. Figure 8.2 shows an example of a tree operating on two numerical attributes and illustrates the corresponding partitioning of the underlying feature space.

Decision trees are well received by users because they are easy to read and interpret. Additionally, the recursive partitioning approach seems to resemble the hierarchical structuring of classification problems by humans.

8.1.2 Construction

From a data analysis point of view it is now of course interesting to know how we can generate such simple, easy to understand structures from real-world data sets. Finding the optimal decision tree for a given set of training examples is nontrivial, but in most cases it is sufficient to find a reasonably small tree, which explains the training data well. Note also that "optimal" is not obvious to define: Do we mean the smallest tree or the one with the best accuracy on the training data. And if we are indeed interested in the smallest tree, does this relate to the number of nodes or the depth or width of the tree? At the end of this chapter we will discuss this issue of "learning bias" in more detail.

Table 8.1 The basic decision tree induction algorithm

Algorithm BuildDecisionTree($\mathcal{D}$, $\mathcal{A}$)

input: training data $\mathcal{D}$, set $\mathcal{A}$ of available attributes
output: a decision tree matching $\mathcal{D}$, using all or a subset of $\mathcal{A}$

```
1     if all elements in D belong to one class
2       return node with corresponding class label
3     elseif A = ∅
4       return node with majority class label in D
5     else
6       select attribute A ∈ A which best classifies D
7       create new node holding decision attribute A
8       for each split v_A of A
9         add new branch below with corresponding test for this split
10        create D(v_A) ⊂ D for which split condition holds
11        if D(v_A) = ∅
12          return node with majority class label in D
13        else
14          add subtree returned by calling
                BuildDecisionTree(D(v_A), A\{A})
15        endif
16      endfor
17      return node.
18    endif
```

The most prominent algorithms therefore do not attempt to optimize a global measure but employ a greedy strategy, that is, they focus on building the tree root-first and then add subsequent branches and splits along those branches for the remainder of the training data—they recursively find the best split at each point. Generally, such an algorithm looks like the one shown in Table 8.1, where $\mathcal{D}$ indicates the available training examples, $\mathcal{C}$ the target attribute, and $\mathcal{A}$ the set of available input attributes.

The algorithm recursively splits up the data and constructs the decision tree starting with its root node. The recursion stops at steps 1 or 3 if a subset of patterns of only one class is encountered (the resulting tree is then simply a leaf carrying that class label) or no further attributes for splits are available (the leaf then carries the label of the majority class in the subset). Otherwise we find the best split for the subset at hand (line 6) and create a new split node. We then recursively call the tree construction method on the subsets created by applying the chosen split (lines 8–14).

Looking at this algorithm, two issues remain unclear:

- How do we determine which of the attributes is the "best" one?
- Which kind of splits are possible?

The second question is already answered above: for nominal attributes, we can either carve out one value, resulting in a binary split or split the set of values into two

or more subsets ultimately leading to a split into as many branches as there exist attribute values. (We will discuss other types of attributes later.)

If we focus on the last option, we are dealing with ID3, the classic decision tree learning algorithm [20].[1] ID3 only constructs decision trees for training data with nominal values and considers solely wide splits, that is, splits into all possible values of an attribute in a single node. This makes the choice of possible splits simpler (only one choice exists); however, we still do not know how to pick the attribute resulting in the "best" split.

Intuitively, a best split would be one which helps us create leaves as soon as possible. Leaves are created only if we reach a branch holding a subset with training patterns, all belonging to one class (or, of course, if the subset is empty, but that is an extreme case). So we need to make sure that we quickly create subsets that have mostly patterns of one class. An often used measure for such an **impurity** is the Shannon entropy, which measures the impurity of a labeled set, based on the column of labels $\mathcal{C}$:

$$H_{\mathcal{D}}(\mathcal{C}) = -\sum_{k \in \text{dom}(\mathcal{C})} \frac{|\mathcal{D}_{\mathcal{C}=k}|}{|\mathcal{D}|} \log \frac{|\mathcal{D}_{\mathcal{C}=k}|}{|\mathcal{D}|}$$

where $|\mathcal{D}_{\mathcal{C}=k}|$ is the number of data in class $\mathcal{C} = k$, with $0 \log 0 := 0$. The entropy ranges from 0 to 1 and is maximal for the case of two classes and an even $50 : 50$ distribution of patterns of those classes. An entropy of exactly 0 tells us that only patterns of the same class exist. Entropy therefore provides us with a measure of how impure (with respect to the class variable $\mathcal{C}$) a data set is.

We can now determine what the best attribute at any given point is: it is the attribute A with the biggest reduction in entropy compared to the original set (recall that our aim is to find pure leaves). We call this reduction also **information gain**:

$$I_{\mathcal{D}}(\mathcal{C}, A) = H_{\mathcal{D}}(\mathcal{C}) - H_{\mathcal{D}}(\mathcal{C}, A),$$

where

$$H_{\mathcal{D}}(\mathcal{C}, A) = \sum_{a \in \text{dom}(A)} \frac{|\mathcal{D}_{A=a}|}{|\mathcal{D}|} H_{\mathcal{D}_{A=a}}(\mathcal{C}),$$

and $\mathcal{D}_{A=a}$ indicates the subset of $\mathcal{D}$ for which attribute A has value a; $H_{\mathcal{D}}(\mathcal{C}, A)$ then denotes the entropy that is left in the subsets of the original data after they have been split according to their values of A. It is interesting to note that the information gain can never be negative, that is, no matter which split we choose, the entropy spread out over the resulting subsets is not going to increase.

However, what happens if we have an attribute ID with unique values for every single example pattern and we use these kinds of splits and the entropy to measure the information gain? Simple, the attribute holding the unique IDs will be chosen

[1] Rumors say that ID3 stands for "Iterative Dichotomiser 3" (from Greek dichotomia, divided), supposedly it was Quinlan's third attempt. Another interpretation of one of the authors is "Induction of Decision 3rees."

first because it results in a node with n branches for $n = |\mathcal{D}|$ training examples: each example has its individual ID resulting in its own branch. The entropy of the sets passed down to all of these branches goes down to zero (since only one element of one class is contained in each set), and the information gain is maximized. This is clearly not desirable in most applications. The reason for choosing this odd split is that we are reducing the entropy at all costs, completely ignoring the costs of actually making the split. In order to compensate for this, we can also compute the **split information**,

$$SI_{\mathcal{D}}(A) = -\sum_{a \in \text{dom}(A)} \frac{|\mathcal{D}_{A=a}|}{|\mathcal{D}|} \log \frac{|\mathcal{D}_{A=a}|}{|\mathcal{D}|},$$

which computes the entropy of distribution of our original set into subsets, and use this to normalize the purely entropy driven information gain,

$$GR_{\mathcal{D}}(\mathcal{C}, A) = \frac{I_{\mathcal{D}}(\mathcal{C}, A)}{SI_{\mathcal{D}}(A)},$$

resulting in *GR*, the **gain ratio** of a given split. The gain ratio penalizes very wide splits, that is, splits with many branches, and biases the selection toward more narrow splits.

Of course, building decision trees on nominal values only is of limited interest. In the following we will touch upon some of the extensions to address this and other limitations. However, the main underlying algorithm always remains the same—a greedy search which attempts to locally maximize some measure of information gain over the available attributes and possible splits at each stage until a certain criteria is reached.

8.1.3 Variations and Issues

Decision trees are a widely researched topic in machine learning and data analysis. There are numerous extensions and variants around which various shortcomings or specific types of data and applications address. In the following we try to briefly summarize some of the more general variations.

8.1.3.1 Numerical Attributes

As mentioned above, the biggest limitation of ID3 is the focus on nominal attributes only. In order to extend this to numerical values, let us first explore possible splits for such variables and worry about how to actually find those splits a tad later. A small example for a numerical attribute (top row) and the corresponding class values (bottom row) is shown in Fig. 8.3.

For convenience, we have sorted the training instances (columns showing values of one input and one class value in this case) according to the numerical attribute. Clearly, splits can divide the range of the numerical attribute "temperature" only into two (or more) ranges. Hence, looking at the sorted series of training instances,

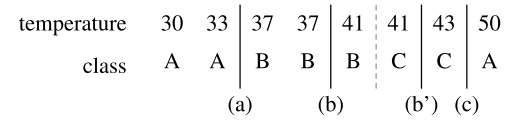


Fig. 8.3 A numerical attribute together with class values and a few possible splits

we can really only divide this into two pieces, a set of patterns to the left of the "cut" and a set to the right. One could imagine performing more than one split at each branching point in the tree, but this can easily be modeled by a series of binary splits. So in the following we will concentrate on the binary split setup.

In addition, in the case of a decision tree, we need to care about splits that occur within class boundaries only—why is that? An intuitive explanation is simple: if we already have instances of one class (e.g., $\mathcal{C} = k$) in one branch of our considered split, it really does not make sense to assign any patterns of that class to the other branch, that is, splitting a uniform subset into even smaller subsets never gives us any advantage. This, of course, depends on the chosen measure of information gain, but in most cases, this assumption holds.

Therefore the only split points we need to consider are the splits indicated in Fig. 8.3. Note that even though we would like to split at the dashed line (i.e., in between the patterns of class B and class C), we cannot do this since the value of temperature is 41 for both instances. So in this particular case we have to consider neighboring splits, e.g., (b) and (b'). This results in four possible splits that we need to investigate further. And suddenly, the problem can be converted back into the issue of looking into finding the one best nominal split: we can simply define four binary variables `temp_leq_35`, `temp_leq_39`, `temp_leq_42`, and `temp_leq_46.5` that take on values of `true` resp. `false` when the value of the original attribute `temperature` lies below resp. above the specific threshold. We can then apply the strategies described in the previous section to determine information gains for each one of these attributes and compare them with other possible splits on nominal or numerical attributes.

Note that using this approach, a numerical attribute may appear several times along the branch of a decision tree, each time being split at a different possible split point. This may make the tree somewhat harder to read. One could circumvent this by adding nominal values that split the numerical attributes in classwise pure ranges only, but, especially for somewhat noisy attributes, this will result in many small bins and subsequently in many nominal values for the resulting variable. As we have seen above, such splits are favored by pure entropy-based information gains and discouraged by other measures. Either way, these splits will be hard to interpret as well. So for many practical applications, it is preferable to restrict splits on numerical attributes to binary divisions.

8.1.3.2 Numerical Target Attributes: Regression Trees

In addition to handling numerical attributes for the feature vector, one is often also interested in dealing with numerical target values. The underlying issue then turns from predicting a class (or the probability of an input vector belonging to a certain

class) to the problem of predicting a continuous output value. We call this regression, and, in case of decision trees, the resulting tree is called a **regression tree**. At the same time when Quinlan was discussing algorithms for construction of classification tree, Breiman and his colleagues described **CART** (Classification And Regression Trees, see [5]).[2] The structure of such a tree is the same as that of the trees discussed above; the only difference is the leaves: instead of class labels (or distributions of class labels), the leaves now contain numerical constants. The tree output is then simply the constant corresponding to the leaf the input pattern ends up in. So the regression process is straightforward, but what about constructing such trees from data? Instead of an entropy-based measurement which aims to measure class impurity, we now need to find a measure for the quality of fit of a tree, or at least a branch. For this, we can use the sum of squares measurement, discussed earlier. The sum of squares for a specific node n may then simply be defined as the squared mean error of fit,

$$SME(\mathcal{D}_n) = \frac{1}{|\mathcal{D}_n|} \sum_{(\mathbf{X},Y)\in\mathcal{D}_n} (Y - f_n)^2,$$

where f_n indicates the constant value assigned to node n, $\mathcal{D}_n$ are the data points ending up at the node, $\mathbf{X}$ is the input feature vector, and Y is the target value. Using this quality measure, we can again apply our decision tree construction algorithm and continue analogously to the classification case: we greedily investigate all possible splits, determine the error on the respective splits, combine them (weighted by the respective sizes of the subsets, of course), and choose the split which results in the biggest reduction of error.

More sophisticated versions of regression trees, so-called **model trees**, not only allow for constants in the leaves but more complex functions on all or a subset of the input variables. However, these trees are much less commonly used, since normal regression trees are usually sufficient to approximate regression problems. In effect, model trees allow one to move some of the complexity from the tree structure into the functions in each leaf, resulting in smaller trees with leaves that are harder to interpret.

8.1.3.3 Pruning

So far we have not really worried much about when to stop growing a tree. Driven to the extreme, the base algorithm described in Sect. 8.1.2 simply continues until the tree cannot be refined any further, that is, no split attributes remain to be used, or the data set at a node contains one pattern or only patterns of the same class resp. output value. As we have discussed, this behavior is most likely not desirable since it leads to overfitting and, in this particular case will also make the interpretation of the resulting tree unnecessarily complex.

This is why pruning of decision trees gains importance quickly in real-world applications. Pruning decision trees comes in two variants:

[2] Quinlan later also developed methods for regression problems, similar to CART.

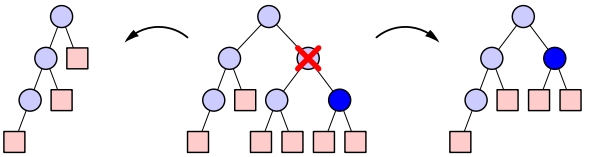


Fig. 8.4 A decision tree (*middle*) with one node (red cross) pruned by subtree replacement (*left*) and subtree raising (*right*)

- **Prepruning** refers to the process of stopping the construction of the decision tree already during the training process, that is, specific splits will not be added since they do not offer sufficient advantage even though they would produce a (albeit small) numerical improvement;
- **Postpruning** refers to the reduction of a decision tree which was built to a clearly overly large size. Here we distinguish two variants; one replaces subtrees with leaves (subtree replacement), and the other one also removes nodes from within the tree itself (subtree raising). The latter is substantially more complex since the instances contained in a node need to be redistributed among the branches of the raised subtree, and its benefits are questionable since in practice trees rarely become large enough to make this approach beneficial.

Figure 8.4 illustrates both approaches.

The important question is, independent of when we prune, how do we determine if a node can be removed (or should not be split in the first place)? Most commonly, one splits the available data into a training and validation data set, where the training set is used to determine possible splits, and the validation data then helps to estimate the split's actual usefulness. Often a sufficient amount of training data is, however, not available, and one needs to rely on other measures. One possibility is statistical tests that help to estimate if expanding a node will likely bring a significant improvement. Quinlan proposes a heuristic which is based on the training data and computes confidence intervals for each node. From these, a standard Bernoulli-based estimate (see Sect. A.3 in the appendix) is used to determine which nodes can be pruned. The statistical foundation of this method is somewhat shaky, but it works well in practice. Other ways for pruning are based on measures for complexity, similar to the split information we discussed above. The Minimum Description Principle (see Sect. 5.5.4.1) can help weigh the expected improvement in accuracy over the required enlargement of the model.

There is also a variant to post pruning which deserves to be mentioned at least briefly, namely rule post pruning. We can easily convert a decision tree into a set of independent rules by simply creating one rule for each leaf of the final decision tree: the rule will then consist of a conjunction of conditions found along the branch back to the root of the tree. We can then apply rule pruning techniques where all conditions in a rule body that do not affect the accuracy of the tree are removed. Note that the final set of rules then needs to be sorted somehow since the condi-

tions of all rules are not necessarily disjunct anymore. One usually sorts them by their estimated accuracy on unseen validation data. We discuss rule generation and processing methods later in this chapter in more detail.

8.1.3.4 Missing Values

Decision trees make it relatively easy to deal with missing values. Instead of simply ignoring rows which contain missing values, we can make use of the remaining information both during training and classification. What all approaches for missing value handling during training have in common is that they use more or less sophisticated ways to estimate the impact of the missing value on the information gain measure. The most basic one simply adds a fraction of each class to each partition if the split attribute's value is missing for that record. During classification, dealing with missing attribute values is straightforward: special treatment is only required if, during tree traversal, a node relies on the value which is missing in the pattern to be classified. Then the remainder of the tree traversal can be simply done in both branches and later on, the results that are encountered in the two (or more, if more than one missing values was encountered) leaves are merged.

8.1.3.5 Additional Notes

Decision trees are one of the most well-known and prominently used examples of more sophisticated data analysis methods. However, one often forgets that they are notoriously unstable. **Stability** means that small changes to the training data, such as removing just one example, can result in drastic changes in the resulting tree. This is mostly due to the greedy nature of the underlying algorithm, which never reconsiders chosen splits but is also part of the general nature of the tree structure: two very similar but not necessarily highly correlated attributes can exchange roles in a decision tree when a few training examples are added or removed, in turn affecting the rest of the tree dramatically.

When better, more stable performance is needed but interpretation is not such an issue, one often refers to **forests of decision trees** or decision stumps, which belong to the family of ensemble methods: instead of building one large tree, a set of differently initialized, much smaller decision trees ("stumps") are created, and the classification (or regression) output is created by committee voting. Forests of trees have the advantage that they are more stable than classical decision trees and often show superior generalization performance. This type of model class can also be used for feature selection wrapper methods, such as backward feature elimination as discussed in Sect. 6.1.1.

There are many other variations of decision trees around, for instance, **fuzzy decision trees** [13] which allow one to process imprecise data and handle degrees of class membership. Other variations allow one to include costs of attributes, that is, they allow one to consider during the coding phase that different attribute values may be obtainable at different costs. The result is a decision tree which attempts to optimize both the quality and the expected cost of processing new instances.

8.2 Bayes Classifiers

Bayes classifiers exploit Bayes' rule and some simplifying assumptions in order to model the conditional probability distributions of different classes given the values of the descriptive attributes. They predict, for each new instance, the most probable class based on this probability model. Bayes classifiers differ mainly in the kind and structure of the simplifying assumptions that are made.

8.2.1 Overview

Let us consider the question: what is the best possible classifier? The obvious and immediate answer seems to be: a classifier that always predicts the correct class, that is, the class the instance under consideration actually belongs to. Although this is certainly the ideal we strive for, it is rarely possible to achieve it in practice.

A fundamental obstacle is that an instantiation of the available descriptive attributes rarely allows us to single out one class as the obtaining one. Rather, there are usually several possible classes, either because there exist one or more hidden (unobserved) attributes that influence the class an instance belongs to, or because there is some random influence which cannot be predicted in principle (see Sect. 5.4 for a more detailed discussion).

Such a situation may be reflected in the available data by the fact that there are contradictory instances, that is, instances that coincide in their values for all descriptive attributes but differ in the class they belong to. Note, however, that the absence of contradictory instances does not guarantee that a perfect classifier is possible: the fact that no contradictory instances were observed (up to now) does not imply that they are impossible. Even if the training data set does not contain contradictory instances, it may still be that future cases, for which we want to predict the class, contradict each other or contradict instances from the training data set.

In a situation where there exists no unique class that can be predicted with certainty, but we still have to predict a single class (and not only a set of possible classes), the best we can do is to predict the class that has the highest probability (provided, of course, that all misclassification costs are equal—unequal misclassification costs are discussed in Sect. 8.2.3). The reason is that this scheme obviously yields (at least on average) the highest number or rate of correct predictions.

If we try to build a classifier that predicts, for any instantiation of the available descriptive attributes, the most probable class, we face two core problems: (1) how can we properly estimate which is the most probable class for a given instantiation? and (2) how can we store all of these estimates in a feasible way, so that they are easily accessible whenever we have to classify a new case? The second problem is actually a more fundamental one: if we are able to estimate, but cannot store efficiently, obtaining the estimates is obviously pointless.

Unfortunately, the most straightforward approach, namely simply storing the most probable class or a probability distribution over the classes for each possible instance is clearly infeasible: from a theoretical point of view, a single metric (numeric) (see Table A.1) attribute would give rise to a supercountably infinite number

of possible instances. In addition, even if we concede that in practice metric attributes are measured with finite precision and thus can have only a finite number of values, the number of possible instances grows exponentially with the number of attributes.

As a consequence, we have to introduce simplifying assumptions. The most common is to assume that the descriptive attributes are conditionally independent given the class (see Sect. A.3.2.3 in the appendix for a definition of this notion), thus reducing the number of needed parameters from the product of the domain sizes to their sum times the number of classes. However, the disadvantage is that this assumption is very strong and not particularly realistic. Therefore the result is also known as the **naive** or even the **idiot's Bayes classifier**. Despite this pejorative name, naive Bayes classifiers perform very well in practice and are highly valued in domains in which large numbers of descriptive attributes have to be taken into account (for example, chemical compound and text document classification).

Other simplifying assumptions concern metric (numeric) attributes. They are usually treated by estimating the parameters of one conditional distribution function per class—most commonly a normal distribution. Of course, this approach limits how well the actual distribution can be fitted but has the advantage that it considerably reduces the number of needed parameters. As a consequence, it may even become possible to abandon the naive conditional independence assumption: if we model all metric attributes jointly with one (multivariate) normal distribution per class, the number of parameters is merely quadratic in the number of (metric) attributes. The result is known as the **full Bayes classifier**. Note, however, that this applies only to metric (numeric) attributes. If categorical attributes are present, assuming conditional independence or something similar may still be necessary.

Extensions of the basic approach deal with mitigating the objectionable conditional independence assumption, selecting an appropriate subset of descriptive attributes to simplify the classifier, or incorporating misclassification costs.

8.2.2 Construction

As outlined in the preceding section, we want to construct a classifier that predicts the most probable class, that is, a classifier that computes its prediction as

$$\text{pred}(\mathbf{x}) = \arg \max_{y \in \text{dom}(Y)} P(y \mid \mathbf{x}).$$

Here $\mathbf{x}$ is an instantiation of the descriptive attributes, and Y is the class attribute. Exploiting **Bayes' rule**, we can rewrite this formula as[3]

$$\text{pred}(\mathbf{x}) = \arg \max_{y \in \text{dom}(Y)} \frac{P(\mathbf{x} \mid y) P(y)}{P(\mathbf{x})}.$$

[3] Since one or more of them may be metric, we may have to use a probability density function f to refer to descriptive attributes, $f(\mathbf{x} \mid y)$. However, we ignore such notational subtleties here.

Since $P(\mathbf{x})$ is independent of the class y, this formula is clearly equivalent to

$$\text{pred}(\mathbf{x}) = \arg \max_{y \in \text{dom}(Y)} P(\mathbf{x} \mid y) P(y).$$

Note that this simplification is possible even if we desire not only to predict the most probable class, but want to report its probability as well. The reason is that by exploiting the **law of total probability** we can compute $P(\mathbf{x})$ for any $\mathbf{x}$ as

$$P(\mathbf{x}) = \sum_{y \in \text{dom}(Y)} P(\mathbf{x} \mid y) P(y).$$

This leads to what is also known as the **extended Bayes' rule**:

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y) P(y)}{\sum_{z \in \text{dom}(Y)} P(\mathbf{x} \mid z) P(z)}.$$

It shows that in order to find the class probabilities, it suffices to compute $P(\mathbf{x} \mid y) P(y)$ for all $y \in \text{dom}(Y)$, and we have to do this anyway in order to find the maximum.

In the formula $\text{pred}(\mathbf{x}) = \arg\max_{y \in \text{dom}(Y)} P(\mathbf{x} \mid y) P(y)$ the factor $P(y)$ is clearly manageable: since the number of classes y is limited, we can store their probabilities explicitly. The problem resides with the other factor, that is, with $P(\mathbf{x} \mid y)$: even if all attributes are categorical, storing the $|\text{dom}(Y)| \cdot \prod_{X \in \mathbf{X}} |\text{dom}(X)|$ needed probabilities cannot be accomplished unless the number of attributes is *very* small.

In order to reach a manageable number of parameters, we make the crucial (but naive) assumption that all descriptive attributes are conditionally independent given the class (see Sect. A.3.3.4 in the appendix for a more detailed treatment of conditional independence). This allows us to rewrite the classification formula as

$$\text{pred}(\mathbf{x}) = \arg \max_{y \in \text{dom}(Y)} P(y) \prod_{X \in \mathbf{X}} P(x \mid y),$$

where x is the element of the data tuple $\mathbf{x}$ that refers to the descriptive attribute X. This is the core classification formula of the **naive Bayes classifier**.

If an attribute X is categorical, the corresponding factor $P(x \mid y)$ is fairly easily manageable: it takes only $|\text{dom}(X)| \cdot |\text{dom}(Y)|$ parameters to store it explicitly. However, if X is metric, a distribution assumption is needed to store the corresponding conditional probability density $f(x \mid y)$. The most common choice is a normal distribution $f(x \mid y) = N(\mu_{X|y}, \sigma^2_{X|y})$ with parameters $\mu_{X|y}$ (mean) and $\sigma^2_{X|y}$ (variance).

Once we have the above classification formula, estimating the model parameters becomes very simple. Given a data set $\mathbf{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, we use

$$\forall y \in \text{dom}(Y): \quad \hat{P}(y) = \frac{\gamma + n_y}{\gamma |\text{dom}(Y)| + n}$$

as an estimate for the prior class probability[4] and

$$\forall y \in \mathrm{dom}(Y) : \forall x \in \mathrm{dom}(X) : \quad \hat{P}(x|y) = \frac{\gamma + n_{xy}}{\gamma |\,\mathrm{dom}(X)| + n_y}$$

as an estimate for the conditional probabilities of the values of a categorical descriptive attribute X given the class. The n_y and n_{xy} in these formulas are defined as

$$n_y = \sum_{i=1}^{n} \tau(y_i = y) \quad \text{and} \quad n_{xy} = \sum_{i=1}^{n} \tau\big(\mathbf{x}_i[X] = x \wedge y_i = y\big),$$

where $\mathbf{x}_i[X]$ is the value that attribute X has in the ith tuple, and y_i is the class of the ith tuple (that is, the value of the class attribute Y). The function τ is a kind of truth function, that is, $\tau(\varphi) = 1$ if φ is true and 0 otherwise. Hence, n_y is simply the number of sample cases belonging to class y, and n_{xy} is the number of sample cases in class y for which the attribute X has the value x.

Finally, γ is a constant that is known as **Laplace correction**. It may be chosen as $\gamma = 0$, thus reducing the procedure to simple maximum likelihood estimation (see Sect. A.4.2.3 in the appendix for more details on maximum likelihood estimation). However, in order to appropriately treat categorical attribute values that do not occur with some class in the given data set but may nevertheless be possible, it is advisable to choose $\gamma > 0$. This also renders the estimation more robust, especially for a small sample size (small data set or small number of cases for a given class). Clearly, the larger the value of γ, the stronger the tendency toward a uniform distribution. Common choices are $\gamma = 1$, $\gamma = \frac{1}{2}$, or $\gamma = \frac{\gamma_0}{|\,\mathrm{dom}(X)| \cdot |\,\mathrm{dom}(Y)|}$, where γ_0 is a user-specified constant that is known as the **equivalent sample size** (since it can be justified with an argument from Bayesian statistics, where it represents the weight of the prior distribution, measured as the size of a sample having the same effect).

For a metric (numeric) attribute X, a (conditional) normal distribution is commonly assumed (see above), that is, it is assumed that

$$f(x \mid y) = N\big(\mu_{X|y}, \sigma^2_{X|y}\big) = \frac{1}{\sqrt{2\pi}\sigma_{X|y}} \exp\left(-\frac{(x - \mu_{X|y})^2}{2\sigma^2_{X|y}}\right).$$

The parameters $\mu_{X|y}$ and $\sigma^2_{X|y}$ of this conditional density function are estimated as

$$\hat{\mu}_{X|y} = \frac{1}{n_y} \sum_{i=1}^{n} \tau(y_i = y) \cdot \mathbf{x}_i[X] \quad \text{and}$$

$$\hat{\sigma}^2_{X|y} = \frac{1}{n'_y} \sum_{i=1}^{n} \tau(y_i = y) \cdot \big(\mathbf{x}_i[X] - \hat{\mu}_{X|y}\big)^2,$$

[4] This is a *prior* probability, because it describes the class probability *before* observing the values of any descriptive attributes.

where either $n'_y = n_y$ (maximum likelihood estimator) or $n'_y = n_y - 1$ (unbiased estimator for the variance) is chosen. (See Sect. A.4.2 in the appendix for more details about estimators and their desirable properties.)

As already briefly mentioned above, the normal distribution assumption also permits us to treat all metric attributes jointly if we assume that their distribution can be modeled with a multivariate normal distribution, that is,

$$f(\mathbf{x}_M \mid y) = \frac{1}{(2\pi)^{\frac{|\mathbf{X}_M|}{2}} \sqrt{|\Sigma_{\mathbf{X}_M|y}|}} \exp\left(-\frac{1}{2}(\mathbf{x}_M - \mu_{\mathbf{X}_M|y})^\top \Sigma^{-1}_{\mathbf{X}_M|y}(\mathbf{x}_M - \mu_{\mathbf{X}_M|y})\right),$$

where $\mathbf{X}_M$ is the set of metric attributes, and $\mathbf{x}_M$ is the vector of values of these attributes. The (class-conditional) mean vectors $\mu_{\mathbf{X}_M|y}$ and the corresponding covariance matrices $\Sigma_{\mathbf{X}_M|y}$ can be estimated from a given data set $\mathbf{X}$ as

$$\forall y \in \mathrm{dom}(Y): \quad \hat{\mu}_{\mathbf{X}_M|y} = \frac{1}{n_y} \sum_{i=1}^{n} \tau(y_i = y) \cdot \mathbf{x}_i[\mathbf{X}_M]$$

$$\text{and} \quad \hat{\Sigma}_{\mathbf{X}_M|y} = \frac{1}{n'_y} \sum_{i=1}^{n} \tau(y_i = y) \times \big(\mathbf{x}_i[\mathbf{X}_M] - \mu_{\mathbf{X}_M|y}\big)\big(\mathbf{x}_i[\mathbf{X}_M] - \mu_{\mathbf{X}_M|y}\big)^\top,$$

with either $n'_y = n_y$ or $n'_y = n_y - 1$ (as above). If all attributes are metric, the result is

$$\mathrm{pred}(\mathbf{x}) = \arg\max_y P(y) f(\mathbf{x} \mid y),$$

that is, the core classification formula of the **full Bayes classifier**.

As a consequence, the classification formula for a **mixed Bayes classifier** is

$$\mathrm{pred}(\mathbf{x}) = \arg\max_y P(y) f(\mathbf{x}_M \mid y) \prod_{X \in \mathbf{X}_C} P(x \mid y),$$

where $\mathbf{X}_C$ is the set of categorical attributes. In a mixed Bayes classifier the metric attributes are modeled with a multivariate normal distribution (as in a full Bayes classifier), while the categorical attributes are treated with the help of the conditional independence assumption (as in a naive Bayes classifier).

In order to illustrate the effect of the conditional independence assumption, we consider a naive Bayes classifier for the well-known iris data [2, 8]. The goal is to predict the iris type (Iris setosa, Iris versicolor, or Iris virginica) from measurements of the length and width of the petals and sepals. Here we confine ourselves to the measurements of the petals, which are most informative with respect to the class. The parameters of a naive Bayes classifier, derived with a normal distribution assumption, are shown in Table 8.2, and its graphical illustration in Fig. 8.5 on the left. In addition, Fig. 8.5 contains, on the right, an illustration of a full Bayes classifier, in which the two descriptive attributes (petal length and width) are modeled with class-conditional bivariate normal distributions.

Table 8.2 A naive Bayes classifier for the iris data. The class-conditional normal distributions are described by $\hat{\mu} \pm \hat{\sigma}$ (that is, expected value $\pm$ standard deviation)

Iris type	Iris setosa	Iris versicolor	Iris virginica
Prior probability	0.333	0.333	0.333
Petal length	1.46 ± 0.17	4.26 ± 0.46	5.55 ± 0.55
Petal width	0.24 ± 0.11	1.33 ± 0.20	2.03 ± 0.27

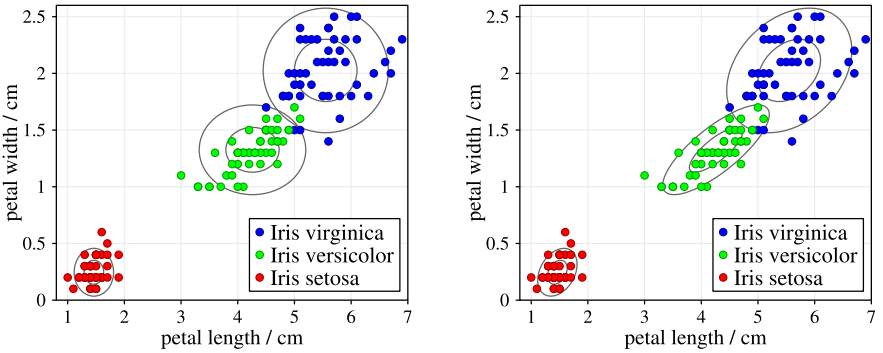


Fig. 8.5 Naive Bayes density functions for the iris data (axes-parallel ellipses, *left*) and density functions that take the covariance of the two measures into account (general ellipses, *right*). The ellipses are the $1\hat{\sigma}$- and $2\hat{\sigma}$-boundaries (lines of equal probability density)

These diagrams show that the conditional independence assumption is expressed by the orientation of the ellipses, which are lines of equal probability density: in a naive Bayes classifier, the major axes of these ellipses are always parallel to the coordinate axes, while a full Bayes classifier allows them to have arbitrary directions. As a consequence, the dependence of the attributes petal length and width can be modeled better, especially for Iris versicolor.

8.2.3 Variations and Issues

Bayes classifiers have been extended in several ways, for example, by mitigating the conditional independence assumption (Sect. 8.2.3.3) and by taking misclassification costs into account (Sect. 8.2.3.5). In addition, we argue in this section why naive Bayes classifiers often perform very well despite the strong independence assumptions they make (Sect. 8.2.3.1), study how full Bayes classifiers are related to linear discriminant analysis (Sect. 8.2.3.2), and consider how missing values are handled in Bayes classifiers (Sect. 8.2.3.4).

8.2.3.1 Performance

The naive assumption that the descriptive attributes are conditionally independent given the class attribute is clearly objectionable: it is rarely (exactly) satisfied in practice. Even in the simple iris classification example studied above it does not

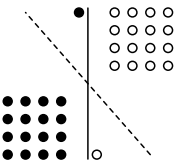


Fig. 8.6 The classification performance of a (naive) Bayes classifier need not get better if additional attributes are used. In this example adding the vertical axis, despite its being informative, yields a worse classification accuracy than using the horizontal axis alone, which allows for a perfect classification of the data points

hold. Nevertheless, naive Bayes classifiers usually perform surprisingly well and are often not much worse than much more sophisticated and complicated classifiers.

The reasons for this behavior are investigated in detail in [7], where it is revealed that the good performance of naive Bayes classifiers is less surprising than one may think at first sight. In a nutshell, this behavior is due to the fact that a classifier is usually evaluated with **accuracy** or **0–1 loss**, that is, it is simply counted how often it makes a correct prediction (on test data). However, in order to make a correct prediction, it is not necessary that the class probabilities are predicted with high accuracy. It suffices that the most probable class receives the highest probability assignment.

For example, in a classification problem with only two classes, the true probability of class 1 given some instantiation of the descriptive attributes may be 94%. If a naive Bayes classifier predicts it to be 51% instead, it wildly underestimates this probability but still yields the correct classification result.

In a large fraction of all cases in which the conditional independence assumption is not satisfied, the effect of this assumption is mainly the distortion of the probability distribution, while the most probable class remains the same for most instantiations. Therefore the classification performance can still be very good, and this is what one actually observes in practice on many (though not all) data sets.

Note also that, even though it sounds highly plausible, it is not the case that a (naive) Bayes classifier can only profit from additional attributes. There are cases where additional attributes actually worsen the prediction performance, even if these additional attributes carry information about the class. An example is shown in Fig. 8.6: a naive Bayes classifier that uses only the horizontal axis to discriminate between cases belonging to class • from cases belonging to class ○ yields a perfect result (solid line). However, if the vertical axis is also used, the classification boundary changes because of the diagonal location of the mass of the data points (dashed line). As a consequence, two instances are misclassified, namely the data point marked with • at the top and the data point marked with ○ at the bottom.

8.2.3.2 Linear Discriminant Analysis

Linear discriminant analysis, as developed by Fisher [8], can be seen as a special case of a full Bayes classifier, which results from additional simplifying assumptions. In the first place, this is a method for discriminating between two classes,

while more classes have to be handled by ensembles of classifiers (see Sect. 9.5). In addition, linear discriminant analysis assumes that the class-conditional multivariate normal distributions have the same shape. Formally, this means that they have the same covariance matrix and differ only in their mean vectors. As a consequence, the class boundary becomes a straight line (in two dimensions) or generally a hyperplane, thus explaining the name *linear* discriminant analysis.

Since there are only two classes, it is common to use a **decision function**, which is simply the difference of the class probability estimates:

$$\mathrm{dec}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{|\mathbf{X}|}{2}}\sqrt{|\Sigma|}}\Bigg(\exp\Bigg(-\frac{1}{2}(\mathbf{x}-\mu_1)^\top \Sigma^{-1}(\mathbf{x}-\mu_1)\Bigg) - \exp\Bigg(-\frac{1}{2}(\mathbf{x}-\mu_0)^\top \Sigma^{-1}(\mathbf{x}-\mu_0)\Bigg)\Bigg).$$

Class 1 is predicted if the decision function is positive, class 0 if it is negative:

$$\mathrm{pred}(\mathbf{x}) = \frac{1}{2}\,\mathrm{sgn}\big(\mathrm{dec}(\mathbf{x})\big) + \frac{1}{2}.$$

In the decision function, μ_0 and μ_1 are the mean vectors for the two classes that are estimated in the same way as for a full Bayes classifier. In order to estimate the covariance matrix, however, the data points from the two classes are pooled:

$$\hat{\Sigma} = \frac{1}{n'}\sum_{i=1}^{n}\big(\mathbf{x}_i[\mathbf{X}] - \hat{\mu}_{y_i}\big)\big(\mathbf{x}_i[\mathbf{X}] - \hat{\mu}_{y_i}\big)^\top,$$

where either $n' = n$ or $n' = n - 1$ (as usually for variance estimation, see Sect. A.4.2.1 in the appendix). That is, the covariance matrix is computed from the difference vectors of the data points to their respective class means.

This pooling of data points is possible because of the assumption that the class-conditional density functions have the same shape. It has the advantage that the covariance estimate becomes more robust (since it is computed from more data points) but also the disadvantage that it misrepresents the density functions if the assumption of equal shape is not valid.

8.2.3.3 Augmented Naive Bayes Classifiers

Bayes classifiers can be seen as a special case of Bayes networks. Although we cannot provide here a comprehensive treatment of Bayes network (an interested reader is referred to, for instance, [4, 14] for an in-depth treatment), we try to capture some basics, because otherwise the name of the most common augmented naive Bayes classifier remains mysterious. The core idea underlying **Bayes networks** is to exploit conditional independence statements that hold in a multidimensional probability distribution, to decompose the distribution function. This already shows the connection to Bayes classifiers, in which we also factorize the joint probability distribution of the descriptive attributes based on the assumption that they are conditionally independent given the class.



Fig. 8.7 A naive Bayes classifier is a Bayes network with a star-like structure (with the class attribute at the center). Additional edges can mitigate the conditional independence assumptions

The other core ingredient of Bayes networks is that, generally, the set of conditional independence statements that hold in a probability distribution has properties that are highly analogous to those of certain separation statements in graphs or networks. As a consequence, the idea suggests itself to use graphs or networks to express these conditional independence statements in a concise and intuitive form. Although we cannot go into the full formal details here, we would like to mention that a naive Bayes classifier is a Bayes network with a star-like structure, see Fig. 8.7 on the left: all edges are directed from the class attribute Y at the center to the descriptive attributes $X_1, \ldots, X_m$. This graph structure expresses the conditional independence assumption by a graph-theoretic criterion that is known as d-separation: any path from a descriptive attribute X_i to another attribute $X_j, i \neq j$, is blocked by the class attribute Y if it is known (details can be found, for instance, in [4]).

Drawing on the theory of representing conditional independencies by graphs or networks, these conditional independence assumptions can be mitigated by adding edges between descriptive attributes that are not conditionally independent given the class, see Fig. 8.7 on the right: the direct connections between these attributes express that even if the class is known, and thus the path through the class attribute is blocked, the two attributes are still dependent on each other. In this case the classification formula of a Bayes classifier is generalized with the standard factorization formula for a Bayes network, namely to

$$\text{pred}(\mathbf{x}) = \arg\max_{y \in Y} P(y) \prod_{X \in \mathbf{X}} P\big(x \mid \text{parents}(X)\big),$$

where $\text{parents}(X)$ denotes the set of parents of the attribute X in the graph structure. Clearly, with a purely star-like structure (as in Fig. 8.7 on the left), this formula coincides with the standard classification formula of a naive Bayes classifier, because in a purely star-like structure the class attribute Y is the only parent of any descriptive attribute. If, however, there are additional edges, a descriptive attribute may have additional parents (other than the class attribute Y) on which it depends and which are needed in order to compute the correct conditional probability.

The most common form of a Bayes classifier that is extended in this way is the **tree-augmented naive Bayes classifier** (TAN) [9, 10]. In this classifier the number of parents of each descriptive attribute is restricted to two: the class attribute Y and at most one descriptive attribute. Since Bayes networks are required to be acyclic

directed graphs (that is, there must not be a directed path connecting a node to itself), this constraint allows at most edges forming a tree to be added to the star structure of a naive Bayes classifier (and hence the name *tree-augmented* naive Bayes classifier).

The standard way to choose the additional edges is to construct a maximum weight spanning tree for the descriptive attributes with conditional mutual information providing the edge weights. This measure is defined as

$$\begin{aligned} I(X_i, X_j \mid Y) &= H(X_i \mid Y) + H(X_j \mid Y) - H(X_i, X_j \mid Y) \\ &= \sum_{x_i \in \mathrm{dom}(X_i)} \sum_{x_j \in \mathrm{dom}(X_j)} \sum_{y \in \mathrm{dom}(Y)} P(x_i, x_j, y) \\ &\quad \times \log_2 \frac{P(x_i, x_j \mid y)}{P(x_i \mid y) \cdot P(x_j \mid y)} \end{aligned}$$

for two descriptive attributes X_i and X_j, conditional on the class attribute Y, where H denotes the Shannon entropy. This procedure has the advantage that it selects those edges that, under the tree constraint, produce a Bayes network that yields the smallest Kullback–Leibler information divergence between the actual multivariate probability distribution and the probability distribution represented by the Bayes network/classifier. The edges may then be directed by choosing an arbitrary descriptive attribute as the root of the added tree and directing all added edges away from it.

More general augmented Bayes classifiers, which allow for more edges than just a directed tree to be added, together with methods for selecting the edges resulting in a good classifier, have been studied, for example, in [16, 23]. They are generally Bayes networks in which one variable representing the class attribute plays a special role, so that all computations are directed toward it.

8.2.3.4 Missing Values

Handling missing values is particularly easy in naive Bayes classifiers. In the construction step, it is convenient that the (conditional) distributions refer to at most two attributes, namely the class and at most one conditioned attribute. Therefore data points that are only partially known (that is, miss some attribute values) can still be used for the estimation: they are useless only for estimating those distributions that involve one of the attributes for which the value is missing. For all other attributes, the data point can be exploited. The only exception is the case for which the class is unknown, since the class is involved in all relevant (conditional) distributions.

In the execution step—that is, when a naive Bayes classifier is employed to compute a prediction for a new case—the factors of the classification formula, which refer to attributes the values of which are unknown, are simply dropped. The prediction computed in this way is the same as that of a naive Bayes classifier which has been constructed on the subspace of those attributes that are known for the sample case under consideration. Hence a naive Bayes classifier can flexibly adapt to whatever subset of the descriptive attributes is known.

The situation is less convenient for a full Bayes classifier: since all descriptive attributes are treated jointly (no conditional independence assumption), a single missing value can already make it impossible to use the sample case in the estimation

of the class-conditional distributions. With a multivariate normal distribution assumption, however, the case may still be used to estimate the covariances of known attribute pairs. One merely has to keep track of the different values of n', which may be different for each element of the covariance matrix. In the execution step integrating the class-conditional multivariate normal distributions over unknown attributes provides a means of computing a prediction despite the fact that values are missing.

8.2.3.5 Misclassification Costs

Up to now we assumed that the costs of misclassifying an instance do not depend on the true class of the instance. In practice, however, these costs usually differ. Consider, for example, medical diagnosis: if a sick patient is misclassified as healthy, the consequences are usually much more severe than if a healthy patient is misclassified as sick. In the former case, the disease is left untreated and thus may develop into a severe state that may cause considerable harm to the patient. In the latter case the actually healthy status of the patient will likely be discovered by future tests.

Another example is the task to select customers for a special mail advertisement. In this case one would like to send the mail only to those customers that will actually buy the advertised product, that is, the class to predict is whether the customer buys or not. In this case misclassifying a nonbuying customer as buying incurs only a relatively small loss, namely, the cost of a useless mailing. However, if a buying customer is misclassified as nonbuying and thus does not receive the mailing (and therefore does not buy, because he/she never gets to know about the offer), the costs are high, because the selling company loses the revenue it could have made.

Since Bayes classifiers predict a class by estimating the probability of the different possible classes and then selecting the most probable one, misclassification costs can fairly easily be incorporated into the process. Suppose that we model the misclassification costs with a cost function $c(y)$, which assigns to each class y the costs of wrongly classifying an instance of this class as belonging to some other class y'. Then the classification formula should be modified to

$$\text{pred}(\mathbf{x}) = \arg \min_{y \in \text{dom}(Y)} c(y)\big(1 - P(y \mid \mathbf{x})\big).$$

In this way a class with nonmaximal probability may be predicted if the costs for misclassifying an instance of it as belonging to some other class are high.

A more complicated situation is where we are given misclassification costs in the form of a matrix $\mathbf{C}$ of size $|\text{dom}(Y)| \times |\text{dom}(Y)|$. Each element c_{yz} of this matrix states the costs of misclassifying an instance of class y as belonging to class z. Naturally, the diagonal elements c_{yy} are all zero (as these refer to a correct classification). In this case the classification formula is modified to

$$\text{pred}(\mathbf{x}) = \arg \min_{y \in \text{dom}(Y)} \sum_{z \in \text{dom}(Y)} c_{zy} P(z \mid \mathbf{x}).$$

With this prediction procedure, the expected costs (under the induced probability model) that result from possible misclassifications are minimized.

Note that both this and the previous modified classification formula reduce to the standard case (that is, predicting the most probable class) where all costs are equal:

$$\begin{aligned}\text{pred}(\mathbf{x}) &= \arg \min_{y \in \text{dom}(Y)} \sum_{z \in \text{dom}(Y)-\{y\}} c \cdot P(z \mid \mathbf{x}) \\ &= \arg \min_{y \in \text{dom}(Y)} c \cdot \big(1 - P(y \mid \mathbf{x})\big) \\ &= \arg \max_{y \in \text{dom}(Y)} P(y \mid \mathbf{x}),\end{aligned}$$

where c denotes the (equal) misclassification costs.

8.3 Regression

Generally, regression is the process of fitting a real-valued function of a given class to a given data set by minimizing some cost functional, most often the sum of squared errors. Most commonly, the fitting function is linear, which allows for a direct solution. Actually, for any polynomial, the solution that minimizes the sum of squared errors can be computed with methods from linear algebra. Transformations, like logistic regression, allow us to use more general function classes and still to obtain a direct solution. In other cases, gradient descent methods can be employed.

8.3.1 Overview

Up to now we considered classification, that is, the task to predict a class from a finite set of possible classes. However, in many applications the quantity to predict is not categorical, but rather metric (numerical), whether it is the price of the shares of a given company, the electrical power consumption in a given area, the demand for a given product, etc. In such cases classification techniques may, in principle, still be applied, but doing so requires us to discretize the quantity to predict into a finite set of intervals, which are then considered as the classes. Clearly, however, a better approach is to use a prediction method that can yield a numeric output.

The main problem of such an approach is that (physical) measurement values rarely show the exact relationship between the considered quantities, because they are inevitably afflicted by errors. If one wants to determine the relationship between the considered quantities nevertheless, at least approximately, one faces the task to find a function that fits the given data as well as possible, so that the measurement errors are "neutralized." Naturally, for such an approach, one should possess at least a conjecture about how the target attribute (in statistics also called the **response variable**) depends on the descriptive attributes (in statistics also called the **explanatory** or **regressor variables**), so that one can choose a (parameterized) function class and thus can reduce the problem to a parameter estimation task. This choice is a critical issue: if a chosen function class does not fit the data (for example, if one tries to fit a

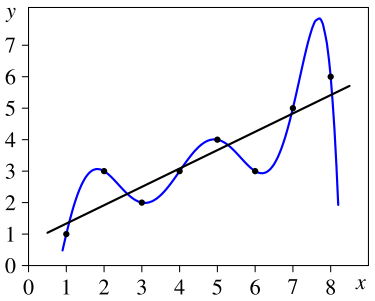


Fig. 8.8 The function class for regression has to be chosen with care. If a very complex function is chosen, a perfect fit can be achieved, but this fit does not allow for reliable inter- or extrapolations. A simpler function, although it fits the training data less well, is usually a much better predictor for new function values

linear function to nonlinear data), the result can be completely useless, because the function cannot, in principle, be made to fit the data.[5]

Generally, one has to be particularly careful not to choose a function class with too many degrees of freedom (too many free parameters), as this invites overfitting. For example, any set of n data points with one explanatory and one response variable can be fitted perfectly with a polynomial of degree $n-1$ (and thus n free parameters). An example with eight data points is shown in Fig. 8.8 (blue curve, polynomial of degree 7). Clearly, even though the blue curve fits the data points perfectly, it is completely useless for interpolation or extrapolation purposes: it is not likely that a data point with a value for the explanatory variable (x-axis) other than those of the given data points actually has a value for the response variable (y-axis) so that it lies on the blue curve. This is particularly obvious for points with an x-value beyond 8. A better fit of this data set can be obtained with a simple straight line (shown in black). Intuitively, interpolating or extrapolating the values of the response variable based on this straight line are much more likely to yield a useful result.

In order to deal with the unavoidable measurement errors and to achieve a good fit to the data within the limitations imposed by the choice of the function class, we have to choose a cost function that penalizes (at least large) deviations from the actual values. The most common cost function is the **sum of squared errors** (SSE), with which the approach is also known as the **method of least squares** (OLS for "ordinary least squares"). It has the advantage that for a large family of parameterized function classes—in particular, any polynomial of the explanatory variables—the solution (that is, the set of parameters yielding the least sum of squared errors) can be obtained by taking derivatives and applying simple methods from linear algebra. However, it has the disadvantage that outliers, either in the explanatory or the response variable, can have a distorting effect due to the fact that the errors are squared, and thus outliers strongly influence the estimation. As a consequence, other cost functions—like the **sum of absolute errors** or functions that limit the

[5] For more details see also Sect. 5.4.

contribution of a single data point to the total cost to a certain maximum—are also used.

Once the (parameterized) function class and the cost function have been chosen, the model construction process is straightforward: try to find the set of parameters that identifies the function from the chosen class that minimizes the costs. Whether the result is unique or not and whether it can be computed directly or has to be found by an iterative improvement procedure (like gradient descent) depends mainly on the cost function used. As we will see below, a direct solution can be obtained for any polynomial as the function class and the sum of squared errors as the cost function.

8.3.2 Construction

We start our description of the model construction procedure with the simplest case, finding a linear regression function for a single explanatory variable. In this case one has to determine the parameters a and b of a straight line $y = f(x) = a + bx$. However, due to the unavoidable measurement errors, it will usually not be possible to find a straight line such that all n data points (x_i, y_i), $1 \leq i \leq n$, lie exactly on this straight line. Rather we have to find a straight line from which the given data points deviate as little as possible. Hence it is plausible to determine the parameters a and b in such a way that the sum of squared deviations, that is,

$$F(a, b) = \sum_{i=1}^{n} \big(f(x_i) - y_i\big)^2 = \sum_{i=1}^{n} (a + bx_i - y_i)^2,$$

is minimal. In other words, the y-values that are computed with the linear equation should (in total) deviate as little as possible from the measured values. The reasons for choosing squared deviations are twofold: (1) by using squares the error function becomes continuously differentiable everywhere (whereas the derivative of the sum of absolute errors is discontinuous/does not exist at zero), and (2) the squares weight larger deviations higher, so that individual large deviations are avoided.[6]

A necessary condition for a minimum of the error function $F(a, b)$ is that the partial derivatives of this function with respect to the parameters a and b vanish, that is,

$$\frac{\partial F}{\partial a} = \sum_{i=1}^{n} 2(a + bx_i - y_i) = 0 \quad \text{and} \quad \frac{\partial F}{\partial b} = \sum_{i=1}^{n} 2(a + bx_i - y_i)x_i = 0.$$

As a consequence, we obtain (after a few simple steps) the so-called **normal equations**

$$na + \left(\sum_{i=1}^{n} x_i\right) b = \sum_{i=1}^{n} y_i \quad \text{and} \quad \left(\sum_{i=1}^{n} x_i\right) a + \left(\sum_{i=1}^{n} x_i^2\right) b = \sum_{i=1}^{n} x_i y_i,$$

[6] Note, however, that this second property can also be a disadvantage, as it can make outliers have an overly strong influence on the regression result.

that is, a linear two-equation system with two unknowns a and b. It can be shown that this equation system has a unique solution unless all x-values are identical and that this solution specifies a minimum of the function F. The straight line determined in this way is called **regression line** for the data set $(x_1, y_1), \ldots, (x_n, y_n)$.

Note that finding a regression line can also be seen as a **maximum likelihood estimation** (see Sect. A.4.2.3 in the appendix) of the parameters of the linear model

$$Y = aX + b + \xi,$$

where X is a random variable with arbitrary distribution, and ξ is a normally distributed random variable with mean 0 and arbitrary variance (but independent of the value of X). The parameters that minimize the sum of squared deviations (in y-direction) from the data points maximize the probability of the data given this model class.

In order to illustrate the procedure, we consider a simple example. Let the following data set be given, which consists of eight data points $(x_1, y_1), \ldots, (x_8, y_8)$:

x	1	2	3	4	5	6	7	8
y	1	3	2	3	4	3	5	6

The regression line computed with the above procedure is

$$y = \frac{3}{4} + \frac{7}{12}x.$$

This line is shown in Fig. 8.8 on page 242 in black. Obviously, it provides a reasonable fit to the data and may be used to interpolate between or to extrapolate the data points.

The least squares method is not limited to straight lines but can be extended (at least) to **regression polynomials**. In this case one tries to find a polynomial

$$y = p(x) = a_0 + a_1 x + \cdots + a_m x^m,$$

with a given fixed degree m, which best fits the n data points $(x_1, y_1), \ldots, (x_n, y_n)$. Consequently, we have to minimize the error function

$$F(a_0, a_1, \ldots, a_m) = \sum_{i=1}^{n} \big(p(x_i) - y_i\big)^2 = \sum_{i=1}^{n} \big(a_0 + a_1 x_i + \cdots + a_m x_i^m - y_i\big)^2.$$

In analogy to the linear case, we form the partial derivatives of this function with respect to the parameters a_k, $0 \leq k \leq m$, and equate them to zero (as this is a necessary condition for a minimum). The resulting system of linear equations ($m+1$ unknowns and $m + 1$ equations) can be solved with one of the standard methods of linear algebra (elimination method according to Gauss, inverting the coefficient matrix, etc.).

Note that finding a regression polynomial can also be interpreted as a maximum likelihood estimation of the parameters (as it was possible for linear regression) if one assumes a normally distributed error term ξ that is independent of X and Y.

Furthermore, the least squares method can be used for functions with more than one argument. This case is known as **multivariate regression**. We consider only **multilinear regression**, that is, we are given a data set $((\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n))$ with input vectors $\mathbf{x}_i$ and the corresponding responses y_i, $1 \leq i \leq n$, for which we want to determine the linear regression function

$$y = f(x_1, \ldots, x_m) = a_0 + \sum_{k=1}^{m} a_k x_k.$$

In order to derive the normal equations, it is convenient to write the functional to minimize in matrix form, that is, as

$$F(\mathbf{a}) = (\mathbf{X}\mathbf{a} - \mathbf{y})^\top (\mathbf{X}\mathbf{a} - \mathbf{y}), \tag{8.1}$$

where

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & \ldots & x_{1m} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \ldots & x_{nm} \end{pmatrix} \quad \text{and} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \tag{8.2}$$

represent the data set, and $\mathbf{a} = (a_0, a_1, \ldots, a_m)^\top$ is the vector of coefficients we have to determine. (Note that the ones in the matrix $\mathbf{X}$ refer to the coefficient a_0.) Again a necessary condition for a minimum is that the partial derivatives of this function with respect to the coefficients a_k, $0 \leq k \leq m$, vanish. Using the differential operator ∇, we can write these conditions as

$$\nabla_{\mathbf{a}} F(\mathbf{a}) = \frac{\mathrm{d}}{\mathrm{d}\mathbf{a}} F(\mathbf{a}) = \left(\frac{\partial}{\partial a_0} F(\mathbf{a}), \frac{\partial}{\partial a_1} F(\mathbf{a}), \ldots, \frac{\partial}{\partial a_m} F(\mathbf{a}) \right) = \mathbf{0}.$$

The derivative can most easily be computed by realizing that formally the differential operator

$$\nabla_{\mathbf{a}} = \left(\frac{\partial}{\partial a_0}, \frac{\partial}{\partial a_1}, \ldots, \frac{\partial}{\partial a_m} \right)$$

behaves like a vector (as one can easily confirm by writing out the computations). As a consequence, we obtain

$$\begin{aligned} \mathbf{0} &= \nabla_{\mathbf{a}} (\mathbf{X}\mathbf{a} - \mathbf{y})^\top (\mathbf{X}\mathbf{a} - \mathbf{y}) \\ &= \big(\nabla_{\mathbf{a}} (\mathbf{X}\mathbf{a} - \mathbf{y})\big)^\top (\mathbf{X}\mathbf{a} - \mathbf{y}) + \big((\mathbf{X}\mathbf{a} - \mathbf{y})^\top \big(\nabla_{\mathbf{a}} (\mathbf{X}\mathbf{a} - \mathbf{y})\big)\big)^\top \\ &= \big(\nabla_{\mathbf{a}} (\mathbf{X}\mathbf{a} - \mathbf{y})\big)^\top (\mathbf{X}\mathbf{a} - \mathbf{y}) + \big(\nabla_{\mathbf{a}} (\mathbf{X}\mathbf{a} - \mathbf{y})\big)^\top (\mathbf{X}\mathbf{a} - \mathbf{y}) \\ &= 2\mathbf{X}^\top (\mathbf{X}\mathbf{a} - \mathbf{y}) \\ &= 2\mathbf{X}^\top \mathbf{X}\mathbf{a} - 2\mathbf{X}^\top \mathbf{y}, \end{aligned}$$

from which we immediately obtain the system of normal equations

$$\mathbf{X}^\top \mathbf{X}\mathbf{a} = \mathbf{X}^\top \mathbf{y}.$$

This system is (uniquely) solvable iff $\mathbf{X}^\top \mathbf{X}$ is invertible. In this case we have

$$\mathbf{a} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}.$$

The expression $(\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top$ is also known as the (Moore–Penrose) **pseudoinverse** of the (usually nonsquare) matrix $\mathbf{X}$ [1]. With this pseudoinverse matrix, the solution of the regression task can be written down immediately.

It should be clear that the least squares method can also be extended to polynomials in multiple explanatory variables. In this case one starts most conveniently from the above matrix representation but also enters into the matrix $\mathbf{X}$ all monomials (that is, products of powers of the regressors, up to a given degree) that can be formed from the explanatory variables. Formally, these can be seen as additional inputs. In this way the derivation of the normal equations remains unchanged and can be used directly to find the solution.

8.3.3 Variations and Issues

As we have seen above, an analytical solution of the least squares problem can easily be obtained for polynomials. However, by exploiting transformations (for example, the logit-transformation), the regression approach can also be applied for other functions classes (Sect. 8.3.3.1). As least squares regression is sensitive to outliers, several more robust methods have been developed (Sect. 8.3.3.3). Finally, methods to automatically select the function class, at least from a wider family of functions of different complexity, may be worth considering (Sect. 8.3.3.4).

8.3.3.1 Transformations

In certain special cases the procedure described above can also be used to find other regression functions. In order for this to be possible, one has to find a suitable transformation which reduces the problem to the problem of finding a regression line or regression polynomial. For example, regression functions of the form

$$y = ax^b$$

can be found by finding a regression line. By simply taking the (natural) logarithm of this equation, we arrive at

$$\ln y = \ln a + b \cdot \ln x.$$

This equation can be handled by finding a regression line if we take the (natural) logarithms of the data points (x_i, y_i), $1 \leq i \leq n$, set $a' = \ln a$, and then carry out all computation with these transformed values.

It should be noted, though, that with such an approach, only the sum of squared errors in the transformed space (coordinates $x' = \ln x$ and $y' = \ln y$) is minimized, but not necessarily also the sum of squared errors in the original space (coordinates x and y). Nevertheless, this approach often yields good results. In addition, one may use the result only as a good starting point for a subsequent gradient descent (see generally Sect. 5.3, also Sect. 9.2 on training artificial neural network, and Sect. 8.3.3.2 below) in the original space, with which the solution can be improved, and the true minimum in the original space may be obtained.

For practical purposes, it is important that one can transform the **logistic function**,

$$y = \frac{y_{\max}}{1 + e^{a+bx}},$$

where $y_{\max}$, a, and b are constants, to the linear or polynomial case (so-called **logistic regression**). The logistic function is relevant for many applications, because it describes growth processes with a limit, for example, the growth of an animal population with a habitat of limited size or the sales of a (new) product with a limited market. In addition, it is popular in artificial neural networks (especially multilayer perceptrons, see Sect. 9.2), where it is often used as the activation function of the neurons.

In order to linearize the logistic function, we first take the reciprocal values

$$\frac{1}{y} = \frac{1 + e^{a+bx}}{y_{\max}}.$$

As a consequence, we have

$$\frac{y_{\max} - y}{y} = e^{a+bx}.$$

Taking the (natural) logarithm of this equation yields

$$\ln\left(\frac{y_{\max} - y}{y}\right) = a + bx.$$

Obviously, this equation can be handled by finding a regression line if we transform the y-values of all data points according to the left-hand side of the above equation. (Note that the value of $y_{\max}$, which mainly scales the values, must be known.) This transformation is known under the name **logit transformation** and can be seen as an inverse of the logistic function. By finding a regression line for the transformed data points, we obtain a logistic regression curve for the original data points.

However, note again that this procedure minimizes the sum of squared errors in the transformed space (coordinates x and $z = \ln(\frac{y_{\max}-y}{y})$), but not necessarily also the sum of squared errors in the original space (coordinates x and y). If one needs to know parameter values that actually describe the minimum of the sum of squared errors in the original space, one may (as above) use the obtained solution

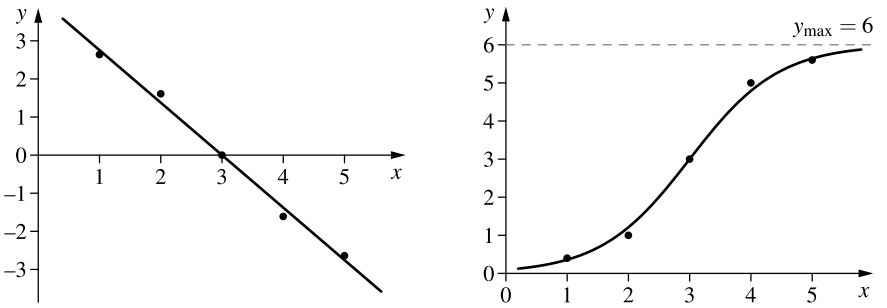


Fig. 8.9 Transformed data (*left*) and original data (*right*) as well as the regression line (*left*) and the corresponding logistic regression curve (*right*) computed with the method of least squares

as a starting point, which is then improved with a gradient descent (see Sect. 8.3.3.2 below).

In order to illustrate the procedure, we consider a simple example. Let the following simple data set be given that consists of five data points $(x_1, y_1), \ldots, (x_5, y_5)$:

x	1	2	3	4	5
y	0.4	1.0	3.0	5.0	5.6

By transforming these data points with the logit transformation using $y_{\max} = 6$ and finding a regression line for the result, we obtain

$$z \approx 4.133 - 1.3775x,$$

and thus, for the original data, the logistic regression curve is

$$y \approx \frac{6}{1 + e^{4.133 - 1.3775x}}.$$

These two regression functions are shown together with the (transformed or original) data points in Fig. 8.9.

8.3.3.2 Gradient Descent

If a transformation was applied in order to reduce a more complex functional relationship to the polynomial case, the sum of squared errors is, as emphasized above, minimized only in the transformed space, but not necessarily in the original space. Hence there is room for improvement, which may be achieved by using the solution that is obtained by solving the system of normal equations in the transformed space only as an initial point in the parameter space of the regression model. This initial solution is then iteratively improved by gradient descent (see Sect. 5.3 for a general treatment), that is, by repeatedly computing the gradient of the objective function (here, the sum of squared errors) at the current point in the parameter space and then making a small step in this direction.

If the functional relationship is logistic (as in the example in the previous section), this procedure is actually equivalent to training a multilayer perceptron without a hidden layer (so actually a two-layer perceptron, with only an input layer and one output neuron) with standard error backpropagation, provided, of course, that the activation function of the output neuron is logistic. Details for this special case can be found in Sect. 9.2.

8.3.3.3 Robust Regression

As pointed out above, solutions of (ordinary) least squares regression can be strongly affected by outliers. More robust results can usually be obtained by minimizing the **sum of absolute deviations** (least absolute deviations, LAD). However, this approach has the disadvantage of not being analytically solvable (like least squares) and thus has to be addressed with iterative methods right from the start. In addition, least absolute deviation solutions can be unstable in the sense that small changes in the data can lead to "jumps" (discontinuous changes) of the solution parameters, while least squares solutions always changes "smoothly" (continuously). Finally, severe outliers can still have a distorting effect on the solution.

As a consequence, more sophisticated regression methods, categorized as **robust regression**, have been developed in order to improve robustness. Among these are **M-estimation** and **S-estimation** for regression and **least trimmed squares** (LTS), the latter of which simply uses a subset of at least half the size of the data set (with the exact size to be specified by a user) that yields the smallest sum of squared errors.

As an example for robust regression, we take a closer look at M-estimators. Let us revisit the linear regression problem as it was formulated in (8.1) and (8.2). We rewrite the error function (8.1), i.e., the sum of squared errors, to be minimized in the form

$$\sum_{i=1}^{n} \rho(e_i) = \sum_{i=1}^{n} \rho\big(\mathbf{x}_i^\top \mathbf{a} - y_i\big), \tag{8.3}$$

where $\rho(e_i) = e_i^2$, and e_i is the (signed) error of the regression function at the ith point. Is this the only reasonable choice for the function ρ? The answer is definitely no. However, ρ should satisfy at least some reasonable restrictions. First, ρ should always be positive, except for the case $e_i = 0$. Then we should have $\rho(e_i) = 0$. The sign of the error e_i should not matter for ρ, and ρ should be increasing when the absolute value of the error increases. These requirements can be formalized in the following way:

$$\rho(e) \geq 0, \tag{8.4}$$

$$\rho(0) = 0, \tag{8.5}$$

$$\rho(e) = \rho(-e), \tag{8.6}$$

$$\rho(e_i) \geq \rho(e_j) \quad \text{if } |e_i| \geq |e_j|. \tag{8.7}$$

Table 8.3 Error measures ρ for different approaches

Method	$\rho(e)$
Least squares	e^2
Huber	$\begin{cases} \frac{1}{2}e^2 & \text{if } \lvert e\rvert \le k, \\ k\lvert e\rvert - \frac{1}{2}k^2 & \text{if } \lvert e\rvert > k. \end{cases}$
Tukey's bisquare	$\begin{cases} \frac{k^2}{6}(1-(1-(\frac{e}{k})^2)^3) & \text{if } \lvert e\rvert \le k, \\ \frac{k^2}{6} & \text{if } \lvert e\rvert > k. \end{cases}$

Parameter estimation (here the estimation of the parameter vector b) based on an objective function of the form (8.3) and an error measure satisfying (8.4)–(8.7) is called an M-estimator. The classical least squares approach is based on the quadratic error, i.e., $\rho(e) = e^2$. Table 8.3 provides the error measure ρ for the classical least squares method and for two approaches from robust statistics.

In order to understand the more general setting of an error measure ρ satisfying (8.4)–(8.7), it is useful to consider the derivative of the error measure $\psi = \rho'$.

Taking the derivatives of the objective function (8.3) with respect to the parameters a_i, we obtain the system of $(m+1)$ linear equations

$$\sum_{i=1}^{n} \psi_i\left(\mathbf{x}_i^\top \mathbf{a} - y_i\right)\mathbf{x}_i^\top = 0. \tag{8.8}$$

Defining $w(e) = \psi(e)/e$ and $w_i = w(e_i)$, equation (8.8) can be rewritten in the form

$$\sum_{i=1}^{n} \frac{\psi_i(\mathbf{x}_i^\top \mathbf{a} - y_i)}{e_i} \cdot e_i \cdot \mathbf{x}_i^\top = \sum_{i=1}^{n} w_i \cdot \left(y_i - x_i^\top b\right) \cdot x_i^\top = 0. \tag{8.9}$$

Solving this system of linear equations corresponds to solving a standard least squares problem with (nonfixed) weights in the form

$$\sum_{i=1}^{n} w_i e_i^2. \tag{8.10}$$

However, the weights w_i depend on the residuals e_i, the residuals depend on the coefficients a_i, and the coefficients depend on the weights. Therefore, it is in general not possible to provide an explicit solution to the system of equations. Instead, the following iteration scheme is applied:

1. Choose an initial solution $\mathbf{a}^{(0)}$, for instance, the standard least squares solution setting all weights to $w_i = 1$.
2. In each iteration step t, calculate the residuals $e^{(t-1)}$ and the corresponding weights $w^{(t-1)} = w(e^{(t-1)})$ determined by the previous step.
3. Compute the solution of the weighted least squares problem $\sum_{i=1}^{n} w_i e_i^2$, which leads to

$$\mathbf{a}^{(t)} = \left(\mathbf{X}^\top \mathbf{W}^{(t-1)} X\right)^{-1} \mathbf{X}^\top \mathbf{W}^{(t-1)} \mathbf{y}, \tag{8.11}$$

where $\mathbf{W}$ stands for the diagonal matrix with weights w_i on the diagonal.

Table 8.4 The computation of the weights for the corresponding approaches

Method	$w(e)$
Least squares	1
Huber	$\begin{cases} 1 & \text{if } \lvert e\rvert \le k, \\ k/\lvert e\rvert & \text{if } \lvert e\rvert > k. \end{cases}$
Tukey's bisquare	$\begin{cases} (1-(\frac{e}{k})^2)^2 & \text{if } \lvert e\rvert \le k, \\ 0 & \text{if } \lvert e\rvert > k. \end{cases}$

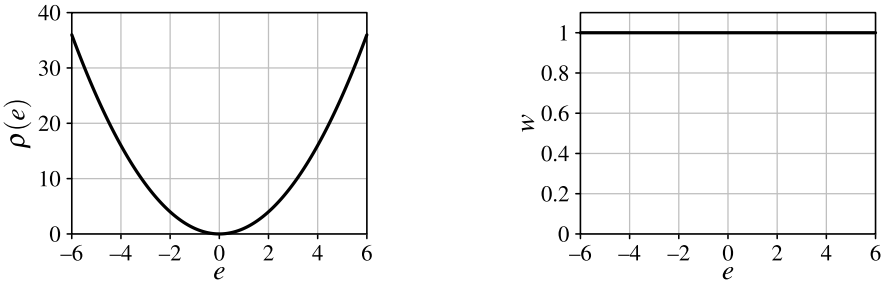


Fig. 8.10 The error measure ρ and the weight w for the standard least squares approach

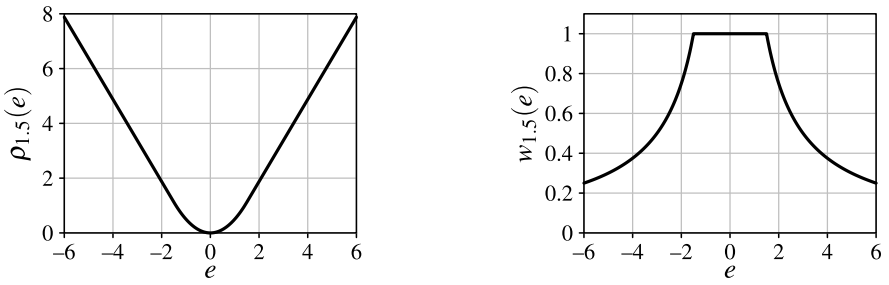


Fig. 8.11 The error measure ρ and the weight w for Huber's approach

Table 8.4 lists the formulae for the weights in the regression scheme based on the error measures listed in Table 8.3.

Figure 8.10 shows the graph of the error measure ρ and the weighting function for the standard least squares approach. The error measure ρ increases in a quadratic manner with increasing distance. The weights are always constant. This means that extreme outliers will have full influence on the regression coefficients and can corrupt the result completely.

In the more robust approach by Huber, the change of the error measure ρ switches from a quadratic increase for small errors to a linear increase for larger errors. As a result, only data points with small errors will have the full influence on the regression coefficients. For extreme outliers, the weights tend to zero. This is illustrated by the corresponding graphs in Fig. 8.11.

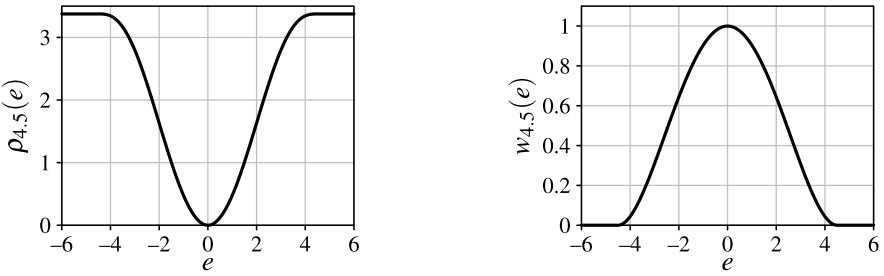
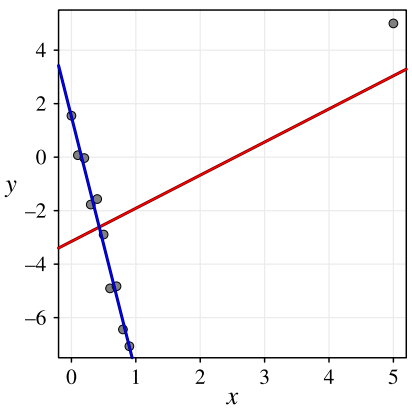


Fig. 8.12 The error measure ρ and the weight w for the bisquare approach

Fig. 8.13 Least squares (*red*) vs. robust regression (*blue*)



Tukey's bisquare approach is even more drastic than Huber's approach. For larger errors, the error measure ρ does not increase at all but remains constant. As a consequence, the weights for outliers drop to zero when they are too far away from the regression curve. This means that extreme outliers have no influence on the regression curve at all. The corresponding graphs for the error measure and the weights are shown in Fig. 8.12.

To illustrate how robust regression works, consider the simple regression problem in Fig. 8.13. There is one outlier that leads to the red regression line that neither fits the outlier nor the other points. With robust regression, for instance, based on Tukey's ρ-function, we obtain the blue regression line that simply ignores the outlier.

An additional result in robust regression is the computed weights for the data points. The weights for the regression problem in Fig. 8.13 are plotted in Fig. 8.14. All weights, except one, have a value close to 1. The right-most weight with the value close to 0 is the weight for the outlier. In this way, the outlier can be identified by robust regression. This applies also to the case of multivariate regression where we cannot simply plot the regression function as in Fig. 8.13. But the weights can still be computed and plotted. We can also take a closer look at the data points with low weights for the regression function. They might be exceptional cases or even erroneous measurements.

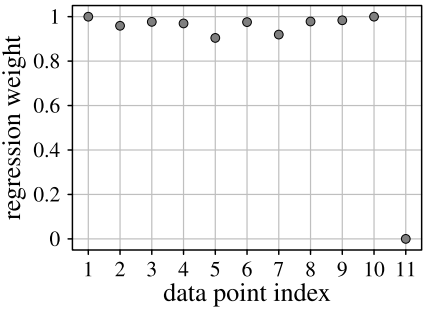


Fig. 8.14 The weights for the robust regression function in Fig. 8.13

8.3.3.4 Function Selection

In their basic form, regression approaches require a conjecture about the form of the functional relationship between the involved variables, so that the task only consists in estimating its parameters. However, sometimes several different function classes present themselves as reasonable hypotheses. For example, one may be reasonably sure that the relationship can be described by a polynomial but does not know the degree. In such a case one can try to determine the function class automatically with an approach that is based on the minimum description length principle.

The core idea of this principle is to imagine that the training data has to be transmitted as efficiently as possible (that is, using a message of minimal length) from a sender to a receiver, possibly using a model to code the data. In this way it becomes possible to measure the complexity of the model (which, if used, needs to be specified in the message, so that the receiver can decode the data) and the data on the same scale, namely as the length of a message describing it. Therefore one goes through the different functions in the hypothesized family, builds the best model for each function, and then determines which model, if used to code the data, yields the shortest message (model description plus data description).

Applied to regression, this general principle takes the form of functions that sum different numbers of monomials (products of powers of the regressors). The coefficients of these monomials are determined to build the model (using the techniques described above). The model description consists of a specification of the monomials that are used in the function and an encoding of the corresponding coefficients (for which a finite precision has to be fixed) plus a variance for the deviations of the response values from the function. The data is described by specifying how far the value of the response variable differs from the value predicted by the model, assuming a normal distribution with the specified variance for coding.

Another approach to select an appropriate function is based on bias/variance decomposition (see Sect. 5.4.5): a more complex function has a lower bias, because it can adapt better to the actual function due to its larger number of degrees of freedom (larger number of free parameters). However, it leads to a higher variance, because the more parameters have to be estimated, the less reliable these estimates become. As a consequence, we would like to minimize the total, that is, lower the bias only as far as the gain is not eaten up by an increased variance.

8.3.4 Two-Class Problems

Even though mainly directed at numeric prediction problems, regression, especially logistic regression, is also a popular method for solving classification problems with two classes. That is, each given data point is assigned to one of two classes, and we desire to find a function (a so-called **discrimination function**) that maps the values of the other, descriptive attributes to a class. More precisely, we desire to find a function that models the probability of one of the classes for the different points in the data space. In this way the classification problem is conveniently mapped to a numeric prediction problem, and, in addition, the classifier yields not only a class but also a confidence value, namely the class probability. This approach is particularly popular in the finance industry to model the credit-worthiness of customers.

Formally the problem to solve can be described as follows: let Y be a class attribute with the domain $\mathrm{dom}(Y) = \{c_1, c_2\}$, and $\mathbf{X} = (X_1, \ldots, X_m)$ an m-dimensional random vector. Furthermore, let

$$P(Y = c_1 \mid \mathbf{X} = \mathbf{x}) = p(\mathbf{x}) \quad \text{and thus}$$
$$P(Y = c_2 \mid \mathbf{X} = \mathbf{x}) = 1 - p(\mathbf{x}).$$

Finally, let a data set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ be given, the elements of which are assigned to one of the classes c_1 or c_2 (that is, $y_i = c_1$ or $y_i = c_2$ for $i = 1, \ldots, n$).

We desire to find a reasonably simple description of the function $p(\mathbf{x})$, the parameters of which have to be estimated from the data set $\mathbf{X}$. A common approach is to model $p(\mathbf{x})$ as a logistic function, that is, as

$$p(\mathbf{x}) = \frac{1}{1 + e^{a_0 + \mathbf{ax}}} = \frac{1}{1 + \exp(a_0 + \sum_{i=1}^{r} a_i x_i)}.$$

By applying the logit-transformation we discussed in Sect. 8.3.3.1, we obtain

$$\ln\left(\frac{1 - p(\mathbf{x})}{p(\mathbf{x})}\right) = a_0 + \mathbf{ax} = a_0 + \sum_{i=1}^{r} a_i x_i,$$

that is, a multilinear regression problem, which can easily be solved with the techniques introduced above.

What remains to be clarified is how we determine the values $p(\mathbf{x})$ that enter the above equation. If the data space is small enough, so that there are sufficiently many realizations for every possible point (that is, for every possible instantiation of the random variables $X_1, \ldots, X_m$), we may estimate the class probabilities simply as the relative frequencies of the classes (see Sect. A.4.2 in the appendix).

If this is not the case, we may rely on an approach known as **kernel estimation** in order to determine the class probabilities at the data points. The basic idea of such an estimation is to define a kernel function K which describes how strongly a data point influences the estimation of the probability (density) at a neighboring point (see Sect. 9.1.3.1 for a related approach in connection with k-nearest-neighbor

classifiers and Sect. 9.4 on support vector machines). The most common choice is a Gaussian function, that is,

$$K(\mathbf{x}, \mathbf{y}) = \frac{1}{(2\pi\sigma^2)^{\frac{m}{2}}} \exp\left(-\frac{(\mathbf{x}-\mathbf{y})^\top(\mathbf{x}-\mathbf{y})}{2\sigma^2}\right),$$

where the variance σ^2 has to be chosen by a user. With the help of this kernel function, the probability density at a point $\mathbf{x}$ is estimated from an (unclassified) data set $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ as

$$\hat{f}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} K(\mathbf{x}, \mathbf{x}_i).$$

If we deal with a two-class problem, one estimates the class probabilities by relating the probability density resulting from datapoints of one of the classes to the total probability density. That is, we estimate

$$\hat{p}(\mathbf{x}) = \frac{\sum_{i=1}^{n} c(\mathbf{x}_i) K(\mathbf{x}, \mathbf{x}_i)}{\sum_{i=1}^{n} K(\mathbf{x}, \mathbf{x}_i)},$$

where

$$c(\mathbf{x}_i) = \begin{cases} 1 & \text{if } x_i \text{ belongs to class } c_1, \\ 0 & \text{if } x_i \text{ belongs to class } c_2. \end{cases}$$

Solving the resulting regression problem yields a (multidimensional) logistic function, which describes the probability of one of the two classes for the points of the data space. For this function, one has to choose a threshold value: if the function value exceeds the threshold, the class the function refers to is predicted; otherwise the other class is predicted. Note that with such a threshold a linear separation of the input space is described (see also Sect. 9.4).

If this method is used in finance to assess the credit-worthiness of a customer, one of the classes means that the loan applied for is granted, while the other means that the application is declined. As a consequence, several threshold values are chosen, which refer to different loan conditions (interest rate, liquidation rate, required securities, loan duration, etc.)

8.3.5 Regularization for Logistic Regression

We have already seen in Sect. 5.5.4 that regularization is often used to compromise between model complexity and model fit. It is often applied to halt the growing complexity of the model during training and thus avoid overfitting. Regularization is a general mathematical technique that introduces a penalty term for more complex models during training as to avoid overfitting.

The same challenge between simplicity of the model and overfitting can be found also in the logistic regression. And in the logistic regression, as in many other machine learning algorithms, regularization techniques can be applied to find the optimal point in between these two extreme situations. In this section, we will cover a

few common regularization techniques applied to the training of the logistic regression model and we will explain the differences among them.

According to Ian Goodfellow [11] "Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error". In other words: regularization can be used to train models that generalize better on unseen data, by preventing the algorithm from overfitting the training data set.

The most common regularization techniques are **Gauss**, **Laplace**, **L1**, and **L2** that are explained below. However, it can be proven that L2 and Gauss or L1 and Laplace regularization have an equivalent impact on the logistic regression algorithm. There are two approaches to attain the regularization effect:

- adding a regularization term to the log-likelihood function LL,
- the Bayesian view of regularization.

Often, in a logistic regression avoiding an excessive growth of the values of the coefficient vector β also avoids overfitting. Thus, the regularization goal consists of penalizing systems with too high values for the coefficients β.

To calculate the coefficients of a logistic regression, the negative of the log-likelihood function, also called the objective function, must be minimized. The first approach penalizes high coefficients β by adding a regularization term $R(\beta)$ to the objective function multiplied by a parameter $\lambda > 0$:

$$\hat{\beta} = \min_{\beta} \{-LL(\beta, y, \mathbf{X}) + \lambda R(\beta)\},$$

where y stands for the dependent (target) variable and $\mathbf{X}$ for the independent variables.

The two common regularization terms, which are added to penalize high coefficients, are the L_1-norm or the Euclidean norm L_2 divided by 2, which motivates the names L1 and L2 regularization. Division by 2 is used for convenience reasons. In some derivations of the L2 regularization, it makes it easier to calculate the gradient. However, it is only a constant value that can be compensated by the choice of the parameter λ.

The L1 regularization term is defined via the L_1-norm as follows, i.e., as the sum of the absolute values of the coefficients, corresponding to the Manhattan distance,

$$R(\beta) = \frac{1}{2}\|\beta\|_2^2 = \frac{1}{2}\sum_i \beta_i^2.$$

The second approach assumes a given prior probability density of the coefficients and uses the Maximum A Posteriori estimate (MAP) approach [19]. For example, we assume the coefficients to be Gaussian distributed with mean 0 and variance σ^2 or Laplace distributed with variance σ^2. In this case we can control the impact of the regularization through the choice of the variance. Smaller values lead to smaller coefficients. Here, however, small values of σ^2 can lead to underfitting.



Fig. 8.15 Plots in the upper part show the performance measures for the different priors. All three performance measures increase if regularization is used. In the lower part the coefficients for the different priors are plotted versus the number of features. The plots show the different impacts of Gaussian and Laplace priors on the coefficients and that regularization in general leads to smaller coefficients

The two mentioned approaches are closely related and, with the correct choice of the control parameters λ and σ^2, lead to equivalent results for the logistic regression algorithm.

Is regularization really necessary? To understand this part, we designed a little experiment, where we used a subset of the Internet Advertisement data set from the UCI Machine Learning Repository. The subset contains more input features (680) than samples (120), thus favoring overfitting. We then train three logistic regression models with different regularization options: uniform prior probability density, i.e., no regularization, Laplace prior with variance $\sigma^2 = 0.1$, and Gaussian prior with variance $\sigma^2 = 0.1$.

In Fig. 8.15 the results are displayed. The upper part shows different performance measures for the three models, e.g., the accuracies, Cohen's kappa, and the ROC curve. In general, we can say that for the considered example, with a data set favoring overfitting, the regularized models perform much better. For example, the

accuracy increases from 87.2% to 93.9% for Gauss and to 94.8% for Laplace. We also get higher values for Cohen's kappa and for the AUC value.

So far, we have seen that Gauss and Laplace regularization lead to a comparable improvement on performance. But do they produce also similar models? Taking a closer look at the regression coefficients, we discover some differences.

In the lower part of Fig. 8.15, the values of the coefficients versus the number of features are displayed for the different priors. Notice that the plots have different ranges on the y-axis. The two upper plots show the coefficients for Laplace and Gaussian priors. In the case of a Laplace prior many of the coefficients are zero. Indeed, Laplace regularization leads to sparse coefficient vectors and logistic regression with Laplace prior performs some form of feature selection [15, 19]. In the case of a Gaussian prior we do not get sparse coefficients, but smaller coefficients than without regularization. In other words, Gauss prior leads to smaller values, while Laplace leads to sparse coefficient vectors with a few higher values.

The two lower line plots show the coefficients of logistic regression without regularization and all coefficients in comparison with each other. The plots show that regularization leads to smaller coefficient values, as we would expect, bearing in mind that regularization penalizes high coefficients.

8.4 Rule learning

The last type of explanation-based methods that we are discussing in this chapter are rule learning methods. In contrast to association rule learners as discussed before, we are now concentrating on algorithms that generate sets of rules which explain all of the training data. Hence we are interested in a global rule system vs. association rules that are a collection of local models. These local models can be highly redundant (and they usually are), and we expect a global rule system to explain the data reasonably free of overlaps. Rule systems are one of the easiest ways (if not the easiest) to express knowledge in a human-readable form. By the end of this section we hope to have given a better understanding of what these types of methods can achieve and why they are still relatively unknown to data analysis practitioners.

Rule learning algorithms can be roughly divided into two types of methods. Simple, "if this fact is true, then that fact holds"-type rules and more complex rules which allow one to include variables, something along the lines of "if x has wings then x is a bird".[7] The first type of rules are called propositional rules and will be discussed first. Afterwards we will then discuss first-order rules or the field of inductive logic programming as this area of rule learning is often also called.

8.4.1 Propositional Rules

Propositional rules are rules consisting of atomic facts and combinations of those using logical operators. In contrast to first-order logic (see next section), no variables are allowed to be part of those rules. A simple propositional classification rule could

[7] Note that we are not saying much about the truthfulness or precision of rules at this stage.

look like this:

$$\text{IF } x_1 \leq 10 \text{ AND } x_3 = \textit{red} \text{ THEN class A.}$$

Note that we have an **antecedent** part of the rule (to the left of THEN) indicating the conditions to be fulfilled in order for the **consequent** part (to the right) to be true. As with typical implications, we do not know anything about the truth value of the consequent if the antecedent does not hold. The atomic facts of propositional rules more commonly occurring in data analysis tasks are constraints on individual attributes:

- constraints on numerical attributes, such as greater/smaller (-or-equal) than a constant, equality (usually for integer values) or containment in given intervals;
- constraints on nominal attributes, such as checks for equality or containment in a set of possible values;
- constraints on ordinal attributes, which add range checks to the list of possible constraints on numeric attributes.

The rule above shows examples for a numerical constraint and a constraint on a nominal attribute. How can we now find such rules given a set of training instances?

8.4.1.1 Extracting Rules from Decision Trees

One very straight forward way to find propositional rules was already presented earlier in this chapter: we can simply train a decision tree and extract the rules from the resulting representation. The tree shown in Fig. 8.2 can also be interpreted as four (mutually exclusive—we will get back to this in a second) rules:

R_a: IF temperature ≤ 20 THEN class "cold"

R_b: IF temperature > 20 AND humidity ≤ 50 THEN class "comfortable"

R_c: IF temperature $\in (20, 35]$AND humidity > 50 THEN class "comfortable"

R_d: IF temperature > 35 AND humidity > 50 THEN class "unbearable"

Those rules are simply generated by traversing the tree from each leaf up to the root and collecting the conditions along the way. Note that in rule R_c we collapsed the two tests on temperature into one. Since the rules are disjunctive and disjunction is commutative, this does not alter the antecedent.

Rules extracted from decision trees have two interesting properties:

- *mutual exclusivity* meaning that a pattern will be explained by one rule only. This stems from the origin of the rules: they were generated from a hierarchical structure where each branch partitions the feature space into two disjoint partitions.
- *unorderness* meaning that the rules are extracted in arbitrary order from the tree, that is, no rule has preference over any other one. This is not a problem since only one (and exactly one!) rule will apply to a training pattern. However, as we will see later, rules can also overlap, and then a conflict avoidance strategy may

be needed: one such strategy requires rules to be ordered, and the first on that matches is the one creating the response.

Creating rules from decision trees is straightforward and uses well-established and efficient training algorithms. However, we inherit all of the disadvantages of decision tree learning algorithms (most notably with respect to their notorious instability!), and the rules can also be quite redundant. We can sometimes avoid this redundancy by transforming the rule set into a set of ordered rules:

R_1: IF temperature ≤ 20 THEN class "cold"
R_2: IF humidity ≤ 50 THEN class "comfortable"
R_3: IF temperature ≤ 35 THEN class "comfortable"
R_4: IF true THEN class "unbearable"

Now we really need to apply these rules in the correct order, but then they still represent exactly the same function as the original decision tree and the unordered rule set shown before. This type of conversion only works because we have an unbalanced tree—no branch going to the left actually carries a subtree. So we could simply assign labels for those branches and carry the rest forward to the next rule describing the branch to the right. In general, simplifying rule sets extracting from decision trees are considerably more complex.

8.4.1.2 Extracting Propositional Rules

Real-world propositional Rule Learners can essentially be categorized by looking at two properties, supported attribute types and learning strategy. There are a number of algorithms that learn rules on nominal attributes only and a second class which concentrates on learning rules for numerical attributes. The second categorization looks at the strategy: most if not all rule learning methods are either specializing or generalizing, that is, they start with very specific rules and make them more general until they fit the training data or they start with one very general rule and iteratively divide and/or specialize the rules until no further conflicts with the training data are encountered. Specializing and generalizing rules are concepts which are more formally defined in [17], however, for the purpose of this chapter, it suffices to understand that we can increase (or decrease) a rule's support and thereby increase (or decrease) the set of training patterns that fall within the influence of that rule's constraint.

The first type of rule learners generally starts with a set of extremely small special rules. In the extreme, these will be rules centered on one or more training examples having constraints that limit numerical values to the exact value of that instance and a precise match of the nominal value. So, for a training instance $(\mathbf{v}, k)$ with

$$\mathbf{v} = (12, 3.5, \text{red}),$$

such a rule would look like

$$\text{IF } x_1 = 12 \text{ AND } x_2 = 3.5 \text{ AND } x_3 = \text{red THEN class } k.$$

There are two typical generalization operators that can now be applied iteratively. If we start with one rule, we will attempt to make this rule cover more training instances by finding one (usually the closest) example that can be included in this rule without including any other example. A second training instance $(\mathbf{v}_2, k)$ with

$$\mathbf{v}_2 = (12.3, 3.5, \text{blue})$$

would result in a revised rule

$$\text{IF } x_1 \in [12, 12.3] \text{ AND } x_2 = 3.5 \text{AND } x_3 = \text{red OR blue THEN class } k.$$

Alternatively, we can also combine two rules into a new one, paying attention that we are not accidentally including a third rule of different class. The first approach (extending rules by covering one additional example) can be seen as a special case of the second approach (merging two rules) since we can always model a single training example as a (very) specific rule.

More generally, we can regard these training algorithms as a heuristic greedy search which iteratively tries to make rules more general by either merging two rules of the original set or by enlarging one rule to cover an additional training example.[8] The biggest question is then which two rules (or one rule and training example) to pick in the next step and how to do the generalization in order to merge the rules (or cover the additional example). As with all greedy algorithms, no matter how these two questions are answered, the resulting algorithm will usually not return the optimal set of rules, but in most cases the result will be relatively close to the optimum.

Specializing Rule Learners operate exactly the opposite way. They start with very general rules, in the extreme with one rule of the following form:

$$\text{IF true THEN class } k$$

for each class k. Then they iteratively attempt to avoid misclassifications by specializing these rules, i.e., adding new or narrowing existing constraints.

We have now, of course, not yet really talked about learning more than one rule; all we know is how to generate *one* rule for a data set. However, it is quite optimistic to assume that one simple rule will be sufficient to explain a complex real-world data set. Most rule learning algorithms hence wrap this "one rule learning" approach into an outer loop, which tries to construct an entire set of rules. This outer loop often employs a set covering strategy, also known as **sequential covering** and generically looks as shown in Table 8.5.

The initialization (steps 1 and 2) creates an empty set of rules R and sets the training instances which are still to be explained to the entire data set. The while loop runs into the performance of the rule set R, reaches a certain threshold p_{min} and iteratively creates a new rule r, adds it to the rule base R, and removes the

[8] Note that this is a substantial deviation from the abstract concepts of rule learners in Mitchell's version space setup: real-world rule learners usually do not investigate *all* more general (or more specific) rules but only a subset of those chosen by the employed heuristic(s).

Table 8.5 The basic algorithm to learn a set of rules

Algorithm BuildRuleSet($\mathcal{D}$, $p_{\min}$)

input: training data $\mathcal{D}$
parameter: performance threshold $p_{\min}$
output: a rule set R matching $\mathcal{D}$ with performance $\geq p_{\min}$

```
1   R = ∅
2   D_rest = D
3   while (Performance(R, D_rest) < p_min)
4     r = FindOneGoodRule(D_rest)
5     R = R ∪ {r}
6     D_rest = D_rest − covered(r, D_rest)
7   endwhile
8   return R
```

instances it covers from the "still to be explained" instances. Once the threshold is reached, the resulting rule set is returned.

The biggest variations in existing implementations of this base algorithm are the chosen error measure to measure the performance of a given rule (or sets of rules) and the strategy to find one good rule which optimizes this performance criterion. One of the earlier and still very prominent methods is called CN2 [6] and uses a simple generalizing rule searching heuristic as shown in Table 8.6.

This routine essentially performs a search for all hypothesis starting with a general one (line 1 and 2) and iteratively specializing them (line 4). During each iteration, the so far best hypothesis is remembered (line 5). There are two heuristics involved controlling the specialization (line 4) and the termination criteria; the latter is done in line 6, where all newly generated hypotheses which do not fulfill a validity criteria are eliminated. For this, CN2 uses a significance test on the data set. The specialization in line 4 returns only consistent, maximally specific hypotheses. In line 8, finally, a rule is assembled, assigning to the chosen antecedent the majority class of the patterns covered by the best hypothesis.

CN2 therefore not only returns 100% correct hypotheses but also rules which do make some errors on the training data—the amount of this error is controlled by the statistical significance test in the update-routing used in line 6. The CN2 algorithm essentially performs a beam search with variable beam width (controlled by the significance test) and evaluates all hypotheses generated within the beam based on the performance criteria used also in the main algorithm.

We have not yet specified which attribute types we are dealing with, the "specialized" routine just assumes that we know how to specialize (or generalize in other types of algorithms) our rules. For nominal attributes it is pretty straightforward: specialization removes individual values from the set of allowed ones, and generalization adds some. In the extreme we have either only one value left (leaving none does not make much sense as it results in a rule that is never valid) or all possible values, resulting in a constraint that is always true. But what about numerical values? The most specific case is easy as it translates an equality to one exact numerical value. Also the most generic case is simple: it results in an interval from minus to

Table 8.6 The algorithm to learn one rule as used in CN2

Algorithm FindOneGoodRule($\mathcal{D}_{\text{rest}}$)

input: (subset of) training data $\mathcal{D}_{\text{rest}}$
output: one good rule r explaining some instance of the training data

```
1   h_best = true
2   H_candidates = {h_best}
3   while H_candidates ≠ ∅
4     H_candidates = specialize(H_candidates)
5     h_best = arg max_{h ∈ H_candidates ∪ {h_best}} {Performance(h, D_rest)}
6     update(H_candidates)
7   endwhile
8   return 'IF h_best THEN arg max_k {|covered_k(h_best, D_rest)|}'
```

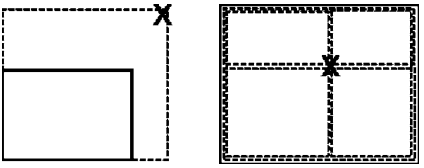


Fig. 8.16 A simple example for two-dimensional rule generalization (*left*) and specialization (*right*)

plus infinity. But also generalizing a numerical constraint so that it contains a value it previously did not contain is simple: we only have to expand the interval so that it uses this new value either as new lower or upper bound, depending on which side of the interval the value lied. Specializing is a bit more complicated as we have two options. In order to move a specific value out of a given numerical interval, we can either move the lower or upper bound to be just above (or below) the given value. This is one point where heuristics start playing a role. However, the much bigger impact of heuristics happens during multidimensional specialization operations. Figure 8.16 illustrates this for the case of two numerical attributes. On the left we show how we can generalize a given rule (solid line) to cover an additional training instance (cross). The new rule (dashed lines) is given without any ambiguity. On the right we show a given rule (solid line) and four possibilities to avoid a given conflict (cross). Note that the different rectangles are not drawn right on top of each other as they should be but slightly apart for better visibility. It is obvious that we have a number of choices and this number will not decrease with an increase in dimensionality.

However, there is one big problem with all of the current propositional rule induction approaches following the above schema: for real-world data sets, they tend to generate an enormous amount of rules. Each one of these rules is, of course, interpretable and hence fulfills the requirements of the methods described in this chapter. However, the overall set of rules is often a way too large to be even remotely comprehensible. Some approaches have been proposed to learn hierarchical rule models or find rules with outlier models, but those have not yet gained great prominence, in part also because they rely on a set of heuristics which are hard to control. In the following section we discuss a few approaches that try to address this problem.

8.4.1.3 Other Types of Propositional Rule Learners

In order to handle the larger number of rules, various approaches have been proposed. Some simply prune rules by their importance, which is usually measured by the number of patterns they cover. Others include the minimum description length principle to balance the complexity (e.g., length) of a rule against the amount of data it covers. Similarly to decision tree learning, we can of course here also employ pruning strategies to reduce collapse several rules into one or completely eliminate them. Other approaches attempt to further reduce the number of rules by looking at their discriminative power between classes or other measures of interestingness. The most prominent such measure is the **J-measure** [12], which essentially estimates how dissimilar the a priori and a posteriori beliefs about the rule's consequent are. Only if these two likelihoods are substantially different, a rule is potentially interesting. In the J-measure this difference is additionally weighted by the generality of the rule (probability that the rule's conditional part holds), because the rule becomes the more interesting the more often it applies. See Chap. 5 for further details on some of these techniques.

Note that in the toy setup, as used in the Version Spaces, another very elegant way to dramatically reduce the number of matching rules was presented: by only reporting the most general and most specific rules which cover the training data, the entire part of the lattice in between these two boundaries is reported as well. Unfortunately, this cannot as easily be applied to rules stemming from real-world data since coverage will hardly ever be exactly 100%. However, the setup described in Sect. 7.6.3.3, where we describe a propositional rule learning system finding association rules, uses a related strategy. Also, here it is hard to dig through the resulting list of association rules. However, looking at closed or maximum itemsets (and the resulting association rules) drastically reduces the set of rules to consider. So we, in effect, report only the most specific rules describing a local aspect of the training data.

One of the reasons why propositional rule learners tend to produce excessive numbers of rules, especially in numerical feature spaces, is the sharp boundaries they attempt to impose. If training instances of different class are not separable by axes parallel lines in some reasonably large local area, the rule learning algorithms are forced to introduce many small rules to model these relationships. Often the precise nature of these boundaries is not interesting or, worse, caused only by noisy data in the first place. To be able to incorporate this imprecision into the learning process, a large number of fuzzy rule learners have been introduced. They are all using the notion of fuzzy sets which allows one to model degrees of membership (to rules in this case) and hence results in the ability to model gray zones where areas of different classes overlap. Fuzzy rule systems have the interesting side effect that it is possible to express a fuzzy rule system as a system of differentiable equations of the norm and membership functions are used accordingly. Then other adaptation methods can be applied, often motivated by work in the neural network community. The resulting Neuro-Fuzzy systems allow one, for instance, to use expert knowledge to initialize a set of fuzzy rules and then employ a gradient descent training algorithm to update those rules to better fit training data. In addition Takagi–Sugeno–Kang-type fuzzy rules also allow one to build regression systems based on

fuzzy antecedents and (local) regression consequents. We refer to [3] for more details on fuzzy rules and the corresponding training methods and [18] for details on Neuro-Fuzzy systems.

8.4.2 Inductive Logic Programming or First-Order Rules

Propositional rules are quite limited in their expressive power. If we want to express a rule of the form

$$\text{IF } x \text{ is Father of } y \text{ AND } y \text{ is female THEN } y \text{ is Daughter of } x$$

using propositional rules, we would need to enumerate all possible values for x and y and put those pairs into individual rules. In order to express such types of rules, we need to introduce variables. First-order rules allow one to do just that and are based on only a few base constructs:

- constants, such as Bob, Luise, red, green,
- variables, such as x and y in the example above,
- predicates, such as is_father(x, y), which produce truth values as a result, and
- functions, such as age(x), which produce constants.

From this we can construct:

- terms which are constants, variables, or functions applied to a term,
- literals, which are predicates (or negations of predicates) applied to any set of terms,
- ground literals, which are literals that do not contain a variable,
- clauses, which are disjunctions of literals whose variables are universally quantified, and
- horn clause, which are clauses with at most one positive literal.

The latter is especially interesting because any disjunction with at most one positive literal can be written as

$$\begin{aligned} & H \vee \neg L_1 \vee \cdots \vee \neg L_n \\ & \quad \hat{=} H \Leftarrow (L_1 \wedge \cdots \wedge L_n) \\ & \quad \hat{=} \text{IF } L_1 \text{ AND} \ldots \text{AND } L_2 \text{ THEN } H. \end{aligned}$$

So horn clauses express rules and H, the *head* of the rule, is the consequent, and the L_i together form the *body* or consequent.

Horn clauses are also used to express Prolog programs, which is also the reason why learning first-order rules is often referred to as **inductive logic programming** (ILP) , because we can see this as learning (Prolog) programs, not only sets of rules.

We talk about *substitutions* which denote any replacement, called *binding* of variables in a literal with appropriate constants. A rule body is satisfied if at least one binding exists that satisfies the literals. A few example rules are:

- simple rules
  IF x is Parent of y AND x is male THEN x is Father of y
- existentially qualified variables (z in this case)
  IF y is Parent of z AND z is Parent of x THEN x is Grandchild of y
- Recursive rules
  IF x is Parent of z AND z is Ancestor of y THEN x is Ancestor of y
  IF x is Parent of y THEN x is Ancestor of y

Inductive Logic Programming allows one to learn concepts also from data spread out over several multirelation databases—without the need for previous integration as discussed in Sect. 6.5. But how do we learn such rules from data? A number of learning methods have been proposed recently, and we will explain one of earlier algorithms in more detail in the following section.

FOIL, the First Order Inductive Learning method developed by Quinlan [22], operates very similar to the rule learners we have discussed in the previous section. It also follows the strategy of a sequential covering approach—the only difference being the inner FindOneGoodRule() routine. It creates not quite Horn clauses but something very similar besides two differences:

- the rules learned by FOIL are more restrictive in that they do not allow literals to contain function symbols (this reduces the hypothesis space dramatically!), and
- FOIL rules are more expressive than Horn clauses because they allow literals in the body to be negated.

The FindOneGoodRule() routine of FOIL again specializes candidate rules greedily. It does so by adding literals one by one where these new literals can be one of the following:

- `P(v1, ..., vr)`, where `P` is any predicate name occurring in the set of available predicates. At least one of the variables must already be present in the original rule, the others can be either new or existing;
- `Equal(x, y)`, where x and y are variables already present in the rule; or
- the negation of either of the above forms of literals.

How does FOIL now pick the "best" specialization to continue? For this, Quinlan introduced a measure (FoilGain) which essentially estimates the utility of the new added literal by comparing the number of positive and negative bindings of the original and the extended rules.

This is, of course, a system that does not scale well with large, real-world data sets. ILP systems are in general not heavily used for real data analysis. They are more interesting for concept learning in data sets with purely nominal values and/or structured data sets. However, some of the ideas are used in other applications such as molecular fragment mining.

8.5 Finding Explanations in Practice

Finding explanations is, similar to finding patterns, a central piece of any data science software. The main difference is the way the extracted model is presented to the user: earlier tools produced long lists of rules or other ASCII representations, but nowadays tools increasingly offer interactive views which allow one to select individual parts of an explanation, e.g., a leaf in a decision tree, and propagate this selection to other views on the underlying data—see Sect. 4.8 for a more detailed discussion of this type of visual brushing.

8.5.1 Decision Trees

In this example we train a decision tree to predict people income (> 50K or ≤ 50K) on the adult data set. Before starting with the details of the decision tree implementation in KNIME Analytics Platform, we would like to remark that all supervised algorithms are implemented through a **Learner–Predictor** construct. The Learner node trains the model on the input data and outputs it at the output port. The Predictor node gets the model at one input port and applies it to the input data, producing the corresponding predictions at the output port.

Let's dig now a bit deeper into the decision tree nodes. When constructing decision trees, a number of options are available in the *Decision Tree Learner* node. KNIME Analytics Platform allows one to adjust the information gain criteria and a number of other options. Figure 8.17 (left) shows the dialog of the native KNIME Decision Tree Learner node.

We need to first and foremost select the target attribute (class—it has to be a nominal attribute, a string in this case). Afterwards, we can choose between two different ways to compute the information gain (Gini index and Gain ratio) and if a pruning of the tree is to be performed (the node either offers to skip pruning or performs a minimum description length (MDL)-based pruning). Most of the remaining options are self-explanatory. Noteworthy is the last option *Number threads*, which allows one to control how many threads KNIME Analytics Platform can use to execute the learning method in parallel on, e.g., a multicore machine. After training the decision tree on the training set, the model is fed into the *Decision Tree Predictor* node to produce the predictions on the test set (Fig. 8.18).

Note that the connection between the model creation and model consumption node has different port icons indicating that, along this connection, a model is transported instead of a data table.

Once the node is executed, the resulting decision tree can be displayed, either via the View in the context menu of the Decision Tree Learner node or the View in the context menu of the Decision Tree Predictor node, or via the View of the dedicated *Decision Tree View* node. Figure 8.17 (right) shows the resulting tree from the Decision Tree Learner node. The nodes of the tree show the data distribution colored by sex, along with the split attribute and the majority class. The numbers in each node show the absolute number and the percentage of training data rows correctly classified over all the training data rows falling into this branch. A similar interactive view of the decision tree is also offered by the Decision Tree View node.



Fig. 8.17 Configuration window of the Decision Tree Learner node (*left*) along with the tree view (*right*) after execution showing the decision tree trained for the adult data set

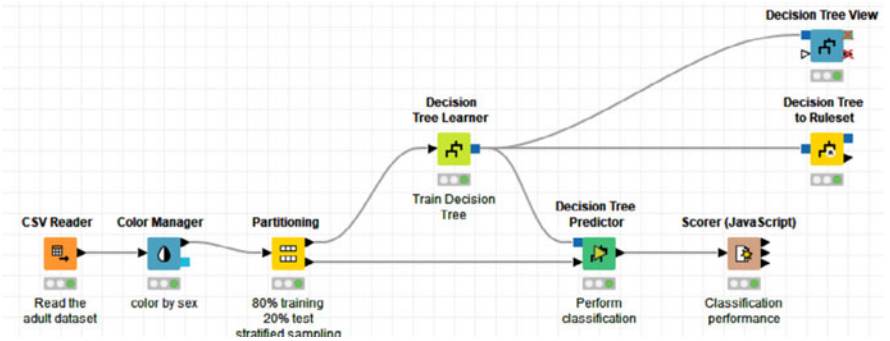


Fig. 8.18 Workflow training a decision tree to classify Income on the adult data set. The decision tree is built on the training data via the Learner node and applied to the test data via the Predictor node. Notice the Color Manager node to color the data distribution in each node of the tree in the view

Finally, the last node of the workflow (Fig. 8.18) translates the splits of the tree into a set of classic IF–THEN rules.

8.5.2 Naïve Bayes

Many other explanation finding methods are available in KNIME Analytics Platform, among them, Naïve Bayes, as described in Sect. 8.2. The corresponding node produces the class-conditional probabilities or normal distributions which can be

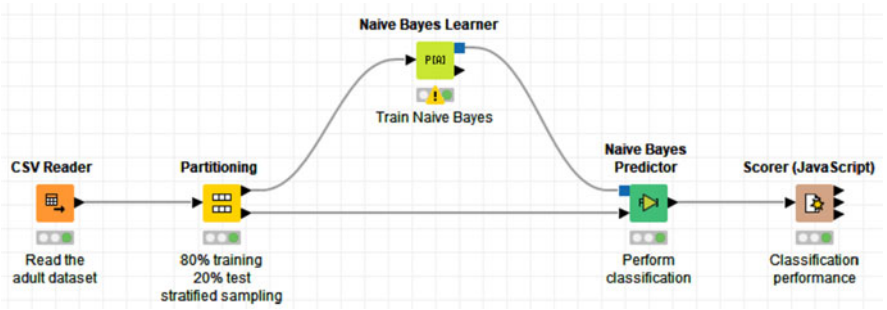


Fig. 8.19 Example workflow for the Naïve Bayes nodes

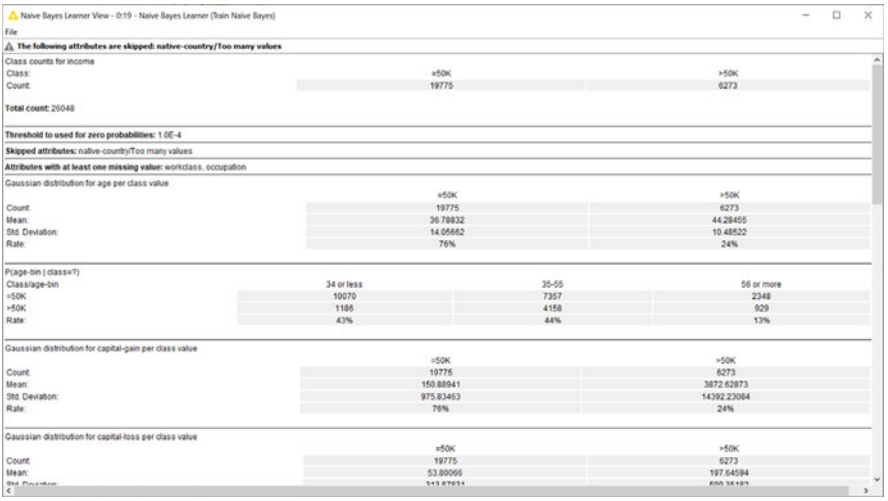


Fig. 8.20 View of the Naïve Bayes Learner node showing the conditional probabilities and distributions involved in the decision process

inspected in the node's view. An example workflow is shown in Fig. 8.19. This workflow implements the same classification task on Income column of the adult data set as in the decision tree section. Again, notice the Learner–Predictor schema. Figure 8.20 shows the model view for the adult data set.

8.5.3 Logistic Regression

KNIME Analytics Platform also provides nodes for Linear, Polynomial, and Logistic Regression. Those methods are all accessible via individual nodes, and their dialogs (and inline documentation) should explain the available options sufficiently. In Fig. 8.21 you can see the training and application of a logistic regression model to solve the classification task using column Income as target column from the adult

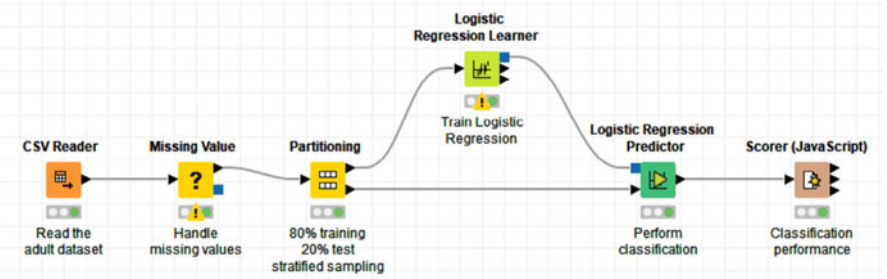


Fig. 8.21 Training and application of a logistic regression model. Notice the Missing Value node to fix possible missing values in the data

data set. Besides the Learner–Predictor schema, notice here the Missing Value node. This is necessary since logistic regression cannot deal with missing values in the training data. The Logistic Regression Learner node has three output ports: the top one outputs the model; the one in the middle the coefficients and the statistics; the one at the bottom the final value of the log-likelihood and the number of iterations required.

8.6 Further Reading

Explanation Finding Methods are almost always the centerpiece of data mining books, so any book on this topic will likely cover most, if not all, of what we discussed in this chapter. For Decision trees, there are two main directions:

- Classification and Regression Trees (CART), which are covered in much more detail in the following book: Hastie, T., Tibshirani, R. and Friedman, J.H.: *Elements of Statistical Learning* (Springer, 2001).
- Quinlan's c4.5 algorithm and subsequent improvements, with the original text book describing c4.5r8: J.R. Quinlan: *Induction of Decision Trees* (Springer, 1986).

For Bayes classifiers and regression, any more statistically oriented book offers more background; these (and much more) with a more statistical view are also described in the book by Hastie, Tibshirani, and Friedman.

Tom Mitchell's excellent *Machine Learning* book (McGraw Hill, 1997) introduces the concept of version space learning and also describes decision tree induction, among others. In order to get a better feeling for how learning systems work in general, this is highly recommended reading, especially the first half. For fuzzy rule learning systems, we recommend the chapter on "Fuzzy Logic" in *Intelligent Data Analysis: An Introduction*, M.R. Berthold and D.J. Hand (Eds), published by Springer Verlag (2003) and the book on *Foundations of Neuro-Fuzzy Systems* by D. Nauck, F. Klawonn, and R. Kruse (Wiley, 1997). For inductive logic programming, Peter Flach and Nada Lavrac contributed a nice chapter in the *Intelligent Data Analysis* book edited by Berthold and Hand.

References

1. Albert, A.: Regression and the Moore–Penrose Pseudoinverse. Academic Press, New York (1972)
2. Anderson, E.: The irises of the Gaspe Peninsula. Bull. Am. Iris Soc. **59**, 2–5 (1935)
3. Berthold, M.R.: Fuzzy logic. In: Berthold, M.R., Hand, D.J. (eds.) Intelligent Data Analysis: An Introduction, 2nd edn. Springer, Berlin (2003)
4. Borgelt, C., Steinbrecher, M., Kruse, R.: Graphical Models—Representations for Learning, Reasoning and Data Mining, 2nd edn. Wiley, Chichester (2009)
5. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: CART: Classification and Regression Trees. Wadsworth, Belmont (1983)
6. Clark, P., Niblett, T.: The CN2 induction algorithm. Mach. Learn. **3**(4), 261–283 (1989)
7. Domingos, P., Pazzani, M.: On the optimality of the simple Bayesian classifier under zero-one loss. Mach. Learn. **29**, 103–137 (1997)
8. Fisher, R.A.: The use of multiple measurements in taxonomic problems. Ann. Eugen. **7**(2), 179–188 (1936)
9. Friedman, N., Goldszmidt, M.: Building classifiers using Bayesian networks. In: Proc. 13th Nat. Conf. on Artificial Intelligence (AAAI'96, Portland, OR, USA), pp. 1277–1284. AAAI Press, Menlo Park (1996)
10. Geiger, D.: An entropy-based learning algorithm of Bayesian conditional trees. In: Proc. 8th Conf. on Uncertainty in Artificial Intelligence (UAI'92, Stanford, CA, USA), pp. 92–97. Morgan Kaufmann, San Mateo (1992)
11. Goodfellow, I., Bengio, Y. Courville, A.: Deep Learning, The MIT Press, London (2017)
12. Goodman, R.M., Smyth, P.: An information-theoretic model for rule-based expert systems. In: Int. Symposium in Information Theory (Kobe, Japan) (1988)
13. Janikow, C.Z.: Fuzzy decision trees: issues and methods. IEEE Trans. Syst. Man Cybern., Part B **28**(1), 1–14 (1998)
14. Jensen, F.V., Nielsen, T.D.: Bayesian Networks and Decision Graphs, 2nd edn. Springer, London (2007)
15. Jurafsky, D., Martin, J.H.: Speech and Language Processing, 2nd edn. Prentice Hall, Englewood Cliffs (2008)
16. Larrañaga, P., Poza, M., Yurramendi, Y., Murga, R., Kuijpers, C.: Structural learning of Bayesian networks by genetic algorithms: a performance analysis of control parameters. IEEE Trans. Pattern Anal. Mach. Intell. **18**, 912–926 (1996)
17. Mitchell, T.: Machine Learning. McGraw-Hill, New York (1997)
18. Nauck, D., Klawonn, F., Kruse, R.: Neuro-Fuzzy Systems. Wiley, Chichester (1997)
19. Ng, A.: Feature selection, L_1 vs L_2 regularization, and rotational invariance. In: ICML '04 Proceedings of the Twenty-First International Conference on Machine Learning (2004)
20. Quinlan, J.R.: Induction of decision trees. Mach. Learn. **1**(1), 81–106 (1986)
21. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo (1993)
22. Quinlan, J.R., Cameron-Jones, R.M.: FOIL: a midterm report. In: Proc. European Conference on Machine Learning. Lecture Notes in Computer Science, vol. 667, pp. 3–20. Springer, Berlin (1993)
23. Sahami, M.: Learning limited dependence Bayesian classifiers. In: Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD'96, Portland, OR, USA), pp. 335–338. AAAI Press, Menlo Park (1996)